DIGITAL ENROLLMENT MANAGEMENT SYSTEM

Line-by-Line Source Code Explanation

Senior High School Enrollment System

Java source files and SQL database script

This guide follows the actual .java and .sql files inside the uploaded project archive. Each source line is shown with a plain-English explanation. Repeated syntax such as braces, imports, and common JDBC operations is explained in context. Blank lines are identified as spacing.

Important: The database connection file contains a configured local database address and credentials. Keep credentials private and change them before using the application outside a controlled school/lab environment. The explanation describes what the source code does; it does not certify that a live MySQL server is running.

How to Read This Guide

• **Source line:** The original line is displayed in a shaded code block.

• **Explanation:** A plain-English description of the line's role in the program.

• **Java files:** Covered in alphabetical order, including GUI screens, services, data-access classes, and user models.

• **SQL file:** The database schema script is covered after the Java files.

• For a multi-line expression, the explanation may refer to the surrounding block because Java syntax often spans several lines.

Project files included: 29 Java source files and 1 SQL file(s).

AdminDashboard.java

Project path: src/AdminDashboard.java

```
0001 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0002 | import javax.swing.*;
```

Explanation: Imports Swing GUI classes used to build the application's windows and controls.

```
0003 | import java.awt.*;
```

Explanation: Imports AWT classes used for layout, dimensions, colors, fonts, or UI events.

```
0004 | import java.sql.*;
```

Explanation: Imports JDBC classes used to communicate with a relational database.

```
0005 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0006 | public class AdminDashboard extends JFrame {
```

Explanation: Declares the AdminDashboard type, which groups related data and behavior.

```
0007 |     private User user;
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0008 |     private DbConnection db = new DbConnection();
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0009 |     private ReportDAO reportDAO = new ReportDAO();
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0010 |     private FeeDAO feeDAO = new FeeDAO();
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0011 |     private UserDAO userDAO = new UserDAO();
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0012 |     private AuthService authService = new AuthService();
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0013 |     private AdminSectionActions sectionActions =
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0014 |             new AdminSectionActions(this);
```

Explanation: Creates an instance of the specified class or object.

```
0015 |     private JTextArea outputArea = new JTextArea();
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0016 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0017 |     public AdminDashboard(User user) {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0018 |         this.user = user;
```

Explanation: Assigns or updates a variable with the value or result on the right-hand side.

```
0019 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0020 |         setTitle("Student Enrollment System - Admin Dashboard");
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0021 |         setSize(1000, 650);
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0022 |         setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0023 |         setLocationRelativeTo(null);
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0024 |         setLayout(new BorderLayout(10, 10));
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0025 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0026 |         JLabel heading = new JLabel(
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0027 |                 "ADMIN DASHBOARD", JLabel.CENTER);
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0028 |         heading.setFont(new Font("Arial", Font.BOLD, 22));
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0029 |         add(heading, BorderLayout.NORTH);
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0030 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0031 |         outputArea.setEditable(false);
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0032 |         outputArea.setFont(new Font("Monospaced", Font.PLAIN, 13));
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0033 |         add(new JScrollPane(outputArea), BorderLayout.CENTER);
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0034 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0035 |         JPanel menuPanel = new JPanel(
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0036 |                 new GridLayout(0, 1, 6, 6));
```

Explanation: Creates an instance of the specified class or object.

```
0037 |         menuPanel.setBorder(
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0038 |                 BorderFactory.createEmptyBorder(10, 10, 10, 10));
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0039 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0040 |         addButton(menuPanel, "Create SHS Section", sectionActions::addSection);
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0041 |         addButton(menuPanel, "View SHS Sections", this::viewSections);
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0042 |         addButton(menuPanel, "Update Section", sectionActions::updateSection);
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0043 |         addButton(menuPanel, "Delete Section", sectionActions::deleteSection);
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0044 |         addButton(menuPanel, "Add School Year", this::addSchoolYear);
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0045 |         addButton(menuPanel, "View School Years", this::viewSchoolYears);
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0046 |         addButton(menuPanel, "Add Strand", this::addStrand);
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0047 |         addButton(menuPanel, "View Strands", this::viewStrands);
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0048 |         addButton(menuPanel, "Add Fee", this::addFee);
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0049 |         addButton(menuPanel, "View Fees", this::viewFees);
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0050 |         addButton(menuPanel, "Create Staff/Cashier Account",
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0051 |                 this::createStaffAccount);
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0052 |         addButton(menuPanel, "Enrollment Report", this::viewEnrollmentReport);
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0053 |         addButton(menuPanel, "Logout", this::logout);
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0054 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0055 |         add(new JScrollPane(menuPanel), BorderLayout.WEST);
```

Explanation: Adds a component to a Swing container so it becomes part of the displayed interface.

```
0056 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0057 |         viewSections();
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0058 |     }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0059 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0060 |     private void addButton(
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0061 |             JPanel panel, String text, Runnable action) {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0062 |         JButton button = new JButton(text);
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0063 |         button.addActionListener(e -&gt; action.run());
```

Explanation: Registers an event handler that runs when the associated GUI control is activated.

```
0064 |         panel.add(button);
```

Explanation: Adds a component to a Swing container so it becomes part of the displayed interface.

```
0065 |     }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0066 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0067 |     public void viewSections() {
```

Explanation: Declares the viewSections method and its return type/parameters; the method performs the operation described by its name and body.

```
0068 |         try {
```

Explanation: Begins a protected block for code that may throw an exception; resources may be managed by try-with-resources.

```
0069 |             outputArea.setText(reportDAO.getSectionsReport());
```

Explanation: Sets the displayed text of a GUI component.

```
0070 |         } catch (SQLException ex) {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0071 |             showError("Could not load sections: " + ex.getMessage());
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0072 |         }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0073 |     }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0074 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0075 |     private void addSchoolYear() {
```

Explanation: Declares the addSchoolYear method and its return type/parameters; the method performs the operation described by its name and body.

```
0076 |         String year = JOptionPane.showInputDialog(
```

Explanation: Displays a Swing dialog to communicate a message, warning, confirmation, or input prompt to the user.

```
0077 |                 this, "Enter school year (example: 2026-2027):");
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0078 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0079 |         if (year == null || year.trim().isEmpty()) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0080 |             return;
```

Explanation: Ends the current method without returning a value.

```
0081 |         }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0082 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0083 |         String sql =
```

Explanation: Stores the SQL command as a string so the program can execute it against the database.

```
0084 |                 "INSERT INTO school_years (year_label, is_active) "
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0085 |                         + "VALUES (?, FALSE)";
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0086 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0087 |         Connection conn = null;
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0088 |         PreparedStatement ps = null;
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0089 |         try {
```

Explanation: Begins a protected block for code that may throw an exception; resources may be managed by try-with-resources.

```
0090 |             conn = db.getConnection();
```

Explanation: Assigns or updates a variable with the value or result on the right-hand side.

```
0091 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0092 |             if (conn == null) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0093 |                 throw new SQLException("Could not connect to the database.");
```

Explanation: Raises an exception so the caller or an enclosing catch block can handle the failure.

```
0094 |             }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0095 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0096 |             ps = conn.prepareStatement(sql);
```

Explanation: Creates a parameterized SQL statement; placeholders can be filled separately to reduce SQL-injection risk.

```
0097 |             ps.setString(1, year.trim());
```

Explanation: Binds a Java value to a placeholder in the prepared SQL statement; the parameter index is one-based.

```
0098 |             ps.executeUpdate();
```

Explanation: Executes an INSERT, UPDATE, or DELETE statement and returns the number of affected rows.

```
0099 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0100 |             JOptionPane.showMessageDialog(
```

Explanation: Displays a Swing dialog to communicate a message, warning, confirmation, or input prompt to the user.

```
0101 |                     this, "School year added successfully.");
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0102 |         } catch (Exception ex) {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0103 |             showError("Could not add school year: " + ex.getMessage());
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0104 |         } finally {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0105 |             try {
```

Explanation: Begins a protected block for code that may throw an exception; resources may be managed by try-with-resources.

```
0106 |                 if (ps != null) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0107 |                     ps.close();
```

Explanation: Closes the associated resource, such as a JDBC ResultSet, Statement, or Connection.

```
0108 |                 }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0109 |             } catch (Exception closeEx) {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0110 |                 System.out.println(closeEx);
```

Explanation: Prints a message or value to the console, commonly for status output or error reporting.

```
0111 |             }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0112 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0113 |             try {
```

Explanation: Begins a protected block for code that may throw an exception; resources may be managed by try-with-resources.

```
0114 |                 if (conn != null) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0115 |                     conn.close();
```

Explanation: Closes the associated resource, such as a JDBC ResultSet, Statement, or Connection.

```
0116 |                 }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0117 |             } catch (Exception closeEx) {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0118 |                 System.out.println(closeEx);
```

Explanation: Prints a message or value to the console, commonly for status output or error reporting.

```
0119 |             }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0120 |         }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0121 |     }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0122 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0123 |     private void viewSchoolYears() {
```

Explanation: Declares the viewSchoolYears method and its return type/parameters; the method performs the operation described by its name and body.

```
0124 |         String sql =
```

Explanation: Stores the SQL command as a string so the program can execute it against the database.

```
0125 |                 "SELECT school_year_id, year_label, is_active "
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0126 |                         + "FROM school_years ORDER BY school_year_id DESC";
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0127 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0128 |         StringBuilder result = new StringBuilder("SCHOOL YEARS\n\n");
```

Explanation: Creates a mutable string builder so text can be assembled efficiently.

```
0129 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0130 |         Connection conn = null;
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0131 |         PreparedStatement ps = null;
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0132 |         ResultSet rs = null;
```

Explanation: Declares or assigns a ResultSet that holds rows returned by a database query.

```
0133 |         try {
```

Explanation: Begins a protected block for code that may throw an exception; resources may be managed by try-with-resources.

```
0134 |             conn = db.getConnection();
```

Explanation: Assigns or updates a variable with the value or result on the right-hand side.

```
0135 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0136 |             if (conn == null) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0137 |                 throw new SQLException("Could not connect to the database.");
```

Explanation: Raises an exception so the caller or an enclosing catch block can handle the failure.

```
0138 |             }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0139 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0140 |             ps = conn.prepareStatement(sql);
```

Explanation: Creates a parameterized SQL statement; placeholders can be filled separately to reduce SQL-injection risk.

```
0141 |             rs = ps.executeQuery();
```

Explanation: Executes a SQL query that returns rows, usually a SELECT, and stores them in a ResultSet.

```
0142 |             while (rs.next()) {
```

Explanation: Repeats the following block while the condition remains true.

```
0143 |                 result.append("ID: ")
```

Explanation: Appends the provided value to the current StringBuilder or collection.

```
0144 |                         .append(rs.getInt("school_year_id"))
```

Explanation: Reads a column value from the current database result row using the requested Java type.

```
0145 |                         .append("\nSchool Year: ")
```

Explanation: Appends the provided value to the current StringBuilder or collection.

```
0146 |                         .append(rs.getString("year_label"))
```

Explanation: Reads a column value from the current database result row using the requested Java type.

```
0147 |                         .append("\nActive: ")
```

Explanation: Appends the provided value to the current StringBuilder or collection.

```
0148 |                         .append(rs.getBoolean("is_active"))
```

Explanation: Appends the provided value to the current StringBuilder or collection.

```
0149 |                         .append("\n------------------------------\n");
```

Explanation: Appends the provided value to the current StringBuilder or collection.

```
0150 |             }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0151 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0152 |             outputArea.setText(result.toString());
```

Explanation: Sets the displayed text of a GUI component.

```
0153 |         } catch (Exception ex) {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0154 |             showError("Could not load school years: " + ex.getMessage());
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0155 |         } finally {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0156 |             try {
```

Explanation: Begins a protected block for code that may throw an exception; resources may be managed by try-with-resources.

```
0157 |                 if (rs != null) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0158 |                     rs.close();
```

Explanation: Closes the associated resource, such as a JDBC ResultSet, Statement, or Connection.

```
0159 |                 }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0160 |             } catch (Exception closeEx) {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0161 |                 System.out.println(closeEx);
```

Explanation: Prints a message or value to the console, commonly for status output or error reporting.

```
0162 |             }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0163 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0164 |             try {
```

Explanation: Begins a protected block for code that may throw an exception; resources may be managed by try-with-resources.

```
0165 |                 if (ps != null) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0166 |                     ps.close();
```

Explanation: Closes the associated resource, such as a JDBC ResultSet, Statement, or Connection.

```
0167 |                 }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0168 |             } catch (Exception closeEx) {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0169 |                 System.out.println(closeEx);
```

Explanation: Prints a message or value to the console, commonly for status output or error reporting.

```
0170 |             }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0171 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0172 |             try {
```

Explanation: Begins a protected block for code that may throw an exception; resources may be managed by try-with-resources.

```
0173 |                 if (conn != null) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0174 |                     conn.close();
```

Explanation: Closes the associated resource, such as a JDBC ResultSet, Statement, or Connection.

```
0175 |                 }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0176 |             } catch (Exception closeEx) {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0177 |                 System.out.println(closeEx);
```

Explanation: Prints a message or value to the console, commonly for status output or error reporting.

```
0178 |             }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0179 |         }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0180 |     }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0181 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0182 |     private void addStrand() {
```

Explanation: Declares the addStrand method and its return type/parameters; the method performs the operation described by its name and body.

```
0183 |         JTextField codeField = new JTextField();
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0184 |         JTextField nameField = new JTextField();
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0185 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0186 |         JPanel form = new JPanel(new GridLayout(0, 2, 6, 6));
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0187 |         form.add(new JLabel("Strand Code:"));
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0188 |         form.add(codeField);
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0189 |         form.add(new JLabel("Strand Name:"));
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0190 |         form.add(nameField);
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0191 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0192 |         int choice = JOptionPane.showConfirmDialog(
```

Explanation: Displays a Swing dialog to communicate a message, warning, confirmation, or input prompt to the user.

```
0193 |                 this, form, "Add Strand",
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0194 |                 JOptionPane.OK_CANCEL_OPTION);
```

Explanation: Displays a Swing dialog to communicate a message, warning, confirmation, or input prompt to the user.

```
0195 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0196 |         if (choice != JOptionPane.OK_OPTION) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0197 |             return;
```

Explanation: Ends the current method without returning a value.

```
0198 |         }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0199 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0200 |         String code = codeField.getText().trim();
```

Explanation: Reads text currently entered or displayed by a GUI component.

```
0201 |         String name = nameField.getText().trim();
```

Explanation: Reads text currently entered or displayed by a GUI component.

```
0202 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0203 |         if (code.isEmpty() || name.isEmpty()) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0204 |             showError("Enter both strand code and strand name.");
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0205 |             return;
```

Explanation: Ends the current method without returning a value.

```
0206 |         }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0207 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0208 |         String sql =
```

Explanation: Stores the SQL command as a string so the program can execute it against the database.

```
0209 |                 "INSERT INTO strands (strand_code, strand_name) "
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0210 |                         + "VALUES (?, ?)";
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0211 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0212 |         Connection conn = null;
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0213 |         PreparedStatement ps = null;
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0214 |         try {
```

Explanation: Begins a protected block for code that may throw an exception; resources may be managed by try-with-resources.

```
0215 |             conn = db.getConnection();
```

Explanation: Assigns or updates a variable with the value or result on the right-hand side.

```
0216 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0217 |             if (conn == null) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0218 |                 throw new SQLException("Could not connect to the database.");
```

Explanation: Raises an exception so the caller or an enclosing catch block can handle the failure.

```
0219 |             }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0220 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0221 |             ps = conn.prepareStatement(sql);
```

Explanation: Creates a parameterized SQL statement; placeholders can be filled separately to reduce SQL-injection risk.

```
0222 |             ps.setString(1, code);
```

Explanation: Binds a Java value to a placeholder in the prepared SQL statement; the parameter index is one-based.

```
0223 |             ps.setString(2, name);
```

Explanation: Binds a Java value to a placeholder in the prepared SQL statement; the parameter index is one-based.

```
0224 |             ps.executeUpdate();
```

Explanation: Executes an INSERT, UPDATE, or DELETE statement and returns the number of affected rows.

```
0225 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0226 |             JOptionPane.showMessageDialog(
```

Explanation: Displays a Swing dialog to communicate a message, warning, confirmation, or input prompt to the user.

```
0227 |                     this, "Strand added successfully.");
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0228 |         } catch (Exception ex) {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0229 |             showError("Could not add strand: " + ex.getMessage());
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0230 |         } finally {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0231 |             try {
```

Explanation: Begins a protected block for code that may throw an exception; resources may be managed by try-with-resources.

```
0232 |                 if (ps != null) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0233 |                     ps.close();
```

Explanation: Closes the associated resource, such as a JDBC ResultSet, Statement, or Connection.

```
0234 |                 }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0235 |             } catch (Exception closeEx) {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0236 |                 System.out.println(closeEx);
```

Explanation: Prints a message or value to the console, commonly for status output or error reporting.

```
0237 |             }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0238 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0239 |             try {
```

Explanation: Begins a protected block for code that may throw an exception; resources may be managed by try-with-resources.

```
0240 |                 if (conn != null) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0241 |                     conn.close();
```

Explanation: Closes the associated resource, such as a JDBC ResultSet, Statement, or Connection.

```
0242 |                 }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0243 |             } catch (Exception closeEx) {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0244 |                 System.out.println(closeEx);
```

Explanation: Prints a message or value to the console, commonly for status output or error reporting.

```
0245 |             }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0246 |         }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0247 |     }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0248 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0249 |     private void viewStrands() {
```

Explanation: Declares the viewStrands method and its return type/parameters; the method performs the operation described by its name and body.

```
0250 |         String sql =
```

Explanation: Stores the SQL command as a string so the program can execute it against the database.

```
0251 |                 "SELECT strand_id, strand_code, strand_name "
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0252 |                         + "FROM strands ORDER BY strand_code";
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0253 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0254 |         StringBuilder result = new StringBuilder("STRANDS\n\n");
```

Explanation: Creates a mutable string builder so text can be assembled efficiently.

```
0255 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0256 |         Connection conn = null;
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0257 |         PreparedStatement ps = null;
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0258 |         ResultSet rs = null;
```

Explanation: Declares or assigns a ResultSet that holds rows returned by a database query.

```
0259 |         try {
```

Explanation: Begins a protected block for code that may throw an exception; resources may be managed by try-with-resources.

```
0260 |             conn = db.getConnection();
```

Explanation: Assigns or updates a variable with the value or result on the right-hand side.

```
0261 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0262 |             if (conn == null) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0263 |                 throw new SQLException("Could not connect to the database.");
```

Explanation: Raises an exception so the caller or an enclosing catch block can handle the failure.

```
0264 |             }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0265 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0266 |             ps = conn.prepareStatement(sql);
```

Explanation: Creates a parameterized SQL statement; placeholders can be filled separately to reduce SQL-injection risk.

```
0267 |             rs = ps.executeQuery();
```

Explanation: Executes a SQL query that returns rows, usually a SELECT, and stores them in a ResultSet.

```
0268 |             while (rs.next()) {
```

Explanation: Repeats the following block while the condition remains true.

```
0269 |                 result.append("Strand ID: ")
```

Explanation: Appends the provided value to the current StringBuilder or collection.

```
0270 |                         .append(rs.getInt("strand_id"))
```

Explanation: Reads a column value from the current database result row using the requested Java type.

```
0271 |                         .append("\nCode: ")
```

Explanation: Appends the provided value to the current StringBuilder or collection.

```
0272 |                         .append(rs.getString("strand_code"))
```

Explanation: Reads a column value from the current database result row using the requested Java type.

```
0273 |                         .append("\nName: ")
```

Explanation: Appends the provided value to the current StringBuilder or collection.

```
0274 |                         .append(rs.getString("strand_name"))
```

Explanation: Reads a column value from the current database result row using the requested Java type.

```
0275 |                         .append("\n----------------------------\n");
```

Explanation: Appends the provided value to the current StringBuilder or collection.

```
0276 |             }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0277 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0278 |             outputArea.setText(result.toString());
```

Explanation: Sets the displayed text of a GUI component.

```
0279 |         } catch (Exception ex) {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0280 |             showError("Could not load strands: " + ex.getMessage());
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0281 |         } finally {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0282 |             try {
```

Explanation: Begins a protected block for code that may throw an exception; resources may be managed by try-with-resources.

```
0283 |                 if (rs != null) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0284 |                     rs.close();
```

Explanation: Closes the associated resource, such as a JDBC ResultSet, Statement, or Connection.

```
0285 |                 }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0286 |             } catch (Exception closeEx) {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0287 |                 System.out.println(closeEx);
```

Explanation: Prints a message or value to the console, commonly for status output or error reporting.

```
0288 |             }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0289 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0290 |             try {
```

Explanation: Begins a protected block for code that may throw an exception; resources may be managed by try-with-resources.

```
0291 |                 if (ps != null) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0292 |                     ps.close();
```

Explanation: Closes the associated resource, such as a JDBC ResultSet, Statement, or Connection.

```
0293 |                 }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0294 |             } catch (Exception closeEx) {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0295 |                 System.out.println(closeEx);
```

Explanation: Prints a message or value to the console, commonly for status output or error reporting.

```
0296 |             }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0297 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0298 |             try {
```

Explanation: Begins a protected block for code that may throw an exception; resources may be managed by try-with-resources.

```
0299 |                 if (conn != null) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0300 |                     conn.close();
```

Explanation: Closes the associated resource, such as a JDBC ResultSet, Statement, or Connection.

```
0301 |                 }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0302 |             } catch (Exception closeEx) {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0303 |                 System.out.println(closeEx);
```

Explanation: Prints a message or value to the console, commonly for status output or error reporting.

```
0304 |             }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0305 |         }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0306 |     }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0307 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0308 |     private void addFee() {
```

Explanation: Declares the addFee method and its return type/parameters; the method performs the operation described by its name and body.

```
0309 |         JTextField nameField = new JTextField();
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0310 |         JTextField amountField = new JTextField();
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0311 |         JComboBox&lt;AdminSectionActions.IdOption&gt; strandBox = new JComboBox&lt;&gt;();
```

Explanation: Assigns or updates a variable with the value or result on the right-hand side.

```
0312 |         JComboBox&lt;AdminSectionActions.IdOption&gt; yearBox = new JComboBox&lt;&gt;();
```

Explanation: Assigns or updates a variable with the value or result on the right-hand side.

```
0313 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0314 |         if (!sectionActions.loadOptions(strandBox, yearBox)) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0315 |             return;
```

Explanation: Ends the current method without returning a value.

```
0316 |         }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0317 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0318 |         JPanel form = new JPanel(new GridLayout(0, 2, 6, 6));
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0319 |         form.add(new JLabel("Fee Name:"));
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0320 |         form.add(nameField);
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0321 |         form.add(new JLabel("Amount (PHP):"));
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0322 |         form.add(amountField);
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0323 |         form.add(new JLabel("Strand:"));
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0324 |         form.add(strandBox);
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0325 |         form.add(new JLabel("School Year:"));
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0326 |         form.add(yearBox);
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0327 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0328 |         int choice = JOptionPane.showConfirmDialog(
```

Explanation: Displays a Swing dialog to communicate a message, warning, confirmation, or input prompt to the user.

```
0329 |                 this, form, "Add Fee", JOptionPane.OK_CANCEL_OPTION);
```

Explanation: Displays a Swing dialog to communicate a message, warning, confirmation, or input prompt to the user.

```
0330 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0331 |         if (choice != JOptionPane.OK_OPTION) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0332 |             return;
```

Explanation: Ends the current method without returning a value.

```
0333 |         }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0334 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0335 |         String name = nameField.getText().trim();
```

Explanation: Reads text currently entered or displayed by a GUI component.

```
0336 |         AdminSectionActions.IdOption strand =
```

Explanation: Assigns or updates a variable with the value or result on the right-hand side.

```
0337 |                 (AdminSectionActions.IdOption) strandBox.getSelectedItem();
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0338 |         AdminSectionActions.IdOption year =
```

Explanation: Assigns or updates a variable with the value or result on the right-hand side.

```
0339 |                 (AdminSectionActions.IdOption) yearBox.getSelectedItem();
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0340 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0341 |         if (name.isEmpty() || strand == null || year == null) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0342 |             showError("Enter the fee name and choose a strand and school year.");
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0343 |             return;
```

Explanation: Ends the current method without returning a value.

```
0344 |         }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0345 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0346 |         try {
```

Explanation: Begins a protected block for code that may throw an exception; resources may be managed by try-with-resources.

```
0347 |             double amount = Double.parseDouble(amountField.getText().trim());
```

Explanation: Reads text currently entered or displayed by a GUI component.

```
0348 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0349 |             if (amount &lt;= 0) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0350 |                 showError("The amount must be greater than zero.");
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0351 |                 return;
```

Explanation: Ends the current method without returning a value.

```
0352 |             }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0353 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0354 |             if (feeDAO.addFee(name, amount, year.id, strand.id)) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0355 |                 JOptionPane.showMessageDialog(this, "Fee added successfully.");
```

Explanation: Displays a Swing dialog to communicate a message, warning, confirmation, or input prompt to the user.

```
0356 |                 viewFees();
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0357 |             } else {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0358 |                 showError("The fee could not be saved. Check the console message.");
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0359 |             }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0360 |         } catch (NumberFormatException ex) {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0361 |             showError("The amount must be a valid number.");
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0362 |         }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0363 |     }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0364 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0365 |     private void viewFees() {
```

Explanation: Declares the viewFees method and its return type/parameters; the method performs the operation described by its name and body.

```
0366 |         outputArea.setText("FEES\n\n" + feeDAO.getFees());
```

Explanation: Sets the displayed text of a GUI component.

```
0367 |     }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0368 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0369 |     private void createStaffAccount() {
```

Explanation: Declares the createStaffAccount method and its return type/parameters; the method performs the operation described by its name and body.

```
0370 |         JTextField usernameField = new JTextField();
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0371 |         JTextField emailField = new JTextField();
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0372 |         JPasswordField passwordField = new JPasswordField();
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0373 |         JComboBox&lt;String&gt; roleBox =
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0374 |                 new JComboBox&lt;&gt;(new String[]{"staff", "cashier"});
```

Explanation: Creates an instance of the specified class or object.

```
0375 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0376 |         JPanel form = new JPanel(new GridLayout(0, 2, 6, 6));
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0377 |         form.add(new JLabel("Username:"));
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0378 |         form.add(usernameField);
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0379 |         form.add(new JLabel("Email:"));
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0380 |         form.add(emailField);
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0381 |         form.add(new JLabel("Password:"));
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0382 |         form.add(passwordField);
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0383 |         form.add(new JLabel("Role:"));
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0384 |         form.add(roleBox);
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0385 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0386 |         int choice = JOptionPane.showConfirmDialog(
```

Explanation: Displays a Swing dialog to communicate a message, warning, confirmation, or input prompt to the user.

```
0387 |                 this, form, "Create Staff/Cashier Account",
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0388 |                 JOptionPane.OK_CANCEL_OPTION);
```

Explanation: Displays a Swing dialog to communicate a message, warning, confirmation, or input prompt to the user.

```
0389 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0390 |         if (choice != JOptionPane.OK_OPTION) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0391 |             return;
```

Explanation: Ends the current method without returning a value.

```
0392 |         }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0393 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0394 |         String username = usernameField.getText().trim();
```

Explanation: Reads text currently entered or displayed by a GUI component.

```
0395 |         String email = emailField.getText().trim();
```

Explanation: Reads text currently entered or displayed by a GUI component.

```
0396 |         String password = new String(passwordField.getPassword());
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0397 |         String role = (String) roleBox.getSelectedItem();
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0398 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0399 |         if (username.isEmpty() || email.isEmpty() || password.length() &lt; 6) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0400 |             showError("Enter a username, an email, and a password "
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0401 |                     + "of at least 6 characters.");
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0402 |             return;
```

Explanation: Ends the current method without returning a value.

```
0403 |         }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0404 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0405 |         if (userDAO.getUserId(username) != -1) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0406 |             showError("That username is already used.");
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0407 |             return;
```

Explanation: Ends the current method without returning a value.

```
0408 |         }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0409 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0410 |         String hash = authService.hashPassword(password);
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0411 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0412 |         if (userDAO.addUser(username, hash, email, role)) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0413 |             JOptionPane.showMessageDialog(
```

Explanation: Displays a Swing dialog to communicate a message, warning, confirmation, or input prompt to the user.

```
0414 |                     this, "Account created for " + username + " (" + role + ").");
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0415 |         } else {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0416 |             showError("The account could not be created. "
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0417 |                     + "The email may already be used.");
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0418 |         }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0419 |     }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0420 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0421 |     private void viewEnrollmentReport() {
```

Explanation: Declares the viewEnrollmentReport method and its return type/parameters; the method performs the operation described by its name and body.

```
0422 |         try {
```

Explanation: Begins a protected block for code that may throw an exception; resources may be managed by try-with-resources.

```
0423 |             int total = reportDAO.getTotalEnrollments();
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0424 |             String report = reportDAO.getEnrollmentReport();
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0425 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0426 |             outputArea.setText(
```

Explanation: Sets the displayed text of a GUI component.

```
0427 |                     "ENROLLMENT REPORT\n\n"
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0428 |                             + "Total Enrollments: " + total + "\n\n"
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0429 |                             + report);
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0430 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0431 |         } catch (Exception ex) {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0432 |             showError("Could not load enrollment report: "
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0433 |                     + ex.getMessage());
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0434 |         }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0435 |     }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0436 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0437 |     private void logout() {
```

Explanation: Declares the logout method and its return type/parameters; the method performs the operation described by its name and body.

```
0438 |         int choice = JOptionPane.showConfirmDialog(
```

Explanation: Displays a Swing dialog to communicate a message, warning, confirmation, or input prompt to the user.

```
0439 |                 this, "Are you sure you want to log out?",
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0440 |                 "Confirm Logout", JOptionPane.YES_NO_OPTION);
```

Explanation: Displays a Swing dialog to communicate a message, warning, confirmation, or input prompt to the user.

```
0441 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0442 |         if (choice == JOptionPane.YES_OPTION) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0443 |             dispose();
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0444 |             new LoginFrame().setVisible(true);
```

Explanation: Creates an instance of the specified class or object.

```
0445 |         }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0446 |     }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0447 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0448 |     public static void showError(Component parent, String message) {
```

Explanation: Declares the showError method and its return type/parameters; the method performs the operation described by its name and body.

```
0449 |         JOptionPane.showMessageDialog(
```

Explanation: Displays a Swing dialog to communicate a message, warning, confirmation, or input prompt to the user.

```
0450 |                 parent, message, "Admin Dashboard",
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0451 |                 JOptionPane.ERROR_MESSAGE);
```

Explanation: Displays a Swing dialog to communicate a message, warning, confirmation, or input prompt to the user.

```
0452 |     }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0453 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0454 |     private void showError(String message) {
```

Explanation: Declares the showError method and its return type/parameters; the method performs the operation described by its name and body.

```
0455 |         showError(this, message);
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0456 |     }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0457 | }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

AdminSectionActions.java

Project path: src/AdminSectionActions.java

```
0001 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0002 | import javax.swing.*;
```

Explanation: Imports Swing GUI classes used to build the application's windows and controls.

```
0003 | import java.awt.*;
```

Explanation: Imports AWT classes used for layout, dimensions, colors, fonts, or UI events.

```
0004 | import java.sql.*;
```

Explanation: Imports JDBC classes used to communicate with a relational database.

```
0005 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0006 | public class AdminSectionActions {
```

Explanation: Declares the AdminSectionActions type, which groups related data and behavior.

```
0007 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0008 |     private AdminDashboard dashboard;
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0009 |     private DbConnection db = new DbConnection();
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0010 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0011 |     public AdminSectionActions(AdminDashboard dashboard) {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0012 |         this.dashboard = dashboard;
```

Explanation: Assigns or updates a variable with the value or result on the right-hand side.

```
0013 |     }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0014 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0015 |     public void addSection() {
```

Explanation: Declares the addSection method and its return type/parameters; the method performs the operation described by its name and body.

```
0016 |         JTextField nameField = new JTextField();
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0017 |         JComboBox&lt;Integer&gt; gradeBox =
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0018 |                 new JComboBox&lt;&gt;(new Integer[]{11, 12});
```

Explanation: Creates an instance of the specified class or object.

```
0019 |         JComboBox&lt;IdOption&gt; strandBox = new JComboBox&lt;&gt;();
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0020 |         JComboBox&lt;IdOption&gt; yearBox = new JComboBox&lt;&gt;();
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0021 |         JTextField slotField = new JTextField("40");
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0022 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0023 |         if (!loadOptions(strandBox, yearBox)) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0024 |             return;
```

Explanation: Ends the current method without returning a value.

```
0025 |         }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0026 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0027 |         JPanel form = new JPanel(new GridLayout(0, 2, 6, 6));
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0028 |         form.add(new JLabel("Section Name:"));
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0029 |         form.add(nameField);
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0030 |         form.add(new JLabel("Grade Level:"));
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0031 |         form.add(gradeBox);
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0032 |         form.add(new JLabel("Strand:"));
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0033 |         form.add(strandBox);
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0034 |         form.add(new JLabel("School Year:"));
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0035 |         form.add(yearBox);
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0036 |         form.add(new JLabel("Slot Limit:"));
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0037 |         form.add(slotField);
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0038 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0039 |         int choice = JOptionPane.showConfirmDialog(
```

Explanation: Displays a Swing dialog to communicate a message, warning, confirmation, or input prompt to the user.

```
0040 |                 dashboard, form, "Create SHS Section",
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0041 |                 JOptionPane.OK_CANCEL_OPTION);
```

Explanation: Displays a Swing dialog to communicate a message, warning, confirmation, or input prompt to the user.

```
0042 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0043 |         if (choice != JOptionPane.OK_OPTION) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0044 |             return;
```

Explanation: Ends the current method without returning a value.

```
0045 |         }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0046 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0047 |         String name = nameField.getText().trim();
```

Explanation: Reads text currently entered or displayed by a GUI component.

```
0048 |         IdOption strand = (IdOption) strandBox.getSelectedItem();
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0049 |         IdOption year = (IdOption) yearBox.getSelectedItem();
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0050 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0051 |         if (name.isEmpty() || strand == null || year == null) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0052 |             AdminDashboard.showError(dashboard, "Complete all section details.");
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0053 |             return;
```

Explanation: Ends the current method without returning a value.

```
0054 |         }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0055 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0056 |         try {
```

Explanation: Begins a protected block for code that may throw an exception; resources may be managed by try-with-resources.

```
0057 |             int grade = (Integer) gradeBox.getSelectedItem();
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0058 |             int slots = Integer.parseInt(slotField.getText().trim());
```

Explanation: Reads text currently entered or displayed by a GUI component.

```
0059 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0060 |             if ((grade != 11 &amp;amp;&amp;amp; grade != 12) || slots &amp;lt; 1) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0061 |                 AdminDashboard.showError(dashboard, "Select Grade 11 or 12 and a valid slot limit.");
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0062 |                 return;
```

Explanation: Ends the current method without returning a value.

```
0063 |             }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0064 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0065 |             String sql = "INSERT INTO sections "
```

Explanation: Stores the SQL command as a string so the program can execute it against the database.

```
0066 |                     + "(section_name, grade_level, strand_id, "
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0067 |                     + "school_year_id, slot_limit, is_active) "
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0068 |                     + "VALUES (?, ?, ?, ?, ?, TRUE)";
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0069 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0070 |             Connection conn = null;
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0071 |             PreparedStatement ps = null;
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0072 |             try {
```

Explanation: Begins a protected block for code that may throw an exception; resources may be managed by try-with-resources.

```
0073 |                 conn = db.getConnection();
```

Explanation: Assigns or updates a variable with the value or result on the right-hand side.

```
0074 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0075 |                 if (conn == null) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0076 |                     throw new SQLException("Could not connect to the database.");
```

Explanation: Raises an exception so the caller or an enclosing catch block can handle the failure.

```
0077 |                 }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0078 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0079 |                 ps = conn.prepareStatement(sql);
```

Explanation: Creates a parameterized SQL statement; placeholders can be filled separately to reduce SQL-injection risk.

```
0080 |                 ps.setString(1, name);
```

Explanation: Binds a Java value to a placeholder in the prepared SQL statement; the parameter index is one-based.

```
0081 |                 ps.setInt(2, grade);
```

Explanation: Binds a Java value to a placeholder in the prepared SQL statement; the parameter index is one-based.

```
0082 |                 ps.setInt(3, strand.id);
```

Explanation: Binds a Java value to a placeholder in the prepared SQL statement; the parameter index is one-based.

```
0083 |                 ps.setInt(4, year.id);
```

Explanation: Binds a Java value to a placeholder in the prepared SQL statement; the parameter index is one-based.

```
0084 |                 ps.setInt(5, slots);
```

Explanation: Binds a Java value to a placeholder in the prepared SQL statement; the parameter index is one-based.

```
0085 |                 ps.executeUpdate();
```

Explanation: Executes an INSERT, UPDATE, or DELETE statement and returns the number of affected rows.

```
0086 |             } finally {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0087 |                 try {
```

Explanation: Begins a protected block for code that may throw an exception; resources may be managed by try-with-resources.

```
0088 |                     if (ps != null) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0089 |                         ps.close();
```

Explanation: Closes the associated resource, such as a JDBC ResultSet, Statement, or Connection.

```
0090 |                     }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0091 |                 } catch (Exception closeEx) {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0092 |                     System.out.println(closeEx);
```

Explanation: Prints a message or value to the console, commonly for status output or error reporting.

```
0093 |                 }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0094 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0095 |                 try {
```

Explanation: Begins a protected block for code that may throw an exception; resources may be managed by try-with-resources.

```
0096 |                     if (conn != null) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0097 |                         conn.close();
```

Explanation: Closes the associated resource, such as a JDBC ResultSet, Statement, or Connection.

```
0098 |                     }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0099 |                 } catch (Exception closeEx) {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0100 |                     System.out.println(closeEx);
```

Explanation: Prints a message or value to the console, commonly for status output or error reporting.

```
0101 |                 }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0102 |             }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0103 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0104 |             JOptionPane.showMessageDialog(
```

Explanation: Displays a Swing dialog to communicate a message, warning, confirmation, or input prompt to the user.

```
0105 |                     dashboard, "SHS section created successfully.");
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0106 |             dashboard.viewSections();
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0107 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0108 |         } catch (NumberFormatException ex) {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0109 |             AdminDashboard.showError(dashboard, "Slot limit must be a valid number.");
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0110 |         } catch (SQLException ex) {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0111 |             AdminDashboard.showError(dashboard, "Could not create section: " + ex.getMessage());
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0112 |         }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0113 |     }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0114 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0115 |     public boolean loadOptions(
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0116 |             JComboBox&lt;IdOption&gt; strandBox,
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0117 |             JComboBox&lt;IdOption&gt; yearBox) {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0118 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0119 |         String strandSql =
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0120 |                 "SELECT strand_id, strand_code, strand_name "
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0121 |                         + "FROM strands ORDER BY strand_code";
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0122 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0123 |         String yearSql =
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0124 |                 "SELECT school_year_id, year_label FROM school_years "
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0125 |                         + "ORDER BY school_year_id DESC";
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0126 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0127 |         Connection conn = null;
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0128 |         try {
```

Explanation: Begins a protected block for code that may throw an exception; resources may be managed by try-with-resources.

```
0129 |             conn = db.getConnection();
```

Explanation: Assigns or updates a variable with the value or result on the right-hand side.

```
0130 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0131 |             if (conn == null) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0132 |                 throw new SQLException("Could not connect to the database.");
```

Explanation: Raises an exception so the caller or an enclosing catch block can handle the failure.

```
0133 |             }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0134 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0135 |             try (PreparedStatement ps =
```

Explanation: Begins a protected block for code that may throw an exception; resources may be managed by try-with-resources.

```
0136 |                     conn.prepareStatement(strandSql);
```

Explanation: Creates a parameterized SQL statement; placeholders can be filled separately to reduce SQL-injection risk.

```
0137 |                  ResultSet rs = ps.executeQuery()) {
```

Explanation: Executes a SQL query that returns rows, usually a SELECT, and stores them in a ResultSet.

```
0138 |                 while (rs.next()) {
```

Explanation: Repeats the following block while the condition remains true.

```
0139 |                     strandBox.addItem(new IdOption(
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0140 |                             rs.getInt("strand_id"),
```

Explanation: Reads a column value from the current database result row using the requested Java type.

```
0141 |                             rs.getString("strand_code") + " - "
```

Explanation: Reads a column value from the current database result row using the requested Java type.

```
0142 |                                     + rs.getString("strand_name")));
```

Explanation: Reads a column value from the current database result row using the requested Java type.

```
0143 |                 }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0144 |             }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0145 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0146 |             try (PreparedStatement ps =
```

Explanation: Begins a protected block for code that may throw an exception; resources may be managed by try-with-resources.

```
0147 |                     conn.prepareStatement(yearSql);
```

Explanation: Creates a parameterized SQL statement; placeholders can be filled separately to reduce SQL-injection risk.

```
0148 |                  ResultSet rs = ps.executeQuery()) {
```

Explanation: Executes a SQL query that returns rows, usually a SELECT, and stores them in a ResultSet.

```
0149 |                 while (rs.next()) {
```

Explanation: Repeats the following block while the condition remains true.

```
0150 |                     yearBox.addItem(new IdOption(
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0151 |                             rs.getInt("school_year_id"),
```

Explanation: Reads a column value from the current database result row using the requested Java type.

```
0152 |                             rs.getString("year_label")));
```

Explanation: Reads a column value from the current database result row using the requested Java type.

```
0153 |                 }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0154 |             }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0155 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0156 |             if (strandBox.getItemCount() == 0
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0157 |                     || yearBox.getItemCount() == 0) {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0158 |                 AdminDashboard.showError(dashboard,
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0159 |                         "Add at least one strand and school year first.");
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0160 |                 return false;
```

Explanation: Returns the value or result to the caller and ends the current method.

```
0161 |             }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0162 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0163 |             return true;
```

Explanation: Returns the value or result to the caller and ends the current method.

```
0164 |         } catch (Exception ex) {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0165 |             AdminDashboard.showError(dashboard, "Could not load options: " + ex.getMessage());
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0166 |             return false;
```

Explanation: Returns the value or result to the caller and ends the current method.

```
0167 |         } finally {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0168 |             try {
```

Explanation: Begins a protected block for code that may throw an exception; resources may be managed by try-with-resources.

```
0169 |                 if (conn != null) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0170 |                     conn.close();
```

Explanation: Closes the associated resource, such as a JDBC ResultSet, Statement, or Connection.

```
0171 |                 }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0172 |             } catch (Exception closeEx) {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0173 |                 System.out.println(closeEx);
```

Explanation: Prints a message or value to the console, commonly for status output or error reporting.

```
0174 |             }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0175 |         }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0176 |     }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0177 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0178 |     public void updateSection() {
```

Explanation: Declares the updateSection method and its return type/parameters; the method performs the operation described by its name and body.

```
0179 |         String input = JOptionPane.showInputDialog(
```

Explanation: Displays a Swing dialog to communicate a message, warning, confirmation, or input prompt to the user.

```
0180 |                 dashboard, "Enter the Section ID to update:");
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0181 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0182 |         if (input == null) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0183 |             return;
```

Explanation: Ends the current method without returning a value.

```
0184 |         }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0185 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0186 |         try {
```

Explanation: Begins a protected block for code that may throw an exception; resources may be managed by try-with-resources.

```
0187 |             int id = Integer.parseInt(input.trim());
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0188 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0189 |             String selectSql =
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0190 |                     "SELECT section_name, grade_level, strand_id, "
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0191 |                             + "school_year_id, slot_limit, is_active "
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0192 |                             + "FROM sections WHERE section_id = ? "
```

Explanation: Assigns or updates a variable with the value or result on the right-hand side.

```
0193 |                             + "AND grade_level IN (11, 12)";
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0194 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0195 |             String name;
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0196 |             int grade;
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0197 |             int strandId;
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0198 |             int yearId;
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0199 |             int slots;
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0200 |             boolean active;
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0201 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0202 |             Connection conn = null;
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0203 |             PreparedStatement ps = null;
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0204 |             try {
```

Explanation: Begins a protected block for code that may throw an exception; resources may be managed by try-with-resources.

```
0205 |                 conn = db.getConnection();
```

Explanation: Assigns or updates a variable with the value or result on the right-hand side.

```
0206 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0207 |                 if (conn == null) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0208 |                     throw new SQLException("Could not connect to the database.");
```

Explanation: Raises an exception so the caller or an enclosing catch block can handle the failure.

```
0209 |                 }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0210 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0211 |                 ps = conn.prepareStatement(selectSql);
```

Explanation: Creates a parameterized SQL statement; placeholders can be filled separately to reduce SQL-injection risk.

```
0212 |                 ps.setInt(1, id);
```

Explanation: Binds a Java value to a placeholder in the prepared SQL statement; the parameter index is one-based.

```
0213 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0214 |                 try (ResultSet rs = ps.executeQuery()) {
```

Explanation: Begins a protected block for code that may throw an exception; resources may be managed by try-with-resources.

```
0215 |                     if (!rs.next()) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0216 |                         AdminDashboard.showError(dashboard, "SHS section not found.");
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0217 |                         return;
```

Explanation: Ends the current method without returning a value.

```
0218 |                     }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0219 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0220 |                     name = rs.getString("section_name");
```

Explanation: Reads a column value from the current database result row using the requested Java type.

```
0221 |                     grade = rs.getInt("grade_level");
```

Explanation: Reads a column value from the current database result row using the requested Java type.

```
0222 |                     strandId = rs.getInt("strand_id");
```

Explanation: Reads a column value from the current database result row using the requested Java type.

```
0223 |                     yearId = rs.getInt("school_year_id");
```

Explanation: Reads a column value from the current database result row using the requested Java type.

```
0224 |                     slots = rs.getInt("slot_limit");
```

Explanation: Reads a column value from the current database result row using the requested Java type.

```
0225 |                     active = rs.getBoolean("is_active");
```

Explanation: Assigns or updates a variable with the value or result on the right-hand side.

```
0226 |                 }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0227 |             } finally {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0228 |                 try {
```

Explanation: Begins a protected block for code that may throw an exception; resources may be managed by try-with-resources.

```
0229 |                     if (ps != null) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0230 |                         ps.close();
```

Explanation: Closes the associated resource, such as a JDBC ResultSet, Statement, or Connection.

```
0231 |                     }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0232 |                 } catch (Exception closeEx) {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0233 |                     System.out.println(closeEx);
```

Explanation: Prints a message or value to the console, commonly for status output or error reporting.

```
0234 |                 }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0235 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0236 |                 try {
```

Explanation: Begins a protected block for code that may throw an exception; resources may be managed by try-with-resources.

```
0237 |                     if (conn != null) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0238 |                         conn.close();
```

Explanation: Closes the associated resource, such as a JDBC ResultSet, Statement, or Connection.

```
0239 |                     }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0240 |                 } catch (Exception closeEx) {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0241 |                     System.out.println(closeEx);
```

Explanation: Prints a message or value to the console, commonly for status output or error reporting.

```
0242 |                 }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0243 |             }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0244 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0245 |             JTextField nameField = new JTextField(name);
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0246 |             JComboBox&lt;Integer&gt; gradeBox =
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0247 |                     new JComboBox&lt;&gt;(new Integer[]{11, 12});
```

Explanation: Creates an instance of the specified class or object.

```
0248 |             gradeBox.setSelectedItem(grade);
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0249 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0250 |             JComboBox&lt;IdOption&gt; strandBox = new JComboBox&lt;&gt;();
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0251 |             JComboBox&lt;IdOption&gt; yearBox = new JComboBox&lt;&gt;();
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0252 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0253 |             if (!loadOptions(strandBox, yearBox)) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0254 |                 return;
```

Explanation: Ends the current method without returning a value.

```
0255 |             }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0256 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0257 |             selectOption(strandBox, strandId);
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0258 |             selectOption(yearBox, yearId);
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0259 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0260 |             JTextField slotField = new JTextField(
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0261 |                     String.valueOf(slots));
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0262 |             JCheckBox activeBox = new JCheckBox(
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0263 |                     "Active Section", active);
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0264 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0265 |             JPanel form = new JPanel(new GridLayout(0, 2, 6, 6));
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0266 |             form.add(new JLabel("Section Name:"));
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0267 |             form.add(nameField);
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0268 |             form.add(new JLabel("Grade Level:"));
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0269 |             form.add(gradeBox);
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0270 |             form.add(new JLabel("Strand:"));
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0271 |             form.add(strandBox);
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0272 |             form.add(new JLabel("School Year:"));
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0273 |             form.add(yearBox);
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0274 |             form.add(new JLabel("Slot Limit:"));
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0275 |             form.add(slotField);
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0276 |             form.add(new JLabel("Status:"));
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0277 |             form.add(activeBox);
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0278 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0279 |             int choice = JOptionPane.showConfirmDialog(
```

Explanation: Displays a Swing dialog to communicate a message, warning, confirmation, or input prompt to the user.

```
0280 |                     dashboard, form, "Update SHS Section",
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0281 |                     JOptionPane.OK_CANCEL_OPTION);
```

Explanation: Displays a Swing dialog to communicate a message, warning, confirmation, or input prompt to the user.

```
0282 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0283 |             if (choice != JOptionPane.OK_OPTION) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0284 |                 return;
```

Explanation: Ends the current method without returning a value.

```
0285 |             }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0286 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0287 |             String newName = nameField.getText().trim();
```

Explanation: Reads text currently entered or displayed by a GUI component.

```
0288 |             IdOption strand = (IdOption) strandBox.getSelectedItem();
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0289 |             IdOption year = (IdOption) yearBox.getSelectedItem();
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0290 |             int newGrade = (Integer) gradeBox.getSelectedItem();
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0291 |             int newSlots = Integer.parseInt(slotField.getText().trim());
```

Explanation: Reads text currently entered or displayed by a GUI component.

```
0292 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0293 |             if (newName.isEmpty() || strand == null || year == null
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0294 |                     || newSlots &lt; 1) {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0295 |                 AdminDashboard.showError(dashboard, "Enter valid section details.");
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0296 |                 return;
```

Explanation: Ends the current method without returning a value.

```
0297 |             }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0298 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0299 |             String updateSql =
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0300 |                     "UPDATE sections SET section_name = ?, "
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0301 |                             + "grade_level = ?, strand_id = ?, "
```

Explanation: Assigns or updates a variable with the value or result on the right-hand side.

```
0302 |                             + "school_year_id = ?, slot_limit = ?, "
```

Explanation: Assigns or updates a variable with the value or result on the right-hand side.

```
0303 |                             + "is_active = ? WHERE section_id = ? "
```

Explanation: Assigns or updates a variable with the value or result on the right-hand side.

```
0304 |                             + "AND grade_level IN (11, 12)";
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0305 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0306 |             conn = null;
```

Explanation: Assigns or updates a variable with the value or result on the right-hand side.

```
0307 |             ps = null;
```

Explanation: Assigns or updates a variable with the value or result on the right-hand side.

```
0308 |             try {
```

Explanation: Begins a protected block for code that may throw an exception; resources may be managed by try-with-resources.

```
0309 |                 conn = db.getConnection();
```

Explanation: Assigns or updates a variable with the value or result on the right-hand side.

```
0310 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0311 |                 if (conn == null) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0312 |                     throw new SQLException("Could not connect to the database.");
```

Explanation: Raises an exception so the caller or an enclosing catch block can handle the failure.

```
0313 |                 }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0314 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0315 |                 ps = conn.prepareStatement(updateSql);
```

Explanation: Creates a parameterized SQL statement; placeholders can be filled separately to reduce SQL-injection risk.

```
0316 |                 ps.setString(1, newName);
```

Explanation: Binds a Java value to a placeholder in the prepared SQL statement; the parameter index is one-based.

```
0317 |                 ps.setInt(2, newGrade);
```

Explanation: Binds a Java value to a placeholder in the prepared SQL statement; the parameter index is one-based.

```
0318 |                 ps.setInt(3, strand.id);
```

Explanation: Binds a Java value to a placeholder in the prepared SQL statement; the parameter index is one-based.

```
0319 |                 ps.setInt(4, year.id);
```

Explanation: Binds a Java value to a placeholder in the prepared SQL statement; the parameter index is one-based.

```
0320 |                 ps.setInt(5, newSlots);
```

Explanation: Binds a Java value to a placeholder in the prepared SQL statement; the parameter index is one-based.

```
0321 |                 ps.setBoolean(6, activeBox.isSelected());
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0322 |                 ps.setInt(7, id);
```

Explanation: Binds a Java value to a placeholder in the prepared SQL statement; the parameter index is one-based.

```
0323 |                 ps.executeUpdate();
```

Explanation: Executes an INSERT, UPDATE, or DELETE statement and returns the number of affected rows.

```
0324 |             } finally {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0325 |                 try {
```

Explanation: Begins a protected block for code that may throw an exception; resources may be managed by try-with-resources.

```
0326 |                     if (ps != null) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0327 |                         ps.close();
```

Explanation: Closes the associated resource, such as a JDBC ResultSet, Statement, or Connection.

```
0328 |                     }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0329 |                 } catch (Exception closeEx) {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0330 |                     System.out.println(closeEx);
```

Explanation: Prints a message or value to the console, commonly for status output or error reporting.

```
0331 |                 }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0332 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0333 |                 try {
```

Explanation: Begins a protected block for code that may throw an exception; resources may be managed by try-with-resources.

```
0334 |                     if (conn != null) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0335 |                         conn.close();
```

Explanation: Closes the associated resource, such as a JDBC ResultSet, Statement, or Connection.

```
0336 |                     }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0337 |                 } catch (Exception closeEx) {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0338 |                     System.out.println(closeEx);
```

Explanation: Prints a message or value to the console, commonly for status output or error reporting.

```
0339 |                 }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0340 |             }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0341 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0342 |             JOptionPane.showMessageDialog(
```

Explanation: Displays a Swing dialog to communicate a message, warning, confirmation, or input prompt to the user.

```
0343 |                     dashboard, "Section updated successfully.");
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0344 |             dashboard.viewSections();
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0345 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0346 |         } catch (NumberFormatException ex) {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0347 |             AdminDashboard.showError(dashboard, "Enter valid numeric values.");
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0348 |         } catch (SQLException ex) {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0349 |             AdminDashboard.showError(dashboard, "Could not update section: " + ex.getMessage());
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0350 |         }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0351 |     }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0352 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0353 |     private void selectOption(
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0354 |             JComboBox&lt;IdOption&gt; box, int id) {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0355 |         for (int i = 0; i &lt; box.getItemCount(); i++) {
```

Explanation: Starts a loop that repeats while its loop condition allows; initialization and update control iteration.

```
0356 |             if (box.getItemAt(i).id == id) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0357 |                 box.setSelectedIndex(i);
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0358 |                 return;
```

Explanation: Ends the current method without returning a value.

```
0359 |             }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0360 |         }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0361 |     }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0362 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0363 |     public void deleteSection() {
```

Explanation: Declares the deleteSection method and its return type/parameters; the method performs the operation described by its name and body.

```
0364 |         String input = JOptionPane.showInputDialog(
```

Explanation: Displays a Swing dialog to communicate a message, warning, confirmation, or input prompt to the user.

```
0365 |                 dashboard, "Enter the Section ID to deactivate:");
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0366 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0367 |         if (input == null) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0368 |             return;
```

Explanation: Ends the current method without returning a value.

```
0369 |         }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0370 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0371 |         try {
```

Explanation: Begins a protected block for code that may throw an exception; resources may be managed by try-with-resources.

```
0372 |             int id = Integer.parseInt(input.trim());
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0373 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0374 |             int choice = JOptionPane.showConfirmDialog(
```

Explanation: Displays a Swing dialog to communicate a message, warning, confirmation, or input prompt to the user.

```
0375 |                     dashboard,
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0376 |                     "Deactivate this section? Existing enrollment records "
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0377 |                             + "will be preserved.",
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0378 |                     "Confirm Deactivation",
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0379 |                     JOptionPane.YES_NO_OPTION);
```

Explanation: Displays a Swing dialog to communicate a message, warning, confirmation, or input prompt to the user.

```
0380 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0381 |             if (choice != JOptionPane.YES_OPTION) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0382 |                 return;
```

Explanation: Ends the current method without returning a value.

```
0383 |             }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0384 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0385 |             String sql =
```

Explanation: Stores the SQL command as a string so the program can execute it against the database.

```
0386 |                     "UPDATE sections SET is_active = FALSE "
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0387 |                             + "WHERE section_id = ? "
```

Explanation: Assigns or updates a variable with the value or result on the right-hand side.

```
0388 |                             + "AND grade_level IN (11, 12)";
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0389 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0390 |             Connection conn = null;
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0391 |             PreparedStatement ps = null;
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0392 |             try {
```

Explanation: Begins a protected block for code that may throw an exception; resources may be managed by try-with-resources.

```
0393 |                 conn = db.getConnection();
```

Explanation: Assigns or updates a variable with the value or result on the right-hand side.

```
0394 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0395 |                 if (conn == null) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0396 |                     throw new SQLException("Could not connect to the database.");
```

Explanation: Raises an exception so the caller or an enclosing catch block can handle the failure.

```
0397 |                 }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0398 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0399 |                 ps = conn.prepareStatement(sql);
```

Explanation: Creates a parameterized SQL statement; placeholders can be filled separately to reduce SQL-injection risk.

```
0400 |                 ps.setInt(1, id);
```

Explanation: Binds a Java value to a placeholder in the prepared SQL statement; the parameter index is one-based.

```
0401 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0402 |                 if (ps.executeUpdate() == 0) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0403 |                     AdminDashboard.showError(dashboard, "SHS section not found.");
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0404 |                     return;
```

Explanation: Ends the current method without returning a value.

```
0405 |                 }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0406 |             } finally {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0407 |                 try {
```

Explanation: Begins a protected block for code that may throw an exception; resources may be managed by try-with-resources.

```
0408 |                     if (ps != null) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0409 |                         ps.close();
```

Explanation: Closes the associated resource, such as a JDBC ResultSet, Statement, or Connection.

```
0410 |                     }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0411 |                 } catch (Exception closeEx) {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0412 |                     System.out.println(closeEx);
```

Explanation: Prints a message or value to the console, commonly for status output or error reporting.

```
0413 |                 }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0414 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0415 |                 try {
```

Explanation: Begins a protected block for code that may throw an exception; resources may be managed by try-with-resources.

```
0416 |                     if (conn != null) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0417 |                         conn.close();
```

Explanation: Closes the associated resource, such as a JDBC ResultSet, Statement, or Connection.

```
0418 |                     }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0419 |                 } catch (Exception closeEx) {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0420 |                     System.out.println(closeEx);
```

Explanation: Prints a message or value to the console, commonly for status output or error reporting.

```
0421 |                 }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0422 |             }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0423 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0424 |             JOptionPane.showMessageDialog(
```

Explanation: Displays a Swing dialog to communicate a message, warning, confirmation, or input prompt to the user.

```
0425 |                     dashboard, "Section deactivated successfully.");
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0426 |             dashboard.viewSections();
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0427 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0428 |         } catch (NumberFormatException ex) {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0429 |             AdminDashboard.showError(dashboard, "Section ID must be a number.");
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0430 |         } catch (SQLException ex) {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0431 |             AdminDashboard.showError(dashboard, "Could not deactivate section: " + ex.getMessage());
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0432 |         }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0433 |     }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0434 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0435 |     public static class IdOption {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0436 |         public int id;
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0437 |         public String label;
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0438 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0439 |         public IdOption(int id, String label) {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0440 |             this.id = id;
```

Explanation: Assigns or updates a variable with the value or result on the right-hand side.

```
0441 |             this.label = label;
```

Explanation: Assigns or updates a variable with the value or result on the right-hand side.

```
0442 |         }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0443 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0444 |         @Override
```

Explanation: Applies an annotation to the declaration that follows; the annotation supplies metadata or framework behavior.

```
0445 |         public String toString() {
```

Explanation: Declares the toString method and its return type/parameters; the method performs the operation described by its name and body.

```
0446 |             return label + " (ID: " + id + ")";
```

Explanation: Returns the value or result to the caller and ends the current method.

```
0447 |         }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0448 |     }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0449 | }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

AdminUser.java

Project path: src/AdminUser.java

```
0001 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0002 | public class AdminUser extends User {
```

Explanation: Declares the AdminUser type, which groups related data and behavior.

```
0003 |     public AdminUser(int userId, String username, String email, String role) {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0004 |         super(userId, username, email, "Admin");
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0005 |     }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0006 | }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

App.java

Project path: src/App.java

```
0001 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0002 | import javax.swing.SwingUtilities;
```

Explanation: Imports Swing GUI classes used to build the application's windows and controls.

```
0003 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0004 | public class App {
```

Explanation: Declares the App type, which groups related data and behavior.

```
0005 |     public static void main(String[] args) {
```

Explanation: Declares Java's program entry point; execution begins here when this class is launched.

```
0006 |         DbConnection db = new DbConnection();
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0007 |         db.connect();
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0008 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0009 |         SwingUtilities.invokeLater(() -&gt; {
```

Explanation: Schedules GUI creation on Swing's Event Dispatch Thread, which is the recommended thread for Swing UI work.

```
0010 |             new LoginFrame().setVisible(true);
```

Explanation: Creates an instance of the specified class or object.

```
0011 |         });
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0012 |     }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0013 | }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

AuthService.java

Project path: src/AuthService.java

```
0001 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0002 | import java.security.MessageDigest;
```

Explanation: Imports Java security utilities used here for cryptographic hashing or related operations.

```
0003 | import java.nio.charset.StandardCharsets;
```

Explanation: Imports NIO utilities used for character encoding or byte handling.

```
0004 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0005 | public class AuthService {
```

Explanation: Declares the AuthService type, which groups related data and behavior.

```
0006 |     private UserDAO userDAO = new UserDAO();
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0007 |     private StudentDAO studentDAO = new StudentDAO();
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0008 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0009 |     public String hashPassword(String password) {
```

Explanation: Declares the hashPassword method, which implements this part of the application's authentication or registration workflow.

```
0010 |         try {
```

Explanation: Begins a protected block for code that may throw an exception; resources may be managed by try-with-resources.

```
0011 |             MessageDigest md = MessageDigest.getInstance("SHA-256");
```

Explanation: Creates the requested cryptographic digest implementation, here used for password hashing.

```
0012 |             byte[] bytes = md.digest(password.getBytes(StandardCharsets.UTF_8));
```

Explanation: Computes the hash bytes for the supplied input.

```
0013 |             StringBuilder result = new StringBuilder();
```

Explanation: Creates a mutable string builder so text can be assembled efficiently.

```
0014 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0015 |             for (byte b : bytes) {
```

Explanation: Starts a loop that repeats while its loop condition allows; initialization and update control iteration.

```
0016 |                 result.append(String.format("%02x", b));
```

Explanation: Appends the provided value to the current StringBuilder or collection.

```
0017 |             }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0018 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0019 |             return result.toString();
```

Explanation: Returns the value or result to the caller and ends the current method.

```
0020 |         } catch (Exception e) {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0021 |             throw new RuntimeException("Unable to hash password.", e);
```

Explanation: Raises an exception so the caller or an enclosing catch block can handle the failure.

```
0022 |         }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0023 |     }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0024 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0025 |     public User login(String username, String password) {
```

Explanation: Declares the login method, which implements this part of the application's authentication or registration workflow.

```
0026 |         return userDAO.login(username, hashPassword(password));
```

Explanation: Returns the value or result to the caller and ends the current method.

```
0027 |     }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0028 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0029 |     public boolean register(String username, String password, String email,
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0030 |                             String lrn, String firstName, String middleName,
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0031 |                             String lastName, String previousSchool) {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0032 |         if (username.isEmpty() || password.isEmpty() || email.isEmpty()
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0033 |                 || lrn.isEmpty() || firstName.isEmpty() || lastName.isEmpty()) {
```

Explanation: Checks whether a string or collection has no content.

```
0034 |             return false;
```

Explanation: Returns the value or result to the caller and ends the current method.

```
0035 |         }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0036 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0037 |         if (!userDAO.addUser(username, hashPassword(password), email, "student")) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0038 |             return false;
```

Explanation: Returns the value or result to the caller and ends the current method.

```
0039 |         }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0040 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0041 |         int userId = userDAO.getUserId(username);
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0042 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0043 |         if (userId == -1) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0044 |             return false;
```

Explanation: Returns the value or result to the caller and ends the current method.

```
0045 |         }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0046 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0047 |         return studentDAO.addStudent(userId, lrn, firstName, middleName,
```

Explanation: Returns the value or result to the caller and ends the current method.

```
0048 |                 lastName, previousSchool);
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0049 |     }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0050 | }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

CashierDashboard.java

Project path: src/CashierDashboard.java

```
0001 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0002 | import javax.swing.*;
```

Explanation: Imports Swing GUI classes used to build the application's windows and controls.

```
0003 | import java.awt.*;
```

Explanation: Imports AWT classes used for layout, dimensions, colors, fonts, or UI events.

```
0004 | import java.sql.*;
```

Explanation: Imports JDBC classes used to communicate with a relational database.

```
0005 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0006 | public class CashierDashboard extends JFrame {
```

Explanation: Declares the CashierDashboard type, which groups related data and behavior.

```
0007 |     private User user;
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0008 |     private PaymentDAO paymentDAO = new PaymentDAO();
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0009 |     private DbConnection db = new DbConnection();
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0010 |     private JTextArea outputArea = new JTextArea();
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0011 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0012 |     public CashierDashboard(User user) {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0013 |         this.user = user;
```

Explanation: Assigns or updates a variable with the value or result on the right-hand side.

```
0014 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0015 |         setTitle("Student Enrollment System - Cashier Dashboard");
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0016 |         setSize(900, 550);
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0017 |         setDefaultCloseOperation(JFrame.DISPOSE_ON_CLOSE);
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0018 |         setLocationRelativeTo(null);
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0019 |         setLayout(new BorderLayout(10, 10));
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0020 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0021 |         JLabel heading = new JLabel("CASHIER DASHBOARD - PAYMENTS ONLY");
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0022 |         heading.setFont(new Font("Arial", Font.BOLD, 19));
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0023 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0024 |         JButton pendingButton = new JButton("View Pending Payments");
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0025 |         JButton historyButton = new JButton("View Payment History");
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0026 |         JButton verifyButton = new JButton("Verify Payment");
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0027 |         JButton deleteButton = new JButton("Delete Payment Record");
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0028 |         JButton refreshButton = new JButton("Refresh Payments");
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0029 |         JButton logoutButton = new JButton("Logout");
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0030 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0031 |         pendingButton.setToolTipText(
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0032 |                 "Display payments awaiting cashier verification."
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0033 |         );
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0034 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0035 |         historyButton.setToolTipText(
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0036 |                 "Display payment records and their statuses."
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0037 |         );
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0038 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0039 |         verifyButton.setToolTipText(
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0040 |                 "Mark a pending payment as Paid after verifying the actual payment."
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0041 |         );
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0042 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0043 |         deleteButton.setToolTipText(
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0044 |                 "Delete an eligible payment record after confirmation."
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0045 |         );
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0046 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0047 |         refreshButton.setToolTipText(
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0048 |                 "Reload the latest payment records from MySQL."
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0049 |         );
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0050 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0051 |         JPanel menu = new JPanel(new GridLayout(0, 1, 5, 5));
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0052 |         menu.setBorder(BorderFactory.createEmptyBorder(10, 10, 10, 10));
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0053 |         menu.add(pendingButton);
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0054 |         menu.add(historyButton);
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0055 |         menu.add(verifyButton);
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0056 |         menu.add(deleteButton);
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0057 |         menu.add(refreshButton);
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0058 |         menu.add(logoutButton);
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0059 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0060 |         outputArea.setEditable(false);
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0061 |         outputArea.setFont(new Font("Monospaced", Font.PLAIN, 13));
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0062 |         outputArea.setText(
```

Explanation: Sets the displayed text of a GUI component.

```
0063 |                 "Welcome to the Cashier Dashboard.\n\n" +
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0064 |                 "1. View pending payments.\n" +
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0065 |                 "2. View payment history.\n" +
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0066 |                 "3. Verify payments after checking actual payment.\n" +
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0067 |                 "4. Delete an eligible payment record.\n\n" +
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0068 |                 "Example: Enter Payment ID 1 to verify a pending payment."
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0069 |         );
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0070 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0071 |         JPanel top = new JPanel(new BorderLayout());
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0072 |         top.add(heading, BorderLayout.CENTER);
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0073 |         top.setBorder(BorderFactory.createEmptyBorder(12, 12, 12, 12));
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0074 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0075 |         add(top, BorderLayout.NORTH);
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0076 |         add(new JScrollPane(menu), BorderLayout.WEST);
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0077 |         add(new JScrollPane(outputArea), BorderLayout.CENTER);
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0078 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0079 |         pendingButton.addActionListener(e -&gt; viewPendingPayments());
```

Explanation: Registers an event handler that runs when the associated GUI control is activated.

```
0080 |         historyButton.addActionListener(e -&gt; viewPaymentHistory());
```

Explanation: Registers an event handler that runs when the associated GUI control is activated.

```
0081 |         verifyButton.addActionListener(e -&gt; verifyPayment());
```

Explanation: Registers an event handler that runs when the associated GUI control is activated.

```
0082 |         deleteButton.addActionListener(e -&gt; deletePayment());
```

Explanation: Registers an event handler that runs when the associated GUI control is activated.

```
0083 |         refreshButton.addActionListener(e -&gt; viewPaymentHistory());
```

Explanation: Registers an event handler that runs when the associated GUI control is activated.

```
0084 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0085 |         logoutButton.addActionListener(e -&gt; {
```

Explanation: Registers an event handler that runs when the associated GUI control is activated.

```
0086 |             int answer = JOptionPane.showConfirmDialog(
```

Explanation: Displays a Swing dialog to communicate a message, warning, confirmation, or input prompt to the user.

```
0087 |                     this,
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0088 |                     "Log out of the cashier dashboard?",
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0089 |                     "Confirm Logout",
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0090 |                     JOptionPane.YES_NO_OPTION
```

Explanation: Displays a Swing dialog to communicate a message, warning, confirmation, or input prompt to the user.

```
0091 |             );
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0092 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0093 |             if (answer == JOptionPane.YES_OPTION) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0094 |                 dispose();
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0095 |                 new LoginFrame().setVisible(true);
```

Explanation: Creates an instance of the specified class or object.

```
0096 |             }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0097 |         });
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0098 |     }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0099 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0100 |     private void viewPendingPayments() {
```

Explanation: Declares the viewPendingPayments method and its return type/parameters; the method performs the operation described by its name and body.

```
0101 |         try {
```

Explanation: Begins a protected block for code that may throw an exception; resources may be managed by try-with-resources.

```
0102 |             outputArea.setText(
```

Explanation: Sets the displayed text of a GUI component.

```
0103 |                     "PENDING PAYMENTS\n\n" + paymentDAO.getPendingPayments()
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0104 |             );
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0105 |         } catch (Exception ex) {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0106 |             showError("Could not load pending payments.\n" + ex.getMessage());
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0107 |         }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0108 |     }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0109 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0110 |     private void viewPaymentHistory() {
```

Explanation: Declares the viewPaymentHistory method and its return type/parameters; the method performs the operation described by its name and body.

```
0111 |         try {
```

Explanation: Begins a protected block for code that may throw an exception; resources may be managed by try-with-resources.

```
0112 |             outputArea.setText(
```

Explanation: Sets the displayed text of a GUI component.

```
0113 |                     "PAYMENT HISTORY\n\n" + paymentDAO.getPayments()
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0114 |             );
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0115 |         } catch (Exception ex) {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0116 |             showError("Could not load payment history.\n" + ex.getMessage());
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0117 |         }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0118 |     }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0119 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0120 |     private void verifyPayment() {
```

Explanation: Declares the verifyPayment method and its return type/parameters; the method performs the operation described by its name and body.

```
0121 |         String input = JOptionPane.showInputDialog(
```

Explanation: Displays a Swing dialog to communicate a message, warning, confirmation, or input prompt to the user.

```
0122 |                 this,
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0123 |                 "Enter the Payment ID to verify.\nExample: 1"
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0124 |         );
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0125 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0126 |         if (input == null) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0127 |             return;
```

Explanation: Ends the current method without returning a value.

```
0128 |         }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0129 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0130 |         int paymentId;
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0131 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0132 |         try {
```

Explanation: Begins a protected block for code that may throw an exception; resources may be managed by try-with-resources.

```
0133 |             paymentId = Integer.parseInt(input.trim());
```

Explanation: Assigns or updates a variable with the value or result on the right-hand side.

```
0134 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0135 |             if (paymentId &lt;= 0) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0136 |                 showError("Payment ID must be a positive number.");
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0137 |                 return;
```

Explanation: Ends the current method without returning a value.

```
0138 |             }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0139 |         } catch (NumberFormatException ex) {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0140 |             showError("Payment ID must be a positive number.");
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0141 |             return;
```

Explanation: Ends the current method without returning a value.

```
0142 |         }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0143 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0144 |         int confirm = JOptionPane.showConfirmDialog(
```

Explanation: Displays a Swing dialog to communicate a message, warning, confirmation, or input prompt to the user.

```
0145 |                 this,
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0146 |                 "Have you confirmed that the actual payment was received?\n\n" +
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0147 |                 "Only choose Yes after checking the payment.\n" +
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0148 |                 "Payment ID: " + paymentId,
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0149 |                 "Confirm Payment Verification",
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0150 |                 JOptionPane.YES_NO_OPTION
```

Explanation: Displays a Swing dialog to communicate a message, warning, confirmation, or input prompt to the user.

```
0151 |         );
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0152 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0153 |         if (confirm != JOptionPane.YES_OPTION) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0154 |             return;
```

Explanation: Ends the current method without returning a value.

```
0155 |         }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0156 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0157 |         try {
```

Explanation: Begins a protected block for code that may throw an exception; resources may be managed by try-with-resources.

```
0158 |             String result = paymentDAO.markPaymentPaid(paymentId);
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0159 |             outputArea.setText("PAYMENT VERIFICATION RESULT\n\n" + result);
```

Explanation: Sets the displayed text of a GUI component.

```
0160 |         } catch (Exception ex) {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0161 |             showError("Could not verify payment.\n" + ex.getMessage());
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0162 |         }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0163 |     }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0164 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0165 |     private void deletePayment() {
```

Explanation: Declares the deletePayment method and its return type/parameters; the method performs the operation described by its name and body.

```
0166 |         String input = JOptionPane.showInputDialog(
```

Explanation: Displays a Swing dialog to communicate a message, warning, confirmation, or input prompt to the user.

```
0167 |                 this,
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0168 |                 "Enter the Payment ID you want to delete.\n\n" +
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0169 |                 "Use View Payment History to find the ID.",
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0170 |                 "Delete Payment Record",
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0171 |                 JOptionPane.QUESTION_MESSAGE
```

Explanation: Displays a Swing dialog to communicate a message, warning, confirmation, or input prompt to the user.

```
0172 |         );
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0173 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0174 |         if (input == null) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0175 |             return;
```

Explanation: Ends the current method without returning a value.

```
0176 |         }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0177 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0178 |         int paymentId;
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0179 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0180 |         try {
```

Explanation: Begins a protected block for code that may throw an exception; resources may be managed by try-with-resources.

```
0181 |             paymentId = Integer.parseInt(input.trim());
```

Explanation: Assigns or updates a variable with the value or result on the right-hand side.

```
0182 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0183 |             if (paymentId &lt;= 0) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0184 |                 showError("Payment ID must be a positive number.");
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0185 |                 return;
```

Explanation: Ends the current method without returning a value.

```
0186 |             }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0187 |         } catch (NumberFormatException ex) {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0188 |             showError("Enter a valid numeric Payment ID.");
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0189 |             return;
```

Explanation: Ends the current method without returning a value.

```
0190 |         }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0191 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0192 |         String paymentStatus;
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0193 |         double amountPaid;
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0194 |         String referenceNumber;
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0195 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0196 |         String selectSql =
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0197 |                 "SELECT payment_status, amount_paid, reference_number " +
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0198 |                 "FROM payments WHERE payment_id = ?";
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0199 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0200 |         Connection conn = null;
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0201 |         PreparedStatement ps = null;
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0202 |         try {
```

Explanation: Begins a protected block for code that may throw an exception; resources may be managed by try-with-resources.

```
0203 |             conn = db.getConnection();
```

Explanation: Assigns or updates a variable with the value or result on the right-hand side.

```
0204 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0205 |             if (conn == null) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0206 |                 throw new SQLException("Could not connect to the database.");
```

Explanation: Raises an exception so the caller or an enclosing catch block can handle the failure.

```
0207 |             }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0208 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0209 |             ps = conn.prepareStatement(selectSql);
```

Explanation: Creates a parameterized SQL statement; placeholders can be filled separately to reduce SQL-injection risk.

```
0210 |             ps.setInt(1, paymentId);
```

Explanation: Binds a Java value to a placeholder in the prepared SQL statement; the parameter index is one-based.

```
0211 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0212 |             try (ResultSet rs = ps.executeQuery()) {
```

Explanation: Begins a protected block for code that may throw an exception; resources may be managed by try-with-resources.

```
0213 |                 if (!rs.next()) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0214 |                     showError("No payment record found with ID " + paymentId + ".");
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0215 |                     return;
```

Explanation: Ends the current method without returning a value.

```
0216 |                 }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0217 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0218 |                 paymentStatus = rs.getString("payment_status");
```

Explanation: Reads a column value from the current database result row using the requested Java type.

```
0219 |                 amountPaid = rs.getDouble("amount_paid");
```

Explanation: Reads a column value from the current database result row using the requested Java type.

```
0220 |                 referenceNumber = rs.getString("reference_number");
```

Explanation: Reads a column value from the current database result row using the requested Java type.

```
0221 |             }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0222 |         } catch (Exception ex) {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0223 |             showError(
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0224 |                     "Could not find the payment record.\n" +
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0225 |                     "Check the payments table columns.\n\n" + ex.getMessage()
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0226 |             );
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0227 |             return;
```

Explanation: Ends the current method without returning a value.

```
0228 |         } finally {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0229 |             try {
```

Explanation: Begins a protected block for code that may throw an exception; resources may be managed by try-with-resources.

```
0230 |                 if (ps != null) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0231 |                     ps.close();
```

Explanation: Closes the associated resource, such as a JDBC ResultSet, Statement, or Connection.

```
0232 |                 }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0233 |             } catch (Exception closeEx) {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0234 |                 System.out.println(closeEx);
```

Explanation: Prints a message or value to the console, commonly for status output or error reporting.

```
0235 |             }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0236 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0237 |             try {
```

Explanation: Begins a protected block for code that may throw an exception; resources may be managed by try-with-resources.

```
0238 |                 if (conn != null) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0239 |                     conn.close();
```

Explanation: Closes the associated resource, such as a JDBC ResultSet, Statement, or Connection.

```
0240 |                 }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0241 |             } catch (Exception closeEx) {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0242 |                 System.out.println(closeEx);
```

Explanation: Prints a message or value to the console, commonly for status output or error reporting.

```
0243 |             }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0244 |         }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0245 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0246 |         if (paymentStatus != null &amp;amp;&amp;amp;
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0247 |                 paymentStatus.equalsIgnoreCase("Paid")) {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0248 |             showError(
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0249 |                     "This payment is already marked as Paid.\n\n" +
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0250 |                     "For accurate financial records, paid transactions cannot " +
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0251 |                     "be deleted from the cashier dashboard."
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0252 |             );
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0253 |             return;
```

Explanation: Ends the current method without returning a value.

```
0254 |         }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0255 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0256 |         int confirm = JOptionPane.showConfirmDialog(
```

Explanation: Displays a Swing dialog to communicate a message, warning, confirmation, or input prompt to the user.

```
0257 |                 this,
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0258 |                 "Delete this payment record?\n\n" +
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0259 |                 "Payment ID: " + paymentId + "\n" +
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0260 |                 "Amount Paid: " + amountPaid + "\n" +
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0261 |                 "Reference Number: " +
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0262 |                 (referenceNumber == null ? "None" : referenceNumber) + "\n" +
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0263 |                 "Status: " + paymentStatus + "\n\n" +
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0264 |                 "This action cannot be undone. The database may prevent deletion " +
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0265 |                 "if other records reference this payment.",
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0266 |                 "Confirm Payment Deletion",
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0267 |                 JOptionPane.YES_NO_OPTION,
```

Explanation: Displays a Swing dialog to communicate a message, warning, confirmation, or input prompt to the user.

```
0268 |                 JOptionPane.WARNING_MESSAGE
```

Explanation: Displays a Swing dialog to communicate a message, warning, confirmation, or input prompt to the user.

```
0269 |         );
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0270 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0271 |         if (confirm != JOptionPane.YES_OPTION) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0272 |             return;
```

Explanation: Ends the current method without returning a value.

```
0273 |         }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0274 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0275 |         String deleteSql =
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0276 |                 "DELETE FROM payments " +
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0277 |                 "WHERE payment_id = ? AND LOWER(payment_status) &lt;&gt; 'paid'";
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0278 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0279 |         conn = null;
```

Explanation: Assigns or updates a variable with the value or result on the right-hand side.

```
0280 |         ps = null;
```

Explanation: Assigns or updates a variable with the value or result on the right-hand side.

```
0281 |         try {
```

Explanation: Begins a protected block for code that may throw an exception; resources may be managed by try-with-resources.

```
0282 |             conn = db.getConnection();
```

Explanation: Assigns or updates a variable with the value or result on the right-hand side.

```
0283 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0284 |             if (conn == null) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0285 |                 throw new SQLException("Could not connect to the database.");
```

Explanation: Raises an exception so the caller or an enclosing catch block can handle the failure.

```
0286 |             }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0287 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0288 |             ps = conn.prepareStatement(deleteSql);
```

Explanation: Creates a parameterized SQL statement; placeholders can be filled separately to reduce SQL-injection risk.

```
0289 |             ps.setInt(1, paymentId);
```

Explanation: Binds a Java value to a placeholder in the prepared SQL statement; the parameter index is one-based.

```
0290 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0291 |             int deleted = ps.executeUpdate();
```

Explanation: Executes an INSERT, UPDATE, or DELETE statement and returns the number of affected rows.

```
0292 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0293 |             if (deleted &gt; 0) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0294 |                 outputArea.setText(
```

Explanation: Sets the displayed text of a GUI component.

```
0295 |                         "PAYMENT RECORD DELETED\n\n" +
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0296 |                         "Payment ID: " + paymentId + "\n" +
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0297 |                         "Amount Paid: " + amountPaid + "\n" +
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0298 |                         "Reference Number: " +
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0299 |                         (referenceNumber == null ? "None" : referenceNumber) + "\n" +
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0300 |                         "Previous Status: " + paymentStatus + "\n\n" +
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0301 |                         "The eligible payment record was deleted."
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0302 |                 );
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0303 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0304 |                 JOptionPane.showMessageDialog(
```

Explanation: Displays a Swing dialog to communicate a message, warning, confirmation, or input prompt to the user.

```
0305 |                         this,
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0306 |                         "Payment record deleted successfully."
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0307 |                 );
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0308 |             } else {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0309 |                 showError(
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0310 |                         "The payment was not deleted. It may already be marked Paid " +
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0311 |                         "or may have been changed by another process."
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0312 |                 );
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0313 |             }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0314 |         } catch (Exception ex) {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0315 |             showError(
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0316 |                     "Could not delete the payment record.\n\n" +
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0317 |                     "The payment may be linked to other records, or a database " +
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0318 |                     "constraint may prevent deletion.\n\n" +
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0319 |                     "Preserve payment history and do not remove related records " +
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0320 |                     "just to force deletion.\n\n" +
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0321 |                     ex.getMessage()
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0322 |             );
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0323 |         } finally {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0324 |             try {
```

Explanation: Begins a protected block for code that may throw an exception; resources may be managed by try-with-resources.

```
0325 |                 if (ps != null) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0326 |                     ps.close();
```

Explanation: Closes the associated resource, such as a JDBC ResultSet, Statement, or Connection.

```
0327 |                 }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0328 |             } catch (Exception closeEx) {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0329 |                 System.out.println(closeEx);
```

Explanation: Prints a message or value to the console, commonly for status output or error reporting.

```
0330 |             }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0331 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0332 |             try {
```

Explanation: Begins a protected block for code that may throw an exception; resources may be managed by try-with-resources.

```
0333 |                 if (conn != null) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0334 |                     conn.close();
```

Explanation: Closes the associated resource, such as a JDBC ResultSet, Statement, or Connection.

```
0335 |                 }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0336 |             } catch (Exception closeEx) {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0337 |                 System.out.println(closeEx);
```

Explanation: Prints a message or value to the console, commonly for status output or error reporting.

```
0338 |             }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0339 |         }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0340 |     }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0341 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0342 |     private void showError(String message) {
```

Explanation: Declares the showError method and its return type/parameters; the method performs the operation described by its name and body.

```
0343 |         JOptionPane.showMessageDialog(
```

Explanation: Displays a Swing dialog to communicate a message, warning, confirmation, or input prompt to the user.

```
0344 |                 this,
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0345 |                 message,
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0346 |                 "Cashier Dashboard",
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0347 |                 JOptionPane.ERROR_MESSAGE
```

Explanation: Displays a Swing dialog to communicate a message, warning, confirmation, or input prompt to the user.

```
0348 |         );
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0349 |     }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0350 | }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

DbConnection.java

Project path: src/DbConnection.java

```
0001 | import java.sql.*;
```

Explanation: Imports JDBC classes used to communicate with a relational database.

```
0002 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0003 | public class DbConnection {
```

Explanation: Declares the DbConnection type, which groups related data and behavior.

```
0004 |     String url = "jdbc:mysql://localhost:3307/enrollment_system";
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0005 |     String username = "root";
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0006 |     String password = "120604";
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0007 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0008 |     public Connection getConnection() {
```

Explanation: Declares the getConnection method, typically returning the requested field or resource.

```
0009 |         try {
```

Explanation: Begins a protected block for code that may throw an exception; resources may be managed by try-with-resources.

```
0010 |             Connection conn = DriverManager.getConnection(url, username, password);
```

Explanation: Asks JDBC DriverManager to open a database connection using the configured URL and credentials.

```
0011 |             return conn;
```

Explanation: Returns the value or result to the caller and ends the current method.

```
0012 |         } catch (Exception e) {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0013 |             System.out.println(e);
```

Explanation: Prints a message or value to the console, commonly for status output or error reporting.

```
0014 |             return null;
```

Explanation: Returns the value or result to the caller and ends the current method.

```
0015 |         }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0016 |     }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0017 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0018 |     public void connect() {
```

Explanation: Declares the connect method and its return type/parameters; the method performs the operation described by its name and body.

```
0019 |         Connection conn = null;
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0020 |         Statement st = null;
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0021 |         ResultSet result = null;
```

Explanation: Declares or assigns a ResultSet that holds rows returned by a database query.

```
0022 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0023 |         try {
```

Explanation: Begins a protected block for code that may throw an exception; resources may be managed by try-with-resources.

```
0024 |             conn = getConnection();
```

Explanation: Assigns or updates a variable with the value or result on the right-hand side.

```
0025 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0026 |             if (conn == null) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0027 |                 return;
```

Explanation: Ends the current method without returning a value.

```
0028 |             }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0029 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0030 |             st = conn.createStatement();
```

Explanation: Creates a JDBC Statement used to execute a SQL command.

```
0031 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0032 |             String sql = "SELECT * FROM users";
```

Explanation: Stores the SQL command as a string so the program can execute it against the database.

```
0033 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0034 |             result = st.executeQuery(sql);
```

Explanation: Executes a SQL query that returns rows, usually a SELECT, and stores them in a ResultSet.

```
0035 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0036 |             System.out.println("Database connected successfully!");
```

Explanation: Prints a message or value to the console, commonly for status output or error reporting.

```
0037 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0038 |             while (result.next()) {
```

Explanation: Repeats the following block while the condition remains true.

```
0039 |                 System.out.println("User ID: " + result.getInt("user_id"));
```

Explanation: Reads a column value from the current database result row using the requested Java type.

```
0040 |                 System.out.println("Username: " + result.getString("username"));
```

Explanation: Reads a column value from the current database result row using the requested Java type.

```
0041 |                 System.out.println("Email: " + result.getString("email"));
```

Explanation: Reads a column value from the current database result row using the requested Java type.

```
0042 |                 System.out.println("----------------------");
```

Explanation: Prints a message or value to the console, commonly for status output or error reporting.

```
0043 |             }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0044 |         } catch (Exception e) {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0045 |             System.out.println("Database connection error: " + e.getMessage());
```

Explanation: Prints a message or value to the console, commonly for status output or error reporting.

```
0046 |         } finally {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0047 |             try {
```

Explanation: Begins a protected block for code that may throw an exception; resources may be managed by try-with-resources.

```
0048 |                 if (result != null) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0049 |                     result.close();
```

Explanation: Closes the associated resource, such as a JDBC ResultSet, Statement, or Connection.

```
0050 |                 }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0051 |             } catch (Exception e) {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0052 |                 System.out.println(e);
```

Explanation: Prints a message or value to the console, commonly for status output or error reporting.

```
0053 |             }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0054 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0055 |             try {
```

Explanation: Begins a protected block for code that may throw an exception; resources may be managed by try-with-resources.

```
0056 |                 if (st != null) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0057 |                     st.close();
```

Explanation: Closes the associated resource, such as a JDBC ResultSet, Statement, or Connection.

```
0058 |                 }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0059 |             } catch (Exception e) {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0060 |                 System.out.println(e);
```

Explanation: Prints a message or value to the console, commonly for status output or error reporting.

```
0061 |             }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0062 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0063 |             try {
```

Explanation: Begins a protected block for code that may throw an exception; resources may be managed by try-with-resources.

```
0064 |                 if (conn != null) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0065 |                     conn.close();
```

Explanation: Closes the associated resource, such as a JDBC ResultSet, Statement, or Connection.

```
0066 |                 }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0067 |             } catch (Exception e) {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0068 |                 System.out.println(e);
```

Explanation: Prints a message or value to the console, commonly for status output or error reporting.

```
0069 |             }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0070 |         }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0071 |     }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0072 | }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

DeleteDAO.java

Project path: src/DeleteDAO.java

```
0001 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0002 | import java.sql.Connection;
```

Explanation: Imports JDBC classes used to communicate with a relational database.

```
0003 | import java.sql.PreparedStatement;
```

Explanation: Imports JDBC classes used to communicate with a relational database.

```
0004 | import java.sql.ResultSet;
```

Explanation: Imports JDBC classes used to communicate with a relational database.

```
0005 | import java.sql.SQLException;
```

Explanation: Imports JDBC classes used to communicate with a relational database.

```
0006 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0007 | public class DeleteDAO {
```

Explanation: Declares the DeleteDAO type, which groups related data and behavior.

```
0008 |     private DbConnection db = new DbConnection();
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0009 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0010 |     public boolean deleteOwnRequirement(int requirementId, int userId)
```

Explanation: Declares the deleteOwnRequirement method and its return type/parameters; the method performs the operation described by its name and body.

```
0011 |             throws SQLException {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0012 |         Connection conn = null;
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0013 |         PreparedStatement ps = null;
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0014 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0015 |         try {
```

Explanation: Begins a protected block for code that may throw an exception; resources may be managed by try-with-resources.

```
0016 |             conn = db.getConnection();
```

Explanation: Assigns or updates a variable with the value or result on the right-hand side.

```
0017 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0018 |             if (conn == null) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0019 |                 throw new SQLException("Could not connect to the database.");
```

Explanation: Raises an exception so the caller or an enclosing catch block can handle the failure.

```
0020 |             }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0021 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0022 |             String sql =
```

Explanation: Stores the SQL command as a string so the program can execute it against the database.

```
0023 |                     "DELETE sr FROM student_requirements sr " +
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0024 |                     "JOIN enrollments e ON sr.enrollment_id = e.enrollment_id " +
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0025 |                     "JOIN students s ON e.student_id = s.student_id " +
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0026 |                     "WHERE sr.requirement_id = ? AND s.user_id = ? " +
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0027 |                     "AND LOWER(sr.requirement_status) IN ('submitted', 'incomplete')";
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0028 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0029 |             ps = conn.prepareStatement(sql);
```

Explanation: Creates a parameterized SQL statement; placeholders can be filled separately to reduce SQL-injection risk.

```
0030 |             ps.setInt(1, requirementId);
```

Explanation: Binds a Java value to a placeholder in the prepared SQL statement; the parameter index is one-based.

```
0031 |             ps.setInt(2, userId);
```

Explanation: Binds a Java value to a placeholder in the prepared SQL statement; the parameter index is one-based.

```
0032 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0033 |             int rowDeleted = ps.executeUpdate();
```

Explanation: Executes an INSERT, UPDATE, or DELETE statement and returns the number of affected rows.

```
0034 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0035 |             return rowDeleted &gt; 0;
```

Explanation: Returns the value or result to the caller and ends the current method.

```
0036 |         } catch (SQLException e) {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0037 |             throw e;
```

Explanation: Raises an exception so the caller or an enclosing catch block can handle the failure.

```
0038 |         } catch (Exception e) {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0039 |             throw new SQLException(e.getMessage(), e);
```

Explanation: Raises an exception so the caller or an enclosing catch block can handle the failure.

```
0040 |         } finally {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0041 |             try {
```

Explanation: Begins a protected block for code that may throw an exception; resources may be managed by try-with-resources.

```
0042 |                 if (ps != null) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0043 |                     ps.close();
```

Explanation: Closes the associated resource, such as a JDBC ResultSet, Statement, or Connection.

```
0044 |                 }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0045 |             } catch (Exception e) {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0046 |                 System.out.println(e);
```

Explanation: Prints a message or value to the console, commonly for status output or error reporting.

```
0047 |             }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0048 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0049 |             try {
```

Explanation: Begins a protected block for code that may throw an exception; resources may be managed by try-with-resources.

```
0050 |                 if (conn != null) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0051 |                     conn.close();
```

Explanation: Closes the associated resource, such as a JDBC ResultSet, Statement, or Connection.

```
0052 |                 }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0053 |             } catch (Exception e) {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0054 |                 System.out.println(e);
```

Explanation: Prints a message or value to the console, commonly for status output or error reporting.

```
0055 |             }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0056 |         }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0057 |     }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0058 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0059 |     public boolean deleteRequirement(int requirementId)
```

Explanation: Declares the deleteRequirement method and its return type/parameters; the method performs the operation described by its name and body.

```
0060 |             throws SQLException {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0061 |         Connection conn = null;
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0062 |         PreparedStatement ps = null;
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0063 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0064 |         try {
```

Explanation: Begins a protected block for code that may throw an exception; resources may be managed by try-with-resources.

```
0065 |             conn = db.getConnection();
```

Explanation: Assigns or updates a variable with the value or result on the right-hand side.

```
0066 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0067 |             if (conn == null) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0068 |                 throw new SQLException("Could not connect to the database.");
```

Explanation: Raises an exception so the caller or an enclosing catch block can handle the failure.

```
0069 |             }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0070 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0071 |             String sql =
```

Explanation: Stores the SQL command as a string so the program can execute it against the database.

```
0072 |                     "DELETE FROM student_requirements " +
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0073 |                     "WHERE requirement_id = ? " +
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0074 |                     "AND LOWER(requirement_status) IN ('submitted', 'incomplete')";
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0075 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0076 |             ps = conn.prepareStatement(sql);
```

Explanation: Creates a parameterized SQL statement; placeholders can be filled separately to reduce SQL-injection risk.

```
0077 |             ps.setInt(1, requirementId);
```

Explanation: Binds a Java value to a placeholder in the prepared SQL statement; the parameter index is one-based.

```
0078 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0079 |             int rowDeleted = ps.executeUpdate();
```

Explanation: Executes an INSERT, UPDATE, or DELETE statement and returns the number of affected rows.

```
0080 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0081 |             return rowDeleted &gt; 0;
```

Explanation: Returns the value or result to the caller and ends the current method.

```
0082 |         } catch (SQLException e) {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0083 |             throw e;
```

Explanation: Raises an exception so the caller or an enclosing catch block can handle the failure.

```
0084 |         } catch (Exception e) {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0085 |             throw new SQLException(e.getMessage(), e);
```

Explanation: Raises an exception so the caller or an enclosing catch block can handle the failure.

```
0086 |         } finally {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0087 |             try {
```

Explanation: Begins a protected block for code that may throw an exception; resources may be managed by try-with-resources.

```
0088 |                 if (ps != null) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0089 |                     ps.close();
```

Explanation: Closes the associated resource, such as a JDBC ResultSet, Statement, or Connection.

```
0090 |                 }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0091 |             } catch (Exception e) {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0092 |                 System.out.println(e);
```

Explanation: Prints a message or value to the console, commonly for status output or error reporting.

```
0093 |             }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0094 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0095 |             try {
```

Explanation: Begins a protected block for code that may throw an exception; resources may be managed by try-with-resources.

```
0096 |                 if (conn != null) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0097 |                     conn.close();
```

Explanation: Closes the associated resource, such as a JDBC ResultSet, Statement, or Connection.

```
0098 |                 }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0099 |             } catch (Exception e) {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0100 |                 System.out.println(e);
```

Explanation: Prints a message or value to the console, commonly for status output or error reporting.

```
0101 |             }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0102 |         }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0103 |     }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0104 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0105 |     public boolean cancelPayment(int paymentId)
```

Explanation: Declares the cancelPayment method and its return type/parameters; the method performs the operation described by its name and body.

```
0106 |             throws SQLException {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0107 |         Connection conn = null;
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0108 |         PreparedStatement ps = null;
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0109 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0110 |         try {
```

Explanation: Begins a protected block for code that may throw an exception; resources may be managed by try-with-resources.

```
0111 |             conn = db.getConnection();
```

Explanation: Assigns or updates a variable with the value or result on the right-hand side.

```
0112 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0113 |             if (conn == null) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0114 |                 throw new SQLException("Could not connect to the database.");
```

Explanation: Raises an exception so the caller or an enclosing catch block can handle the failure.

```
0115 |             }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0116 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0117 |             String sql =
```

Explanation: Stores the SQL command as a string so the program can execute it against the database.

```
0118 |                     "UPDATE payments SET payment_status = 'Cancelled' " +
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0119 |                     "WHERE payment_id = ? " +
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0120 |                     "AND LOWER(payment_status) = 'pending'";
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0121 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0122 |             ps = conn.prepareStatement(sql);
```

Explanation: Creates a parameterized SQL statement; placeholders can be filled separately to reduce SQL-injection risk.

```
0123 |             ps.setInt(1, paymentId);
```

Explanation: Binds a Java value to a placeholder in the prepared SQL statement; the parameter index is one-based.

```
0124 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0125 |             int rowUpdated = ps.executeUpdate();
```

Explanation: Executes an INSERT, UPDATE, or DELETE statement and returns the number of affected rows.

```
0126 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0127 |             return rowUpdated &gt; 0;
```

Explanation: Returns the value or result to the caller and ends the current method.

```
0128 |         } catch (SQLException e) {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0129 |             throw e;
```

Explanation: Raises an exception so the caller or an enclosing catch block can handle the failure.

```
0130 |         } catch (Exception e) {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0131 |             throw new SQLException(e.getMessage(), e);
```

Explanation: Raises an exception so the caller or an enclosing catch block can handle the failure.

```
0132 |         } finally {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0133 |             try {
```

Explanation: Begins a protected block for code that may throw an exception; resources may be managed by try-with-resources.

```
0134 |                 if (ps != null) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0135 |                     ps.close();
```

Explanation: Closes the associated resource, such as a JDBC ResultSet, Statement, or Connection.

```
0136 |                 }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0137 |             } catch (Exception e) {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0138 |                 System.out.println(e);
```

Explanation: Prints a message or value to the console, commonly for status output or error reporting.

```
0139 |             }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0140 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0141 |             try {
```

Explanation: Begins a protected block for code that may throw an exception; resources may be managed by try-with-resources.

```
0142 |                 if (conn != null) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0143 |                     conn.close();
```

Explanation: Closes the associated resource, such as a JDBC ResultSet, Statement, or Connection.

```
0144 |                 }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0145 |             } catch (Exception e) {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0146 |                 System.out.println(e);
```

Explanation: Prints a message or value to the console, commonly for status output or error reporting.

```
0147 |             }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0148 |         }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0149 |     }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0150 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0151 |     public boolean cancelEnrollment(int enrollmentId, int userId)
```

Explanation: Declares the cancelEnrollment method and its return type/parameters; the method performs the operation described by its name and body.

```
0152 |             throws SQLException {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0153 |         Connection conn = null;
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0154 |         PreparedStatement ps = null;
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0155 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0156 |         try {
```

Explanation: Begins a protected block for code that may throw an exception; resources may be managed by try-with-resources.

```
0157 |             conn = db.getConnection();
```

Explanation: Assigns or updates a variable with the value or result on the right-hand side.

```
0158 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0159 |             if (conn == null) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0160 |                 throw new SQLException("Could not connect to the database.");
```

Explanation: Raises an exception so the caller or an enclosing catch block can handle the failure.

```
0161 |             }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0162 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0163 |             String sql =
```

Explanation: Stores the SQL command as a string so the program can execute it against the database.

```
0164 |                     "UPDATE enrollments e " +
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0165 |                     "JOIN students s ON e.student_id = s.student_id " +
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0166 |                     "SET e.enrollment_status = 'Cancelled' " +
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0167 |                     "WHERE e.enrollment_id = ? AND s.user_id = ? " +
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0168 |                     "AND LOWER(e.enrollment_status) IN ('pending', 'rejected')";
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0169 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0170 |             ps = conn.prepareStatement(sql);
```

Explanation: Creates a parameterized SQL statement; placeholders can be filled separately to reduce SQL-injection risk.

```
0171 |             ps.setInt(1, enrollmentId);
```

Explanation: Binds a Java value to a placeholder in the prepared SQL statement; the parameter index is one-based.

```
0172 |             ps.setInt(2, userId);
```

Explanation: Binds a Java value to a placeholder in the prepared SQL statement; the parameter index is one-based.

```
0173 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0174 |             int rowUpdated = ps.executeUpdate();
```

Explanation: Executes an INSERT, UPDATE, or DELETE statement and returns the number of affected rows.

```
0175 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0176 |             return rowUpdated &gt; 0;
```

Explanation: Returns the value or result to the caller and ends the current method.

```
0177 |         } catch (SQLException e) {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0178 |             throw e;
```

Explanation: Raises an exception so the caller or an enclosing catch block can handle the failure.

```
0179 |         } catch (Exception e) {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0180 |             throw new SQLException(e.getMessage(), e);
```

Explanation: Raises an exception so the caller or an enclosing catch block can handle the failure.

```
0181 |         } finally {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0182 |             try {
```

Explanation: Begins a protected block for code that may throw an exception; resources may be managed by try-with-resources.

```
0183 |                 if (ps != null) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0184 |                     ps.close();
```

Explanation: Closes the associated resource, such as a JDBC ResultSet, Statement, or Connection.

```
0185 |                 }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0186 |             } catch (Exception e) {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0187 |                 System.out.println(e);
```

Explanation: Prints a message or value to the console, commonly for status output or error reporting.

```
0188 |             }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0189 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0190 |             try {
```

Explanation: Begins a protected block for code that may throw an exception; resources may be managed by try-with-resources.

```
0191 |                 if (conn != null) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0192 |                     conn.close();
```

Explanation: Closes the associated resource, such as a JDBC ResultSet, Statement, or Connection.

```
0193 |                 }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0194 |             } catch (Exception e) {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0195 |                 System.out.println(e);
```

Explanation: Prints a message or value to the console, commonly for status output or error reporting.

```
0196 |             }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0197 |         }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0198 |     }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0199 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0200 |     public boolean deactivateUser(int userId)
```

Explanation: Declares the deactivateUser method and its return type/parameters; the method performs the operation described by its name and body.

```
0201 |             throws SQLException {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0202 |         Connection conn = null;
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0203 |         PreparedStatement ps = null;
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0204 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0205 |         try {
```

Explanation: Begins a protected block for code that may throw an exception; resources may be managed by try-with-resources.

```
0206 |             conn = db.getConnection();
```

Explanation: Assigns or updates a variable with the value or result on the right-hand side.

```
0207 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0208 |             if (conn == null) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0209 |                 throw new SQLException("Could not connect to the database.");
```

Explanation: Raises an exception so the caller or an enclosing catch block can handle the failure.

```
0210 |             }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0211 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0212 |             String sql =
```

Explanation: Stores the SQL command as a string so the program can execute it against the database.

```
0213 |                     "UPDATE users SET is_active = 0 WHERE user_id = ?";
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0214 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0215 |             ps = conn.prepareStatement(sql);
```

Explanation: Creates a parameterized SQL statement; placeholders can be filled separately to reduce SQL-injection risk.

```
0216 |             ps.setInt(1, userId);
```

Explanation: Binds a Java value to a placeholder in the prepared SQL statement; the parameter index is one-based.

```
0217 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0218 |             int rowUpdated = ps.executeUpdate();
```

Explanation: Executes an INSERT, UPDATE, or DELETE statement and returns the number of affected rows.

```
0219 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0220 |             return rowUpdated &gt; 0;
```

Explanation: Returns the value or result to the caller and ends the current method.

```
0221 |         } catch (SQLException e) {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0222 |             throw e;
```

Explanation: Raises an exception so the caller or an enclosing catch block can handle the failure.

```
0223 |         } catch (Exception e) {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0224 |             throw new SQLException(e.getMessage(), e);
```

Explanation: Raises an exception so the caller or an enclosing catch block can handle the failure.

```
0225 |         } finally {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0226 |             try {
```

Explanation: Begins a protected block for code that may throw an exception; resources may be managed by try-with-resources.

```
0227 |                 if (ps != null) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0228 |                     ps.close();
```

Explanation: Closes the associated resource, such as a JDBC ResultSet, Statement, or Connection.

```
0229 |                 }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0230 |             } catch (Exception e) {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0231 |                 System.out.println(e);
```

Explanation: Prints a message or value to the console, commonly for status output or error reporting.

```
0232 |             }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0233 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0234 |             try {
```

Explanation: Begins a protected block for code that may throw an exception; resources may be managed by try-with-resources.

```
0235 |                 if (conn != null) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0236 |                     conn.close();
```

Explanation: Closes the associated resource, such as a JDBC ResultSet, Statement, or Connection.

```
0237 |                 }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0238 |             } catch (Exception e) {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0239 |                 System.out.println(e);
```

Explanation: Prints a message or value to the console, commonly for status output or error reporting.

```
0240 |             }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0241 |         }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0242 |     }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0243 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0244 |     public String deletePayment(int paymentId) throws SQLException {
```

Explanation: Declares the deletePayment method and its return type/parameters; the method performs the operation described by its name and body.

```
0245 |         if (paymentId &lt;= 0) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0246 |             throw new SQLException("Enter a valid Payment ID.");
```

Explanation: Raises an exception so the caller or an enclosing catch block can handle the failure.

```
0247 |         }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0248 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0249 |         try (Connection conn = new DbConnection().getConnection()) {
```

Explanation: Begins a protected block for code that may throw an exception; resources may be managed by try-with-resources.

```
0250 |             if (conn == null) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0251 |                 throw new SQLException("Could not connect to the database.");
```

Explanation: Raises an exception so the caller or an enclosing catch block can handle the failure.

```
0252 |             }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0253 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0254 |             conn.setAutoCommit(false);
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0255 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0256 |             try {
```

Explanation: Begins a protected block for code that may throw an exception; resources may be managed by try-with-resources.

```
0257 |                 String status;
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0258 |                 int requestId;
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0259 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0260 |                 String findSql =
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0261 |                         "SELECT request_id, payment_status FROM payments " +
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0262 |                         "WHERE payment_id = ? FOR UPDATE";
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0263 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0264 |                 try (PreparedStatement ps = conn.prepareStatement(findSql)) {
```

Explanation: Begins a protected block for code that may throw an exception; resources may be managed by try-with-resources.

```
0265 |                     ps.setInt(1, paymentId);
```

Explanation: Binds a Java value to a placeholder in the prepared SQL statement; the parameter index is one-based.

```
0266 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0267 |                     try (ResultSet rs = ps.executeQuery()) {
```

Explanation: Begins a protected block for code that may throw an exception; resources may be managed by try-with-resources.

```
0268 |                         if (!rs.next()) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0269 |                             throw new SQLException("Payment ID was not found.");
```

Explanation: Raises an exception so the caller or an enclosing catch block can handle the failure.

```
0270 |                         }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0271 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0272 |                         requestId = rs.getInt("request_id");
```

Explanation: Reads a column value from the current database result row using the requested Java type.

```
0273 |                         status = rs.getString("payment_status");
```

Explanation: Reads a column value from the current database result row using the requested Java type.

```
0274 |                     }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0275 |                 }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0276 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0277 |                 if ("Paid".equalsIgnoreCase(status)) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0278 |                     throw new SQLException(
```

Explanation: Raises an exception so the caller or an enclosing catch block can handle the failure.

```
0279 |                             "Paid payments cannot be deleted. Preserve verified transaction history.");
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0280 |                 }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0281 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0282 |                 String deleteSql =
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0283 |                         "DELETE FROM payments WHERE payment_id = ? " +
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0284 |                         "AND (payment_status IS NULL OR payment_status &lt;&gt; 'Paid')";
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0285 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0286 |                 try (PreparedStatement ps = conn.prepareStatement(deleteSql)) {
```

Explanation: Begins a protected block for code that may throw an exception; resources may be managed by try-with-resources.

```
0287 |                     ps.setInt(1, paymentId);
```

Explanation: Binds a Java value to a placeholder in the prepared SQL statement; the parameter index is one-based.

```
0288 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0289 |                     if (ps.executeUpdate() != 1) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0290 |                         throw new SQLException("Payment record could not be deleted.");
```

Explanation: Raises an exception so the caller or an enclosing catch block can handle the failure.

```
0291 |                     }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0292 |                 }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0293 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0294 |                 if (requestId &gt; 0) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0295 |                     String resetSql =
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0296 |                             "UPDATE payment_requests SET request_status = 'Open' " +
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0297 |                             "WHERE request_id = ? AND request_status = 'Payment Pending' " +
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0298 |                             "AND NOT EXISTS (" +
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0299 |                             "SELECT 1 FROM payments WHERE request_id = ? " +
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0300 |                             "AND payment_status = 'Pending')";
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0301 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0302 |                     try (PreparedStatement ps = conn.prepareStatement(resetSql)) {
```

Explanation: Begins a protected block for code that may throw an exception; resources may be managed by try-with-resources.

```
0303 |                         ps.setInt(1, requestId);
```

Explanation: Binds a Java value to a placeholder in the prepared SQL statement; the parameter index is one-based.

```
0304 |                         ps.setInt(2, requestId);
```

Explanation: Binds a Java value to a placeholder in the prepared SQL statement; the parameter index is one-based.

```
0305 |                         ps.executeUpdate();
```

Explanation: Executes an INSERT, UPDATE, or DELETE statement and returns the number of affected rows.

```
0306 |                     }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0307 |                 }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0308 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0309 |                 conn.commit();
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0310 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0311 |                 return "Payment record deleted successfully.\nPayment ID: " + paymentId;
```

Explanation: Returns the value or result to the caller and ends the current method.

```
0312 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0313 |             } catch (Exception ex) {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0314 |                 conn.rollback();
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0315 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0316 |                 if (ex instanceof SQLException) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0317 |                     throw (SQLException) ex;
```

Explanation: Raises an exception so the caller or an enclosing catch block can handle the failure.

```
0318 |                 }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0319 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0320 |                 throw new SQLException("Could not delete payment.", ex);
```

Explanation: Raises an exception so the caller or an enclosing catch block can handle the failure.

```
0321 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0322 |             } finally {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0323 |                 conn.setAutoCommit(true);
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0324 |             }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0325 |         }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0326 |     }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0327 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0328 |     public String deletePaymentRequest(int requestId) throws SQLException {
```

Explanation: Declares the deletePaymentRequest method and its return type/parameters; the method performs the operation described by its name and body.

```
0329 |         if (requestId &lt;= 0) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0330 |             throw new SQLException("Enter a valid Request ID.");
```

Explanation: Raises an exception so the caller or an enclosing catch block can handle the failure.

```
0331 |         }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0332 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0333 |         try (Connection conn = new DbConnection().getConnection()) {
```

Explanation: Begins a protected block for code that may throw an exception; resources may be managed by try-with-resources.

```
0334 |             if (conn == null) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0335 |                 throw new SQLException("Could not connect to the database.");
```

Explanation: Raises an exception so the caller or an enclosing catch block can handle the failure.

```
0336 |             }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0337 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0338 |             conn.setAutoCommit(false);
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0339 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0340 |             try {
```

Explanation: Begins a protected block for code that may throw an exception; resources may be managed by try-with-resources.

```
0341 |                 String status;
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0342 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0343 |                 String findSql =
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0344 |                         "SELECT request_status FROM payment_requests " +
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0345 |                         "WHERE request_id = ? FOR UPDATE";
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0346 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0347 |                 try (PreparedStatement ps = conn.prepareStatement(findSql)) {
```

Explanation: Begins a protected block for code that may throw an exception; resources may be managed by try-with-resources.

```
0348 |                     ps.setInt(1, requestId);
```

Explanation: Binds a Java value to a placeholder in the prepared SQL statement; the parameter index is one-based.

```
0349 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0350 |                     try (ResultSet rs = ps.executeQuery()) {
```

Explanation: Begins a protected block for code that may throw an exception; resources may be managed by try-with-resources.

```
0351 |                         if (!rs.next()) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0352 |                             throw new SQLException("Payment request was not found.");
```

Explanation: Raises an exception so the caller or an enclosing catch block can handle the failure.

```
0353 |                         }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0354 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0355 |                         status = rs.getString("request_status");
```

Explanation: Reads a column value from the current database result row using the requested Java type.

```
0356 |                     }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0357 |                 }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0358 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0359 |                 if ("Paid".equalsIgnoreCase(status)) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0360 |                     throw new SQLException("Paid payment requests cannot be deleted.");
```

Explanation: Raises an exception so the caller or an enclosing catch block can handle the failure.

```
0361 |                 }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0362 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0363 |                 String paymentCheckSql =
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0364 |                         "SELECT COUNT(*) FROM payments WHERE request_id = ?";
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0365 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0366 |                 try (PreparedStatement ps = conn.prepareStatement(paymentCheckSql)) {
```

Explanation: Begins a protected block for code that may throw an exception; resources may be managed by try-with-resources.

```
0367 |                     ps.setInt(1, requestId);
```

Explanation: Binds a Java value to a placeholder in the prepared SQL statement; the parameter index is one-based.

```
0368 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0369 |                     try (ResultSet rs = ps.executeQuery()) {
```

Explanation: Begins a protected block for code that may throw an exception; resources may be managed by try-with-resources.

```
0370 |                         if (rs.next() &amp;amp;&amp;amp; rs.getInt(1) &amp;gt; 0) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0371 |                             throw new SQLException(
```

Explanation: Raises an exception so the caller or an enclosing catch block can handle the failure.

```
0372 |                                     "This request has payment records. Resolve or delete those records first.");
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0373 |                         }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0374 |                     }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0375 |                 }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0376 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0377 |                 String detailSql =
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0378 |                         "DELETE FROM payment_request_details WHERE request_id = ?";
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0379 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0380 |                 try (PreparedStatement ps = conn.prepareStatement(detailSql)) {
```

Explanation: Begins a protected block for code that may throw an exception; resources may be managed by try-with-resources.

```
0381 |                     ps.setInt(1, requestId);
```

Explanation: Binds a Java value to a placeholder in the prepared SQL statement; the parameter index is one-based.

```
0382 |                     ps.executeUpdate();
```

Explanation: Executes an INSERT, UPDATE, or DELETE statement and returns the number of affected rows.

```
0383 |                 }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0384 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0385 |                 String deleteSql =
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0386 |                         "DELETE FROM payment_requests WHERE request_id = ?";
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0387 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0388 |                 try (PreparedStatement ps = conn.prepareStatement(deleteSql)) {
```

Explanation: Begins a protected block for code that may throw an exception; resources may be managed by try-with-resources.

```
0389 |                     ps.setInt(1, requestId);
```

Explanation: Binds a Java value to a placeholder in the prepared SQL statement; the parameter index is one-based.

```
0390 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0391 |                     if (ps.executeUpdate() != 1) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0392 |                         throw new SQLException("Payment request was not deleted.");
```

Explanation: Raises an exception so the caller or an enclosing catch block can handle the failure.

```
0393 |                     }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0394 |                 }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0395 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0396 |                 conn.commit();
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0397 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0398 |                 return "Payment request deleted successfully.\nRequest ID: " + requestId;
```

Explanation: Returns the value or result to the caller and ends the current method.

```
0399 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0400 |             } catch (Exception ex) {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0401 |                 conn.rollback();
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0402 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0403 |                 if (ex instanceof SQLException) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0404 |                     throw (SQLException) ex;
```

Explanation: Raises an exception so the caller or an enclosing catch block can handle the failure.

```
0405 |                 }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0406 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0407 |                 throw new SQLException("Could not delete payment request.", ex);
```

Explanation: Raises an exception so the caller or an enclosing catch block can handle the failure.

```
0408 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0409 |             } finally {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0410 |                 conn.setAutoCommit(true);
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0411 |             }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0412 |         }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0413 |     }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0414 | }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

EnrollmentDAO.java

Project path: src/EnrollmentDAO.java

```
0001 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0002 | import java.sql.*;
```

Explanation: Imports JDBC classes used to communicate with a relational database.

```
0003 | import java.util.ArrayList;
```

Explanation: Imports Java utility classes such as collections, lists, maps, or helper types.

```
0004 | import java.util.List;
```

Explanation: Imports Java utility classes such as collections, lists, maps, or helper types.

```
0005 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0006 | public class EnrollmentDAO {
```

Explanation: Declares the EnrollmentDAO type, which groups related data and behavior.

```
0007 |     private DbConnection db = new DbConnection();
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0008 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0009 |     public boolean apply(int studentId, int schoolYearId, int strandId,
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0010 |                          int gradeLevel, String enrolleeType) {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0011 |         Connection conn = null;
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0012 |         PreparedStatement ps = null;
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0013 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0014 |         try {
```

Explanation: Begins a protected block for code that may throw an exception; resources may be managed by try-with-resources.

```
0015 |             conn = db.getConnection();
```

Explanation: Assigns or updates a variable with the value or result on the right-hand side.

```
0016 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0017 |             if (conn == null) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0018 |                 return false;
```

Explanation: Returns the value or result to the caller and ends the current method.

```
0019 |             }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0020 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0021 |             String sql = "INSERT INTO enrollments "
```

Explanation: Stores the SQL command as a string so the program can execute it against the database.

```
0022 |                     + "(student_id, school_year_id, strand_id, year_level, "
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0023 |                     + "enrollee_type, enrollment_status) "
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0024 |                     + "VALUES (?, ?, ?, ?, ?, 'Pending')";
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0025 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0026 |             ps = conn.prepareStatement(sql);
```

Explanation: Creates a parameterized SQL statement; placeholders can be filled separately to reduce SQL-injection risk.

```
0027 |             ps.setInt(1, studentId);
```

Explanation: Binds a Java value to a placeholder in the prepared SQL statement; the parameter index is one-based.

```
0028 |             ps.setInt(2, schoolYearId);
```

Explanation: Binds a Java value to a placeholder in the prepared SQL statement; the parameter index is one-based.

```
0029 |             ps.setInt(3, strandId);
```

Explanation: Binds a Java value to a placeholder in the prepared SQL statement; the parameter index is one-based.

```
0030 |             ps.setInt(4, gradeLevel);
```

Explanation: Binds a Java value to a placeholder in the prepared SQL statement; the parameter index is one-based.

```
0031 |             ps.setString(5, enrolleeType);
```

Explanation: Binds a Java value to a placeholder in the prepared SQL statement; the parameter index is one-based.

```
0032 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0033 |             int rowInserted = ps.executeUpdate();
```

Explanation: Executes an INSERT, UPDATE, or DELETE statement and returns the number of affected rows.

```
0034 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0035 |             return rowInserted &gt; 0;
```

Explanation: Returns the value or result to the caller and ends the current method.

```
0036 |         } catch (Exception e) {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0037 |             System.out.println("Enrollment error: " + e.getMessage());
```

Explanation: Prints a message or value to the console, commonly for status output or error reporting.

```
0038 |             return false;
```

Explanation: Returns the value or result to the caller and ends the current method.

```
0039 |         } finally {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0040 |             try {
```

Explanation: Begins a protected block for code that may throw an exception; resources may be managed by try-with-resources.

```
0041 |                 if (ps != null) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0042 |                     ps.close();
```

Explanation: Closes the associated resource, such as a JDBC ResultSet, Statement, or Connection.

```
0043 |                 }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0044 |             } catch (Exception e) {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0045 |                 System.out.println(e);
```

Explanation: Prints a message or value to the console, commonly for status output or error reporting.

```
0046 |             }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0047 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0048 |             try {
```

Explanation: Begins a protected block for code that may throw an exception; resources may be managed by try-with-resources.

```
0049 |                 if (conn != null) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0050 |                     conn.close();
```

Explanation: Closes the associated resource, such as a JDBC ResultSet, Statement, or Connection.

```
0051 |                 }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0052 |             } catch (Exception e) {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0053 |                 System.out.println(e);
```

Explanation: Prints a message or value to the console, commonly for status output or error reporting.

```
0054 |             }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0055 |         }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0056 |     }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0057 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0058 |     public String getMyEnrollments(int studentId) {
```

Explanation: Declares the getMyEnrollments method, typically returning the requested field or resource.

```
0059 |         Connection conn = null;
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0060 |         PreparedStatement ps = null;
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0061 |         ResultSet rs = null;
```

Explanation: Declares or assigns a ResultSet that holds rows returned by a database query.

```
0062 |         StringBuilder result = new StringBuilder();
```

Explanation: Creates a mutable string builder so text can be assembled efficiently.

```
0063 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0064 |         try {
```

Explanation: Begins a protected block for code that may throw an exception; resources may be managed by try-with-resources.

```
0065 |             conn = db.getConnection();
```

Explanation: Assigns or updates a variable with the value or result on the right-hand side.

```
0066 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0067 |             if (conn == null) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0068 |                 return "Error: Could not connect to the database.";
```

Explanation: Returns the value or result to the caller and ends the current method.

```
0069 |             }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0070 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0071 |             String sql = "SELECT e.enrollment_id, sy.year_label, s.strand_code, "
```

Explanation: Stores the SQL command as a string so the program can execute it against the database.

```
0072 |                     + "e.year_level, e.enrollee_type, e.enrollment_status "
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0073 |                     + "FROM enrollments e "
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0074 |                     + "JOIN school_years sy ON e.school_year_id = sy.school_year_id "
```

Explanation: Assigns or updates a variable with the value or result on the right-hand side.

```
0075 |                     + "JOIN strands s ON e.strand_id = s.strand_id "
```

Explanation: Assigns or updates a variable with the value or result on the right-hand side.

```
0076 |                     + "WHERE e.student_id = ?";
```

Explanation: Assigns or updates a variable with the value or result on the right-hand side.

```
0077 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0078 |             ps = conn.prepareStatement(sql);
```

Explanation: Creates a parameterized SQL statement; placeholders can be filled separately to reduce SQL-injection risk.

```
0079 |             ps.setInt(1, studentId);
```

Explanation: Binds a Java value to a placeholder in the prepared SQL statement; the parameter index is one-based.

```
0080 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0081 |             rs = ps.executeQuery();
```

Explanation: Executes a SQL query that returns rows, usually a SELECT, and stores them in a ResultSet.

```
0082 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0083 |             while (rs.next()) {
```

Explanation: Repeats the following block while the condition remains true.

```
0084 |                 result.append("Application ID: ")
```

Explanation: Appends the provided value to the current StringBuilder or collection.

```
0085 |                         .append(rs.getInt("enrollment_id"))
```

Explanation: Reads a column value from the current database result row using the requested Java type.

```
0086 |                         .append("\nSchool Year: ")
```

Explanation: Appends the provided value to the current StringBuilder or collection.

```
0087 |                         .append(rs.getString("year_label"))
```

Explanation: Reads a column value from the current database result row using the requested Java type.

```
0088 |                         .append("\nStrand: ")
```

Explanation: Appends the provided value to the current StringBuilder or collection.

```
0089 |                         .append(rs.getString("strand_code"))
```

Explanation: Reads a column value from the current database result row using the requested Java type.

```
0090 |                         .append("\nGrade: ")
```

Explanation: Appends the provided value to the current StringBuilder or collection.

```
0091 |                         .append(rs.getInt("year_level"))
```

Explanation: Reads a column value from the current database result row using the requested Java type.

```
0092 |                         .append("\nType: ")
```

Explanation: Appends the provided value to the current StringBuilder or collection.

```
0093 |                         .append(rs.getString("enrollee_type"))
```

Explanation: Reads a column value from the current database result row using the requested Java type.

```
0094 |                         .append("\nStatus: ")
```

Explanation: Appends the provided value to the current StringBuilder or collection.

```
0095 |                         .append(rs.getString("enrollment_status"))
```

Explanation: Reads a column value from the current database result row using the requested Java type.

```
0096 |                         .append("\n------------------------\n");
```

Explanation: Appends the provided value to the current StringBuilder or collection.

```
0097 |             }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0098 |         } catch (Exception e) {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0099 |             return "Error: " + e.getMessage();
```

Explanation: Returns the value or result to the caller and ends the current method.

```
0100 |         } finally {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0101 |             try {
```

Explanation: Begins a protected block for code that may throw an exception; resources may be managed by try-with-resources.

```
0102 |                 if (rs != null) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0103 |                     rs.close();
```

Explanation: Closes the associated resource, such as a JDBC ResultSet, Statement, or Connection.

```
0104 |                 }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0105 |             } catch (Exception e) {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0106 |                 System.out.println(e);
```

Explanation: Prints a message or value to the console, commonly for status output or error reporting.

```
0107 |             }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0108 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0109 |             try {
```

Explanation: Begins a protected block for code that may throw an exception; resources may be managed by try-with-resources.

```
0110 |                 if (ps != null) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0111 |                     ps.close();
```

Explanation: Closes the associated resource, such as a JDBC ResultSet, Statement, or Connection.

```
0112 |                 }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0113 |             } catch (Exception e) {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0114 |                 System.out.println(e);
```

Explanation: Prints a message or value to the console, commonly for status output or error reporting.

```
0115 |             }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0116 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0117 |             try {
```

Explanation: Begins a protected block for code that may throw an exception; resources may be managed by try-with-resources.

```
0118 |                 if (conn != null) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0119 |                     conn.close();
```

Explanation: Closes the associated resource, such as a JDBC ResultSet, Statement, or Connection.

```
0120 |                 }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0121 |             } catch (Exception e) {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0122 |                 System.out.println(e);
```

Explanation: Prints a message or value to the console, commonly for status output or error reporting.

```
0123 |             }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0124 |         }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0125 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0126 |         return result.length() == 0
```

Explanation: Returns the value or result to the caller and ends the current method.

```
0127 |                 ? "No enrollment applications found."
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0128 |                 : result.toString();
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0129 |     }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0130 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0131 |     public String getAllEnrollments() {
```

Explanation: Declares the getAllEnrollments method, typically returning the requested field or resource.

```
0132 |         Connection conn = null;
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0133 |         PreparedStatement ps = null;
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0134 |         ResultSet rs = null;
```

Explanation: Declares or assigns a ResultSet that holds rows returned by a database query.

```
0135 |         StringBuilder result = new StringBuilder();
```

Explanation: Creates a mutable string builder so text can be assembled efficiently.

```
0136 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0137 |         try {
```

Explanation: Begins a protected block for code that may throw an exception; resources may be managed by try-with-resources.

```
0138 |             conn = db.getConnection();
```

Explanation: Assigns or updates a variable with the value or result on the right-hand side.

```
0139 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0140 |             if (conn == null) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0141 |                 return "Error: Could not connect to the database.";
```

Explanation: Returns the value or result to the caller and ends the current method.

```
0142 |             }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0143 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0144 |             String sql = "SELECT e.enrollment_id, st.first_name, st.last_name, "
```

Explanation: Stores the SQL command as a string so the program can execute it against the database.

```
0145 |                     + "s.strand_code, e.year_level, e.enrollee_type, "
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0146 |                     + "e.enrollment_status FROM enrollments e "
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0147 |                     + "JOIN students st ON e.student_id = st.student_id "
```

Explanation: Assigns or updates a variable with the value or result on the right-hand side.

```
0148 |                     + "JOIN strands s ON e.strand_id = s.strand_id "
```

Explanation: Assigns or updates a variable with the value or result on the right-hand side.

```
0149 |                     + "ORDER BY e.enrollment_id DESC";
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0150 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0151 |             ps = conn.prepareStatement(sql);
```

Explanation: Creates a parameterized SQL statement; placeholders can be filled separately to reduce SQL-injection risk.

```
0152 |             rs = ps.executeQuery();
```

Explanation: Executes a SQL query that returns rows, usually a SELECT, and stores them in a ResultSet.

```
0153 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0154 |             while (rs.next()) {
```

Explanation: Repeats the following block while the condition remains true.

```
0155 |                 result.append("Application ID: ")
```

Explanation: Appends the provided value to the current StringBuilder or collection.

```
0156 |                         .append(rs.getInt("enrollment_id"))
```

Explanation: Reads a column value from the current database result row using the requested Java type.

```
0157 |                         .append("\nStudent: ")
```

Explanation: Appends the provided value to the current StringBuilder or collection.

```
0158 |                         .append(rs.getString("first_name")).append(" ")
```

Explanation: Reads a column value from the current database result row using the requested Java type.

```
0159 |                         .append(rs.getString("last_name"))
```

Explanation: Reads a column value from the current database result row using the requested Java type.

```
0160 |                         .append("\nStrand: ")
```

Explanation: Appends the provided value to the current StringBuilder or collection.

```
0161 |                         .append(rs.getString("strand_code"))
```

Explanation: Reads a column value from the current database result row using the requested Java type.

```
0162 |                         .append("\nGrade: ")
```

Explanation: Appends the provided value to the current StringBuilder or collection.

```
0163 |                         .append(rs.getInt("year_level"))
```

Explanation: Reads a column value from the current database result row using the requested Java type.

```
0164 |                         .append("\nType: ")
```

Explanation: Appends the provided value to the current StringBuilder or collection.

```
0165 |                         .append(rs.getString("enrollee_type"))
```

Explanation: Reads a column value from the current database result row using the requested Java type.

```
0166 |                         .append("\nStatus: ")
```

Explanation: Appends the provided value to the current StringBuilder or collection.

```
0167 |                         .append(rs.getString("enrollment_status"))
```

Explanation: Reads a column value from the current database result row using the requested Java type.

```
0168 |                         .append("\n-------------------------\n");
```

Explanation: Appends the provided value to the current StringBuilder or collection.

```
0169 |             }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0170 |         } catch (Exception e) {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0171 |             return "Error: " + e.getMessage();
```

Explanation: Returns the value or result to the caller and ends the current method.

```
0172 |         } finally {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0173 |             try {
```

Explanation: Begins a protected block for code that may throw an exception; resources may be managed by try-with-resources.

```
0174 |                 if (rs != null) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0175 |                     rs.close();
```

Explanation: Closes the associated resource, such as a JDBC ResultSet, Statement, or Connection.

```
0176 |                 }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0177 |             } catch (Exception e) {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0178 |                 System.out.println(e);
```

Explanation: Prints a message or value to the console, commonly for status output or error reporting.

```
0179 |             }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0180 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0181 |             try {
```

Explanation: Begins a protected block for code that may throw an exception; resources may be managed by try-with-resources.

```
0182 |                 if (ps != null) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0183 |                     ps.close();
```

Explanation: Closes the associated resource, such as a JDBC ResultSet, Statement, or Connection.

```
0184 |                 }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0185 |             } catch (Exception e) {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0186 |                 System.out.println(e);
```

Explanation: Prints a message or value to the console, commonly for status output or error reporting.

```
0187 |             }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0188 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0189 |             try {
```

Explanation: Begins a protected block for code that may throw an exception; resources may be managed by try-with-resources.

```
0190 |                 if (conn != null) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0191 |                     conn.close();
```

Explanation: Closes the associated resource, such as a JDBC ResultSet, Statement, or Connection.

```
0192 |                 }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0193 |             } catch (Exception e) {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0194 |                 System.out.println(e);
```

Explanation: Prints a message or value to the console, commonly for status output or error reporting.

```
0195 |             }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0196 |         }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0197 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0198 |         return result.length() == 0
```

Explanation: Returns the value or result to the caller and ends the current method.

```
0199 |                 ? "No applications found."
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0200 |                 : result.toString();
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0201 |     }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0202 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0203 |     public boolean updateStatus(int enrollmentId, String status) {
```

Explanation: Declares the updateStatus method and its return type/parameters; the method performs the operation described by its name and body.

```
0204 |         Connection conn = null;
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0205 |         PreparedStatement ps = null;
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0206 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0207 |         try {
```

Explanation: Begins a protected block for code that may throw an exception; resources may be managed by try-with-resources.

```
0208 |             conn = db.getConnection();
```

Explanation: Assigns or updates a variable with the value or result on the right-hand side.

```
0209 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0210 |             if (conn == null) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0211 |                 return false;
```

Explanation: Returns the value or result to the caller and ends the current method.

```
0212 |             }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0213 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0214 |             String sql = "UPDATE enrollments SET enrollment_status = ? "
```

Explanation: Stores the SQL command as a string so the program can execute it against the database.

```
0215 |                     + "WHERE enrollment_id = ?";
```

Explanation: Assigns or updates a variable with the value or result on the right-hand side.

```
0216 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0217 |             ps = conn.prepareStatement(sql);
```

Explanation: Creates a parameterized SQL statement; placeholders can be filled separately to reduce SQL-injection risk.

```
0218 |             ps.setString(1, status);
```

Explanation: Binds a Java value to a placeholder in the prepared SQL statement; the parameter index is one-based.

```
0219 |             ps.setInt(2, enrollmentId);
```

Explanation: Binds a Java value to a placeholder in the prepared SQL statement; the parameter index is one-based.

```
0220 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0221 |             int rowUpdated = ps.executeUpdate();
```

Explanation: Executes an INSERT, UPDATE, or DELETE statement and returns the number of affected rows.

```
0222 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0223 |             return rowUpdated &gt; 0;
```

Explanation: Returns the value or result to the caller and ends the current method.

```
0224 |         } catch (Exception e) {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0225 |             System.out.println("Status update error: " + e.getMessage());
```

Explanation: Prints a message or value to the console, commonly for status output or error reporting.

```
0226 |             return false;
```

Explanation: Returns the value or result to the caller and ends the current method.

```
0227 |         } finally {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0228 |             try {
```

Explanation: Begins a protected block for code that may throw an exception; resources may be managed by try-with-resources.

```
0229 |                 if (ps != null) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0230 |                     ps.close();
```

Explanation: Closes the associated resource, such as a JDBC ResultSet, Statement, or Connection.

```
0231 |                 }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0232 |             } catch (Exception e) {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0233 |                 System.out.println(e);
```

Explanation: Prints a message or value to the console, commonly for status output or error reporting.

```
0234 |             }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0235 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0236 |             try {
```

Explanation: Begins a protected block for code that may throw an exception; resources may be managed by try-with-resources.

```
0237 |                 if (conn != null) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0238 |                     conn.close();
```

Explanation: Closes the associated resource, such as a JDBC ResultSet, Statement, or Connection.

```
0239 |                 }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0240 |             } catch (Exception e) {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0241 |                 System.out.println(e);
```

Explanation: Prints a message or value to the console, commonly for status output or error reporting.

```
0242 |             }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0243 |         }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0244 |     }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0245 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0246 |     public static class SectionOption {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0247 |         public int sectionId;
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0248 |         public String sectionName;
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0249 |         public int capacity;
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0250 |         public int currentCount;
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0251 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0252 |         @Override
```

Explanation: Applies an annotation to the declaration that follows; the annotation supplies metadata or framework behavior.

```
0253 |         public String toString() {
```

Explanation: Declares the toString method and its return type/parameters; the method performs the operation described by its name and body.

```
0254 |             return sectionName + " ("
```

Explanation: Returns the value or result to the caller and ends the current method.

```
0255 |                     + currentCount + "/" + capacity + " slots)";
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0256 |         }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0257 |     }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0258 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0259 |     public List&lt;SectionOption&gt; getAvailableSections(int yearLevel, int strandId,
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0260 |             int schoolYearId, Integer currentSectionId) throws SQLException {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0261 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0262 |         List&lt;SectionOption&gt; sections = new ArrayList&lt;&gt;();
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0263 |         Connection conn = null;
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0264 |         PreparedStatement ps = null;
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0265 |         ResultSet rs = null;
```

Explanation: Declares or assigns a ResultSet that holds rows returned by a database query.

```
0266 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0267 |         try {
```

Explanation: Begins a protected block for code that may throw an exception; resources may be managed by try-with-resources.

```
0268 |             conn = db.getConnection();
```

Explanation: Assigns or updates a variable with the value or result on the right-hand side.

```
0269 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0270 |             if (conn == null) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0271 |                 throw new SQLException("Could not connect to the database.");
```

Explanation: Raises an exception so the caller or an enclosing catch block can handle the failure.

```
0272 |             }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0273 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0274 |             String sql =
```

Explanation: Stores the SQL command as a string so the program can execute it against the database.

```
0275 |                     "SELECT sec.section_id, sec.section_name, "
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0276 |                             + "sec.slot_limit, "
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0277 |                             + "(SELECT COUNT(*) FROM enrollments e "
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0278 |                             + "WHERE e.section_id = sec.section_id "
```

Explanation: Assigns or updates a variable with the value or result on the right-hand side.

```
0279 |                             + "AND e.enrollment_status &lt;&gt; 'Rejected') "
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0280 |                             + "AS current_count "
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0281 |                             + "FROM sections sec "
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0282 |                             + "WHERE sec.grade_level = ? "
```

Explanation: Assigns or updates a variable with the value or result on the right-hand side.

```
0283 |                             + "AND sec.strand_id = ? "
```

Explanation: Assigns or updates a variable with the value or result on the right-hand side.

```
0284 |                             + "AND sec.school_year_id = ? "
```

Explanation: Assigns or updates a variable with the value or result on the right-hand side.

```
0285 |                             + "AND sec.is_active = TRUE "
```

Explanation: Assigns or updates a variable with the value or result on the right-hand side.

```
0286 |                             + "ORDER BY sec.section_name";
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0287 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0288 |             ps = conn.prepareStatement(sql);
```

Explanation: Creates a parameterized SQL statement; placeholders can be filled separately to reduce SQL-injection risk.

```
0289 |             ps.setInt(1, yearLevel);
```

Explanation: Binds a Java value to a placeholder in the prepared SQL statement; the parameter index is one-based.

```
0290 |             ps.setInt(2, strandId);
```

Explanation: Binds a Java value to a placeholder in the prepared SQL statement; the parameter index is one-based.

```
0291 |             ps.setInt(3, schoolYearId);
```

Explanation: Binds a Java value to a placeholder in the prepared SQL statement; the parameter index is one-based.

```
0292 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0293 |             rs = ps.executeQuery();
```

Explanation: Executes a SQL query that returns rows, usually a SELECT, and stores them in a ResultSet.

```
0294 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0295 |             while (rs.next()) {
```

Explanation: Repeats the following block while the condition remains true.

```
0296 |                 int capacity = rs.getInt("slot_limit");
```

Explanation: Reads a column value from the current database result row using the requested Java type.

```
0297 |                 int count = rs.getInt("current_count");
```

Explanation: Reads a column value from the current database result row using the requested Java type.

```
0298 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0299 |                 if (currentSectionId != null
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0300 |                         &amp;&amp; currentSectionId
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0301 |                         == rs.getInt("section_id")) {
```

Explanation: Reads a column value from the current database result row using the requested Java type.

```
0302 |                     count--;
```

Explanation: Increments or decrements a counter by one, commonly used to track loop progress.

```
0303 |                 }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0304 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0305 |                 if (count &lt; capacity) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0306 |                     SectionOption section = new SectionOption();
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0307 |                     section.sectionId = rs.getInt("section_id");
```

Explanation: Reads a column value from the current database result row using the requested Java type.

```
0308 |                     section.sectionName =
```

Explanation: Assigns or updates a variable with the value or result on the right-hand side.

```
0309 |                             rs.getString("section_name");
```

Explanation: Reads a column value from the current database result row using the requested Java type.

```
0310 |                     section.capacity = capacity;
```

Explanation: Assigns or updates a variable with the value or result on the right-hand side.

```
0311 |                     section.currentCount = Math.max(count, 0);
```

Explanation: Assigns or updates a variable with the value or result on the right-hand side.

```
0312 |                     sections.add(section);
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0313 |                 }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0314 |             }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0315 |         } catch (SQLException e) {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0316 |             throw e;
```

Explanation: Raises an exception so the caller or an enclosing catch block can handle the failure.

```
0317 |         } catch (Exception e) {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0318 |             throw new SQLException(e.getMessage(), e);
```

Explanation: Raises an exception so the caller or an enclosing catch block can handle the failure.

```
0319 |         } finally {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0320 |             try {
```

Explanation: Begins a protected block for code that may throw an exception; resources may be managed by try-with-resources.

```
0321 |                 if (rs != null) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0322 |                     rs.close();
```

Explanation: Closes the associated resource, such as a JDBC ResultSet, Statement, or Connection.

```
0323 |                 }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0324 |             } catch (Exception e) {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0325 |                 System.out.println(e);
```

Explanation: Prints a message or value to the console, commonly for status output or error reporting.

```
0326 |             }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0327 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0328 |             try {
```

Explanation: Begins a protected block for code that may throw an exception; resources may be managed by try-with-resources.

```
0329 |                 if (ps != null) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0330 |                     ps.close();
```

Explanation: Closes the associated resource, such as a JDBC ResultSet, Statement, or Connection.

```
0331 |                 }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0332 |             } catch (Exception e) {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0333 |                 System.out.println(e);
```

Explanation: Prints a message or value to the console, commonly for status output or error reporting.

```
0334 |             }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0335 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0336 |             try {
```

Explanation: Begins a protected block for code that may throw an exception; resources may be managed by try-with-resources.

```
0337 |                 if (conn != null) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0338 |                     conn.close();
```

Explanation: Closes the associated resource, such as a JDBC ResultSet, Statement, or Connection.

```
0339 |                 }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0340 |             } catch (Exception e) {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0341 |                 System.out.println(e);
```

Explanation: Prints a message or value to the console, commonly for status output or error reporting.

```
0342 |             }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0343 |         }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0344 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0345 |         return sections;
```

Explanation: Returns the value or result to the caller and ends the current method.

```
0346 |     }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0347 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0348 |     public void assignSection(int enrollmentId, int sectionId, int yearLevel,
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0349 |             int strandId, int schoolYearId) throws SQLException {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0350 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0351 |         String sectionSql =
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0352 |                 "SELECT grade_level, strand_id, school_year_id, "
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0353 |                         + "slot_limit, is_active "
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0354 |                         + "FROM sections WHERE section_id = ?";
```

Explanation: Assigns or updates a variable with the value or result on the right-hand side.

```
0355 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0356 |         String countSql =
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0357 |                 "SELECT COUNT(*) FROM enrollments "
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0358 |                         + "WHERE section_id = ? "
```

Explanation: Assigns or updates a variable with the value or result on the right-hand side.

```
0359 |                         + "AND enrollment_status &lt;&gt; 'Rejected' "
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0360 |                         + "AND enrollment_id &lt;&gt; ?";
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0361 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0362 |         String updateSql =
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0363 |                 "UPDATE enrollments SET section_id = ? "
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0364 |                         + "WHERE enrollment_id = ? "
```

Explanation: Assigns or updates a variable with the value or result on the right-hand side.

```
0365 |                         + "AND year_level IN (11, 12) "
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0366 |                         + "AND strand_id = ? "
```

Explanation: Assigns or updates a variable with the value or result on the right-hand side.

```
0367 |                         + "AND school_year_id = ?";
```

Explanation: Assigns or updates a variable with the value or result on the right-hand side.

```
0368 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0369 |         try (Connection conn = db.getConnection()) {
```

Explanation: Begins a protected block for code that may throw an exception; resources may be managed by try-with-resources.

```
0370 |             if (conn == null) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0371 |                 throw new SQLException("Could not connect to the database.");
```

Explanation: Raises an exception so the caller or an enclosing catch block can handle the failure.

```
0372 |             }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0373 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0374 |             conn.setAutoCommit(false);
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0375 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0376 |             try {
```

Explanation: Begins a protected block for code that may throw an exception; resources may be managed by try-with-resources.

```
0377 |                 int grade;
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0378 |                 int sectionStrandId;
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0379 |                 int sectionSchoolYearId;
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0380 |                 int capacity;
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0381 |                 boolean active;
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0382 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0383 |                 try (PreparedStatement ps =
```

Explanation: Begins a protected block for code that may throw an exception; resources may be managed by try-with-resources.

```
0384 |                             conn.prepareStatement(sectionSql)) {
```

Explanation: Creates a parameterized SQL statement; placeholders can be filled separately to reduce SQL-injection risk.

```
0385 |                     ps.setInt(1, sectionId);
```

Explanation: Binds a Java value to a placeholder in the prepared SQL statement; the parameter index is one-based.

```
0386 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0387 |                     try (ResultSet rs = ps.executeQuery()) {
```

Explanation: Begins a protected block for code that may throw an exception; resources may be managed by try-with-resources.

```
0388 |                         if (!rs.next()) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0389 |                             throw new SQLException(
```

Explanation: Raises an exception so the caller or an enclosing catch block can handle the failure.

```
0390 |                                     "Section was not found.");
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0391 |                         }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0392 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0393 |                         grade = rs.getInt("grade_level");
```

Explanation: Reads a column value from the current database result row using the requested Java type.

```
0394 |                         sectionStrandId = rs.getInt("strand_id");
```

Explanation: Reads a column value from the current database result row using the requested Java type.

```
0395 |                         sectionSchoolYearId = rs.getInt("school_year_id");
```

Explanation: Reads a column value from the current database result row using the requested Java type.

```
0396 |                         capacity = rs.getInt("slot_limit");
```

Explanation: Reads a column value from the current database result row using the requested Java type.

```
0397 |                         active = rs.getBoolean("is_active");
```

Explanation: Assigns or updates a variable with the value or result on the right-hand side.

```
0398 |                     }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0399 |                 }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0400 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0401 |                 if (!active
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0402 |                         || grade != yearLevel
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0403 |                         || sectionStrandId != strandId
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0404 |                         || sectionSchoolYearId != schoolYearId) {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0405 |                     throw new SQLException(
```

Explanation: Raises an exception so the caller or an enclosing catch block can handle the failure.

```
0406 |                             "Section does not match the student's "
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0407 |                                     + "grade, strand, or school year.");
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0408 |                 }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0409 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0410 |                 int count;
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0411 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0412 |                 try (PreparedStatement ps =
```

Explanation: Begins a protected block for code that may throw an exception; resources may be managed by try-with-resources.

```
0413 |                             conn.prepareStatement(countSql)) {
```

Explanation: Creates a parameterized SQL statement; placeholders can be filled separately to reduce SQL-injection risk.

```
0414 |                     ps.setInt(1, sectionId);
```

Explanation: Binds a Java value to a placeholder in the prepared SQL statement; the parameter index is one-based.

```
0415 |                     ps.setInt(2, enrollmentId);
```

Explanation: Binds a Java value to a placeholder in the prepared SQL statement; the parameter index is one-based.

```
0416 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0417 |                     try (ResultSet rs = ps.executeQuery()) {
```

Explanation: Begins a protected block for code that may throw an exception; resources may be managed by try-with-resources.

```
0418 |                         rs.next();
```

Explanation: Moves to the next row in a ResultSet, or checks whether another row exists.

```
0419 |                         count = rs.getInt(1);
```

Explanation: Reads a column value from the current database result row using the requested Java type.

```
0420 |                     }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0421 |                 }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0422 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0423 |                 if (count &gt;= capacity) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0424 |                     throw new SQLException(
```

Explanation: Raises an exception so the caller or an enclosing catch block can handle the failure.

```
0425 |                             "This section has reached its slot limit.");
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0426 |                 }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0427 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0428 |                 try (PreparedStatement ps =
```

Explanation: Begins a protected block for code that may throw an exception; resources may be managed by try-with-resources.

```
0429 |                              conn.prepareStatement(updateSql)) {
```

Explanation: Creates a parameterized SQL statement; placeholders can be filled separately to reduce SQL-injection risk.

```
0430 |                     ps.setInt(1, sectionId);
```

Explanation: Binds a Java value to a placeholder in the prepared SQL statement; the parameter index is one-based.

```
0431 |                     ps.setInt(2, enrollmentId);
```

Explanation: Binds a Java value to a placeholder in the prepared SQL statement; the parameter index is one-based.

```
0432 |                     ps.setInt(3, strandId);
```

Explanation: Binds a Java value to a placeholder in the prepared SQL statement; the parameter index is one-based.

```
0433 |                     ps.setInt(4, schoolYearId);
```

Explanation: Binds a Java value to a placeholder in the prepared SQL statement; the parameter index is one-based.

```
0434 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0435 |                     if (ps.executeUpdate() == 0) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0436 |                         throw new SQLException(
```

Explanation: Raises an exception so the caller or an enclosing catch block can handle the failure.

```
0437 |                                 "Section assignment failed.");
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0438 |                     }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0439 |                 }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0440 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0441 |                 conn.commit();
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0442 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0443 |             } catch (SQLException ex) {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0444 |                 conn.rollback();
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0445 |                 throw ex;
```

Explanation: Raises an exception so the caller or an enclosing catch block can handle the failure.

```
0446 |             } finally {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0447 |                 conn.setAutoCommit(true);
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0448 |             }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0449 |         }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0450 |     }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0451 | }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

EnrollmentFormFrame.java

Project path: src/EnrollmentFormFrame.java

```
0001 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0002 | import javax.swing.*;
```

Explanation: Imports Swing GUI classes used to build the application's windows and controls.

```
0003 | import java.awt.*;
```

Explanation: Imports AWT classes used for layout, dimensions, colors, fonts, or UI events.

```
0004 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0005 | public class EnrollmentFormFrame extends JFrame {
```

Explanation: Declares the EnrollmentFormFrame type, which groups related data and behavior.

```
0006 |     private StudentDAO studentDAO = new StudentDAO();
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0007 |     private EnrollmentDAO enrollmentDAO = new EnrollmentDAO();
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0008 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0009 |     public EnrollmentFormFrame(User user) {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0010 |         setTitle("Enrollment Form");
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0011 |         setSize(550, 450);
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0012 |         setDefaultCloseOperation(JFrame.DISPOSE_ON_CLOSE);
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0013 |         setLocationRelativeTo(null);
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0014 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0015 |         JTextArea form = new JTextArea();
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0016 |         form.setEditable(false);
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0017 |         form.setFont(new Font("Monospaced", Font.PLAIN, 14));
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0018 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0019 |         int studentId = studentDAO.getStudentId(user.getUserId());
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0020 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0021 |         StringBuilder text = new StringBuilder();
```

Explanation: Creates a mutable string builder so text can be assembled efficiently.

```
0022 |         text.append("SENIOR HIGH SCHOOL ENROLLMENT FORM\n");
```

Explanation: Appends the provided value to the current StringBuilder or collection.

```
0023 |         text.append("==================================\n");
```

Explanation: Appends the provided value to the current StringBuilder or collection.

```
0024 |         text.append("Student: ")
```

Explanation: Appends the provided value to the current StringBuilder or collection.

```
0025 |                 .append(studentDAO.getStudentName(user.getUserId()))
```

Explanation: Appends the provided value to the current StringBuilder or collection.

```
0026 |                 .append("\n\n");
```

Explanation: Appends the provided value to the current StringBuilder or collection.

```
0027 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0028 |         if (studentId != -1) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0029 |             text.append(enrollmentDAO.getMyEnrollments(studentId));
```

Explanation: Appends the provided value to the current StringBuilder or collection.

```
0030 |         } else {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0031 |             text.append("Student profile not found.");
```

Explanation: Appends the provided value to the current StringBuilder or collection.

```
0032 |         }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0033 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0034 |         form.setText(text.toString());
```

Explanation: Sets the displayed text of a GUI component.

```
0035 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0036 |         JButton printButton = new JButton("Print Enrollment Form");
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0037 |         printButton.addActionListener(e -&gt; {
```

Explanation: Registers an event handler that runs when the associated GUI control is activated.

```
0038 |             try {
```

Explanation: Begins a protected block for code that may throw an exception; resources may be managed by try-with-resources.

```
0039 |                 form.print();
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0040 |             } catch (Exception ex) {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0041 |                 JOptionPane.showMessageDialog(this,
```

Explanation: Displays a Swing dialog to communicate a message, warning, confirmation, or input prompt to the user.

```
0042 |                         "Printing error: " + ex.getMessage());
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0043 |             }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0044 |         });
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0045 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0046 |         add(new JScrollPane(form), BorderLayout.CENTER);
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0047 |         add(printButton, BorderLayout.SOUTH);
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0048 |     }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0049 | }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

EnrollmentService.java

Project path: src/EnrollmentService.java

```
0001 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0002 | public class EnrollmentService {
```

Explanation: Declares the EnrollmentService type, which groups related data and behavior.

```
0003 |     private EnrollmentDAO enrollmentDAO = new EnrollmentDAO();
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0004 |     private StudentDAO studentDAO = new StudentDAO();
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0005 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0006 |     public boolean applyForEnrollment(int userId, int schoolYearId,
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0007 |                                       int strandId, int gradeLevel,
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0008 |                                       String enrolleeType) {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0009 |         if (gradeLevel != 11 &amp;amp;&amp;amp; gradeLevel != 12) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0010 |             return false;
```

Explanation: Returns the value or result to the caller and ends the current method.

```
0011 |         }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0012 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0013 |         int studentId = studentDAO.getStudentId(userId);
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0014 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0015 |         if (studentId == -1) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0016 |             return false;
```

Explanation: Returns the value or result to the caller and ends the current method.

```
0017 |         }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0018 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0019 |         return enrollmentDAO.apply(studentId, schoolYearId, strandId,
```

Explanation: Returns the value or result to the caller and ends the current method.

```
0020 |                 gradeLevel, enrolleeType);
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0021 |     }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0022 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0023 |     public boolean updateEnrollmentStatus(int enrollmentId, String status) {
```

Explanation: Declares the updateEnrollmentStatus method and its return type/parameters; the method performs the operation described by its name and body.

```
0024 |         return enrollmentDAO.updateStatus(enrollmentId, status);
```

Explanation: Returns the value or result to the caller and ends the current method.

```
0025 |     }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0026 | }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

EnrollmentStaff.java

Project path: src/EnrollmentStaff.java

```
0001 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0002 | public class EnrollmentStaff extends User {
```

Explanation: Declares the EnrollmentStaff type, which groups related data and behavior.

```
0003 |     public EnrollmentStaff(int userId, String username, String email, String role) {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0004 |         super(userId, username, email, "Staff");
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0005 |     }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0006 | }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

FeeDAO.java

Project path: src/FeeDAO.java

```
0001 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0002 | import java.sql.*;
```

Explanation: Imports JDBC classes used to communicate with a relational database.

```
0003 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0004 | public class FeeDAO {
```

Explanation: Declares the FeeDAO type, which groups related data and behavior.

```
0005 |     private DbConnection db = new DbConnection();
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0006 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0007 |     public boolean addFee(String feeName, double amount,
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0008 |                           int schoolYearId, int strandId) {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0009 |         Connection conn = null;
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0010 |         PreparedStatement ps = null;
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0011 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0012 |         try {
```

Explanation: Begins a protected block for code that may throw an exception; resources may be managed by try-with-resources.

```
0013 |             conn = db.getConnection();
```

Explanation: Assigns or updates a variable with the value or result on the right-hand side.

```
0014 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0015 |             if (conn == null) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0016 |                 return false;
```

Explanation: Returns the value or result to the caller and ends the current method.

```
0017 |             }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0018 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0019 |             String sql = "INSERT INTO fees "
```

Explanation: Stores the SQL command as a string so the program can execute it against the database.

```
0020 |                     + "(fee_name, amount, school_year_id, strand_id) "
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0021 |                     + "VALUES (?, ?, ?, ?)";
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0022 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0023 |             ps = conn.prepareStatement(sql);
```

Explanation: Creates a parameterized SQL statement; placeholders can be filled separately to reduce SQL-injection risk.

```
0024 |             ps.setString(1, feeName);
```

Explanation: Binds a Java value to a placeholder in the prepared SQL statement; the parameter index is one-based.

```
0025 |             ps.setDouble(2, amount);
```

Explanation: Binds a Java value to a placeholder in the prepared SQL statement; the parameter index is one-based.

```
0026 |             ps.setInt(3, schoolYearId);
```

Explanation: Binds a Java value to a placeholder in the prepared SQL statement; the parameter index is one-based.

```
0027 |             ps.setInt(4, strandId);
```

Explanation: Binds a Java value to a placeholder in the prepared SQL statement; the parameter index is one-based.

```
0028 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0029 |             int rowInserted = ps.executeUpdate();
```

Explanation: Executes an INSERT, UPDATE, or DELETE statement and returns the number of affected rows.

```
0030 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0031 |             return rowInserted &gt; 0;
```

Explanation: Returns the value or result to the caller and ends the current method.

```
0032 |         } catch (Exception e) {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0033 |             System.out.println("Fee error: " + e.getMessage());
```

Explanation: Prints a message or value to the console, commonly for status output or error reporting.

```
0034 |             return false;
```

Explanation: Returns the value or result to the caller and ends the current method.

```
0035 |         } finally {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0036 |             try {
```

Explanation: Begins a protected block for code that may throw an exception; resources may be managed by try-with-resources.

```
0037 |                 if (ps != null) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0038 |                     ps.close();
```

Explanation: Closes the associated resource, such as a JDBC ResultSet, Statement, or Connection.

```
0039 |                 }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0040 |             } catch (Exception e) {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0041 |                 System.out.println(e);
```

Explanation: Prints a message or value to the console, commonly for status output or error reporting.

```
0042 |             }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0043 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0044 |             try {
```

Explanation: Begins a protected block for code that may throw an exception; resources may be managed by try-with-resources.

```
0045 |                 if (conn != null) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0046 |                     conn.close();
```

Explanation: Closes the associated resource, such as a JDBC ResultSet, Statement, or Connection.

```
0047 |                 }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0048 |             } catch (Exception e) {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0049 |                 System.out.println(e);
```

Explanation: Prints a message or value to the console, commonly for status output or error reporting.

```
0050 |             }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0051 |         }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0052 |     }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0053 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0054 |     public String getFees() {
```

Explanation: Declares the getFees method, typically returning the requested field or resource.

```
0055 |         Connection conn = null;
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0056 |         PreparedStatement ps = null;
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0057 |         ResultSet rs = null;
```

Explanation: Declares or assigns a ResultSet that holds rows returned by a database query.

```
0058 |         StringBuilder result = new StringBuilder();
```

Explanation: Creates a mutable string builder so text can be assembled efficiently.

```
0059 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0060 |         try {
```

Explanation: Begins a protected block for code that may throw an exception; resources may be managed by try-with-resources.

```
0061 |             conn = db.getConnection();
```

Explanation: Assigns or updates a variable with the value or result on the right-hand side.

```
0062 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0063 |             if (conn == null) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0064 |                 return "Error: Could not connect to the database.";
```

Explanation: Returns the value or result to the caller and ends the current method.

```
0065 |             }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0066 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0067 |             String sql = "SELECT fee_id, fee_name, amount FROM fees";
```

Explanation: Stores the SQL command as a string so the program can execute it against the database.

```
0068 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0069 |             ps = conn.prepareStatement(sql);
```

Explanation: Creates a parameterized SQL statement; placeholders can be filled separately to reduce SQL-injection risk.

```
0070 |             rs = ps.executeQuery();
```

Explanation: Executes a SQL query that returns rows, usually a SELECT, and stores them in a ResultSet.

```
0071 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0072 |             while (rs.next()) {
```

Explanation: Repeats the following block while the condition remains true.

```
0073 |                 result.append("Fee ID: ").append(rs.getInt("fee_id"))
```

Explanation: Reads a column value from the current database result row using the requested Java type.

```
0074 |                         .append(" | ")
```

Explanation: Appends the provided value to the current StringBuilder or collection.

```
0075 |                         .append(rs.getString("fee_name"))
```

Explanation: Reads a column value from the current database result row using the requested Java type.

```
0076 |                         .append(" | PHP ")
```

Explanation: Appends the provided value to the current StringBuilder or collection.

```
0077 |                         .append(rs.getBigDecimal("amount"))
```

Explanation: Reads a column value from the current database result row using the requested Java type.

```
0078 |                         .append("\n");
```

Explanation: Appends the provided value to the current StringBuilder or collection.

```
0079 |             }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0080 |         } catch (Exception e) {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0081 |             return "Error: " + e.getMessage();
```

Explanation: Returns the value or result to the caller and ends the current method.

```
0082 |         } finally {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0083 |             try {
```

Explanation: Begins a protected block for code that may throw an exception; resources may be managed by try-with-resources.

```
0084 |                 if (rs != null) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0085 |                     rs.close();
```

Explanation: Closes the associated resource, such as a JDBC ResultSet, Statement, or Connection.

```
0086 |                 }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0087 |             } catch (Exception e) {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0088 |                 System.out.println(e);
```

Explanation: Prints a message or value to the console, commonly for status output or error reporting.

```
0089 |             }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0090 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0091 |             try {
```

Explanation: Begins a protected block for code that may throw an exception; resources may be managed by try-with-resources.

```
0092 |                 if (ps != null) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0093 |                     ps.close();
```

Explanation: Closes the associated resource, such as a JDBC ResultSet, Statement, or Connection.

```
0094 |                 }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0095 |             } catch (Exception e) {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0096 |                 System.out.println(e);
```

Explanation: Prints a message or value to the console, commonly for status output or error reporting.

```
0097 |             }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0098 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0099 |             try {
```

Explanation: Begins a protected block for code that may throw an exception; resources may be managed by try-with-resources.

```
0100 |                 if (conn != null) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0101 |                     conn.close();
```

Explanation: Closes the associated resource, such as a JDBC ResultSet, Statement, or Connection.

```
0102 |                 }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0103 |             } catch (Exception e) {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0104 |                 System.out.println(e);
```

Explanation: Prints a message or value to the console, commonly for status output or error reporting.

```
0105 |             }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0106 |         }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0107 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0108 |         return result.length() == 0 ? "No fees found." : result.toString();
```

Explanation: Returns the value or result to the caller and ends the current method.

```
0109 |     }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0110 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0111 |     public double getTotalFees(int schoolYearId, int strandId) {
```

Explanation: Declares the getTotalFees method, typically returning the requested field or resource.

```
0112 |         Connection conn = null;
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0113 |         PreparedStatement ps = null;
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0114 |         ResultSet rs = null;
```

Explanation: Declares or assigns a ResultSet that holds rows returned by a database query.

```
0115 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0116 |         try {
```

Explanation: Begins a protected block for code that may throw an exception; resources may be managed by try-with-resources.

```
0117 |             conn = db.getConnection();
```

Explanation: Assigns or updates a variable with the value or result on the right-hand side.

```
0118 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0119 |             if (conn == null) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0120 |                 return 0;
```

Explanation: Returns the value or result to the caller and ends the current method.

```
0121 |             }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0122 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0123 |             String sql = "SELECT COALESCE(SUM(amount), 0) AS total FROM fees "
```

Explanation: Stores the SQL command as a string so the program can execute it against the database.

```
0124 |                     + "WHERE school_year_id = ? AND strand_id = ?";
```

Explanation: Assigns or updates a variable with the value or result on the right-hand side.

```
0125 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0126 |             ps = conn.prepareStatement(sql);
```

Explanation: Creates a parameterized SQL statement; placeholders can be filled separately to reduce SQL-injection risk.

```
0127 |             ps.setInt(1, schoolYearId);
```

Explanation: Binds a Java value to a placeholder in the prepared SQL statement; the parameter index is one-based.

```
0128 |             ps.setInt(2, strandId);
```

Explanation: Binds a Java value to a placeholder in the prepared SQL statement; the parameter index is one-based.

```
0129 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0130 |             rs = ps.executeQuery();
```

Explanation: Executes a SQL query that returns rows, usually a SELECT, and stores them in a ResultSet.

```
0131 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0132 |             if (rs.next()) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0133 |                 return rs.getDouble("total");
```

Explanation: Returns the value or result to the caller and ends the current method.

```
0134 |             }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0135 |         } catch (Exception e) {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0136 |             System.out.println("Fee total error: " + e.getMessage());
```

Explanation: Prints a message or value to the console, commonly for status output or error reporting.

```
0137 |         } finally {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0138 |             try {
```

Explanation: Begins a protected block for code that may throw an exception; resources may be managed by try-with-resources.

```
0139 |                 if (rs != null) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0140 |                     rs.close();
```

Explanation: Closes the associated resource, such as a JDBC ResultSet, Statement, or Connection.

```
0141 |                 }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0142 |             } catch (Exception e) {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0143 |                 System.out.println(e);
```

Explanation: Prints a message or value to the console, commonly for status output or error reporting.

```
0144 |             }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0145 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0146 |             try {
```

Explanation: Begins a protected block for code that may throw an exception; resources may be managed by try-with-resources.

```
0147 |                 if (ps != null) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0148 |                     ps.close();
```

Explanation: Closes the associated resource, such as a JDBC ResultSet, Statement, or Connection.

```
0149 |                 }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0150 |             } catch (Exception e) {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0151 |                 System.out.println(e);
```

Explanation: Prints a message or value to the console, commonly for status output or error reporting.

```
0152 |             }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0153 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0154 |             try {
```

Explanation: Begins a protected block for code that may throw an exception; resources may be managed by try-with-resources.

```
0155 |                 if (conn != null) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0156 |                     conn.close();
```

Explanation: Closes the associated resource, such as a JDBC ResultSet, Statement, or Connection.

```
0157 |                 }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0158 |             } catch (Exception e) {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0159 |                 System.out.println(e);
```

Explanation: Prints a message or value to the console, commonly for status output or error reporting.

```
0160 |             }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0161 |         }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0162 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0163 |         return 0;
```

Explanation: Returns the value or result to the caller and ends the current method.

```
0164 |     }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0165 | }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

LoginFrame.java

Project path: src/LoginFrame.java

```
0001 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0002 | import javax.swing.*;
```

Explanation: Imports Swing GUI classes used to build the application's windows and controls.

```
0003 | import java.awt.*;
```

Explanation: Imports AWT classes used for layout, dimensions, colors, fonts, or UI events.

```
0004 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0005 | public class LoginFrame extends JFrame {
```

Explanation: Declares the LoginFrame type, which groups related data and behavior.

```
0006 |     private JTextField usernameField;
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0007 |     private JPasswordField passwordField;
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0008 |     private AuthService authService = new AuthService();
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0009 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0010 |     public LoginFrame() {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0011 |         setTitle("Student Enrollment System - Login");
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0012 |         setSize(420, 300);
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0013 |         setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0014 |         setLocationRelativeTo(null);
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0015 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0016 |         JPanel panel = new JPanel(new GridLayout(5, 1, 8, 8));
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0017 |         panel.setBorder(BorderFactory.createEmptyBorder(20, 30, 20, 30));
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0018 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0019 |         panel.add(new JLabel("STUDENT ENROLLMENT SYSTEM", JLabel.CENTER));
```

Explanation: Adds a component to a Swing container so it becomes part of the displayed interface.

```
0020 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0021 |         usernameField = new JTextField();
```

Explanation: Assigns or updates a variable with the value or result on the right-hand side.

```
0022 |         passwordField = new JPasswordField();
```

Explanation: Assigns or updates a variable with the value or result on the right-hand side.

```
0023 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0024 |         panel.add(createField("Username:", usernameField));
```

Explanation: Adds a component to a Swing container so it becomes part of the displayed interface.

```
0025 |         panel.add(createField("Password:", passwordField));
```

Explanation: Adds a component to a Swing container so it becomes part of the displayed interface.

```
0026 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0027 |         JButton loginButton = new JButton("Login");
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0028 |         JButton registerButton = new JButton("Register Student Account");
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0029 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0030 |         panel.add(loginButton);
```

Explanation: Adds a component to a Swing container so it becomes part of the displayed interface.

```
0031 |         panel.add(registerButton);
```

Explanation: Adds a component to a Swing container so it becomes part of the displayed interface.

```
0032 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0033 |         add(panel);
```

Explanation: Adds a component to a Swing container so it becomes part of the displayed interface.

```
0034 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0035 |         loginButton.addActionListener(e -&gt; login());
```

Explanation: Registers an event handler that runs when the associated GUI control is activated.

```
0036 |         registerButton.addActionListener(e -&gt;
```

Explanation: Registers an event handler that runs when the associated GUI control is activated.

```
0037 |                 new RegistrationFrame().setVisible(true));
```

Explanation: Creates an instance of the specified class or object.

```
0038 |     }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0039 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0040 |     private JPanel createField(String label, JComponent field) {
```

Explanation: Declares the createField method and its return type/parameters; the method performs the operation described by its name and body.

```
0041 |         JPanel panel = new JPanel(new BorderLayout(8, 0));
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0042 |         panel.add(new JLabel(label), BorderLayout.WEST);
```

Explanation: Adds a component to a Swing container so it becomes part of the displayed interface.

```
0043 |         panel.add(field, BorderLayout.CENTER);
```

Explanation: Adds a component to a Swing container so it becomes part of the displayed interface.

```
0044 |         return panel;
```

Explanation: Returns the value or result to the caller and ends the current method.

```
0045 |     }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0046 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0047 |     private void login() {
```

Explanation: Declares the login method, which implements this part of the application's authentication or registration workflow.

```
0048 |         String username = usernameField.getText().trim();
```

Explanation: Reads text currently entered or displayed by a GUI component.

```
0049 |         String password = new String(passwordField.getPassword());
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0050 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0051 |         if (username.isEmpty() || password.isEmpty()) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0052 |             JOptionPane.showMessageDialog(this,
```

Explanation: Displays a Swing dialog to communicate a message, warning, confirmation, or input prompt to the user.

```
0053 |                     "Enter your username and password.");
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0054 |             return;
```

Explanation: Ends the current method without returning a value.

```
0055 |         }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0056 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0057 |         User user = authService.login(username, password);
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0058 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0059 |         if (user == null) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0060 |             JOptionPane.showMessageDialog(this,
```

Explanation: Displays a Swing dialog to communicate a message, warning, confirmation, or input prompt to the user.

```
0061 |                     "Invalid username or password.");
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0062 |             return;
```

Explanation: Ends the current method without returning a value.

```
0063 |         }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0064 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0065 |         String role = user.getRole().toLowerCase();
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0066 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0067 |         if (role.equals("admin")) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0068 |             new AdminDashboard(user).setVisible(true);
```

Explanation: Creates an instance of the specified class or object.

```
0069 |         } else if (role.equals("staff")) {
```

Explanation: Compares values for equality using the class's equals method.

```
0070 |             new StaffDashboard(user).setVisible(true);
```

Explanation: Creates an instance of the specified class or object.

```
0071 |         } else if (role.equals("cashier")) {
```

Explanation: Compares values for equality using the class's equals method.

```
0072 |             new CashierDashboard(user).setVisible(true);
```

Explanation: Creates an instance of the specified class or object.

```
0073 |         } else if (role.equals("student")) {
```

Explanation: Compares values for equality using the class's equals method.

```
0074 |             new StudentDashboard(user).setVisible(true);
```

Explanation: Creates an instance of the specified class or object.

```
0075 |         } else {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0076 |             JOptionPane.showMessageDialog(this, "Unknown account role.");
```

Explanation: Displays a Swing dialog to communicate a message, warning, confirmation, or input prompt to the user.

```
0077 |             return;
```

Explanation: Ends the current method without returning a value.

```
0078 |         }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0079 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0080 |         dispose();
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0081 |     }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0082 | }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

PaymentDAO.java

Project path: src/PaymentDAO.java

```
0001 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0002 | import java.sql.*;
```

Explanation: Imports JDBC classes used to communicate with a relational database.

```
0003 | import java.util.ArrayList;
```

Explanation: Imports Java utility classes such as collections, lists, maps, or helper types.

```
0004 | import java.util.List;
```

Explanation: Imports Java utility classes such as collections, lists, maps, or helper types.

```
0005 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0006 | public class PaymentDAO {
```

Explanation: Declares the PaymentDAO type, which groups related data and behavior.

```
0007 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0008 |     public String getDiscounts() throws SQLException {
```

Explanation: Declares the getDiscounts method, typically returning the requested field or resource.

```
0009 |         StringBuilder result = new StringBuilder();
```

Explanation: Creates a mutable string builder so text can be assembled efficiently.

```
0010 |         Connection conn = null;
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0011 |         PreparedStatement ps = null;
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0012 |         ResultSet rs = null;
```

Explanation: Declares or assigns a ResultSet that holds rows returned by a database query.

```
0013 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0014 |         try {
```

Explanation: Begins a protected block for code that may throw an exception; resources may be managed by try-with-resources.

```
0015 |             conn = new DbConnection().getConnection();
```

Explanation: Assigns or updates a variable with the value or result on the right-hand side.

```
0016 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0017 |             if (conn == null) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0018 |                 throw new SQLException("Could not connect to the database.");
```

Explanation: Raises an exception so the caller or an enclosing catch block can handle the failure.

```
0019 |             }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0020 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0021 |             String sql = "SELECT discount_id, discount_name, discount_type, discount_value " +
```

Explanation: Stores the SQL command as a string so the program can execute it against the database.

```
0022 |                     "FROM discounts ORDER BY discount_id";
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0023 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0024 |             ps = conn.prepareStatement(sql);
```

Explanation: Creates a parameterized SQL statement; placeholders can be filled separately to reduce SQL-injection risk.

```
0025 |             rs = ps.executeQuery();
```

Explanation: Executes a SQL query that returns rows, usually a SELECT, and stores them in a ResultSet.

```
0026 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0027 |             boolean found = false;
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0028 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0029 |             while (rs.next()) {
```

Explanation: Repeats the following block while the condition remains true.

```
0030 |                 found = true;
```

Explanation: Assigns or updates a variable with the value or result on the right-hand side.

```
0031 |                 result.append("Discount ID: ").append(rs.getInt("discount_id"))
```

Explanation: Reads a column value from the current database result row using the requested Java type.

```
0032 |                         .append("\nName: ").append(rs.getString("discount_name"))
```

Explanation: Reads a column value from the current database result row using the requested Java type.

```
0033 |                         .append("\nType: ").append(rs.getString("discount_type"))
```

Explanation: Reads a column value from the current database result row using the requested Java type.

```
0034 |                         .append("\nValue: ").append(rs.getBigDecimal("discount_value"))
```

Explanation: Reads a column value from the current database result row using the requested Java type.

```
0035 |                         .append("\n-----------------------------\n");
```

Explanation: Appends the provided value to the current StringBuilder or collection.

```
0036 |             }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0037 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0038 |             if (!found) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0039 |                 result.append("No discounts are available.");
```

Explanation: Appends the provided value to the current StringBuilder or collection.

```
0040 |             }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0041 |         } catch (SQLException e) {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0042 |             throw e;
```

Explanation: Raises an exception so the caller or an enclosing catch block can handle the failure.

```
0043 |         } catch (Exception e) {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0044 |             throw new SQLException(e.getMessage(), e);
```

Explanation: Raises an exception so the caller or an enclosing catch block can handle the failure.

```
0045 |         } finally {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0046 |             try {
```

Explanation: Begins a protected block for code that may throw an exception; resources may be managed by try-with-resources.

```
0047 |                 if (rs != null) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0048 |                     rs.close();
```

Explanation: Closes the associated resource, such as a JDBC ResultSet, Statement, or Connection.

```
0049 |                 }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0050 |             } catch (Exception e) {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0051 |                 System.out.println(e);
```

Explanation: Prints a message or value to the console, commonly for status output or error reporting.

```
0052 |             }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0053 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0054 |             try {
```

Explanation: Begins a protected block for code that may throw an exception; resources may be managed by try-with-resources.

```
0055 |                 if (ps != null) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0056 |                     ps.close();
```

Explanation: Closes the associated resource, such as a JDBC ResultSet, Statement, or Connection.

```
0057 |                 }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0058 |             } catch (Exception e) {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0059 |                 System.out.println(e);
```

Explanation: Prints a message or value to the console, commonly for status output or error reporting.

```
0060 |             }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0061 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0062 |             try {
```

Explanation: Begins a protected block for code that may throw an exception; resources may be managed by try-with-resources.

```
0063 |                 if (conn != null) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0064 |                     conn.close();
```

Explanation: Closes the associated resource, such as a JDBC ResultSet, Statement, or Connection.

```
0065 |                 }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0066 |             } catch (Exception e) {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0067 |                 System.out.println(e);
```

Explanation: Prints a message or value to the console, commonly for status output or error reporting.

```
0068 |             }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0069 |         }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0070 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0071 |         return result.toString();
```

Explanation: Returns the value or result to the caller and ends the current method.

```
0072 |     }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0073 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0074 |     public String createPaymentRequest(int enrollmentId, List&lt;Integer&gt; discountIds)
```

Explanation: Declares the createPaymentRequest method and its return type/parameters; the method performs the operation described by its name and body.

```
0075 |             throws SQLException {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0076 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0077 |         if (discountIds == null) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0078 |             discountIds = new ArrayList&lt;&gt;();
```

Explanation: Assigns or updates a variable with the value or result on the right-hand side.

```
0079 |         }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0080 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0081 |         try (Connection conn = new DbConnection().getConnection()) {
```

Explanation: Begins a protected block for code that may throw an exception; resources may be managed by try-with-resources.

```
0082 |             if (conn == null) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0083 |                 throw new SQLException("Could not connect to the database.");
```

Explanation: Raises an exception so the caller or an enclosing catch block can handle the failure.

```
0084 |             }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0085 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0086 |             conn.setAutoCommit(false);
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0087 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0088 |             try {
```

Explanation: Begins a protected block for code that may throw an exception; resources may be managed by try-with-resources.

```
0089 |                 int schoolYearId;
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0090 |                 int strandId;
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0091 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0092 |                 String enrollmentSql =
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0093 |                         "SELECT school_year_id, strand_id FROM enrollments " +
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0094 |                         "WHERE enrollment_id = ? FOR UPDATE";
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0095 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0096 |                 try (PreparedStatement ps = conn.prepareStatement(enrollmentSql)) {
```

Explanation: Begins a protected block for code that may throw an exception; resources may be managed by try-with-resources.

```
0097 |                     ps.setInt(1, enrollmentId);
```

Explanation: Binds a Java value to a placeholder in the prepared SQL statement; the parameter index is one-based.

```
0098 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0099 |                     try (ResultSet rs = ps.executeQuery()) {
```

Explanation: Begins a protected block for code that may throw an exception; resources may be managed by try-with-resources.

```
0100 |                         if (!rs.next()) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0101 |                             throw new SQLException("Enrollment ID was not found.");
```

Explanation: Raises an exception so the caller or an enclosing catch block can handle the failure.

```
0102 |                         }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0103 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0104 |                         schoolYearId = rs.getInt("school_year_id");
```

Explanation: Reads a column value from the current database result row using the requested Java type.

```
0105 |                         strandId = rs.getInt("strand_id");
```

Explanation: Reads a column value from the current database result row using the requested Java type.

```
0106 |                     }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0107 |                 }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0108 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0109 |                 List&lt;FeeItem&gt; items = new ArrayList&lt;&gt;();
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0110 |                 double total = 0.0;
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0111 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0112 |                 String feeSql =
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0113 |                         "SELECT fee_name, amount FROM fees " +
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0114 |                         "WHERE school_year_id = ? AND strand_id = ?";
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0115 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0116 |                 try (PreparedStatement ps = conn.prepareStatement(feeSql)) {
```

Explanation: Begins a protected block for code that may throw an exception; resources may be managed by try-with-resources.

```
0117 |                     ps.setInt(1, schoolYearId);
```

Explanation: Binds a Java value to a placeholder in the prepared SQL statement; the parameter index is one-based.

```
0118 |                     ps.setInt(2, strandId);
```

Explanation: Binds a Java value to a placeholder in the prepared SQL statement; the parameter index is one-based.

```
0119 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0120 |                     try (ResultSet rs = ps.executeQuery()) {
```

Explanation: Begins a protected block for code that may throw an exception; resources may be managed by try-with-resources.

```
0121 |                         while (rs.next()) {
```

Explanation: Repeats the following block while the condition remains true.

```
0122 |                             String name = rs.getString("fee_name");
```

Explanation: Reads a column value from the current database result row using the requested Java type.

```
0123 |                             double amount = rs.getDouble("amount");
```

Explanation: Reads a column value from the current database result row using the requested Java type.

```
0124 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0125 |                             items.add(new FeeItem(name, "Fee", amount));
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0126 |                             total += amount;
```

Explanation: Assigns or updates a variable with the value or result on the right-hand side.

```
0127 |                         }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0128 |                     }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0129 |                 }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0130 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0131 |                 if (items.isEmpty()) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0132 |                     throw new SQLException(
```

Explanation: Raises an exception so the caller or an enclosing catch block can handle the failure.

```
0133 |                             "No fees are configured for this enrollment's school year and strand.");
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0134 |                 }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0135 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0136 |                 for (Integer discountId : discountIds) {
```

Explanation: Starts a loop that repeats while its loop condition allows; initialization and update control iteration.

```
0137 |                     if (discountId == null) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0138 |                         throw new SQLException("A discount ID cannot be empty.");
```

Explanation: Raises an exception so the caller or an enclosing catch block can handle the failure.

```
0139 |                     }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0140 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0141 |                     String discountSql =
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0142 |                             "SELECT discount_name, discount_type, discount_value " +
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0143 |                             "FROM discounts WHERE discount_id = ?";
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0144 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0145 |                     try (PreparedStatement ps = conn.prepareStatement(discountSql)) {
```

Explanation: Begins a protected block for code that may throw an exception; resources may be managed by try-with-resources.

```
0146 |                         ps.setInt(1, discountId);
```

Explanation: Binds a Java value to a placeholder in the prepared SQL statement; the parameter index is one-based.

```
0147 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0148 |                         try (ResultSet rs = ps.executeQuery()) {
```

Explanation: Begins a protected block for code that may throw an exception; resources may be managed by try-with-resources.

```
0149 |                             if (!rs.next()) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0150 |                                 throw new SQLException(
```

Explanation: Raises an exception so the caller or an enclosing catch block can handle the failure.

```
0151 |                                         "Discount ID " + discountId + " was not found.");
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0152 |                             }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0153 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0154 |                             String name = rs.getString("discount_name");
```

Explanation: Reads a column value from the current database result row using the requested Java type.

```
0155 |                             String type = rs.getString("discount_type");
```

Explanation: Reads a column value from the current database result row using the requested Java type.

```
0156 |                             double value = rs.getDouble("discount_value");
```

Explanation: Reads a column value from the current database result row using the requested Java type.

```
0157 |                             double discountAmount;
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0158 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0159 |                             if ("percentage".equalsIgnoreCase(type)
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0160 |                                     || "percent".equalsIgnoreCase(type)) {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0161 |                                 if (value &lt; 0 || value &gt; 100) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0162 |                                     throw new SQLException(
```

Explanation: Raises an exception so the caller or an enclosing catch block can handle the failure.

```
0163 |                                             "Percentage discounts must be between 0 and 100.");
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0164 |                                 }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0165 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0166 |                                 discountAmount = total * value / 100.0;
```

Explanation: Assigns or updates a variable with the value or result on the right-hand side.

```
0167 |                             } else if ("fixed".equalsIgnoreCase(type)
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0168 |                                     || "amount".equalsIgnoreCase(type)) {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0169 |                                 if (value &lt; 0) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0170 |                                     throw new SQLException("Discount value cannot be negative.");
```

Explanation: Raises an exception so the caller or an enclosing catch block can handle the failure.

```
0171 |                                 }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0172 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0173 |                                 discountAmount = value;
```

Explanation: Assigns or updates a variable with the value or result on the right-hand side.

```
0174 |                             } else {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0175 |                                 throw new SQLException(
```

Explanation: Raises an exception so the caller or an enclosing catch block can handle the failure.

```
0176 |                                         "Unknown discount type for " + name +
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0177 |                                         ". Use Percentage or Fixed.");
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0178 |                             }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0179 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0180 |                             discountAmount = Math.min(discountAmount, total);
```

Explanation: Assigns or updates a variable with the value or result on the right-hand side.

```
0181 |                             discountAmount = Math.round(discountAmount * 100.0) / 100.0;
```

Explanation: Assigns or updates a variable with the value or result on the right-hand side.

```
0182 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0183 |                             items.add(new FeeItem(name, "Discount", -discountAmount));
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0184 |                             total = Math.max(0.0, total - discountAmount);
```

Explanation: Assigns or updates a variable with the value or result on the right-hand side.

```
0185 |                         }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0186 |                     }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0187 |                 }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0188 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0189 |                 total = Math.round(total * 100.0) / 100.0;
```

Explanation: Assigns or updates a variable with the value or result on the right-hand side.

```
0190 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0191 |                 String requestSql =
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0192 |                         "INSERT INTO payment_requests " +
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0193 |                         "(enrollment_id, amount_due, description, request_status) " +
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0194 |                         "VALUES (?, ?, ?, 'Open')";
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0195 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0196 |                 int requestId;
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0197 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0198 |                 try (PreparedStatement ps = conn.prepareStatement(
```

Explanation: Begins a protected block for code that may throw an exception; resources may be managed by try-with-resources.

```
0199 |                         requestSql, Statement.RETURN_GENERATED_KEYS)) {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0200 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0201 |                     ps.setInt(1, enrollmentId);
```

Explanation: Binds a Java value to a placeholder in the prepared SQL statement; the parameter index is one-based.

```
0202 |                     ps.setDouble(2, total);
```

Explanation: Binds a Java value to a placeholder in the prepared SQL statement; the parameter index is one-based.

```
0203 |                     ps.setString(3, "Enrollment fees and selected discounts");
```

Explanation: Binds a Java value to a placeholder in the prepared SQL statement; the parameter index is one-based.

```
0204 |                     ps.executeUpdate();
```

Explanation: Executes an INSERT, UPDATE, or DELETE statement and returns the number of affected rows.

```
0205 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0206 |                     try (ResultSet keys = ps.getGeneratedKeys()) {
```

Explanation: Begins a protected block for code that may throw an exception; resources may be managed by try-with-resources.

```
0207 |                         if (!keys.next()) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0208 |                             throw new SQLException("Could not create the payment request.");
```

Explanation: Raises an exception so the caller or an enclosing catch block can handle the failure.

```
0209 |                         }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0210 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0211 |                         requestId = keys.getInt(1);
```

Explanation: Reads a column value from the current database result row using the requested Java type.

```
0212 |                     }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0213 |                 }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0214 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0215 |                 String detailSql =
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0216 |                         "INSERT INTO payment_request_details " +
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0217 |                         "(request_id, item_name, item_type, amount) VALUES (?, ?, ?, ?)";
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0218 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0219 |                 try (PreparedStatement ps = conn.prepareStatement(detailSql)) {
```

Explanation: Begins a protected block for code that may throw an exception; resources may be managed by try-with-resources.

```
0220 |                     for (FeeItem item : items) {
```

Explanation: Starts a loop that repeats while its loop condition allows; initialization and update control iteration.

```
0221 |                         ps.setInt(1, requestId);
```

Explanation: Binds a Java value to a placeholder in the prepared SQL statement; the parameter index is one-based.

```
0222 |                         ps.setString(2, item.name);
```

Explanation: Binds a Java value to a placeholder in the prepared SQL statement; the parameter index is one-based.

```
0223 |                         ps.setString(3, item.type);
```

Explanation: Binds a Java value to a placeholder in the prepared SQL statement; the parameter index is one-based.

```
0224 |                         ps.setDouble(4, item.amount);
```

Explanation: Binds a Java value to a placeholder in the prepared SQL statement; the parameter index is one-based.

```
0225 |                         ps.addBatch();
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0226 |                     }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0227 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0228 |                     ps.executeBatch();
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0229 |                 }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0230 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0231 |                 conn.commit();
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0232 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0233 |                 return "Payment request created successfully.\n\n" +
```

Explanation: Returns the value or result to the caller and ends the current method.

```
0234 |                         "Request ID: " + requestId + "\n" +
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0235 |                         "Enrollment ID: " + enrollmentId + "\n" +
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0236 |                         "Total Amount Due: PHP " + String.format("%.2f", total) + "\n" +
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0237 |                         "Status: Open";
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0238 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0239 |             } catch (Exception ex) {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0240 |                 conn.rollback();
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0241 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0242 |                 if (ex instanceof SQLException) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0243 |                     throw (SQLException) ex;
```

Explanation: Raises an exception so the caller or an enclosing catch block can handle the failure.

```
0244 |                 }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0245 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0246 |                 throw new SQLException("Could not create payment request.", ex);
```

Explanation: Raises an exception so the caller or an enclosing catch block can handle the failure.

```
0247 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0248 |             } finally {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0249 |                 conn.setAutoCommit(true);
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0250 |             }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0251 |         }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0252 |     }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0253 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0254 |     public String getPaymentsForStudent(int userId) throws SQLException {
```

Explanation: Declares the getPaymentsForStudent method, typically returning the requested field or resource.

```
0255 |         String sql =
```

Explanation: Stores the SQL command as a string so the program can execute it against the database.

```
0256 |                 "SELECT p.payment_id, p.request_id, p.enrollment_id, " +
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0257 |                 "p.amount_paid, p.payment_method, p.reference_number, " +
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0258 |                 "p.payment_status, p.paid_at " +
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0259 |                 "FROM payments p " +
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0260 |                 "JOIN enrollments e ON p.enrollment_id = e.enrollment_id " +
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0261 |                 "JOIN students s ON e.student_id = s.student_id " +
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0262 |                 "WHERE s.user_id = ? ORDER BY p.payment_id DESC";
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0263 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0264 |         return readPayments(sql, userId);
```

Explanation: Returns the value or result to the caller and ends the current method.

```
0265 |     }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0266 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0267 |     public String getPendingPayments() throws SQLException {
```

Explanation: Declares the getPendingPayments method, typically returning the requested field or resource.

```
0268 |         String sql =
```

Explanation: Stores the SQL command as a string so the program can execute it against the database.

```
0269 |                 "SELECT p.payment_id, p.request_id, p.enrollment_id, " +
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0270 |                 "p.amount_paid, p.payment_method, p.reference_number, " +
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0271 |                 "p.payment_status, p.paid_at FROM payments p " +
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0272 |                 "WHERE p.payment_status = 'Pending' ORDER BY p.payment_id DESC";
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0273 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0274 |         return readPayments(sql, 0);
```

Explanation: Returns the value or result to the caller and ends the current method.

```
0275 |     }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0276 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0277 |     public String getPayments() throws SQLException {
```

Explanation: Declares the getPayments method, typically returning the requested field or resource.

```
0278 |         String sql =
```

Explanation: Stores the SQL command as a string so the program can execute it against the database.

```
0279 |                 "SELECT p.payment_id, p.request_id, p.enrollment_id, " +
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0280 |                 "p.amount_paid, p.payment_method, p.reference_number, " +
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0281 |                 "p.payment_status, p.paid_at FROM payments p " +
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0282 |                 "ORDER BY p.payment_id DESC";
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0283 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0284 |         return readPayments(sql, 0);
```

Explanation: Returns the value or result to the caller and ends the current method.

```
0285 |     }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0286 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0287 |     private String readPayments(String sql, int userId) throws SQLException {
```

Explanation: Declares the readPayments method and its return type/parameters; the method performs the operation described by its name and body.

```
0288 |         StringBuilder result = new StringBuilder();
```

Explanation: Creates a mutable string builder so text can be assembled efficiently.

```
0289 |         boolean found = false;
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0290 |         Connection conn = null;
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0291 |         PreparedStatement ps = null;
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0292 |         ResultSet rs = null;
```

Explanation: Declares or assigns a ResultSet that holds rows returned by a database query.

```
0293 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0294 |         try {
```

Explanation: Begins a protected block for code that may throw an exception; resources may be managed by try-with-resources.

```
0295 |             conn = new DbConnection().getConnection();
```

Explanation: Assigns or updates a variable with the value or result on the right-hand side.

```
0296 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0297 |             if (conn == null) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0298 |                 throw new SQLException("Could not connect to the database.");
```

Explanation: Raises an exception so the caller or an enclosing catch block can handle the failure.

```
0299 |             }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0300 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0301 |             ps = conn.prepareStatement(sql);
```

Explanation: Creates a parameterized SQL statement; placeholders can be filled separately to reduce SQL-injection risk.

```
0302 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0303 |             if (sql.contains("s.user_id = ?")) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0304 |                 ps.setInt(1, userId);
```

Explanation: Binds a Java value to a placeholder in the prepared SQL statement; the parameter index is one-based.

```
0305 |             }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0306 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0307 |             rs = ps.executeQuery();
```

Explanation: Executes a SQL query that returns rows, usually a SELECT, and stores them in a ResultSet.

```
0308 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0309 |             while (rs.next()) {
```

Explanation: Repeats the following block while the condition remains true.

```
0310 |                 found = true;
```

Explanation: Assigns or updates a variable with the value or result on the right-hand side.

```
0311 |                 result.append("Payment ID: ").append(rs.getInt("payment_id"))
```

Explanation: Reads a column value from the current database result row using the requested Java type.

```
0312 |                         .append("\nRequest ID: ").append(rs.getObject("request_id"))
```

Explanation: Appends the provided value to the current StringBuilder or collection.

```
0313 |                         .append("\nEnrollment ID: ").append(rs.getInt("enrollment_id"))
```

Explanation: Reads a column value from the current database result row using the requested Java type.

```
0314 |                         .append("\nAmount: PHP ")
```

Explanation: Appends the provided value to the current StringBuilder or collection.

```
0315 |                         .append(String.format("%.2f", rs.getDouble("amount_paid")))
```

Explanation: Reads a column value from the current database result row using the requested Java type.

```
0316 |                         .append("\nMethod: ").append(rs.getString("payment_method"))
```

Explanation: Reads a column value from the current database result row using the requested Java type.

```
0317 |                         .append("\nReference: ").append(rs.getString("reference_number"))
```

Explanation: Reads a column value from the current database result row using the requested Java type.

```
0318 |                         .append("\nStatus: ").append(rs.getString("payment_status"))
```

Explanation: Reads a column value from the current database result row using the requested Java type.

```
0319 |                         .append("\nDate: ").append(rs.getTimestamp("paid_at"))
```

Explanation: Appends the provided value to the current StringBuilder or collection.

```
0320 |                         .append("\n----------------------------\n");
```

Explanation: Appends the provided value to the current StringBuilder or collection.

```
0321 |             }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0322 |         } catch (SQLException e) {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0323 |             throw e;
```

Explanation: Raises an exception so the caller or an enclosing catch block can handle the failure.

```
0324 |         } catch (Exception e) {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0325 |             throw new SQLException(e.getMessage(), e);
```

Explanation: Raises an exception so the caller or an enclosing catch block can handle the failure.

```
0326 |         } finally {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0327 |             try {
```

Explanation: Begins a protected block for code that may throw an exception; resources may be managed by try-with-resources.

```
0328 |                 if (rs != null) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0329 |                     rs.close();
```

Explanation: Closes the associated resource, such as a JDBC ResultSet, Statement, or Connection.

```
0330 |                 }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0331 |             } catch (Exception e) {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0332 |                 System.out.println(e);
```

Explanation: Prints a message or value to the console, commonly for status output or error reporting.

```
0333 |             }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0334 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0335 |             try {
```

Explanation: Begins a protected block for code that may throw an exception; resources may be managed by try-with-resources.

```
0336 |                 if (ps != null) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0337 |                     ps.close();
```

Explanation: Closes the associated resource, such as a JDBC ResultSet, Statement, or Connection.

```
0338 |                 }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0339 |             } catch (Exception e) {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0340 |                 System.out.println(e);
```

Explanation: Prints a message or value to the console, commonly for status output or error reporting.

```
0341 |             }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0342 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0343 |             try {
```

Explanation: Begins a protected block for code that may throw an exception; resources may be managed by try-with-resources.

```
0344 |                 if (conn != null) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0345 |                     conn.close();
```

Explanation: Closes the associated resource, such as a JDBC ResultSet, Statement, or Connection.

```
0346 |                 }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0347 |             } catch (Exception e) {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0348 |                 System.out.println(e);
```

Explanation: Prints a message or value to the console, commonly for status output or error reporting.

```
0349 |             }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0350 |         }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0351 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0352 |         if (!found) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0353 |             result.append("No payment records found.");
```

Explanation: Appends the provided value to the current StringBuilder or collection.

```
0354 |         }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0355 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0356 |         return result.toString();
```

Explanation: Returns the value or result to the caller and ends the current method.

```
0357 |     }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0358 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0359 |     public String markPaymentPaid(int paymentId) throws SQLException {
```

Explanation: Declares the markPaymentPaid method and its return type/parameters; the method performs the operation described by its name and body.

```
0360 |         try (Connection conn = new DbConnection().getConnection()) {
```

Explanation: Begins a protected block for code that may throw an exception; resources may be managed by try-with-resources.

```
0361 |             if (conn == null) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0362 |                 throw new SQLException("Could not connect to the database.");
```

Explanation: Raises an exception so the caller or an enclosing catch block can handle the failure.

```
0363 |             }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0364 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0365 |             conn.setAutoCommit(false);
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0366 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0367 |             try {
```

Explanation: Begins a protected block for code that may throw an exception; resources may be managed by try-with-resources.

```
0368 |                 int requestId;
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0369 |                 String status;
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0370 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0371 |                 String findSql =
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0372 |                         "SELECT request_id, payment_status FROM payments " +
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0373 |                         "WHERE payment_id = ? FOR UPDATE";
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0374 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0375 |                 try (PreparedStatement ps = conn.prepareStatement(findSql)) {
```

Explanation: Begins a protected block for code that may throw an exception; resources may be managed by try-with-resources.

```
0376 |                     ps.setInt(1, paymentId);
```

Explanation: Binds a Java value to a placeholder in the prepared SQL statement; the parameter index is one-based.

```
0377 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0378 |                     try (ResultSet rs = ps.executeQuery()) {
```

Explanation: Begins a protected block for code that may throw an exception; resources may be managed by try-with-resources.

```
0379 |                         if (!rs.next()) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0380 |                             throw new SQLException("Payment ID was not found.");
```

Explanation: Raises an exception so the caller or an enclosing catch block can handle the failure.

```
0381 |                         }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0382 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0383 |                         requestId = rs.getInt("request_id");
```

Explanation: Reads a column value from the current database result row using the requested Java type.

```
0384 |                         status = rs.getString("payment_status");
```

Explanation: Reads a column value from the current database result row using the requested Java type.

```
0385 |                     }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0386 |                 }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0387 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0388 |                 if ("Paid".equalsIgnoreCase(status)) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0389 |                     throw new SQLException("This payment is already marked as Paid.");
```

Explanation: Raises an exception so the caller or an enclosing catch block can handle the failure.

```
0390 |                 }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0391 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0392 |                 String updatePaymentSql =
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0393 |                         "UPDATE payments SET payment_status = 'Paid', " +
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0394 |                         "paid_at = CURRENT_TIMESTAMP WHERE payment_id = ?";
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0395 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0396 |                 try (PreparedStatement ps = conn.prepareStatement(updatePaymentSql)) {
```

Explanation: Begins a protected block for code that may throw an exception; resources may be managed by try-with-resources.

```
0397 |                     ps.setInt(1, paymentId);
```

Explanation: Binds a Java value to a placeholder in the prepared SQL statement; the parameter index is one-based.

```
0398 |                     ps.executeUpdate();
```

Explanation: Executes an INSERT, UPDATE, or DELETE statement and returns the number of affected rows.

```
0399 |                 }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0400 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0401 |                 String updateRequestSql =
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0402 |                         "UPDATE payment_requests SET request_status = 'Paid' " +
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0403 |                         "WHERE request_id = ?";
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0404 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0405 |                 try (PreparedStatement ps = conn.prepareStatement(updateRequestSql)) {
```

Explanation: Begins a protected block for code that may throw an exception; resources may be managed by try-with-resources.

```
0406 |                     ps.setInt(1, requestId);
```

Explanation: Binds a Java value to a placeholder in the prepared SQL statement; the parameter index is one-based.

```
0407 |                     ps.executeUpdate();
```

Explanation: Executes an INSERT, UPDATE, or DELETE statement and returns the number of affected rows.

```
0408 |                 }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0409 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0410 |                 conn.commit();
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0411 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0412 |                 return "Payment verified successfully.\n" +
```

Explanation: Returns the value or result to the caller and ends the current method.

```
0413 |                         "Payment ID: " + paymentId + "\n" +
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0414 |                         "Request ID: " + requestId + "\n" +
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0415 |                         "Status: Paid";
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0416 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0417 |             } catch (Exception ex) {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0418 |                 conn.rollback();
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0419 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0420 |                 if (ex instanceof SQLException) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0421 |                     throw (SQLException) ex;
```

Explanation: Raises an exception so the caller or an enclosing catch block can handle the failure.

```
0422 |                 }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0423 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0424 |                 throw new SQLException("Could not verify payment.", ex);
```

Explanation: Raises an exception so the caller or an enclosing catch block can handle the failure.

```
0425 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0426 |             } finally {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0427 |                 conn.setAutoCommit(true);
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0428 |             }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0429 |         }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0430 |     }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0431 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0432 |     private static class FeeItem {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0433 |         private String name;
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0434 |         private String type;
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0435 |         private double amount;
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0436 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0437 |         public FeeItem(String name, String type, double amount) {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0438 |             this.name = name;
```

Explanation: Assigns or updates a variable with the value or result on the right-hand side.

```
0439 |             this.type = type;
```

Explanation: Assigns or updates a variable with the value or result on the right-hand side.

```
0440 |             this.amount = amount;
```

Explanation: Assigns or updates a variable with the value or result on the right-hand side.

```
0441 |         }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0442 |     }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0443 | }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

RegistrationFrame.java

Project path: src/RegistrationFrame.java

```
0001 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0002 | import javax.swing.*;
```

Explanation: Imports Swing GUI classes used to build the application's windows and controls.

```
0003 | import java.awt.*;
```

Explanation: Imports AWT classes used for layout, dimensions, colors, fonts, or UI events.

```
0004 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0005 | public class RegistrationFrame extends JFrame {
```

Explanation: Declares the RegistrationFrame type, which groups related data and behavior.

```
0006 |     private JTextField usernameField = new JTextField();
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0007 |     private JTextField emailField = new JTextField();
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0008 |     private JPasswordField passwordField = new JPasswordField();
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0009 |     private JTextField lrnField = new JTextField();
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0010 |     private JTextField firstNameField = new JTextField();
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0011 |     private JTextField middleNameField = new JTextField();
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0012 |     private JTextField lastNameField = new JTextField();
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0013 |     private JTextField previousSchoolField = new JTextField();
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0014 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0015 |     private AuthService authService = new AuthService();
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0016 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0017 |     public RegistrationFrame() {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0018 |         setTitle("Student Registration");
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0019 |         setSize(500, 500);
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0020 |         setDefaultCloseOperation(JFrame.DISPOSE_ON_CLOSE);
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0021 |         setLocationRelativeTo(null);
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0022 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0023 |         JPanel panel = new JPanel(new GridLayout(9, 2, 8, 8));
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0024 |         panel.setBorder(BorderFactory.createEmptyBorder(15, 20, 15, 20));
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0025 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0026 |         panel.add(new JLabel("Username:"));
```

Explanation: Adds a component to a Swing container so it becomes part of the displayed interface.

```
0027 |         panel.add(usernameField);
```

Explanation: Adds a component to a Swing container so it becomes part of the displayed interface.

```
0028 |         panel.add(new JLabel("Email:"));
```

Explanation: Adds a component to a Swing container so it becomes part of the displayed interface.

```
0029 |         panel.add(emailField);
```

Explanation: Adds a component to a Swing container so it becomes part of the displayed interface.

```
0030 |         panel.add(new JLabel("Password:"));
```

Explanation: Adds a component to a Swing container so it becomes part of the displayed interface.

```
0031 |         panel.add(passwordField);
```

Explanation: Adds a component to a Swing container so it becomes part of the displayed interface.

```
0032 |         panel.add(new JLabel("LRN:"));
```

Explanation: Adds a component to a Swing container so it becomes part of the displayed interface.

```
0033 |         panel.add(lrnField);
```

Explanation: Adds a component to a Swing container so it becomes part of the displayed interface.

```
0034 |         panel.add(new JLabel("First Name:"));
```

Explanation: Adds a component to a Swing container so it becomes part of the displayed interface.

```
0035 |         panel.add(firstNameField);
```

Explanation: Adds a component to a Swing container so it becomes part of the displayed interface.

```
0036 |         panel.add(new JLabel("Middle Name:"));
```

Explanation: Adds a component to a Swing container so it becomes part of the displayed interface.

```
0037 |         panel.add(middleNameField);
```

Explanation: Adds a component to a Swing container so it becomes part of the displayed interface.

```
0038 |         panel.add(new JLabel("Last Name:"));
```

Explanation: Adds a component to a Swing container so it becomes part of the displayed interface.

```
0039 |         panel.add(lastNameField);
```

Explanation: Adds a component to a Swing container so it becomes part of the displayed interface.

```
0040 |         panel.add(new JLabel("Previous School:"));
```

Explanation: Adds a component to a Swing container so it becomes part of the displayed interface.

```
0041 |         panel.add(previousSchoolField);
```

Explanation: Adds a component to a Swing container so it becomes part of the displayed interface.

```
0042 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0043 |         JButton registerButton = new JButton("Register");
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0044 |         JButton cancelButton = new JButton("Cancel");
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0045 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0046 |         panel.add(registerButton);
```

Explanation: Adds a component to a Swing container so it becomes part of the displayed interface.

```
0047 |         panel.add(cancelButton);
```

Explanation: Adds a component to a Swing container so it becomes part of the displayed interface.

```
0048 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0049 |         add(panel);
```

Explanation: Adds a component to a Swing container so it becomes part of the displayed interface.

```
0050 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0051 |         registerButton.addActionListener(e -&gt; register());
```

Explanation: Registers an event handler that runs when the associated GUI control is activated.

```
0052 |         cancelButton.addActionListener(e -&gt; dispose());
```

Explanation: Registers an event handler that runs when the associated GUI control is activated.

```
0053 |     }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0054 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0055 |     private void register() {
```

Explanation: Declares the register method, which implements this part of the application's authentication or registration workflow.

```
0056 |         String username = usernameField.getText().trim();
```

Explanation: Reads text currently entered or displayed by a GUI component.

```
0057 |         String email = emailField.getText().trim();
```

Explanation: Reads text currently entered or displayed by a GUI component.

```
0058 |         String password = new String(passwordField.getPassword());
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0059 |         String lrn = lrnField.getText().trim();
```

Explanation: Reads text currently entered or displayed by a GUI component.

```
0060 |         String firstName = firstNameField.getText().trim();
```

Explanation: Reads text currently entered or displayed by a GUI component.

```
0061 |         String middleName = middleNameField.getText().trim();
```

Explanation: Reads text currently entered or displayed by a GUI component.

```
0062 |         String lastName = lastNameField.getText().trim();
```

Explanation: Reads text currently entered or displayed by a GUI component.

```
0063 |         String previousSchool = previousSchoolField.getText().trim();
```

Explanation: Reads text currently entered or displayed by a GUI component.

```
0064 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0065 |         if (username.isEmpty() || email.isEmpty() || password.isEmpty()
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0066 |                 || lrn.isEmpty() || firstName.isEmpty() || lastName.isEmpty()) {
```

Explanation: Checks whether a string or collection has no content.

```
0067 |             JOptionPane.showMessageDialog(this,
```

Explanation: Displays a Swing dialog to communicate a message, warning, confirmation, or input prompt to the user.

```
0068 |                     "Please fill in all required fields.");
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0069 |             return;
```

Explanation: Ends the current method without returning a value.

```
0070 |         }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0071 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0072 |         boolean success = authService.register(
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0073 |                 username, password, email, lrn, firstName,
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0074 |                 middleName, lastName, previousSchool);
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0075 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0076 |         if (success) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0077 |             JOptionPane.showMessageDialog(this,
```

Explanation: Displays a Swing dialog to communicate a message, warning, confirmation, or input prompt to the user.

```
0078 |                     "Registration successful. You can now log in.");
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0079 |             dispose();
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0080 |         } else {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0081 |             JOptionPane.showMessageDialog(this,
```

Explanation: Displays a Swing dialog to communicate a message, warning, confirmation, or input prompt to the user.

```
0082 |                     "Registration failed. Check for duplicate username, email, or LRN.");
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0083 |         }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0084 |     }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0085 | }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

ReportDAO.java

Project path: src/ReportDAO.java

```
0001 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0002 | import java.sql.*;
```

Explanation: Imports JDBC classes used to communicate with a relational database.

```
0003 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0004 | public class ReportDAO {
```

Explanation: Declares the ReportDAO type, which groups related data and behavior.

```
0005 |     private DbConnection db = new DbConnection();
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0006 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0007 |     public String getEnrollmentReport() {
```

Explanation: Declares the getEnrollmentReport method, typically returning the requested field or resource.

```
0008 |         Connection conn = null;
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0009 |         PreparedStatement ps = null;
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0010 |         ResultSet rs = null;
```

Explanation: Declares or assigns a ResultSet that holds rows returned by a database query.

```
0011 |         StringBuilder result = new StringBuilder();
```

Explanation: Creates a mutable string builder so text can be assembled efficiently.

```
0012 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0013 |         try {
```

Explanation: Begins a protected block for code that may throw an exception; resources may be managed by try-with-resources.

```
0014 |             conn = db.getConnection();
```

Explanation: Assigns or updates a variable with the value or result on the right-hand side.

```
0015 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0016 |             if (conn == null) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0017 |                 return "Report error: Could not connect to the database.";
```

Explanation: Returns the value or result to the caller and ends the current method.

```
0018 |             }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0019 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0020 |             String sql = "SELECT s.strand_code, e.year_level, "
```

Explanation: Stores the SQL command as a string so the program can execute it against the database.

```
0021 |                     + "e.enrollment_status, COUNT(*) AS total "
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0022 |                     + "FROM enrollments e "
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0023 |                     + "JOIN strands s ON e.strand_id = s.strand_id "
```

Explanation: Assigns or updates a variable with the value or result on the right-hand side.

```
0024 |                     + "GROUP BY s.strand_code, e.year_level, e.enrollment_status "
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0025 |                     + "ORDER BY s.strand_code, e.year_level";
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0026 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0027 |             ps = conn.prepareStatement(sql);
```

Explanation: Creates a parameterized SQL statement; placeholders can be filled separately to reduce SQL-injection risk.

```
0028 |             rs = ps.executeQuery();
```

Explanation: Executes a SQL query that returns rows, usually a SELECT, and stores them in a ResultSet.

```
0029 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0030 |             while (rs.next()) {
```

Explanation: Repeats the following block while the condition remains true.

```
0031 |                 result.append("Strand: ").append(rs.getString("strand_code"))
```

Explanation: Reads a column value from the current database result row using the requested Java type.

```
0032 |                         .append(" | Grade: ").append(rs.getInt("year_level"))
```

Explanation: Reads a column value from the current database result row using the requested Java type.

```
0033 |                         .append(" | Status: ")
```

Explanation: Appends the provided value to the current StringBuilder or collection.

```
0034 |                         .append(rs.getString("enrollment_status"))
```

Explanation: Reads a column value from the current database result row using the requested Java type.

```
0035 |                         .append(" | Total: ").append(rs.getInt("total"))
```

Explanation: Reads a column value from the current database result row using the requested Java type.

```
0036 |                         .append("\n");
```

Explanation: Appends the provided value to the current StringBuilder or collection.

```
0037 |             }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0038 |         } catch (Exception e) {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0039 |             return "Report error: " + e.getMessage();
```

Explanation: Returns the value or result to the caller and ends the current method.

```
0040 |         } finally {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0041 |             try {
```

Explanation: Begins a protected block for code that may throw an exception; resources may be managed by try-with-resources.

```
0042 |                 if (rs != null) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0043 |                     rs.close();
```

Explanation: Closes the associated resource, such as a JDBC ResultSet, Statement, or Connection.

```
0044 |                 }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0045 |             } catch (Exception e) {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0046 |                 System.out.println(e);
```

Explanation: Prints a message or value to the console, commonly for status output or error reporting.

```
0047 |             }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0048 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0049 |             try {
```

Explanation: Begins a protected block for code that may throw an exception; resources may be managed by try-with-resources.

```
0050 |                 if (ps != null) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0051 |                     ps.close();
```

Explanation: Closes the associated resource, such as a JDBC ResultSet, Statement, or Connection.

```
0052 |                 }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0053 |             } catch (Exception e) {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0054 |                 System.out.println(e);
```

Explanation: Prints a message or value to the console, commonly for status output or error reporting.

```
0055 |             }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0056 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0057 |             try {
```

Explanation: Begins a protected block for code that may throw an exception; resources may be managed by try-with-resources.

```
0058 |                 if (conn != null) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0059 |                     conn.close();
```

Explanation: Closes the associated resource, such as a JDBC ResultSet, Statement, or Connection.

```
0060 |                 }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0061 |             } catch (Exception e) {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0062 |                 System.out.println(e);
```

Explanation: Prints a message or value to the console, commonly for status output or error reporting.

```
0063 |             }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0064 |         }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0065 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0066 |         return result.length() == 0
```

Explanation: Returns the value or result to the caller and ends the current method.

```
0067 |                 ? "No enrollment records found."
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0068 |                 : result.toString();
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0069 |     }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0070 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0071 |     public int getTotalEnrollments() {
```

Explanation: Declares the getTotalEnrollments method, typically returning the requested field or resource.

```
0072 |         Connection conn = null;
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0073 |         PreparedStatement ps = null;
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0074 |         ResultSet rs = null;
```

Explanation: Declares or assigns a ResultSet that holds rows returned by a database query.

```
0075 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0076 |         try {
```

Explanation: Begins a protected block for code that may throw an exception; resources may be managed by try-with-resources.

```
0077 |             conn = db.getConnection();
```

Explanation: Assigns or updates a variable with the value or result on the right-hand side.

```
0078 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0079 |             if (conn == null) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0080 |                 return 0;
```

Explanation: Returns the value or result to the caller and ends the current method.

```
0081 |             }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0082 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0083 |             String sql = "SELECT COUNT(*) AS total FROM enrollments";
```

Explanation: Stores the SQL command as a string so the program can execute it against the database.

```
0084 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0085 |             ps = conn.prepareStatement(sql);
```

Explanation: Creates a parameterized SQL statement; placeholders can be filled separately to reduce SQL-injection risk.

```
0086 |             rs = ps.executeQuery();
```

Explanation: Executes a SQL query that returns rows, usually a SELECT, and stores them in a ResultSet.

```
0087 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0088 |             if (rs.next()) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0089 |                 return rs.getInt("total");
```

Explanation: Returns the value or result to the caller and ends the current method.

```
0090 |             }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0091 |         } catch (Exception e) {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0092 |             System.out.println("Report error: " + e.getMessage());
```

Explanation: Prints a message or value to the console, commonly for status output or error reporting.

```
0093 |         } finally {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0094 |             try {
```

Explanation: Begins a protected block for code that may throw an exception; resources may be managed by try-with-resources.

```
0095 |                 if (rs != null) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0096 |                     rs.close();
```

Explanation: Closes the associated resource, such as a JDBC ResultSet, Statement, or Connection.

```
0097 |                 }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0098 |             } catch (Exception e) {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0099 |                 System.out.println(e);
```

Explanation: Prints a message or value to the console, commonly for status output or error reporting.

```
0100 |             }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0101 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0102 |             try {
```

Explanation: Begins a protected block for code that may throw an exception; resources may be managed by try-with-resources.

```
0103 |                 if (ps != null) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0104 |                     ps.close();
```

Explanation: Closes the associated resource, such as a JDBC ResultSet, Statement, or Connection.

```
0105 |                 }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0106 |             } catch (Exception e) {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0107 |                 System.out.println(e);
```

Explanation: Prints a message or value to the console, commonly for status output or error reporting.

```
0108 |             }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0109 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0110 |             try {
```

Explanation: Begins a protected block for code that may throw an exception; resources may be managed by try-with-resources.

```
0111 |                 if (conn != null) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0112 |                     conn.close();
```

Explanation: Closes the associated resource, such as a JDBC ResultSet, Statement, or Connection.

```
0113 |                 }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0114 |             } catch (Exception e) {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0115 |                 System.out.println(e);
```

Explanation: Prints a message or value to the console, commonly for status output or error reporting.

```
0116 |             }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0117 |         }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0118 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0119 |         return 0;
```

Explanation: Returns the value or result to the caller and ends the current method.

```
0120 |     }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0121 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0122 |     public String getEnrollmentCountReport(String type) throws SQLException {
```

Explanation: Declares the getEnrollmentCountReport method, typically returning the requested field or resource.

```
0123 |         String sql;
```

Explanation: Stores the SQL command as a string so the program can execute it against the database.

```
0124 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0125 |         if ("strand".equals(type)) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0126 |             sql = "SELECT str.strand_code AS report_name, COUNT(*) AS total "
```

Explanation: Assigns or updates a variable with the value or result on the right-hand side.

```
0127 |                     + "FROM enrollments e "
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0128 |                     + "JOIN strands str ON e.strand_id = str.strand_id "
```

Explanation: Assigns or updates a variable with the value or result on the right-hand side.

```
0129 |                     + "WHERE e.year_level IN (11, 12) "
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0130 |                     + "GROUP BY str.strand_code "
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0131 |                     + "ORDER BY str.strand_code";
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0132 |         } else if ("grade".equals(type)) {
```

Explanation: Compares values for equality using the class's equals method.

```
0133 |             sql = "SELECT e.year_level AS report_name, COUNT(*) AS total "
```

Explanation: Assigns or updates a variable with the value or result on the right-hand side.

```
0134 |                     + "FROM enrollments e "
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0135 |                     + "WHERE e.year_level IN (11, 12) "
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0136 |                     + "GROUP BY e.year_level ORDER BY e.year_level";
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0137 |         } else {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0138 |             sql = "SELECT COALESCE(sec.section_name, "
```

Explanation: Assigns or updates a variable with the value or result on the right-hand side.

```
0139 |                     + "'Not Assigned') AS report_name, COUNT(*) AS total "
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0140 |                     + "FROM enrollments e "
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0141 |                     + "LEFT JOIN sections sec "
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0142 |                     + "ON e.section_id = sec.section_id "
```

Explanation: Assigns or updates a variable with the value or result on the right-hand side.

```
0143 |                     + "WHERE e.year_level IN (11, 12) "
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0144 |                     + "GROUP BY sec.section_id, sec.section_name "
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0145 |                     + "ORDER BY report_name";
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0146 |         }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0147 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0148 |         StringBuilder result = new StringBuilder(
```

Explanation: Creates a mutable string builder so text can be assembled efficiently.

```
0149 |                 "ENROLLEES BY " + type.toUpperCase() + "\n\n");
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0150 |         Connection conn = null;
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0151 |         PreparedStatement ps = null;
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0152 |         ResultSet rs = null;
```

Explanation: Declares or assigns a ResultSet that holds rows returned by a database query.

```
0153 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0154 |         try {
```

Explanation: Begins a protected block for code that may throw an exception; resources may be managed by try-with-resources.

```
0155 |             conn = db.getConnection();
```

Explanation: Assigns or updates a variable with the value or result on the right-hand side.

```
0156 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0157 |             if (conn == null) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0158 |                 throw new SQLException("Could not connect to the database.");
```

Explanation: Raises an exception so the caller or an enclosing catch block can handle the failure.

```
0159 |             }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0160 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0161 |             ps = conn.prepareStatement(sql);
```

Explanation: Creates a parameterized SQL statement; placeholders can be filled separately to reduce SQL-injection risk.

```
0162 |             rs = ps.executeQuery();
```

Explanation: Executes a SQL query that returns rows, usually a SELECT, and stores them in a ResultSet.

```
0163 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0164 |             boolean found = false;
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0165 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0166 |             while (rs.next()) {
```

Explanation: Repeats the following block while the condition remains true.

```
0167 |                 found = true;
```

Explanation: Assigns or updates a variable with the value or result on the right-hand side.

```
0168 |                 result.append(rs.getString("report_name"))
```

Explanation: Reads a column value from the current database result row using the requested Java type.

```
0169 |                         .append(": ")
```

Explanation: Appends the provided value to the current StringBuilder or collection.

```
0170 |                         .append(rs.getInt("total"))
```

Explanation: Reads a column value from the current database result row using the requested Java type.

```
0171 |                         .append(" student(s)\n");
```

Explanation: Appends the provided value to the current StringBuilder or collection.

```
0172 |             }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0173 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0174 |             if (!found) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0175 |                 result.append("No records found.");
```

Explanation: Appends the provided value to the current StringBuilder or collection.

```
0176 |             }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0177 |         } catch (SQLException e) {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0178 |             throw e;
```

Explanation: Raises an exception so the caller or an enclosing catch block can handle the failure.

```
0179 |         } catch (Exception e) {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0180 |             throw new SQLException(e.getMessage(), e);
```

Explanation: Raises an exception so the caller or an enclosing catch block can handle the failure.

```
0181 |         } finally {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0182 |             try {
```

Explanation: Begins a protected block for code that may throw an exception; resources may be managed by try-with-resources.

```
0183 |                 if (rs != null) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0184 |                     rs.close();
```

Explanation: Closes the associated resource, such as a JDBC ResultSet, Statement, or Connection.

```
0185 |                 }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0186 |             } catch (Exception e) {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0187 |                 System.out.println(e);
```

Explanation: Prints a message or value to the console, commonly for status output or error reporting.

```
0188 |             }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0189 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0190 |             try {
```

Explanation: Begins a protected block for code that may throw an exception; resources may be managed by try-with-resources.

```
0191 |                 if (ps != null) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0192 |                     ps.close();
```

Explanation: Closes the associated resource, such as a JDBC ResultSet, Statement, or Connection.

```
0193 |                 }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0194 |             } catch (Exception e) {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0195 |                 System.out.println(e);
```

Explanation: Prints a message or value to the console, commonly for status output or error reporting.

```
0196 |             }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0197 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0198 |             try {
```

Explanation: Begins a protected block for code that may throw an exception; resources may be managed by try-with-resources.

```
0199 |                 if (conn != null) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0200 |                     conn.close();
```

Explanation: Closes the associated resource, such as a JDBC ResultSet, Statement, or Connection.

```
0201 |                 }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0202 |             } catch (Exception e) {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0203 |                 System.out.println(e);
```

Explanation: Prints a message or value to the console, commonly for status output or error reporting.

```
0204 |             }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0205 |         }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0206 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0207 |         return result.toString();
```

Explanation: Returns the value or result to the caller and ends the current method.

```
0208 |     }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0209 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0210 |     public String getIncompleteRequirementsReport() throws SQLException {
```

Explanation: Declares the getIncompleteRequirementsReport method, typically returning the requested field or resource.

```
0211 |         String sql =
```

Explanation: Stores the SQL command as a string so the program can execute it against the database.

```
0212 |                 "SELECT sr.requirement_id, st.student_id, st.lrn, st.first_name, "
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0213 |                         + "st.last_name, e.year_level, "
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0214 |                         + "str.strand_code, sr.requirement_name, "
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0215 |                         + "sr.requirement_status "
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0216 |                         + "FROM student_requirements sr "
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0217 |                         + "JOIN enrollments e "
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0218 |                         + "ON sr.enrollment_id = e.enrollment_id "
```

Explanation: Assigns or updates a variable with the value or result on the right-hand side.

```
0219 |                         + "JOIN students st ON e.student_id = st.student_id "
```

Explanation: Assigns or updates a variable with the value or result on the right-hand side.

```
0220 |                         + "JOIN strands str ON e.strand_id = str.strand_id "
```

Explanation: Assigns or updates a variable with the value or result on the right-hand side.

```
0221 |                         + "WHERE e.year_level IN (11, 12) "
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0222 |                         + "AND (sr.requirement_status IS NULL "
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0223 |                         + "OR sr.requirement_status &lt;&gt; 'Complete') "
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0224 |                         + "ORDER BY st.last_name, st.first_name";
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0225 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0226 |         StringBuilder result = new StringBuilder(
```

Explanation: Creates a mutable string builder so text can be assembled efficiently.

```
0227 |                 "INCOMPLETE REQUIREMENTS\n\n");
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0228 |         Connection conn = null;
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0229 |         PreparedStatement ps = null;
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0230 |         ResultSet rs = null;
```

Explanation: Declares or assigns a ResultSet that holds rows returned by a database query.

```
0231 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0232 |         try {
```

Explanation: Begins a protected block for code that may throw an exception; resources may be managed by try-with-resources.

```
0233 |             conn = db.getConnection();
```

Explanation: Assigns or updates a variable with the value or result on the right-hand side.

```
0234 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0235 |             if (conn == null) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0236 |                 throw new SQLException("Could not connect to the database.");
```

Explanation: Raises an exception so the caller or an enclosing catch block can handle the failure.

```
0237 |             }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0238 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0239 |             ps = conn.prepareStatement(sql);
```

Explanation: Creates a parameterized SQL statement; placeholders can be filled separately to reduce SQL-injection risk.

```
0240 |             rs = ps.executeQuery();
```

Explanation: Executes a SQL query that returns rows, usually a SELECT, and stores them in a ResultSet.

```
0241 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0242 |             boolean found = false;
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0243 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0244 |             while (rs.next()) {
```

Explanation: Repeats the following block while the condition remains true.

```
0245 |                 found = true;
```

Explanation: Assigns or updates a variable with the value or result on the right-hand side.

```
0246 |                 result.append("Requirement ID: ")
```

Explanation: Appends the provided value to the current StringBuilder or collection.

```
0247 |                         .append(rs.getInt("requirement_id"))
```

Explanation: Reads a column value from the current database result row using the requested Java type.

```
0248 |                         .append("\nStudent ID: ")
```

Explanation: Appends the provided value to the current StringBuilder or collection.

```
0249 |                         .append(rs.getInt("student_id"))
```

Explanation: Reads a column value from the current database result row using the requested Java type.

```
0250 |                         .append("\nLRN: ")
```

Explanation: Appends the provided value to the current StringBuilder or collection.

```
0251 |                         .append(rs.getString("lrn"))
```

Explanation: Reads a column value from the current database result row using the requested Java type.

```
0252 |                         .append("\nName: ")
```

Explanation: Appends the provided value to the current StringBuilder or collection.

```
0253 |                         .append(rs.getString("first_name"))
```

Explanation: Reads a column value from the current database result row using the requested Java type.

```
0254 |                         .append(" ")
```

Explanation: Appends the provided value to the current StringBuilder or collection.

```
0255 |                         .append(rs.getString("last_name"))
```

Explanation: Reads a column value from the current database result row using the requested Java type.

```
0256 |                         .append("\nGrade: ")
```

Explanation: Appends the provided value to the current StringBuilder or collection.

```
0257 |                         .append(rs.getInt("year_level"))
```

Explanation: Reads a column value from the current database result row using the requested Java type.

```
0258 |                         .append("\nStrand: ")
```

Explanation: Appends the provided value to the current StringBuilder or collection.

```
0259 |                         .append(rs.getString("strand_code"))
```

Explanation: Reads a column value from the current database result row using the requested Java type.

```
0260 |                         .append("\nRequirement: ")
```

Explanation: Appends the provided value to the current StringBuilder or collection.

```
0261 |                         .append(rs.getString("requirement_name"))
```

Explanation: Reads a column value from the current database result row using the requested Java type.

```
0262 |                         .append("\nStatus: ")
```

Explanation: Appends the provided value to the current StringBuilder or collection.

```
0263 |                         .append(rs.getString("requirement_status"))
```

Explanation: Reads a column value from the current database result row using the requested Java type.

```
0264 |                         .append("\n--------------------------------------\n");
```

Explanation: Appends the provided value to the current StringBuilder or collection.

```
0265 |             }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0266 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0267 |             if (!found) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0268 |                 result.append("No incomplete requirements found.");
```

Explanation: Appends the provided value to the current StringBuilder or collection.

```
0269 |             }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0270 |         } catch (SQLException e) {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0271 |             throw e;
```

Explanation: Raises an exception so the caller or an enclosing catch block can handle the failure.

```
0272 |         } catch (Exception e) {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0273 |             throw new SQLException(e.getMessage(), e);
```

Explanation: Raises an exception so the caller or an enclosing catch block can handle the failure.

```
0274 |         } finally {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0275 |             try {
```

Explanation: Begins a protected block for code that may throw an exception; resources may be managed by try-with-resources.

```
0276 |                 if (rs != null) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0277 |                     rs.close();
```

Explanation: Closes the associated resource, such as a JDBC ResultSet, Statement, or Connection.

```
0278 |                 }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0279 |             } catch (Exception e) {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0280 |                 System.out.println(e);
```

Explanation: Prints a message or value to the console, commonly for status output or error reporting.

```
0281 |             }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0282 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0283 |             try {
```

Explanation: Begins a protected block for code that may throw an exception; resources may be managed by try-with-resources.

```
0284 |                 if (ps != null) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0285 |                     ps.close();
```

Explanation: Closes the associated resource, such as a JDBC ResultSet, Statement, or Connection.

```
0286 |                 }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0287 |             } catch (Exception e) {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0288 |                 System.out.println(e);
```

Explanation: Prints a message or value to the console, commonly for status output or error reporting.

```
0289 |             }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0290 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0291 |             try {
```

Explanation: Begins a protected block for code that may throw an exception; resources may be managed by try-with-resources.

```
0292 |                 if (conn != null) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0293 |                     conn.close();
```

Explanation: Closes the associated resource, such as a JDBC ResultSet, Statement, or Connection.

```
0294 |                 }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0295 |             } catch (Exception e) {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0296 |                 System.out.println(e);
```

Explanation: Prints a message or value to the console, commonly for status output or error reporting.

```
0297 |             }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0298 |         }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0299 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0300 |         return result.toString();
```

Explanation: Returns the value or result to the caller and ends the current method.

```
0301 |     }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0302 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0303 |     public String getSectionsReport() throws SQLException {
```

Explanation: Declares the getSectionsReport method, typically returning the requested field or resource.

```
0304 |         String sql =
```

Explanation: Stores the SQL command as a string so the program can execute it against the database.

```
0305 |                 "SELECT sec.section_id, sec.section_name, "
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0306 |                         + "sec.grade_level, st.strand_code, "
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0307 |                         + "sy.year_label, sec.slot_limit, sec.is_active, "
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0308 |                         + "(SELECT COUNT(*) FROM enrollments e "
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0309 |                         + "WHERE e.section_id = sec.section_id "
```

Explanation: Assigns or updates a variable with the value or result on the right-hand side.

```
0310 |                         + "AND e.school_year_id = sec.school_year_id) "
```

Explanation: Assigns or updates a variable with the value or result on the right-hand side.

```
0311 |                         + "AS enrolled_count "
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0312 |                         + "FROM sections sec "
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0313 |                         + "JOIN strands st ON sec.strand_id = st.strand_id "
```

Explanation: Assigns or updates a variable with the value or result on the right-hand side.

```
0314 |                         + "JOIN school_years sy "
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0315 |                         + "ON sec.school_year_id = sy.school_year_id "
```

Explanation: Assigns or updates a variable with the value or result on the right-hand side.

```
0316 |                         + "WHERE sec.grade_level IN (11, 12) "
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0317 |                         + "ORDER BY sec.grade_level, sec.section_name";
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0318 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0319 |         StringBuilder result = new StringBuilder();
```

Explanation: Creates a mutable string builder so text can be assembled efficiently.

```
0320 |         result.append("SENIOR HIGH SCHOOL SECTIONS\n\n");
```

Explanation: Appends the provided value to the current StringBuilder or collection.

```
0321 |         Connection conn = null;
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0322 |         PreparedStatement ps = null;
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0323 |         ResultSet rs = null;
```

Explanation: Declares or assigns a ResultSet that holds rows returned by a database query.

```
0324 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0325 |         try {
```

Explanation: Begins a protected block for code that may throw an exception; resources may be managed by try-with-resources.

```
0326 |             conn = db.getConnection();
```

Explanation: Assigns or updates a variable with the value or result on the right-hand side.

```
0327 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0328 |             if (conn == null) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0329 |                 throw new SQLException("Could not connect to the database.");
```

Explanation: Raises an exception so the caller or an enclosing catch block can handle the failure.

```
0330 |             }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0331 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0332 |             ps = conn.prepareStatement(sql);
```

Explanation: Creates a parameterized SQL statement; placeholders can be filled separately to reduce SQL-injection risk.

```
0333 |             rs = ps.executeQuery();
```

Explanation: Executes a SQL query that returns rows, usually a SELECT, and stores them in a ResultSet.

```
0334 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0335 |             boolean found = false;
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0336 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0337 |             while (rs.next()) {
```

Explanation: Repeats the following block while the condition remains true.

```
0338 |                 found = true;
```

Explanation: Assigns or updates a variable with the value or result on the right-hand side.

```
0339 |                 result.append("Section ID: ")
```

Explanation: Appends the provided value to the current StringBuilder or collection.

```
0340 |                         .append(rs.getInt("section_id"))
```

Explanation: Reads a column value from the current database result row using the requested Java type.

```
0341 |                         .append("\nSection: ")
```

Explanation: Appends the provided value to the current StringBuilder or collection.

```
0342 |                         .append(rs.getString("section_name"))
```

Explanation: Reads a column value from the current database result row using the requested Java type.

```
0343 |                         .append("\nGrade Level: ")
```

Explanation: Appends the provided value to the current StringBuilder or collection.

```
0344 |                         .append(rs.getInt("grade_level"))
```

Explanation: Reads a column value from the current database result row using the requested Java type.

```
0345 |                         .append("\nStrand: ")
```

Explanation: Appends the provided value to the current StringBuilder or collection.

```
0346 |                         .append(rs.getString("strand_code"))
```

Explanation: Reads a column value from the current database result row using the requested Java type.

```
0347 |                         .append("\nSchool Year: ")
```

Explanation: Appends the provided value to the current StringBuilder or collection.

```
0348 |                         .append(rs.getString("year_label"))
```

Explanation: Reads a column value from the current database result row using the requested Java type.

```
0349 |                         .append("\nEnrolled: ")
```

Explanation: Appends the provided value to the current StringBuilder or collection.

```
0350 |                         .append(rs.getInt("enrolled_count"))
```

Explanation: Reads a column value from the current database result row using the requested Java type.

```
0351 |                         .append("/")
```

Explanation: Appends the provided value to the current StringBuilder or collection.

```
0352 |                         .append(rs.getInt("slot_limit"))
```

Explanation: Reads a column value from the current database result row using the requested Java type.

```
0353 |                         .append("\nActive: ")
```

Explanation: Appends the provided value to the current StringBuilder or collection.

```
0354 |                         .append(rs.getBoolean("is_active"))
```

Explanation: Appends the provided value to the current StringBuilder or collection.

```
0355 |                         .append("\n----------------------------------------\n");
```

Explanation: Appends the provided value to the current StringBuilder or collection.

```
0356 |             }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0357 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0358 |             if (!found) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0359 |                 result.append("No Grade 11 or Grade 12 sections found.");
```

Explanation: Appends the provided value to the current StringBuilder or collection.

```
0360 |             }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0361 |         } catch (SQLException e) {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0362 |             throw e;
```

Explanation: Raises an exception so the caller or an enclosing catch block can handle the failure.

```
0363 |         } catch (Exception e) {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0364 |             throw new SQLException(e.getMessage(), e);
```

Explanation: Raises an exception so the caller or an enclosing catch block can handle the failure.

```
0365 |         } finally {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0366 |             try {
```

Explanation: Begins a protected block for code that may throw an exception; resources may be managed by try-with-resources.

```
0367 |                 if (rs != null) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0368 |                     rs.close();
```

Explanation: Closes the associated resource, such as a JDBC ResultSet, Statement, or Connection.

```
0369 |                 }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0370 |             } catch (Exception e) {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0371 |                 System.out.println(e);
```

Explanation: Prints a message or value to the console, commonly for status output or error reporting.

```
0372 |             }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0373 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0374 |             try {
```

Explanation: Begins a protected block for code that may throw an exception; resources may be managed by try-with-resources.

```
0375 |                 if (ps != null) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0376 |                     ps.close();
```

Explanation: Closes the associated resource, such as a JDBC ResultSet, Statement, or Connection.

```
0377 |                 }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0378 |             } catch (Exception e) {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0379 |                 System.out.println(e);
```

Explanation: Prints a message or value to the console, commonly for status output or error reporting.

```
0380 |             }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0381 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0382 |             try {
```

Explanation: Begins a protected block for code that may throw an exception; resources may be managed by try-with-resources.

```
0383 |                 if (conn != null) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0384 |                     conn.close();
```

Explanation: Closes the associated resource, such as a JDBC ResultSet, Statement, or Connection.

```
0385 |                 }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0386 |             } catch (Exception e) {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0387 |                 System.out.println(e);
```

Explanation: Prints a message or value to the console, commonly for status output or error reporting.

```
0388 |             }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0389 |         }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0390 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0391 |         return result.toString();
```

Explanation: Returns the value or result to the caller and ends the current method.

```
0392 |     }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0393 | }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

RequirementDAO.java

Project path: src/RequirementDAO.java

```
0001 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0002 | import java.sql.*;
```

Explanation: Imports JDBC classes used to communicate with a relational database.

```
0003 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0004 | public class RequirementDAO {
```

Explanation: Declares the RequirementDAO type, which groups related data and behavior.

```
0005 |     private DbConnection db = new DbConnection();
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0006 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0007 |     public String getRequirements() {
```

Explanation: Declares the getRequirements method, typically returning the requested field or resource.

```
0008 |         Connection conn = null;
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0009 |         PreparedStatement ps = null;
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0010 |         ResultSet rs = null;
```

Explanation: Declares or assigns a ResultSet that holds rows returned by a database query.

```
0011 |         StringBuilder result = new StringBuilder();
```

Explanation: Creates a mutable string builder so text can be assembled efficiently.

```
0012 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0013 |         try {
```

Explanation: Begins a protected block for code that may throw an exception; resources may be managed by try-with-resources.

```
0014 |             conn = db.getConnection();
```

Explanation: Assigns or updates a variable with the value or result on the right-hand side.

```
0015 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0016 |             if (conn == null) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0017 |                 return "Error: Could not connect to the database.";
```

Explanation: Returns the value or result to the caller and ends the current method.

```
0018 |             }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0019 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0020 |             String sql = "SELECT requirement_id, requirement_name FROM requirements";
```

Explanation: Stores the SQL command as a string so the program can execute it against the database.

```
0021 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0022 |             ps = conn.prepareStatement(sql);
```

Explanation: Creates a parameterized SQL statement; placeholders can be filled separately to reduce SQL-injection risk.

```
0023 |             rs = ps.executeQuery();
```

Explanation: Executes a SQL query that returns rows, usually a SELECT, and stores them in a ResultSet.

```
0024 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0025 |             while (rs.next()) {
```

Explanation: Repeats the following block while the condition remains true.

```
0026 |                 result.append(rs.getInt("requirement_id"))
```

Explanation: Reads a column value from the current database result row using the requested Java type.

```
0027 |                         .append(" - ")
```

Explanation: Appends the provided value to the current StringBuilder or collection.

```
0028 |                         .append(rs.getString("requirement_name"))
```

Explanation: Reads a column value from the current database result row using the requested Java type.

```
0029 |                         .append("\n");
```

Explanation: Appends the provided value to the current StringBuilder or collection.

```
0030 |             }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0031 |         } catch (Exception e) {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0032 |             return "Error: " + e.getMessage();
```

Explanation: Returns the value or result to the caller and ends the current method.

```
0033 |         } finally {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0034 |             try {
```

Explanation: Begins a protected block for code that may throw an exception; resources may be managed by try-with-resources.

```
0035 |                 if (rs != null) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0036 |                     rs.close();
```

Explanation: Closes the associated resource, such as a JDBC ResultSet, Statement, or Connection.

```
0037 |                 }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0038 |             } catch (Exception e) {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0039 |                 System.out.println(e);
```

Explanation: Prints a message or value to the console, commonly for status output or error reporting.

```
0040 |             }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0041 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0042 |             try {
```

Explanation: Begins a protected block for code that may throw an exception; resources may be managed by try-with-resources.

```
0043 |                 if (ps != null) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0044 |                     ps.close();
```

Explanation: Closes the associated resource, such as a JDBC ResultSet, Statement, or Connection.

```
0045 |                 }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0046 |             } catch (Exception e) {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0047 |                 System.out.println(e);
```

Explanation: Prints a message or value to the console, commonly for status output or error reporting.

```
0048 |             }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0049 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0050 |             try {
```

Explanation: Begins a protected block for code that may throw an exception; resources may be managed by try-with-resources.

```
0051 |                 if (conn != null) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0052 |                     conn.close();
```

Explanation: Closes the associated resource, such as a JDBC ResultSet, Statement, or Connection.

```
0053 |                 }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0054 |             } catch (Exception e) {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0055 |                 System.out.println(e);
```

Explanation: Prints a message or value to the console, commonly for status output or error reporting.

```
0056 |             }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0057 |         }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0058 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0059 |         return result.length() == 0 ? "No requirements found." : result.toString();
```

Explanation: Returns the value or result to the caller and ends the current method.

```
0060 |     }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0061 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0062 |     public boolean addRequirement(String name) {
```

Explanation: Declares the addRequirement method and its return type/parameters; the method performs the operation described by its name and body.

```
0063 |         Connection conn = null;
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0064 |         PreparedStatement ps = null;
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0065 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0066 |         try {
```

Explanation: Begins a protected block for code that may throw an exception; resources may be managed by try-with-resources.

```
0067 |             conn = db.getConnection();
```

Explanation: Assigns or updates a variable with the value or result on the right-hand side.

```
0068 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0069 |             if (conn == null) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0070 |                 return false;
```

Explanation: Returns the value or result to the caller and ends the current method.

```
0071 |             }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0072 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0073 |             String sql = "INSERT INTO requirements (requirement_name) VALUES (?)";
```

Explanation: Stores the SQL command as a string so the program can execute it against the database.

```
0074 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0075 |             ps = conn.prepareStatement(sql);
```

Explanation: Creates a parameterized SQL statement; placeholders can be filled separately to reduce SQL-injection risk.

```
0076 |             ps.setString(1, name);
```

Explanation: Binds a Java value to a placeholder in the prepared SQL statement; the parameter index is one-based.

```
0077 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0078 |             int rowInserted = ps.executeUpdate();
```

Explanation: Executes an INSERT, UPDATE, or DELETE statement and returns the number of affected rows.

```
0079 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0080 |             return rowInserted &gt; 0;
```

Explanation: Returns the value or result to the caller and ends the current method.

```
0081 |         } catch (Exception e) {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0082 |             System.out.println("Requirement error: " + e.getMessage());
```

Explanation: Prints a message or value to the console, commonly for status output or error reporting.

```
0083 |             return false;
```

Explanation: Returns the value or result to the caller and ends the current method.

```
0084 |         } finally {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0085 |             try {
```

Explanation: Begins a protected block for code that may throw an exception; resources may be managed by try-with-resources.

```
0086 |                 if (ps != null) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0087 |                     ps.close();
```

Explanation: Closes the associated resource, such as a JDBC ResultSet, Statement, or Connection.

```
0088 |                 }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0089 |             } catch (Exception e) {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0090 |                 System.out.println(e);
```

Explanation: Prints a message or value to the console, commonly for status output or error reporting.

```
0091 |             }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0092 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0093 |             try {
```

Explanation: Begins a protected block for code that may throw an exception; resources may be managed by try-with-resources.

```
0094 |                 if (conn != null) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0095 |                     conn.close();
```

Explanation: Closes the associated resource, such as a JDBC ResultSet, Statement, or Connection.

```
0096 |                 }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0097 |             } catch (Exception e) {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0098 |                 System.out.println(e);
```

Explanation: Prints a message or value to the console, commonly for status output or error reporting.

```
0099 |             }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0100 |         }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0101 |     }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0102 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0103 |     public boolean submitRequirement(int enrollmentId, int requirementId,
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0104 |                                       String status) {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0105 |         Connection conn = null;
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0106 |         PreparedStatement ps = null;
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0107 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0108 |         try {
```

Explanation: Begins a protected block for code that may throw an exception; resources may be managed by try-with-resources.

```
0109 |             conn = db.getConnection();
```

Explanation: Assigns or updates a variable with the value or result on the right-hand side.

```
0110 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0111 |             if (conn == null) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0112 |                 return false;
```

Explanation: Returns the value or result to the caller and ends the current method.

```
0113 |             }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0114 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0115 |             String sql = "INSERT INTO submitted_requirements "
```

Explanation: Stores the SQL command as a string so the program can execute it against the database.

```
0116 |                     + "(enrollment_id, requirement_id, status) VALUES (?, ?, ?) "
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0117 |                     + "ON DUPLICATE KEY UPDATE status = VALUES(status)";
```

Explanation: Assigns or updates a variable with the value or result on the right-hand side.

```
0118 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0119 |             ps = conn.prepareStatement(sql);
```

Explanation: Creates a parameterized SQL statement; placeholders can be filled separately to reduce SQL-injection risk.

```
0120 |             ps.setInt(1, enrollmentId);
```

Explanation: Binds a Java value to a placeholder in the prepared SQL statement; the parameter index is one-based.

```
0121 |             ps.setInt(2, requirementId);
```

Explanation: Binds a Java value to a placeholder in the prepared SQL statement; the parameter index is one-based.

```
0122 |             ps.setString(3, status);
```

Explanation: Binds a Java value to a placeholder in the prepared SQL statement; the parameter index is one-based.

```
0123 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0124 |             int rowInserted = ps.executeUpdate();
```

Explanation: Executes an INSERT, UPDATE, or DELETE statement and returns the number of affected rows.

```
0125 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0126 |             return rowInserted &gt; 0;
```

Explanation: Returns the value or result to the caller and ends the current method.

```
0127 |         } catch (Exception e) {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0128 |             System.out.println("Requirement submission error: " + e.getMessage());
```

Explanation: Prints a message or value to the console, commonly for status output or error reporting.

```
0129 |             return false;
```

Explanation: Returns the value or result to the caller and ends the current method.

```
0130 |         } finally {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0131 |             try {
```

Explanation: Begins a protected block for code that may throw an exception; resources may be managed by try-with-resources.

```
0132 |                 if (ps != null) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0133 |                     ps.close();
```

Explanation: Closes the associated resource, such as a JDBC ResultSet, Statement, or Connection.

```
0134 |                 }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0135 |             } catch (Exception e) {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0136 |                 System.out.println(e);
```

Explanation: Prints a message or value to the console, commonly for status output or error reporting.

```
0137 |             }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0138 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0139 |             try {
```

Explanation: Begins a protected block for code that may throw an exception; resources may be managed by try-with-resources.

```
0140 |                 if (conn != null) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0141 |                     conn.close();
```

Explanation: Closes the associated resource, such as a JDBC ResultSet, Statement, or Connection.

```
0142 |                 }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0143 |             } catch (Exception e) {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0144 |                 System.out.println(e);
```

Explanation: Prints a message or value to the console, commonly for status output or error reporting.

```
0145 |             }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0146 |         }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0147 |     }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0148 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0149 |     public String getSubmittedRequirements(int enrollmentId) {
```

Explanation: Declares the getSubmittedRequirements method, typically returning the requested field or resource.

```
0150 |         Connection conn = null;
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0151 |         PreparedStatement ps = null;
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0152 |         ResultSet rs = null;
```

Explanation: Declares or assigns a ResultSet that holds rows returned by a database query.

```
0153 |         StringBuilder result = new StringBuilder();
```

Explanation: Creates a mutable string builder so text can be assembled efficiently.

```
0154 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0155 |         try {
```

Explanation: Begins a protected block for code that may throw an exception; resources may be managed by try-with-resources.

```
0156 |             conn = db.getConnection();
```

Explanation: Assigns or updates a variable with the value or result on the right-hand side.

```
0157 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0158 |             if (conn == null) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0159 |                 return "Error: Could not connect to the database.";
```

Explanation: Returns the value or result to the caller and ends the current method.

```
0160 |             }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0161 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0162 |             String sql = "SELECT r.requirement_name, sr.status "
```

Explanation: Stores the SQL command as a string so the program can execute it against the database.

```
0163 |                     + "FROM submitted_requirements sr "
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0164 |                     + "JOIN requirements r ON sr.requirement_id = r.requirement_id "
```

Explanation: Assigns or updates a variable with the value or result on the right-hand side.

```
0165 |                     + "WHERE sr.enrollment_id = ?";
```

Explanation: Assigns or updates a variable with the value or result on the right-hand side.

```
0166 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0167 |             ps = conn.prepareStatement(sql);
```

Explanation: Creates a parameterized SQL statement; placeholders can be filled separately to reduce SQL-injection risk.

```
0168 |             ps.setInt(1, enrollmentId);
```

Explanation: Binds a Java value to a placeholder in the prepared SQL statement; the parameter index is one-based.

```
0169 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0170 |             rs = ps.executeQuery();
```

Explanation: Executes a SQL query that returns rows, usually a SELECT, and stores them in a ResultSet.

```
0171 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0172 |             while (rs.next()) {
```

Explanation: Repeats the following block while the condition remains true.

```
0173 |                 result.append(rs.getString("requirement_name"))
```

Explanation: Reads a column value from the current database result row using the requested Java type.

```
0174 |                         .append(": ")
```

Explanation: Appends the provided value to the current StringBuilder or collection.

```
0175 |                         .append(rs.getString("status"))
```

Explanation: Reads a column value from the current database result row using the requested Java type.

```
0176 |                         .append("\n");
```

Explanation: Appends the provided value to the current StringBuilder or collection.

```
0177 |             }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0178 |         } catch (Exception e) {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0179 |             return "Error: " + e.getMessage();
```

Explanation: Returns the value or result to the caller and ends the current method.

```
0180 |         } finally {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0181 |             try {
```

Explanation: Begins a protected block for code that may throw an exception; resources may be managed by try-with-resources.

```
0182 |                 if (rs != null) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0183 |                     rs.close();
```

Explanation: Closes the associated resource, such as a JDBC ResultSet, Statement, or Connection.

```
0184 |                 }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0185 |             } catch (Exception e) {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0186 |                 System.out.println(e);
```

Explanation: Prints a message or value to the console, commonly for status output or error reporting.

```
0187 |             }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0188 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0189 |             try {
```

Explanation: Begins a protected block for code that may throw an exception; resources may be managed by try-with-resources.

```
0190 |                 if (ps != null) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0191 |                     ps.close();
```

Explanation: Closes the associated resource, such as a JDBC ResultSet, Statement, or Connection.

```
0192 |                 }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0193 |             } catch (Exception e) {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0194 |                 System.out.println(e);
```

Explanation: Prints a message or value to the console, commonly for status output or error reporting.

```
0195 |             }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0196 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0197 |             try {
```

Explanation: Begins a protected block for code that may throw an exception; resources may be managed by try-with-resources.

```
0198 |                 if (conn != null) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0199 |                     conn.close();
```

Explanation: Closes the associated resource, such as a JDBC ResultSet, Statement, or Connection.

```
0200 |                 }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0201 |             } catch (Exception e) {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0202 |                 System.out.println(e);
```

Explanation: Prints a message or value to the console, commonly for status output or error reporting.

```
0203 |             }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0204 |         }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0205 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0206 |         return result.length() == 0
```

Explanation: Returns the value or result to the caller and ends the current method.

```
0207 |                 ? "No requirement records found."
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0208 |                 : result.toString();
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0209 |     }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0210 | }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

StaffDashboard.java

Project path: src/StaffDashboard.java

```
0001 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0002 | import javax.swing.*;
```

Explanation: Imports Swing GUI classes used to build the application's windows and controls.

```
0003 | import java.awt.*;
```

Explanation: Imports AWT classes used for layout, dimensions, colors, fonts, or UI events.

```
0004 | import java.io.File;
```

Explanation: Imports a class or package so its types can be referenced in this source file.

```
0005 | import java.sql.*;
```

Explanation: Imports JDBC classes used to communicate with a relational database.

```
0006 | import java.util.ArrayList;
```

Explanation: Imports Java utility classes such as collections, lists, maps, or helper types.

```
0007 | import java.util.List;
```

Explanation: Imports Java utility classes such as collections, lists, maps, or helper types.

```
0008 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0009 | public class StaffDashboard extends JFrame {
```

Explanation: Declares the StaffDashboard type, which groups related data and behavior.

```
0010 |     private User user;
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0011 |     private DbConnection db = new DbConnection();
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0012 |     private PaymentDAO paymentDAO = new PaymentDAO();
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0013 |     private ReportDAO reportDAO = new ReportDAO();
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0014 |     private StaffEnrollmentActions enrollmentActions =
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0015 |             new StaffEnrollmentActions(this);
```

Explanation: Creates an instance of the specified class or object.

```
0016 |     private JTextArea outputArea = new JTextArea();
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0017 |     private JLabel helpLabel =
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0018 |             new JLabel("Choose a feature from the menu.");
```

Explanation: Creates an instance of the specified class or object.

```
0019 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0020 |     public StaffDashboard(User user) {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0021 |         this.user = user;
```

Explanation: Assigns or updates a variable with the value or result on the right-hand side.

```
0022 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0023 |         setTitle("Student Enrollment System - Staff Dashboard");
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0024 |         setSize(1050, 650);
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0025 |         setDefaultCloseOperation(JFrame.DISPOSE_ON_CLOSE);
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0026 |         setLocationRelativeTo(null);
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0027 |         setLayout(new BorderLayout(10, 10));
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0028 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0029 |         JLabel heading = new JLabel(
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0030 |                 "STAFF DASHBOARD", JLabel.CENTER);
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0031 |         heading.setFont(new Font("Arial", Font.BOLD, 22));
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0032 |         add(heading, BorderLayout.NORTH);
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0033 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0034 |         outputArea.setEditable(false);
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0035 |         outputArea.setFont(new Font("Monospaced", Font.PLAIN, 13));
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0036 |         add(new JScrollPane(outputArea), BorderLayout.CENTER);
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0037 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0038 |         JPanel menuPanel = new JPanel(
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0039 |                 new GridLayout(0, 1, 5, 5));
```

Explanation: Creates an instance of the specified class or object.

```
0040 |         menuPanel.setBorder(
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0041 |                 BorderFactory.createEmptyBorder(10, 10, 10, 10));
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0042 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0043 |         addButton(menuPanel, "View Enrollment Records",
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0044 |                 this::viewEnrollmentRecords);
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0045 |         addButton(menuPanel, "Update Enrollment Status",
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0046 |                 enrollmentActions::updateEnrollmentStatus);
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0047 |         addButton(menuPanel, "Assign Section",
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0048 |                 enrollmentActions::assignSection);
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0049 |         addButton(menuPanel, "Delete Enrollment Record",
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0050 |                 enrollmentActions::deleteEnrollmentRecord);
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0051 |         addButton(menuPanel, "Delete Student Requirement",
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0052 |                 this::deleteStudentRequirement);
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0053 |         addButton(menuPanel, "Review Requirement",
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0054 |                 this::reviewRequirement);
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0055 |         addButton(menuPanel, "View Discounts",
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0056 |                 this::viewDiscounts);
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0057 |         addButton(menuPanel, "Create Payment Request",
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0058 |                 enrollmentActions::createPaymentRequest);
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0059 |         addButton(menuPanel, "View Payments",
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0060 |                 this::viewPayments);
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0061 |         addButton(menuPanel, "Report: Enrollees by Strand",
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0062 |                 () -&gt; showEnrollmentReport("strand"));
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0063 |         addButton(menuPanel, "Report: Enrollees by Grade",
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0064 |                 () -&gt; showEnrollmentReport("grade"));
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0065 |         addButton(menuPanel, "Report: Enrollees by Section",
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0066 |                 () -&gt; showEnrollmentReport("section"));
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0067 |         addButton(menuPanel, "Incomplete Requirements",
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0068 |                 this::showIncompleteRequirements);
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0069 |         addButton(menuPanel, "Dashboard Guide",
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0070 |                 this::showGuide);
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0071 |         addButton(menuPanel, "Logout", this::logout);
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0072 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0073 |         add(new JScrollPane(menuPanel), BorderLayout.WEST);
```

Explanation: Adds a component to a Swing container so it becomes part of the displayed interface.

```
0074 |         add(helpLabel, BorderLayout.SOUTH);
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0075 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0076 |         viewEnrollmentRecords();
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0077 |     }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0078 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0079 |     private void addButton(
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0080 |             JPanel panel, String text, Runnable action) {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0081 |         JButton button = new JButton(text);
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0082 |         button.addActionListener(e -&gt; action.run());
```

Explanation: Registers an event handler that runs when the associated GUI control is activated.

```
0083 |         panel.add(button);
```

Explanation: Adds a component to a Swing container so it becomes part of the displayed interface.

```
0084 |     }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0085 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0086 |     public void viewEnrollmentRecords() {
```

Explanation: Declares the viewEnrollmentRecords method and its return type/parameters; the method performs the operation described by its name and body.

```
0087 |         String sql =
```

Explanation: Stores the SQL command as a string so the program can execute it against the database.

```
0088 |                 "SELECT e.enrollment_id, st.student_id, st.lrn, "
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0089 |                         + "st.first_name, st.last_name, "
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0090 |                         + "e.year_level, str.strand_code, "
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0091 |                         + "sy.year_label, sec.section_name, "
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0092 |                         + "e.enrollment_status "
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0093 |                         + "FROM enrollments e "
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0094 |                         + "JOIN students st ON e.student_id = st.student_id "
```

Explanation: Assigns or updates a variable with the value or result on the right-hand side.

```
0095 |                         + "JOIN strands str ON e.strand_id = str.strand_id "
```

Explanation: Assigns or updates a variable with the value or result on the right-hand side.

```
0096 |                         + "JOIN school_years sy "
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0097 |                         + "ON e.school_year_id = sy.school_year_id "
```

Explanation: Assigns or updates a variable with the value or result on the right-hand side.

```
0098 |                         + "LEFT JOIN sections sec "
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0099 |                         + "ON e.section_id = sec.section_id "
```

Explanation: Assigns or updates a variable with the value or result on the right-hand side.

```
0100 |                         + "WHERE e.year_level IN (11, 12) "
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0101 |                         + "ORDER BY e.enrollment_id DESC";
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0102 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0103 |         StringBuilder result =
```

Explanation: Creates a mutable string builder so text can be assembled efficiently.

```
0104 |                 new StringBuilder("SHS ENROLLMENT RECORDS\n\n");
```

Explanation: Creates an instance of the specified class or object.

```
0105 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0106 |         Connection conn = null;
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0107 |         PreparedStatement ps = null;
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0108 |         ResultSet rs = null;
```

Explanation: Declares or assigns a ResultSet that holds rows returned by a database query.

```
0109 |         try {
```

Explanation: Begins a protected block for code that may throw an exception; resources may be managed by try-with-resources.

```
0110 |             conn = db.getConnection();
```

Explanation: Assigns or updates a variable with the value or result on the right-hand side.

```
0111 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0112 |             if (conn == null) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0113 |                 throw new SQLException("Could not connect to the database.");
```

Explanation: Raises an exception so the caller or an enclosing catch block can handle the failure.

```
0114 |             }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0115 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0116 |             ps = conn.prepareStatement(sql);
```

Explanation: Creates a parameterized SQL statement; placeholders can be filled separately to reduce SQL-injection risk.

```
0117 |             rs = ps.executeQuery();
```

Explanation: Executes a SQL query that returns rows, usually a SELECT, and stores them in a ResultSet.

```
0118 |             boolean found = false;
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0119 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0120 |             while (rs.next()) {
```

Explanation: Repeats the following block while the condition remains true.

```
0121 |                 found = true;
```

Explanation: Assigns or updates a variable with the value or result on the right-hand side.

```
0122 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0123 |                 String section = rs.getString("section_name");
```

Explanation: Reads a column value from the current database result row using the requested Java type.

```
0124 |                 if (section == null) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0125 |                     section = "Not Assigned";
```

Explanation: Assigns or updates a variable with the value or result on the right-hand side.

```
0126 |                 }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0127 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0128 |                 result.append("Enrollment ID: ")
```

Explanation: Appends the provided value to the current StringBuilder or collection.

```
0129 |                         .append(rs.getInt("enrollment_id"))
```

Explanation: Reads a column value from the current database result row using the requested Java type.

```
0130 |                         .append("\nStudent ID: ")
```

Explanation: Appends the provided value to the current StringBuilder or collection.

```
0131 |                         .append(rs.getInt("student_id"))
```

Explanation: Reads a column value from the current database result row using the requested Java type.

```
0132 |                         .append("\nLRN: ")
```

Explanation: Appends the provided value to the current StringBuilder or collection.

```
0133 |                         .append(rs.getString("lrn"))
```

Explanation: Reads a column value from the current database result row using the requested Java type.

```
0134 |                         .append("\nStudent: ")
```

Explanation: Appends the provided value to the current StringBuilder or collection.

```
0135 |                         .append(rs.getString("first_name"))
```

Explanation: Reads a column value from the current database result row using the requested Java type.

```
0136 |                         .append(" ")
```

Explanation: Appends the provided value to the current StringBuilder or collection.

```
0137 |                         .append(rs.getString("last_name"))
```

Explanation: Reads a column value from the current database result row using the requested Java type.

```
0138 |                         .append("\nGrade: ")
```

Explanation: Appends the provided value to the current StringBuilder or collection.

```
0139 |                         .append(rs.getInt("year_level"))
```

Explanation: Reads a column value from the current database result row using the requested Java type.

```
0140 |                         .append("\nStrand: ")
```

Explanation: Appends the provided value to the current StringBuilder or collection.

```
0141 |                         .append(rs.getString("strand_code"))
```

Explanation: Reads a column value from the current database result row using the requested Java type.

```
0142 |                         .append("\nSchool Year: ")
```

Explanation: Appends the provided value to the current StringBuilder or collection.

```
0143 |                         .append(rs.getString("year_label"))
```

Explanation: Reads a column value from the current database result row using the requested Java type.

```
0144 |                         .append("\nSection: ")
```

Explanation: Appends the provided value to the current StringBuilder or collection.

```
0145 |                         .append(section)
```

Explanation: Appends the provided value to the current StringBuilder or collection.

```
0146 |                         .append("\nStatus: ")
```

Explanation: Appends the provided value to the current StringBuilder or collection.

```
0147 |                         .append(rs.getString("enrollment_status"))
```

Explanation: Reads a column value from the current database result row using the requested Java type.

```
0148 |                         .append("\n----------------------------------------\n");
```

Explanation: Appends the provided value to the current StringBuilder or collection.

```
0149 |             }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0150 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0151 |             if (!found) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0152 |                 result.append("No SHS enrollment records found.");
```

Explanation: Appends the provided value to the current StringBuilder or collection.

```
0153 |             }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0154 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0155 |             outputArea.setText(result.toString());
```

Explanation: Sets the displayed text of a GUI component.

```
0156 |             helpLabel.setText("Enrollment records loaded.");
```

Explanation: Sets the displayed text of a GUI component.

```
0157 |         } catch (Exception ex) {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0158 |             showError("Could not load enrollment records: "
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0159 |                     + ex.getMessage());
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0160 |         } finally {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0161 |             try {
```

Explanation: Begins a protected block for code that may throw an exception; resources may be managed by try-with-resources.

```
0162 |                 if (rs != null) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0163 |                     rs.close();
```

Explanation: Closes the associated resource, such as a JDBC ResultSet, Statement, or Connection.

```
0164 |                 }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0165 |             } catch (Exception closeEx) {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0166 |                 System.out.println(closeEx);
```

Explanation: Prints a message or value to the console, commonly for status output or error reporting.

```
0167 |             }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0168 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0169 |             try {
```

Explanation: Begins a protected block for code that may throw an exception; resources may be managed by try-with-resources.

```
0170 |                 if (ps != null) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0171 |                     ps.close();
```

Explanation: Closes the associated resource, such as a JDBC ResultSet, Statement, or Connection.

```
0172 |                 }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0173 |             } catch (Exception closeEx) {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0174 |                 System.out.println(closeEx);
```

Explanation: Prints a message or value to the console, commonly for status output or error reporting.

```
0175 |             }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0176 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0177 |             try {
```

Explanation: Begins a protected block for code that may throw an exception; resources may be managed by try-with-resources.

```
0178 |                 if (conn != null) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0179 |                     conn.close();
```

Explanation: Closes the associated resource, such as a JDBC ResultSet, Statement, or Connection.

```
0180 |                 }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0181 |             } catch (Exception closeEx) {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0182 |                 System.out.println(closeEx);
```

Explanation: Prints a message or value to the console, commonly for status output or error reporting.

```
0183 |             }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0184 |         }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0185 |     }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0186 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0187 |     private void deleteStudentRequirement() {
```

Explanation: Declares the deleteStudentRequirement method and its return type/parameters; the method performs the operation described by its name and body.

```
0188 |         String input = JOptionPane.showInputDialog(
```

Explanation: Displays a Swing dialog to communicate a message, warning, confirmation, or input prompt to the user.

```
0189 |                 this, "Enter the Requirement ID to delete:");
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0190 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0191 |         if (input == null) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0192 |             return;
```

Explanation: Ends the current method without returning a value.

```
0193 |         }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0194 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0195 |         try {
```

Explanation: Begins a protected block for code that may throw an exception; resources may be managed by try-with-resources.

```
0196 |             int id = Integer.parseInt(input.trim());
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0197 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0198 |             int choice = JOptionPane.showConfirmDialog(
```

Explanation: Displays a Swing dialog to communicate a message, warning, confirmation, or input prompt to the user.

```
0199 |                     this,
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0200 |                     "Delete this requirement record and its file?",
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0201 |                     "Confirm Deletion",
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0202 |                     JOptionPane.YES_NO_OPTION);
```

Explanation: Displays a Swing dialog to communicate a message, warning, confirmation, or input prompt to the user.

```
0203 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0204 |             if (choice != JOptionPane.YES_OPTION) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0205 |                 return;
```

Explanation: Ends the current method without returning a value.

```
0206 |             }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0207 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0208 |             String selectSql =
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0209 |                     "SELECT sr.file_path "
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0210 |                             + "FROM student_requirements sr "
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0211 |                             + "JOIN enrollments e "
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0212 |                             + "ON sr.enrollment_id = e.enrollment_id "
```

Explanation: Assigns or updates a variable with the value or result on the right-hand side.

```
0213 |                             + "WHERE sr.requirement_id = ? "
```

Explanation: Assigns or updates a variable with the value or result on the right-hand side.

```
0214 |                             + "AND e.year_level IN (11, 12)";
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0215 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0216 |             String path;
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0217 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0218 |             Connection conn = null;
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0219 |             PreparedStatement ps = null;
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0220 |             try {
```

Explanation: Begins a protected block for code that may throw an exception; resources may be managed by try-with-resources.

```
0221 |                 conn = db.getConnection();
```

Explanation: Assigns or updates a variable with the value or result on the right-hand side.

```
0222 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0223 |                 if (conn == null) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0224 |                     throw new SQLException("Could not connect to the database.");
```

Explanation: Raises an exception so the caller or an enclosing catch block can handle the failure.

```
0225 |                 }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0226 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0227 |                 ps = conn.prepareStatement(selectSql);
```

Explanation: Creates a parameterized SQL statement; placeholders can be filled separately to reduce SQL-injection risk.

```
0228 |                 ps.setInt(1, id);
```

Explanation: Binds a Java value to a placeholder in the prepared SQL statement; the parameter index is one-based.

```
0229 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0230 |                 try (ResultSet rs = ps.executeQuery()) {
```

Explanation: Begins a protected block for code that may throw an exception; resources may be managed by try-with-resources.

```
0231 |                     if (!rs.next()) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0232 |                         showError("Requirement not found.");
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0233 |                         return;
```

Explanation: Ends the current method without returning a value.

```
0234 |                     }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0235 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0236 |                     path = rs.getString("file_path");
```

Explanation: Reads a column value from the current database result row using the requested Java type.

```
0237 |                 }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0238 |             } finally {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0239 |                 try {
```

Explanation: Begins a protected block for code that may throw an exception; resources may be managed by try-with-resources.

```
0240 |                     if (ps != null) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0241 |                         ps.close();
```

Explanation: Closes the associated resource, such as a JDBC ResultSet, Statement, or Connection.

```
0242 |                     }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0243 |                 } catch (Exception closeEx) {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0244 |                     System.out.println(closeEx);
```

Explanation: Prints a message or value to the console, commonly for status output or error reporting.

```
0245 |                 }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0246 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0247 |                 try {
```

Explanation: Begins a protected block for code that may throw an exception; resources may be managed by try-with-resources.

```
0248 |                     if (conn != null) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0249 |                         conn.close();
```

Explanation: Closes the associated resource, such as a JDBC ResultSet, Statement, or Connection.

```
0250 |                     }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0251 |                 } catch (Exception closeEx) {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0252 |                     System.out.println(closeEx);
```

Explanation: Prints a message or value to the console, commonly for status output or error reporting.

```
0253 |                 }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0254 |             }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0255 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0256 |             String deleteSql =
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0257 |                     "DELETE sr FROM student_requirements sr "
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0258 |                             + "JOIN enrollments e "
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0259 |                             + "ON sr.enrollment_id = e.enrollment_id "
```

Explanation: Assigns or updates a variable with the value or result on the right-hand side.

```
0260 |                             + "WHERE sr.requirement_id = ? "
```

Explanation: Assigns or updates a variable with the value or result on the right-hand side.

```
0261 |                             + "AND e.year_level IN (11, 12)";
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0262 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0263 |             conn = null;
```

Explanation: Assigns or updates a variable with the value or result on the right-hand side.

```
0264 |             ps = null;
```

Explanation: Assigns or updates a variable with the value or result on the right-hand side.

```
0265 |             try {
```

Explanation: Begins a protected block for code that may throw an exception; resources may be managed by try-with-resources.

```
0266 |                 conn = db.getConnection();
```

Explanation: Assigns or updates a variable with the value or result on the right-hand side.

```
0267 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0268 |                 if (conn == null) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0269 |                     throw new SQLException("Could not connect to the database.");
```

Explanation: Raises an exception so the caller or an enclosing catch block can handle the failure.

```
0270 |                 }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0271 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0272 |                 ps = conn.prepareStatement(deleteSql);
```

Explanation: Creates a parameterized SQL statement; placeholders can be filled separately to reduce SQL-injection risk.

```
0273 |                 ps.setInt(1, id);
```

Explanation: Binds a Java value to a placeholder in the prepared SQL statement; the parameter index is one-based.

```
0274 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0275 |                 if (ps.executeUpdate() == 0) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0276 |                     showError("Requirement was not deleted.");
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0277 |                     return;
```

Explanation: Ends the current method without returning a value.

```
0278 |                 }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0279 |             } finally {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0280 |                 try {
```

Explanation: Begins a protected block for code that may throw an exception; resources may be managed by try-with-resources.

```
0281 |                     if (ps != null) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0282 |                         ps.close();
```

Explanation: Closes the associated resource, such as a JDBC ResultSet, Statement, or Connection.

```
0283 |                     }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0284 |                 } catch (Exception closeEx) {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0285 |                     System.out.println(closeEx);
```

Explanation: Prints a message or value to the console, commonly for status output or error reporting.

```
0286 |                 }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0287 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0288 |                 try {
```

Explanation: Begins a protected block for code that may throw an exception; resources may be managed by try-with-resources.

```
0289 |                     if (conn != null) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0290 |                         conn.close();
```

Explanation: Closes the associated resource, such as a JDBC ResultSet, Statement, or Connection.

```
0291 |                     }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0292 |                 } catch (Exception closeEx) {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0293 |                     System.out.println(closeEx);
```

Explanation: Prints a message or value to the console, commonly for status output or error reporting.

```
0294 |                 }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0295 |             }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0296 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0297 |             if (path != null &amp;amp;&amp;amp; !path.trim().isEmpty()) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0298 |                 File file = new File(path);
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0299 |                 if (file.isFile()) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0300 |                     file.delete();
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0301 |                 }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0302 |             }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0303 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0304 |             JOptionPane.showMessageDialog(
```

Explanation: Displays a Swing dialog to communicate a message, warning, confirmation, or input prompt to the user.

```
0305 |                     this, "Requirement record deleted.");
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0306 |             viewEnrollmentRecords();
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0307 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0308 |         } catch (NumberFormatException ex) {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0309 |             showError("Requirement ID must be numeric.");
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0310 |         } catch (SQLException ex) {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0311 |             showError("Could not delete requirement: "
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0312 |                     + ex.getMessage());
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0313 |         }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0314 |     }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0315 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0316 |     private void reviewRequirement() {
```

Explanation: Declares the reviewRequirement method and its return type/parameters; the method performs the operation described by its name and body.

```
0317 |         String input = JOptionPane.showInputDialog(
```

Explanation: Displays a Swing dialog to communicate a message, warning, confirmation, or input prompt to the user.

```
0318 |                 this, "Enter the Requirement ID to review:\n"
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0319 |                         + "(Use Incomplete Requirements to see the IDs.)");
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0320 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0321 |         if (input == null || input.trim().isEmpty()) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0322 |             return;
```

Explanation: Ends the current method without returning a value.

```
0323 |         }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0324 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0325 |         int id;
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0326 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0327 |         try {
```

Explanation: Begins a protected block for code that may throw an exception; resources may be managed by try-with-resources.

```
0328 |             id = Integer.parseInt(input.trim());
```

Explanation: Assigns or updates a variable with the value or result on the right-hand side.

```
0329 |         } catch (NumberFormatException ex) {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0330 |             showError("Requirement ID must be numeric.");
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0331 |             return;
```

Explanation: Ends the current method without returning a value.

```
0332 |         }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0333 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0334 |         String[] statuses = {"Complete", "Incomplete"};
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0335 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0336 |         String status = (String) JOptionPane.showInputDialog(
```

Explanation: Displays a Swing dialog to communicate a message, warning, confirmation, or input prompt to the user.

```
0337 |                 this,
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0338 |                 "Mark this requirement as:",
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0339 |                 "Review Requirement",
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0340 |                 JOptionPane.QUESTION_MESSAGE,
```

Explanation: Displays a Swing dialog to communicate a message, warning, confirmation, or input prompt to the user.

```
0341 |                 null,
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0342 |                 statuses,
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0343 |                 statuses[0]);
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0344 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0345 |         if (status == null) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0346 |             return;
```

Explanation: Ends the current method without returning a value.

```
0347 |         }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0348 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0349 |         String sql =
```

Explanation: Stores the SQL command as a string so the program can execute it against the database.

```
0350 |                 "UPDATE student_requirements sr "
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0351 |                         + "JOIN enrollments e "
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0352 |                         + "ON sr.enrollment_id = e.enrollment_id "
```

Explanation: Assigns or updates a variable with the value or result on the right-hand side.

```
0353 |                         + "SET sr.requirement_status = ? "
```

Explanation: Assigns or updates a variable with the value or result on the right-hand side.

```
0354 |                         + "WHERE sr.requirement_id = ? "
```

Explanation: Assigns or updates a variable with the value or result on the right-hand side.

```
0355 |                         + "AND e.year_level IN (11, 12)";
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0356 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0357 |         Connection conn = null;
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0358 |         PreparedStatement ps = null;
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0359 |         try {
```

Explanation: Begins a protected block for code that may throw an exception; resources may be managed by try-with-resources.

```
0360 |             conn = db.getConnection();
```

Explanation: Assigns or updates a variable with the value or result on the right-hand side.

```
0361 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0362 |             if (conn == null) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0363 |                 throw new SQLException("Could not connect to the database.");
```

Explanation: Raises an exception so the caller or an enclosing catch block can handle the failure.

```
0364 |             }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0365 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0366 |             ps = conn.prepareStatement(sql);
```

Explanation: Creates a parameterized SQL statement; placeholders can be filled separately to reduce SQL-injection risk.

```
0367 |             ps.setString(1, status);
```

Explanation: Binds a Java value to a placeholder in the prepared SQL statement; the parameter index is one-based.

```
0368 |             ps.setInt(2, id);
```

Explanation: Binds a Java value to a placeholder in the prepared SQL statement; the parameter index is one-based.

```
0369 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0370 |             if (ps.executeUpdate() &gt; 0) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0371 |                 JOptionPane.showMessageDialog(
```

Explanation: Displays a Swing dialog to communicate a message, warning, confirmation, or input prompt to the user.

```
0372 |                         this, "Requirement marked as " + status + ".");
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0373 |                 showIncompleteRequirements();
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0374 |             } else {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0375 |                 showError("Requirement not found.");
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0376 |             }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0377 |         } catch (Exception ex) {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0378 |             showError("Could not update requirement: "
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0379 |                     + ex.getMessage());
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0380 |         } finally {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0381 |             try {
```

Explanation: Begins a protected block for code that may throw an exception; resources may be managed by try-with-resources.

```
0382 |                 if (ps != null) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0383 |                     ps.close();
```

Explanation: Closes the associated resource, such as a JDBC ResultSet, Statement, or Connection.

```
0384 |                 }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0385 |             } catch (Exception closeEx) {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0386 |                 System.out.println(closeEx);
```

Explanation: Prints a message or value to the console, commonly for status output or error reporting.

```
0387 |             }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0388 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0389 |             try {
```

Explanation: Begins a protected block for code that may throw an exception; resources may be managed by try-with-resources.

```
0390 |                 if (conn != null) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0391 |                     conn.close();
```

Explanation: Closes the associated resource, such as a JDBC ResultSet, Statement, or Connection.

```
0392 |                 }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0393 |             } catch (Exception closeEx) {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0394 |                 System.out.println(closeEx);
```

Explanation: Prints a message or value to the console, commonly for status output or error reporting.

```
0395 |             }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0396 |         }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0397 |     }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0398 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0399 |     private void viewDiscounts() {
```

Explanation: Declares the viewDiscounts method and its return type/parameters; the method performs the operation described by its name and body.

```
0400 |         try {
```

Explanation: Begins a protected block for code that may throw an exception; resources may be managed by try-with-resources.

```
0401 |             outputArea.setText(
```

Explanation: Sets the displayed text of a GUI component.

```
0402 |                     "DISCOUNTS\n\n" + paymentDAO.getDiscounts());
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0403 |         } catch (Exception ex) {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0404 |             showError("Could not load discounts: " + ex.getMessage());
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0405 |         }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0406 |     }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0407 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0408 |     private void viewPayments() {
```

Explanation: Declares the viewPayments method and its return type/parameters; the method performs the operation described by its name and body.

```
0409 |         try {
```

Explanation: Begins a protected block for code that may throw an exception; resources may be managed by try-with-resources.

```
0410 |             outputArea.setText(
```

Explanation: Sets the displayed text of a GUI component.

```
0411 |                     "PAYMENT RECORDS\n\n" + paymentDAO.getPayments());
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0412 |         } catch (Exception ex) {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0413 |             showError("Could not load payments: " + ex.getMessage());
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0414 |         }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0415 |     }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0416 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0417 |     private void showEnrollmentReport(String type) {
```

Explanation: Declares the showEnrollmentReport method and its return type/parameters; the method performs the operation described by its name and body.

```
0418 |         try {
```

Explanation: Begins a protected block for code that may throw an exception; resources may be managed by try-with-resources.

```
0419 |             outputArea.setText(reportDAO.getEnrollmentCountReport(type));
```

Explanation: Sets the displayed text of a GUI component.

```
0420 |         } catch (SQLException ex) {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0421 |             showError("Could not generate report: " + ex.getMessage());
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0422 |         }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0423 |     }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0424 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0425 |     private void showIncompleteRequirements() {
```

Explanation: Declares the showIncompleteRequirements method and its return type/parameters; the method performs the operation described by its name and body.

```
0426 |         try {
```

Explanation: Begins a protected block for code that may throw an exception; resources may be managed by try-with-resources.

```
0427 |             outputArea.setText(reportDAO.getIncompleteRequirementsReport());
```

Explanation: Sets the displayed text of a GUI component.

```
0428 |         } catch (SQLException ex) {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0429 |             showError("Could not load incomplete requirements: "
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0430 |                     + ex.getMessage());
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0431 |         }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0432 |     }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0433 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0434 |     private void showGuide() {
```

Explanation: Declares the showGuide method and its return type/parameters; the method performs the operation described by its name and body.

```
0435 |         outputArea.setText(
```

Explanation: Sets the displayed text of a GUI component.

```
0436 |                 "STAFF DASHBOARD GUIDE\n\n"
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0437 |                         + "View enrollment records for Grade 11 and 12.\n"
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0438 |                         + "Update status using the allowed options.\n"
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0439 |                         + "Review submitted requirements (Complete or Incomplete).\n"
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0440 |                         + "Assign students to matching active sections.\n"
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0441 |                         + "Review payments and discounts.\n"
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0442 |                         + "Generate enrollment reports by strand, grade, "
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0443 |                         + "or section.\n"
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0444 |                         + "Review students with incomplete requirements.");
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0445 |     }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0446 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0447 |     private void logout() {
```

Explanation: Declares the logout method and its return type/parameters; the method performs the operation described by its name and body.

```
0448 |         int choice = JOptionPane.showConfirmDialog(
```

Explanation: Displays a Swing dialog to communicate a message, warning, confirmation, or input prompt to the user.

```
0449 |                 this, "Are you sure you want to log out?",
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0450 |                 "Confirm Logout", JOptionPane.YES_NO_OPTION);
```

Explanation: Displays a Swing dialog to communicate a message, warning, confirmation, or input prompt to the user.

```
0451 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0452 |         if (choice == JOptionPane.YES_OPTION) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0453 |             dispose();
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0454 |             new LoginFrame().setVisible(true);
```

Explanation: Creates an instance of the specified class or object.

```
0455 |         }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0456 |     }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0457 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0458 |     public static void showError(java.awt.Component parent, String message) {
```

Explanation: Declares the showError method and its return type/parameters; the method performs the operation described by its name and body.

```
0459 |         JOptionPane.showMessageDialog(
```

Explanation: Displays a Swing dialog to communicate a message, warning, confirmation, or input prompt to the user.

```
0460 |                 parent, message, "Staff Dashboard",
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0461 |                 JOptionPane.ERROR_MESSAGE);
```

Explanation: Displays a Swing dialog to communicate a message, warning, confirmation, or input prompt to the user.

```
0462 |     }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0463 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0464 |     private void showError(String message) {
```

Explanation: Declares the showError method and its return type/parameters; the method performs the operation described by its name and body.

```
0465 |         showError(this, message);
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0466 |     }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0467 | }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

StaffEnrollmentActions.java

Project path: src/StaffEnrollmentActions.java

```
0001 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0002 | import javax.swing.*;
```

Explanation: Imports Swing GUI classes used to build the application's windows and controls.

```
0003 | import java.sql.*;
```

Explanation: Imports JDBC classes used to communicate with a relational database.

```
0004 | import java.util.ArrayList;
```

Explanation: Imports Java utility classes such as collections, lists, maps, or helper types.

```
0005 | import java.util.List;
```

Explanation: Imports Java utility classes such as collections, lists, maps, or helper types.

```
0006 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0007 | public class StaffEnrollmentActions {
```

Explanation: Declares the StaffEnrollmentActions type, which groups related data and behavior.

```
0008 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0009 |     private StaffDashboard dashboard;
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0010 |     private DbConnection db = new DbConnection();
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0011 |     private EnrollmentDAO enrollmentDAO = new EnrollmentDAO();
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0012 |     private PaymentDAO paymentDAO = new PaymentDAO();
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0013 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0014 |     public StaffEnrollmentActions(StaffDashboard dashboard) {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0015 |         this.dashboard = dashboard;
```

Explanation: Assigns or updates a variable with the value or result on the right-hand side.

```
0016 |     }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0017 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0018 |     public void updateEnrollmentStatus() {
```

Explanation: Declares the updateEnrollmentStatus method and its return type/parameters; the method performs the operation described by its name and body.

```
0019 |         EnrollmentOption enrollment = selectEnrollment();
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0020 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0021 |         if (enrollment == null) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0022 |             return;
```

Explanation: Ends the current method without returning a value.

```
0023 |         }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0024 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0025 |         String[] statuses = {
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0026 |                 "Pending", "Enrolled", "Rejected"
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0027 |         };
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0028 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0029 |         String status = (String) JOptionPane.showInputDialog(
```

Explanation: Displays a Swing dialog to communicate a message, warning, confirmation, or input prompt to the user.

```
0030 |                 dashboard,
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0031 |                 "Select the new enrollment status:",
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0032 |                 "Update Enrollment Status",
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0033 |                 JOptionPane.QUESTION_MESSAGE,
```

Explanation: Displays a Swing dialog to communicate a message, warning, confirmation, or input prompt to the user.

```
0034 |                 null,
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0035 |                 statuses,
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0036 |                 statuses[0]);
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0037 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0038 |         if (status == null) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0039 |             return;
```

Explanation: Ends the current method without returning a value.

```
0040 |         }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0041 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0042 |         if (status.equals("Enrolled") &amp;&amp; enrollment.sectionId == null) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0043 |             StaffDashboard.showError(dashboard,
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0044 |                     "Assign a section first, then mark the student as Enrolled.");
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0045 |             return;
```

Explanation: Ends the current method without returning a value.

```
0046 |         }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0047 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0048 |         String sql =
```

Explanation: Stores the SQL command as a string so the program can execute it against the database.

```
0049 |                 "UPDATE enrollments SET enrollment_status = ? "
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0050 |                         + "WHERE enrollment_id = ? "
```

Explanation: Assigns or updates a variable with the value or result on the right-hand side.

```
0051 |                         + "AND year_level IN (11, 12)";
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0052 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0053 |         Connection conn = null;
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0054 |         PreparedStatement ps = null;
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0055 |         try {
```

Explanation: Begins a protected block for code that may throw an exception; resources may be managed by try-with-resources.

```
0056 |             conn = db.getConnection();
```

Explanation: Assigns or updates a variable with the value or result on the right-hand side.

```
0057 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0058 |             if (conn == null) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0059 |                 throw new SQLException("Could not connect to the database.");
```

Explanation: Raises an exception so the caller or an enclosing catch block can handle the failure.

```
0060 |             }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0061 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0062 |             ps = conn.prepareStatement(sql);
```

Explanation: Creates a parameterized SQL statement; placeholders can be filled separately to reduce SQL-injection risk.

```
0063 |             ps.setString(1, status);
```

Explanation: Binds a Java value to a placeholder in the prepared SQL statement; the parameter index is one-based.

```
0064 |             ps.setInt(2, enrollment.enrollmentId);
```

Explanation: Binds a Java value to a placeholder in the prepared SQL statement; the parameter index is one-based.

```
0065 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0066 |             if (ps.executeUpdate() &gt; 0) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0067 |                 JOptionPane.showMessageDialog(
```

Explanation: Displays a Swing dialog to communicate a message, warning, confirmation, or input prompt to the user.

```
0068 |                         dashboard, "Enrollment status updated.");
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0069 |                 dashboard.viewEnrollmentRecords();
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0070 |             } else {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0071 |                 StaffDashboard.showError(dashboard, "Enrollment record was not updated.");
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0072 |             }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0073 |         } catch (Exception ex) {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0074 |             StaffDashboard.showError(dashboard, "Could not update status: "
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0075 |                     + ex.getMessage()
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0076 |                     + "\nCheck the allowed values of "
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0077 |                     + "enrollments_chk_2.");
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0078 |         } finally {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0079 |             try {
```

Explanation: Begins a protected block for code that may throw an exception; resources may be managed by try-with-resources.

```
0080 |                 if (ps != null) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0081 |                     ps.close();
```

Explanation: Closes the associated resource, such as a JDBC ResultSet, Statement, or Connection.

```
0082 |                 }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0083 |             } catch (Exception closeEx) {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0084 |                 System.out.println(closeEx);
```

Explanation: Prints a message or value to the console, commonly for status output or error reporting.

```
0085 |             }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0086 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0087 |             try {
```

Explanation: Begins a protected block for code that may throw an exception; resources may be managed by try-with-resources.

```
0088 |                 if (conn != null) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0089 |                     conn.close();
```

Explanation: Closes the associated resource, such as a JDBC ResultSet, Statement, or Connection.

```
0090 |                 }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0091 |             } catch (Exception closeEx) {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0092 |                 System.out.println(closeEx);
```

Explanation: Prints a message or value to the console, commonly for status output or error reporting.

```
0093 |             }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0094 |         }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0095 |     }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0096 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0097 |     public void assignSection() {
```

Explanation: Declares the assignSection method and its return type/parameters; the method performs the operation described by its name and body.

```
0098 |         EnrollmentOption enrollment = selectEnrollment();
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0099 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0100 |         if (enrollment == null) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0101 |             return;
```

Explanation: Ends the current method without returning a value.

```
0102 |         }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0103 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0104 |         List&lt;EnrollmentDAO.SectionOption&gt; sections;
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0105 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0106 |         try {
```

Explanation: Begins a protected block for code that may throw an exception; resources may be managed by try-with-resources.

```
0107 |             sections = enrollmentDAO.getAvailableSections(
```

Explanation: Assigns or updates a variable with the value or result on the right-hand side.

```
0108 |                     enrollment.yearLevel, enrollment.strandId,
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0109 |                     enrollment.schoolYearId, enrollment.sectionId);
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0110 |         } catch (SQLException ex) {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0111 |             StaffDashboard.showError(dashboard,
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0112 |                     "Could not load sections: " + ex.getMessage());
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0113 |             return;
```

Explanation: Ends the current method without returning a value.

```
0114 |         }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0115 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0116 |         if (sections.isEmpty()) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0117 |             StaffDashboard.showError(dashboard,
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0118 |                     "No active sections with available slots "
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0119 |                     + "match this student's grade, strand, "
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0120 |                     + "and school year.");
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0121 |             return;
```

Explanation: Ends the current method without returning a value.

```
0122 |         }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0123 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0124 |         JComboBox&lt;EnrollmentDAO.SectionOption&gt; sectionBox = new JComboBox&lt;&gt;();
```

Explanation: Assigns or updates a variable with the value or result on the right-hand side.

```
0125 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0126 |         for (EnrollmentDAO.SectionOption section : sections) {
```

Explanation: Starts a loop that repeats while its loop condition allows; initialization and update control iteration.

```
0127 |             sectionBox.addItem(section);
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0128 |         }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0129 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0130 |         int choice = JOptionPane.showConfirmDialog(
```

Explanation: Displays a Swing dialog to communicate a message, warning, confirmation, or input prompt to the user.

```
0131 |                 dashboard, sectionBox,
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0132 |                 "Assign Section to " + enrollment.studentName,
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0133 |                 JOptionPane.OK_CANCEL_OPTION);
```

Explanation: Displays a Swing dialog to communicate a message, warning, confirmation, or input prompt to the user.

```
0134 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0135 |         if (choice != JOptionPane.OK_OPTION) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0136 |             return;
```

Explanation: Ends the current method without returning a value.

```
0137 |         }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0138 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0139 |         EnrollmentDAO.SectionOption selected =
```

Explanation: Assigns or updates a variable with the value or result on the right-hand side.

```
0140 |                 (EnrollmentDAO.SectionOption) sectionBox.getSelectedItem();
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0141 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0142 |         if (selected == null) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0143 |             return;
```

Explanation: Ends the current method without returning a value.

```
0144 |         }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0145 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0146 |         try {
```

Explanation: Begins a protected block for code that may throw an exception; resources may be managed by try-with-resources.

```
0147 |             enrollmentDAO.assignSection(
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0148 |                     enrollment.enrollmentId, selected.sectionId,
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0149 |                     enrollment.yearLevel, enrollment.strandId,
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0150 |                     enrollment.schoolYearId);
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0151 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0152 |             JOptionPane.showMessageDialog(
```

Explanation: Displays a Swing dialog to communicate a message, warning, confirmation, or input prompt to the user.

```
0153 |                     dashboard, "Section assigned successfully.");
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0154 |             dashboard.viewEnrollmentRecords();
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0155 |         } catch (SQLException ex) {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0156 |             StaffDashboard.showError(dashboard,
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0157 |                     "Could not assign section: " + ex.getMessage());
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0158 |         }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0159 |     }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0160 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0161 |     public EnrollmentOption selectEnrollment() {
```

Explanation: Declares the selectEnrollment method and its return type/parameters; the method performs the operation described by its name and body.

```
0162 |         List&lt;EnrollmentOption&gt; enrollments =
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0163 |                 loadEnrollmentOptions();
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0164 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0165 |         if (enrollments.isEmpty()) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0166 |             StaffDashboard.showError(dashboard, "No SHS enrollment records are available.");
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0167 |             return null;
```

Explanation: Returns the value or result to the caller and ends the current method.

```
0168 |         }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0169 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0170 |         JComboBox&lt;EnrollmentOption&gt; enrollmentBox =
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0171 |                 new JComboBox&lt;&gt;();
```

Explanation: Creates an instance of the specified class or object.

```
0172 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0173 |         for (EnrollmentOption enrollment : enrollments) {
```

Explanation: Starts a loop that repeats while its loop condition allows; initialization and update control iteration.

```
0174 |             enrollmentBox.addItem(enrollment);
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0175 |         }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0176 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0177 |         int choice = JOptionPane.showConfirmDialog(
```

Explanation: Displays a Swing dialog to communicate a message, warning, confirmation, or input prompt to the user.

```
0178 |                 dashboard, enrollmentBox, "Select Enrollment",
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0179 |                 JOptionPane.OK_CANCEL_OPTION);
```

Explanation: Displays a Swing dialog to communicate a message, warning, confirmation, or input prompt to the user.

```
0180 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0181 |         if (choice != JOptionPane.OK_OPTION) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0182 |             return null;
```

Explanation: Returns the value or result to the caller and ends the current method.

```
0183 |         }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0184 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0185 |         return (EnrollmentOption) enrollmentBox.getSelectedItem();
```

Explanation: Returns the value or result to the caller and ends the current method.

```
0186 |     }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0187 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0188 |     private List&lt;EnrollmentOption&gt; loadEnrollmentOptions() {
```

Explanation: Declares the loadEnrollmentOptions method and its return type/parameters; the method performs the operation described by its name and body.

```
0189 |         List&lt;EnrollmentOption&gt; enrollments = new ArrayList&lt;&gt;();
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0190 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0191 |         String sql =
```

Explanation: Stores the SQL command as a string so the program can execute it against the database.

```
0192 |                 "SELECT e.enrollment_id, st.student_id, st.lrn, "
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0193 |                         + "st.first_name, st.last_name, "
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0194 |                         + "e.year_level, e.strand_id, str.strand_code, "
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0195 |                         + "e.school_year_id, e.section_id, "
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0196 |                         + "sec.section_name "
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0197 |                         + "FROM enrollments e "
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0198 |                         + "JOIN students st ON e.student_id = st.student_id "
```

Explanation: Assigns or updates a variable with the value or result on the right-hand side.

```
0199 |                         + "JOIN strands str ON e.strand_id = str.strand_id "
```

Explanation: Assigns or updates a variable with the value or result on the right-hand side.

```
0200 |                         + "LEFT JOIN sections sec "
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0201 |                         + "ON e.section_id = sec.section_id "
```

Explanation: Assigns or updates a variable with the value or result on the right-hand side.

```
0202 |                         + "WHERE e.year_level IN (11, 12) "
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0203 |                         + "AND e.enrollment_status &lt;&gt; 'Rejected' "
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0204 |                         + "ORDER BY st.last_name, st.first_name";
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0205 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0206 |         Connection conn = null;
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0207 |         PreparedStatement ps = null;
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0208 |         ResultSet rs = null;
```

Explanation: Declares or assigns a ResultSet that holds rows returned by a database query.

```
0209 |         try {
```

Explanation: Begins a protected block for code that may throw an exception; resources may be managed by try-with-resources.

```
0210 |             conn = db.getConnection();
```

Explanation: Assigns or updates a variable with the value or result on the right-hand side.

```
0211 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0212 |             if (conn == null) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0213 |                 throw new SQLException("Could not connect to the database.");
```

Explanation: Raises an exception so the caller or an enclosing catch block can handle the failure.

```
0214 |             }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0215 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0216 |             ps = conn.prepareStatement(sql);
```

Explanation: Creates a parameterized SQL statement; placeholders can be filled separately to reduce SQL-injection risk.

```
0217 |             rs = ps.executeQuery();
```

Explanation: Executes a SQL query that returns rows, usually a SELECT, and stores them in a ResultSet.

```
0218 |             while (rs.next()) {
```

Explanation: Repeats the following block while the condition remains true.

```
0219 |                 EnrollmentOption item = new EnrollmentOption();
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0220 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0221 |                 item.enrollmentId = rs.getInt("enrollment_id");
```

Explanation: Reads a column value from the current database result row using the requested Java type.

```
0222 |                 item.studentId = rs.getInt("student_id");
```

Explanation: Reads a column value from the current database result row using the requested Java type.

```
0223 |                 item.strandId = rs.getInt("strand_id");
```

Explanation: Reads a column value from the current database result row using the requested Java type.

```
0224 |                 item.schoolYearId = rs.getInt("school_year_id");
```

Explanation: Reads a column value from the current database result row using the requested Java type.

```
0225 |                 item.yearLevel = rs.getInt("year_level");
```

Explanation: Reads a column value from the current database result row using the requested Java type.

```
0226 |                 item.studentName = rs.getString("first_name")
```

Explanation: Reads a column value from the current database result row using the requested Java type.

```
0227 |                         + " " + rs.getString("last_name");
```

Explanation: Reads a column value from the current database result row using the requested Java type.

```
0228 |                 item.lrn = rs.getString("lrn");
```

Explanation: Reads a column value from the current database result row using the requested Java type.

```
0229 |                 item.strandName = rs.getString("strand_code");
```

Explanation: Reads a column value from the current database result row using the requested Java type.

```
0230 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0231 |                 int sectionId = rs.getInt("section_id");
```

Explanation: Reads a column value from the current database result row using the requested Java type.

```
0232 |                 if (!rs.wasNull()) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0233 |                     item.sectionId = sectionId;
```

Explanation: Assigns or updates a variable with the value or result on the right-hand side.

```
0234 |                 }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0235 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0236 |                 item.sectionName = rs.getString("section_name");
```

Explanation: Reads a column value from the current database result row using the requested Java type.

```
0237 |                 if (item.sectionName == null) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0238 |                     item.sectionName = "Not Assigned";
```

Explanation: Assigns or updates a variable with the value or result on the right-hand side.

```
0239 |                 }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0240 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0241 |                 enrollments.add(item);
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0242 |             }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0243 |         } catch (Exception ex) {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0244 |             StaffDashboard.showError(dashboard, "Could not load enrollment options: "
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0245 |                     + ex.getMessage());
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0246 |         } finally {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0247 |             try {
```

Explanation: Begins a protected block for code that may throw an exception; resources may be managed by try-with-resources.

```
0248 |                 if (rs != null) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0249 |                     rs.close();
```

Explanation: Closes the associated resource, such as a JDBC ResultSet, Statement, or Connection.

```
0250 |                 }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0251 |             } catch (Exception closeEx) {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0252 |                 System.out.println(closeEx);
```

Explanation: Prints a message or value to the console, commonly for status output or error reporting.

```
0253 |             }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0254 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0255 |             try {
```

Explanation: Begins a protected block for code that may throw an exception; resources may be managed by try-with-resources.

```
0256 |                 if (ps != null) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0257 |                     ps.close();
```

Explanation: Closes the associated resource, such as a JDBC ResultSet, Statement, or Connection.

```
0258 |                 }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0259 |             } catch (Exception closeEx) {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0260 |                 System.out.println(closeEx);
```

Explanation: Prints a message or value to the console, commonly for status output or error reporting.

```
0261 |             }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0262 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0263 |             try {
```

Explanation: Begins a protected block for code that may throw an exception; resources may be managed by try-with-resources.

```
0264 |                 if (conn != null) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0265 |                     conn.close();
```

Explanation: Closes the associated resource, such as a JDBC ResultSet, Statement, or Connection.

```
0266 |                 }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0267 |             } catch (Exception closeEx) {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0268 |                 System.out.println(closeEx);
```

Explanation: Prints a message or value to the console, commonly for status output or error reporting.

```
0269 |             }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0270 |         }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0271 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0272 |         return enrollments;
```

Explanation: Returns the value or result to the caller and ends the current method.

```
0273 |     }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0274 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0275 |     public void deleteEnrollmentRecord() {
```

Explanation: Declares the deleteEnrollmentRecord method and its return type/parameters; the method performs the operation described by its name and body.

```
0276 |         EnrollmentOption enrollment = selectEnrollment();
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0277 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0278 |         if (enrollment == null) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0279 |             return;
```

Explanation: Ends the current method without returning a value.

```
0280 |         }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0281 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0282 |         int choice = JOptionPane.showConfirmDialog(
```

Explanation: Displays a Swing dialog to communicate a message, warning, confirmation, or input prompt to the user.

```
0283 |                 dashboard,
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0284 |                 "Delete enrollment for " + enrollment.studentName
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0285 |                         + "?\nRelated records may prevent deletion.",
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0286 |                 "Confirm Deletion",
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0287 |                 JOptionPane.YES_NO_OPTION,
```

Explanation: Displays a Swing dialog to communicate a message, warning, confirmation, or input prompt to the user.

```
0288 |                 JOptionPane.WARNING_MESSAGE);
```

Explanation: Displays a Swing dialog to communicate a message, warning, confirmation, or input prompt to the user.

```
0289 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0290 |         if (choice != JOptionPane.YES_OPTION) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0291 |             return;
```

Explanation: Ends the current method without returning a value.

```
0292 |         }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0293 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0294 |         String sql =
```

Explanation: Stores the SQL command as a string so the program can execute it against the database.

```
0295 |                 "DELETE FROM enrollments WHERE enrollment_id = ? "
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0296 |                         + "AND year_level IN (11, 12)";
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0297 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0298 |         Connection conn = null;
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0299 |         PreparedStatement ps = null;
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0300 |         try {
```

Explanation: Begins a protected block for code that may throw an exception; resources may be managed by try-with-resources.

```
0301 |             conn = db.getConnection();
```

Explanation: Assigns or updates a variable with the value or result on the right-hand side.

```
0302 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0303 |             if (conn == null) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0304 |                 throw new SQLException("Could not connect to the database.");
```

Explanation: Raises an exception so the caller or an enclosing catch block can handle the failure.

```
0305 |             }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0306 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0307 |             ps = conn.prepareStatement(sql);
```

Explanation: Creates a parameterized SQL statement; placeholders can be filled separately to reduce SQL-injection risk.

```
0308 |             ps.setInt(1, enrollment.enrollmentId);
```

Explanation: Binds a Java value to a placeholder in the prepared SQL statement; the parameter index is one-based.

```
0309 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0310 |             if (ps.executeUpdate() &gt; 0) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0311 |                 JOptionPane.showMessageDialog(
```

Explanation: Displays a Swing dialog to communicate a message, warning, confirmation, or input prompt to the user.

```
0312 |                         dashboard, "Enrollment record deleted.");
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0313 |                 dashboard.viewEnrollmentRecords();
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0314 |             } else {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0315 |                 StaffDashboard.showError(dashboard, "Enrollment record was not deleted.");
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0316 |             }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0317 |         } catch (Exception ex) {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0318 |             StaffDashboard.showError(dashboard, "Could not delete enrollment: " + ex.getMessage());
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0319 |         } finally {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0320 |             try {
```

Explanation: Begins a protected block for code that may throw an exception; resources may be managed by try-with-resources.

```
0321 |                 if (ps != null) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0322 |                     ps.close();
```

Explanation: Closes the associated resource, such as a JDBC ResultSet, Statement, or Connection.

```
0323 |                 }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0324 |             } catch (Exception closeEx) {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0325 |                 System.out.println(closeEx);
```

Explanation: Prints a message or value to the console, commonly for status output or error reporting.

```
0326 |             }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0327 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0328 |             try {
```

Explanation: Begins a protected block for code that may throw an exception; resources may be managed by try-with-resources.

```
0329 |                 if (conn != null) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0330 |                     conn.close();
```

Explanation: Closes the associated resource, such as a JDBC ResultSet, Statement, or Connection.

```
0331 |                 }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0332 |             } catch (Exception closeEx) {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0333 |                 System.out.println(closeEx);
```

Explanation: Prints a message or value to the console, commonly for status output or error reporting.

```
0334 |             }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0335 |         }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0336 |     }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0337 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0338 |     public void createPaymentRequest() {
```

Explanation: Declares the createPaymentRequest method and its return type/parameters; the method performs the operation described by its name and body.

```
0339 |         StaffEnrollmentActions.EnrollmentOption enrollment =
```

Explanation: Assigns or updates a variable with the value or result on the right-hand side.

```
0340 |                 selectEnrollment();
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0341 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0342 |         if (enrollment == null) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0343 |             return;
```

Explanation: Ends the current method without returning a value.

```
0344 |         }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0345 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0346 |         String input = JOptionPane.showInputDialog(
```

Explanation: Displays a Swing dialog to communicate a message, warning, confirmation, or input prompt to the user.

```
0347 |                 dashboard,
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0348 |                 "Enter discount IDs separated by commas.\n"
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0349 |                         + "Leave blank if no discount applies.");
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0350 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0351 |         if (input == null) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0352 |             return;
```

Explanation: Ends the current method without returning a value.

```
0353 |         }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0354 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0355 |         List&lt;Integer&gt; discountIds = new ArrayList&lt;&gt;();
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0356 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0357 |         if (!input.trim().isEmpty()) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0358 |             try {
```

Explanation: Begins a protected block for code that may throw an exception; resources may be managed by try-with-resources.

```
0359 |                 for (String value : input.split(",")) {
```

Explanation: Starts a loop that repeats while its loop condition allows; initialization and update control iteration.

```
0360 |                     discountIds.add(Integer.parseInt(value.trim()));
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0361 |                 }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0362 |             } catch (NumberFormatException ex) {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0363 |                 StaffDashboard.showError(dashboard, "Enter valid discount IDs.");
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0364 |                 return;
```

Explanation: Ends the current method without returning a value.

```
0365 |             }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0366 |         }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0367 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0368 |         try {
```

Explanation: Begins a protected block for code that may throw an exception; resources may be managed by try-with-resources.

```
0369 |             paymentDAO.createPaymentRequest(
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0370 |                     enrollment.enrollmentId, discountIds);
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0371 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0372 |             JOptionPane.showMessageDialog(
```

Explanation: Displays a Swing dialog to communicate a message, warning, confirmation, or input prompt to the user.

```
0373 |                     dashboard, "Payment request created.");
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0374 |         } catch (Exception ex) {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0375 |             StaffDashboard.showError(dashboard, "Could not create payment request: "
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0376 |                     + ex.getMessage());
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0377 |         }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0378 |     }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0379 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0380 |     public static class EnrollmentOption {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0381 |         public int enrollmentId;
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0382 |         public int studentId;
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0383 |         public int strandId;
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0384 |         public int schoolYearId;
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0385 |         public int yearLevel;
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0386 |         public Integer sectionId;
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0387 |         public String studentName;
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0388 |         public String lrn;
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0389 |         public String strandName;
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0390 |         public String sectionName;
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0391 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0392 |         @Override
```

Explanation: Applies an annotation to the declaration that follows; the annotation supplies metadata or framework behavior.

```
0393 |         public String toString() {
```

Explanation: Declares the toString method and its return type/parameters; the method performs the operation described by its name and body.

```
0394 |             return "ID: " + enrollmentId
```

Explanation: Returns the value or result to the caller and ends the current method.

```
0395 |                     + " | " + studentName
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0396 |                     + " | Grade " + yearLevel
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0397 |                     + " | " + strandName
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0398 |                     + " | Section: " + sectionName;
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0399 |         }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0400 |     }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0401 | }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

StudentDAO.java

Project path: src/StudentDAO.java

```
0001 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0002 | import java.sql.*;
```

Explanation: Imports JDBC classes used to communicate with a relational database.

```
0003 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0004 | public class StudentDAO {
```

Explanation: Declares the StudentDAO type, which groups related data and behavior.

```
0005 |     private DbConnection db = new DbConnection();
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0006 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0007 |     public boolean addStudent(int userId, String lrn, String firstName,
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0008 |                               String middleName, String lastName,
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0009 |                               String previousSchool) {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0010 |         Connection conn = null;
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0011 |         PreparedStatement ps = null;
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0012 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0013 |         try {
```

Explanation: Begins a protected block for code that may throw an exception; resources may be managed by try-with-resources.

```
0014 |             conn = db.getConnection();
```

Explanation: Assigns or updates a variable with the value or result on the right-hand side.

```
0015 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0016 |             if (conn == null) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0017 |                 return false;
```

Explanation: Returns the value or result to the caller and ends the current method.

```
0018 |             }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0019 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0020 |             String sql = "INSERT INTO students "
```

Explanation: Stores the SQL command as a string so the program can execute it against the database.

```
0021 |                     + "(user_id, lrn, first_name, middle_name, last_name, previous_school) "
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0022 |                     + "VALUES (?, ?, ?, ?, ?, ?)";
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0023 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0024 |             ps = conn.prepareStatement(sql);
```

Explanation: Creates a parameterized SQL statement; placeholders can be filled separately to reduce SQL-injection risk.

```
0025 |             ps.setInt(1, userId);
```

Explanation: Binds a Java value to a placeholder in the prepared SQL statement; the parameter index is one-based.

```
0026 |             ps.setString(2, lrn);
```

Explanation: Binds a Java value to a placeholder in the prepared SQL statement; the parameter index is one-based.

```
0027 |             ps.setString(3, firstName);
```

Explanation: Binds a Java value to a placeholder in the prepared SQL statement; the parameter index is one-based.

```
0028 |             ps.setString(4, middleName);
```

Explanation: Binds a Java value to a placeholder in the prepared SQL statement; the parameter index is one-based.

```
0029 |             ps.setString(5, lastName);
```

Explanation: Binds a Java value to a placeholder in the prepared SQL statement; the parameter index is one-based.

```
0030 |             ps.setString(6, previousSchool);
```

Explanation: Binds a Java value to a placeholder in the prepared SQL statement; the parameter index is one-based.

```
0031 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0032 |             int rowInserted = ps.executeUpdate();
```

Explanation: Executes an INSERT, UPDATE, or DELETE statement and returns the number of affected rows.

```
0033 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0034 |             return rowInserted &gt; 0;
```

Explanation: Returns the value or result to the caller and ends the current method.

```
0035 |         } catch (Exception e) {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0036 |             System.out.println("Student error: " + e.getMessage());
```

Explanation: Prints a message or value to the console, commonly for status output or error reporting.

```
0037 |             return false;
```

Explanation: Returns the value or result to the caller and ends the current method.

```
0038 |         } finally {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0039 |             try {
```

Explanation: Begins a protected block for code that may throw an exception; resources may be managed by try-with-resources.

```
0040 |                 if (ps != null) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0041 |                     ps.close();
```

Explanation: Closes the associated resource, such as a JDBC ResultSet, Statement, or Connection.

```
0042 |                 }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0043 |             } catch (Exception e) {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0044 |                 System.out.println(e);
```

Explanation: Prints a message or value to the console, commonly for status output or error reporting.

```
0045 |             }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0046 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0047 |             try {
```

Explanation: Begins a protected block for code that may throw an exception; resources may be managed by try-with-resources.

```
0048 |                 if (conn != null) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0049 |                     conn.close();
```

Explanation: Closes the associated resource, such as a JDBC ResultSet, Statement, or Connection.

```
0050 |                 }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0051 |             } catch (Exception e) {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0052 |                 System.out.println(e);
```

Explanation: Prints a message or value to the console, commonly for status output or error reporting.

```
0053 |             }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0054 |         }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0055 |     }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0056 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0057 |     public int getStudentId(int userId) {
```

Explanation: Declares the getStudentId method, typically returning the requested field or resource.

```
0058 |         Connection conn = null;
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0059 |         PreparedStatement ps = null;
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0060 |         ResultSet rs = null;
```

Explanation: Declares or assigns a ResultSet that holds rows returned by a database query.

```
0061 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0062 |         try {
```

Explanation: Begins a protected block for code that may throw an exception; resources may be managed by try-with-resources.

```
0063 |             conn = db.getConnection();
```

Explanation: Assigns or updates a variable with the value or result on the right-hand side.

```
0064 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0065 |             if (conn == null) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0066 |                 return -1;
```

Explanation: Returns the value or result to the caller and ends the current method.

```
0067 |             }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0068 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0069 |             String sql = "SELECT student_id FROM students WHERE user_id = ?";
```

Explanation: Stores the SQL command as a string so the program can execute it against the database.

```
0070 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0071 |             ps = conn.prepareStatement(sql);
```

Explanation: Creates a parameterized SQL statement; placeholders can be filled separately to reduce SQL-injection risk.

```
0072 |             ps.setInt(1, userId);
```

Explanation: Binds a Java value to a placeholder in the prepared SQL statement; the parameter index is one-based.

```
0073 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0074 |             rs = ps.executeQuery();
```

Explanation: Executes a SQL query that returns rows, usually a SELECT, and stores them in a ResultSet.

```
0075 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0076 |             if (rs.next()) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0077 |                 return rs.getInt("student_id");
```

Explanation: Returns the value or result to the caller and ends the current method.

```
0078 |             }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0079 |         } catch (Exception e) {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0080 |             System.out.println("Student lookup error: " + e.getMessage());
```

Explanation: Prints a message or value to the console, commonly for status output or error reporting.

```
0081 |         } finally {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0082 |             try {
```

Explanation: Begins a protected block for code that may throw an exception; resources may be managed by try-with-resources.

```
0083 |                 if (rs != null) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0084 |                     rs.close();
```

Explanation: Closes the associated resource, such as a JDBC ResultSet, Statement, or Connection.

```
0085 |                 }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0086 |             } catch (Exception e) {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0087 |                 System.out.println(e);
```

Explanation: Prints a message or value to the console, commonly for status output or error reporting.

```
0088 |             }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0089 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0090 |             try {
```

Explanation: Begins a protected block for code that may throw an exception; resources may be managed by try-with-resources.

```
0091 |                 if (ps != null) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0092 |                     ps.close();
```

Explanation: Closes the associated resource, such as a JDBC ResultSet, Statement, or Connection.

```
0093 |                 }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0094 |             } catch (Exception e) {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0095 |                 System.out.println(e);
```

Explanation: Prints a message or value to the console, commonly for status output or error reporting.

```
0096 |             }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0097 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0098 |             try {
```

Explanation: Begins a protected block for code that may throw an exception; resources may be managed by try-with-resources.

```
0099 |                 if (conn != null) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0100 |                     conn.close();
```

Explanation: Closes the associated resource, such as a JDBC ResultSet, Statement, or Connection.

```
0101 |                 }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0102 |             } catch (Exception e) {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0103 |                 System.out.println(e);
```

Explanation: Prints a message or value to the console, commonly for status output or error reporting.

```
0104 |             }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0105 |         }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0106 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0107 |         return -1;
```

Explanation: Returns the value or result to the caller and ends the current method.

```
0108 |     }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0109 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0110 |     public String getStudentName(int userId) {
```

Explanation: Declares the getStudentName method, typically returning the requested field or resource.

```
0111 |         Connection conn = null;
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0112 |         PreparedStatement ps = null;
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0113 |         ResultSet rs = null;
```

Explanation: Declares or assigns a ResultSet that holds rows returned by a database query.

```
0114 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0115 |         try {
```

Explanation: Begins a protected block for code that may throw an exception; resources may be managed by try-with-resources.

```
0116 |             conn = db.getConnection();
```

Explanation: Assigns or updates a variable with the value or result on the right-hand side.

```
0117 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0118 |             if (conn == null) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0119 |                 return "Student";
```

Explanation: Returns the value or result to the caller and ends the current method.

```
0120 |             }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0121 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0122 |             String sql = "SELECT first_name, last_name FROM students WHERE user_id = ?";
```

Explanation: Stores the SQL command as a string so the program can execute it against the database.

```
0123 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0124 |             ps = conn.prepareStatement(sql);
```

Explanation: Creates a parameterized SQL statement; placeholders can be filled separately to reduce SQL-injection risk.

```
0125 |             ps.setInt(1, userId);
```

Explanation: Binds a Java value to a placeholder in the prepared SQL statement; the parameter index is one-based.

```
0126 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0127 |             rs = ps.executeQuery();
```

Explanation: Executes a SQL query that returns rows, usually a SELECT, and stores them in a ResultSet.

```
0128 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0129 |             if (rs.next()) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0130 |                 return rs.getString("first_name") + " "
```

Explanation: Returns the value or result to the caller and ends the current method.

```
0131 |                         + rs.getString("last_name");
```

Explanation: Reads a column value from the current database result row using the requested Java type.

```
0132 |             }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0133 |         } catch (Exception e) {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0134 |             System.out.println("Student lookup error: " + e.getMessage());
```

Explanation: Prints a message or value to the console, commonly for status output or error reporting.

```
0135 |         } finally {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0136 |             try {
```

Explanation: Begins a protected block for code that may throw an exception; resources may be managed by try-with-resources.

```
0137 |                 if (rs != null) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0138 |                     rs.close();
```

Explanation: Closes the associated resource, such as a JDBC ResultSet, Statement, or Connection.

```
0139 |                 }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0140 |             } catch (Exception e) {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0141 |                 System.out.println(e);
```

Explanation: Prints a message or value to the console, commonly for status output or error reporting.

```
0142 |             }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0143 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0144 |             try {
```

Explanation: Begins a protected block for code that may throw an exception; resources may be managed by try-with-resources.

```
0145 |                 if (ps != null) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0146 |                     ps.close();
```

Explanation: Closes the associated resource, such as a JDBC ResultSet, Statement, or Connection.

```
0147 |                 }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0148 |             } catch (Exception e) {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0149 |                 System.out.println(e);
```

Explanation: Prints a message or value to the console, commonly for status output or error reporting.

```
0150 |             }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0151 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0152 |             try {
```

Explanation: Begins a protected block for code that may throw an exception; resources may be managed by try-with-resources.

```
0153 |                 if (conn != null) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0154 |                     conn.close();
```

Explanation: Closes the associated resource, such as a JDBC ResultSet, Statement, or Connection.

```
0155 |                 }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0156 |             } catch (Exception e) {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0157 |                 System.out.println(e);
```

Explanation: Prints a message or value to the console, commonly for status output or error reporting.

```
0158 |             }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0159 |         }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0160 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0161 |         return "Student";
```

Explanation: Returns the value or result to the caller and ends the current method.

```
0162 |     }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0163 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0164 |     public int findOrCreateStudent(
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0165 |         Connection conn,
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0166 |         int userId,
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0167 |         String lrn,
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0168 |         String firstName,
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0169 |         String middleName,
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0170 |         String lastName,
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0171 |         String previousSchool
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0172 |     ) throws SQLException {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0173 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0174 |         String findSql =
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0175 |             "SELECT student_id, lrn FROM students WHERE user_id = ?";
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0176 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0177 |         try (PreparedStatement ps = conn.prepareStatement(findSql)) {
```

Explanation: Begins a protected block for code that may throw an exception; resources may be managed by try-with-resources.

```
0178 |             ps.setInt(1, userId);
```

Explanation: Binds a Java value to a placeholder in the prepared SQL statement; the parameter index is one-based.

```
0179 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0180 |             try (ResultSet rs = ps.executeQuery()) {
```

Explanation: Begins a protected block for code that may throw an exception; resources may be managed by try-with-resources.

```
0181 |                 if (rs.next()) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0182 |                     int studentId = rs.getInt("student_id");
```

Explanation: Reads a column value from the current database result row using the requested Java type.

```
0183 |                     String existingLrn = rs.getString("lrn");
```

Explanation: Reads a column value from the current database result row using the requested Java type.

```
0184 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0185 |                     if (!lrn.equals(existingLrn)) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0186 |                         try (PreparedStatement check =
```

Explanation: Begins a protected block for code that may throw an exception; resources may be managed by try-with-resources.

```
0187 |                                 conn.prepareStatement(
```

Explanation: Creates a parameterized SQL statement; placeholders can be filled separately to reduce SQL-injection risk.

```
0188 |                                     "SELECT student_id FROM students "
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0189 |                                     + "WHERE lrn = ?")) {
```

Explanation: Assigns or updates a variable with the value or result on the right-hand side.

```
0190 |                             check.setString(1, lrn);
```

Explanation: Binds a Java value to a placeholder in the prepared SQL statement; the parameter index is one-based.

```
0191 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0192 |                             try (ResultSet checkRs = check.executeQuery()) {
```

Explanation: Begins a protected block for code that may throw an exception; resources may be managed by try-with-resources.

```
0193 |                                 if (checkRs.next()) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0194 |                                     throw new SQLException(
```

Explanation: Raises an exception so the caller or an enclosing catch block can handle the failure.

```
0195 |                                     "This LRN is already registered."
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0196 |                                 );
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0197 |                             }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0198 |                         }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0199 |                     }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0200 |                 }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0201 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0202 |                 String updateSql =
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0203 |                     "UPDATE students SET lrn = ?, first_name = ?, "
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0204 |                     + "middle_name = ?, last_name = ?, "
```

Explanation: Assigns or updates a variable with the value or result on the right-hand side.

```
0205 |                     + "previous_school = ? WHERE student_id = ?";
```

Explanation: Assigns or updates a variable with the value or result on the right-hand side.

```
0206 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0207 |                 try (PreparedStatement update =
```

Explanation: Begins a protected block for code that may throw an exception; resources may be managed by try-with-resources.

```
0208 |                         conn.prepareStatement(updateSql)) {
```

Explanation: Creates a parameterized SQL statement; placeholders can be filled separately to reduce SQL-injection risk.

```
0209 |                     update.setString(1, lrn);
```

Explanation: Binds a Java value to a placeholder in the prepared SQL statement; the parameter index is one-based.

```
0210 |                     update.setString(2, firstName);
```

Explanation: Binds a Java value to a placeholder in the prepared SQL statement; the parameter index is one-based.

```
0211 |                     update.setString(
```

Explanation: Binds a Java value to a placeholder in the prepared SQL statement; the parameter index is one-based.

```
0212 |                         3,
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0213 |                         middleName.isEmpty() ? null : middleName
```

Explanation: Checks whether a string or collection has no content.

```
0214 |                     );
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0215 |                     update.setString(4, lastName);
```

Explanation: Binds a Java value to a placeholder in the prepared SQL statement; the parameter index is one-based.

```
0216 |                     update.setString(5, previousSchool);
```

Explanation: Binds a Java value to a placeholder in the prepared SQL statement; the parameter index is one-based.

```
0217 |                     update.setInt(6, studentId);
```

Explanation: Binds a Java value to a placeholder in the prepared SQL statement; the parameter index is one-based.

```
0218 |                         update.executeUpdate();
```

Explanation: Executes an INSERT, UPDATE, or DELETE statement and returns the number of affected rows.

```
0219 |                     }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0220 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0221 |                     return studentId;
```

Explanation: Returns the value or result to the caller and ends the current method.

```
0222 |                 }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0223 |             }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0224 |         }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0225 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0226 |         try (PreparedStatement ps = conn.prepareStatement(
```

Explanation: Begins a protected block for code that may throw an exception; resources may be managed by try-with-resources.

```
0227 |                 "SELECT student_id FROM students WHERE lrn = ?")) {
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0228 |             ps.setString(1, lrn);
```

Explanation: Binds a Java value to a placeholder in the prepared SQL statement; the parameter index is one-based.

```
0229 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0230 |             try (ResultSet rs = ps.executeQuery()) {
```

Explanation: Begins a protected block for code that may throw an exception; resources may be managed by try-with-resources.

```
0231 |                 if (rs.next()) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0232 |                     throw new SQLException(
```

Explanation: Raises an exception so the caller or an enclosing catch block can handle the failure.

```
0233 |                         "This LRN is already registered."
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0234 |                     );
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0235 |                 }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0236 |             }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0237 |         }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0238 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0239 |         String insertSql =
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0240 |             "INSERT INTO students "
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0241 |             + "(user_id, lrn, first_name, middle_name, last_name, "
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0242 |             + "previous_school) VALUES (?, ?, ?, ?, ?, ?)";
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0243 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0244 |         try (PreparedStatement ps = conn.prepareStatement(
```

Explanation: Begins a protected block for code that may throw an exception; resources may be managed by try-with-resources.

```
0245 |                 insertSql, Statement.RETURN_GENERATED_KEYS)) {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0246 |             ps.setInt(1, userId);
```

Explanation: Binds a Java value to a placeholder in the prepared SQL statement; the parameter index is one-based.

```
0247 |             ps.setString(2, lrn);
```

Explanation: Binds a Java value to a placeholder in the prepared SQL statement; the parameter index is one-based.

```
0248 |             ps.setString(3, firstName);
```

Explanation: Binds a Java value to a placeholder in the prepared SQL statement; the parameter index is one-based.

```
0249 |             ps.setString(
```

Explanation: Binds a Java value to a placeholder in the prepared SQL statement; the parameter index is one-based.

```
0250 |                 4,
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0251 |                 middleName.isEmpty() ? null : middleName
```

Explanation: Checks whether a string or collection has no content.

```
0252 |             );
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0253 |             ps.setString(5, lastName);
```

Explanation: Binds a Java value to a placeholder in the prepared SQL statement; the parameter index is one-based.

```
0254 |             ps.setString(6, previousSchool);
```

Explanation: Binds a Java value to a placeholder in the prepared SQL statement; the parameter index is one-based.

```
0255 |             ps.executeUpdate();
```

Explanation: Executes an INSERT, UPDATE, or DELETE statement and returns the number of affected rows.

```
0256 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0257 |             try (ResultSet keys = ps.getGeneratedKeys()) {
```

Explanation: Begins a protected block for code that may throw an exception; resources may be managed by try-with-resources.

```
0258 |                 if (keys.next()) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0259 |                     return keys.getInt(1);
```

Explanation: Returns the value or result to the caller and ends the current method.

```
0260 |                 }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0261 |             }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0262 |         }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0263 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0264 |         throw new SQLException("Could not save student information.");
```

Explanation: Raises an exception so the caller or an enclosing catch block can handle the failure.

```
0265 |     }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0266 | }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

StudentDashboard.java

Project path: src/StudentDashboard.java

```
0001 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0002 | import javax.swing.*;
```

Explanation: Imports Swing GUI classes used to build the application's windows and controls.

```
0003 | import java.awt.*;
```

Explanation: Imports AWT classes used for layout, dimensions, colors, fonts, or UI events.

```
0004 | import java.sql.*;
```

Explanation: Imports JDBC classes used to communicate with a relational database.

```
0005 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0006 | public class StudentDashboard extends JFrame {
```

Explanation: Declares the StudentDashboard type, which groups related data and behavior.

```
0007 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0008 |     public User user;
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0009 |     public JTextArea outputArea = new JTextArea();
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0010 |     public JLabel helpLabel = new JLabel(
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0011 |         "Choose a feature below to manage your enrollment."
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0012 |     );
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0013 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0014 |     private Color backgroundColor = new Color(245, 247, 250);
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0015 |     private Color primaryColor = new Color(24, 55, 90);
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0016 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0017 |     private StudentEnrollmentActions enrollmentActions;
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0018 |     private StudentRequirementActions requirementActions;
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0019 |     private StudentPaymentActions paymentActions;
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0020 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0021 |     public StudentDashboard(User user) {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0022 |         this.user = user;
```

Explanation: Assigns or updates a variable with the value or result on the right-hand side.

```
0023 |         this.enrollmentActions = new StudentEnrollmentActions(this);
```

Explanation: Assigns or updates a variable with the value or result on the right-hand side.

```
0024 |         this.requirementActions = new StudentRequirementActions(this);
```

Explanation: Assigns or updates a variable with the value or result on the right-hand side.

```
0025 |         this.paymentActions = new StudentPaymentActions(this);
```

Explanation: Assigns or updates a variable with the value or result on the right-hand side.

```
0026 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0027 |         setTitle("Student Enrollment System - Student Dashboard");
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0028 |         setSize(950, 650);
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0029 |         setDefaultCloseOperation(JFrame.DISPOSE_ON_CLOSE);
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0030 |         setLocationRelativeTo(null);
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0031 |         setLayout(new BorderLayout(10, 10));
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0032 |         getContentPane().setBackground(backgroundColor);
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0033 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0034 |         JLabel titleLabel = new JLabel(
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0035 |             "STUDENT ENROLLMENT DASHBOARD",
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0036 |             SwingConstants.CENTER
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0037 |         );
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0038 |         titleLabel.setFont(new Font("Arial", Font.BOLD, 23));
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0039 |         titleLabel.setForeground(Color.WHITE);
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0040 |         titleLabel.setOpaque(true);
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0041 |         titleLabel.setBackground(primaryColor);
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0042 |         titleLabel.setBorder(
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0043 |             BorderFactory.createEmptyBorder(20, 10, 20, 10)
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0044 |         );
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0045 |         add(titleLabel, BorderLayout.NORTH);
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0046 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0047 |         JPanel mainPanel = new JPanel(new BorderLayout(10, 10));
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0048 |         mainPanel.setBackground(backgroundColor);
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0049 |         mainPanel.setBorder(
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0050 |             BorderFactory.createEmptyBorder(10, 15, 15, 15)
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0051 |         );
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0052 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0053 |         JPanel buttonPanel = new JPanel(new GridLayout(0, 2, 10, 10));
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0054 |         buttonPanel.setBackground(backgroundColor);
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0055 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0056 |         addButton(buttonPanel, "Submit Enrollment Application",
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0057 |             enrollmentActions::submitEnrollmentApplication);
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0058 |         addButton(buttonPanel, "View My Enrollments",
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0059 |             this::viewMyEnrollments);
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0060 |         addButton(buttonPanel, "Print Enrollment Form",
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0061 |             enrollmentActions::printEnrollmentForm);
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0062 |         addButton(buttonPanel, "Upload Enrollment Requirements",
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0063 |             requirementActions::submitRequirement);
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0064 |         addButton(buttonPanel, "View My Requirements",
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0065 |             this::viewMyRequirements);
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0066 |         addButton(buttonPanel, "Delete My Requirement",
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0067 |             requirementActions::deleteMyRequirement);
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0068 |         addButton(buttonPanel, "View Payment Requests",
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0069 |             paymentActions::viewPaymentRequests);
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0070 |         addButton(buttonPanel, "View Request Breakdown",
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0071 |             paymentActions::viewRequestBreakdown);
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0072 |         addButton(buttonPanel, "Submit Payment",
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0073 |             paymentActions::submitPayment);
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0074 |         addButton(buttonPanel, "View My Payments",
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0075 |             paymentActions::viewMyPayments);
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0076 |         addButton(buttonPanel, "How to Use Dashboard",
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0077 |             this::showInstructions);
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0078 |         addButton(buttonPanel, "Logout", this::logout);
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0079 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0080 |         outputArea.setEditable(false);
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0081 |         outputArea.setFont(new Font("Monospaced", Font.PLAIN, 13));
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0082 |         outputArea.setLineWrap(true);
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0083 |         outputArea.setWrapStyleWord(true);
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0084 |         outputArea.setText(
```

Explanation: Sets the displayed text of a GUI component.

```
0085 |             "Welcome to the Student Enrollment System.\n\n"
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0086 |             + "Select a feature to get started."
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0087 |         );
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0088 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0089 |         JScrollPane scrollPane = new JScrollPane(outputArea);
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0090 |         scrollPane.setPreferredSize(new Dimension(500, 250));
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0091 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0092 |         JPanel contentPanel = new JPanel(new BorderLayout(10, 10));
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0093 |         contentPanel.setBackground(backgroundColor);
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0094 |         contentPanel.add(buttonPanel, BorderLayout.NORTH);
```

Explanation: Adds a component to a Swing container so it becomes part of the displayed interface.

```
0095 |         contentPanel.add(scrollPane, BorderLayout.CENTER);
```

Explanation: Adds a component to a Swing container so it becomes part of the displayed interface.

```
0096 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0097 |         helpLabel.setBorder(
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0098 |             BorderFactory.createEmptyBorder(5, 5, 5, 5)
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0099 |         );
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0100 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0101 |         mainPanel.add(helpLabel, BorderLayout.NORTH);
```

Explanation: Adds a component to a Swing container so it becomes part of the displayed interface.

```
0102 |         mainPanel.add(contentPanel, BorderLayout.CENTER);
```

Explanation: Adds a component to a Swing container so it becomes part of the displayed interface.

```
0103 |         add(mainPanel, BorderLayout.CENTER);
```

Explanation: Adds a component to a Swing container so it becomes part of the displayed interface.

```
0104 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0105 |         setVisible(true);
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0106 |     }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0107 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0108 |     private void addButton(JPanel panel, String text, Runnable action) {
```

Explanation: Declares the addButton method and its return type/parameters; the method performs the operation described by its name and body.

```
0109 |         JButton button = new JButton(text);
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0110 |         button.setFont(new Font("Arial", Font.BOLD, 13));
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0111 |         button.setFocusPainted(false);
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0112 |         button.setBackground(Color.WHITE);
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0113 |         button.setForeground(primaryColor);
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0114 |         button.setBorder(
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0115 |             BorderFactory.createCompoundBorder(
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0116 |                 BorderFactory.createLineBorder(
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0117 |                     new Color(210, 218, 228)
```

Explanation: Creates an instance of the specified class or object.

```
0118 |                 ),
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0119 |                 BorderFactory.createEmptyBorder(10, 8, 10, 8)
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0120 |             )
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0121 |         );
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0122 |         button.addActionListener(e -&gt; action.run());
```

Explanation: Registers an event handler that runs when the associated GUI control is activated.

```
0123 |         panel.add(button);
```

Explanation: Adds a component to a Swing container so it becomes part of the displayed interface.

```
0124 |     }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0125 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0126 |     public Connection getConnection() throws SQLException {
```

Explanation: Declares the getConnection method, typically returning the requested field or resource.

```
0127 |         return new DbConnection().getConnection();
```

Explanation: Returns the value or result to the caller and ends the current method.

```
0128 |     }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0129 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0130 |     private void viewMyEnrollments() {
```

Explanation: Declares the viewMyEnrollments method and its return type/parameters; the method performs the operation described by its name and body.

```
0131 |         String sql =
```

Explanation: Stores the SQL command as a string so the program can execute it against the database.

```
0132 |             "SELECT e.enrollment_id, sy.year_label, "
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0133 |             + "st.strand_code, st.strand_name, e.year_level, "
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0134 |             + "e.enrollee_type, e.enrollment_status, "
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0135 |             + "sec.section_name AS assigned_section, "
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0136 |             + "sec.grade_level AS section_grade_level "
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0137 |             + "FROM enrollments e "
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0138 |             + "JOIN students s ON e.student_id = s.student_id "
```

Explanation: Assigns or updates a variable with the value or result on the right-hand side.

```
0139 |             + "JOIN school_years sy "
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0140 |             + "ON e.school_year_id = sy.school_year_id "
```

Explanation: Assigns or updates a variable with the value or result on the right-hand side.

```
0141 |             + "JOIN strands st ON e.strand_id = st.strand_id "
```

Explanation: Assigns or updates a variable with the value or result on the right-hand side.

```
0142 |             + "LEFT JOIN sections sec ON e.section_id = sec.section_id "
```

Explanation: Assigns or updates a variable with the value or result on the right-hand side.

```
0143 |             + "WHERE s.user_id = ? "
```

Explanation: Assigns or updates a variable with the value or result on the right-hand side.

```
0144 |             + "ORDER BY e.enrollment_id DESC";
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0145 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0146 |         Connection conn = null;
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0147 |         PreparedStatement ps = null;
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0148 |         try {
```

Explanation: Begins a protected block for code that may throw an exception; resources may be managed by try-with-resources.

```
0149 |             conn = getConnection();
```

Explanation: Assigns or updates a variable with the value or result on the right-hand side.

```
0150 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0151 |             if (conn == null) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0152 |                 throw new SQLException("Could not connect to the database.");
```

Explanation: Raises an exception so the caller or an enclosing catch block can handle the failure.

```
0153 |             }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0154 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0155 |             ps = conn.prepareStatement(sql);
```

Explanation: Creates a parameterized SQL statement; placeholders can be filled separately to reduce SQL-injection risk.

```
0156 |             ps.setInt(1, user.getUserId());
```

Explanation: Binds a Java value to a placeholder in the prepared SQL statement; the parameter index is one-based.

```
0157 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0158 |             try (ResultSet rs = ps.executeQuery()) {
```

Explanation: Begins a protected block for code that may throw an exception; resources may be managed by try-with-resources.

```
0159 |                 StringBuilder output =
```

Explanation: Creates a mutable string builder so text can be assembled efficiently.

```
0160 |                     new StringBuilder("MY ENROLLMENT RECORDS\n\n");
```

Explanation: Creates an instance of the specified class or object.

```
0161 |                 boolean found = false;
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0162 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0163 |                 while (rs.next()) {
```

Explanation: Repeats the following block while the condition remains true.

```
0164 |                     found = true;
```

Explanation: Assigns or updates a variable with the value or result on the right-hand side.

```
0165 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0166 |                     output.append("Enrollment ID: ")
```

Explanation: Appends the provided value to the current StringBuilder or collection.

```
0167 |                         .append(rs.getInt("enrollment_id"))
```

Explanation: Reads a column value from the current database result row using the requested Java type.

```
0168 |                         .append("\nSchool Year: ")
```

Explanation: Appends the provided value to the current StringBuilder or collection.

```
0169 |                         .append(rs.getString("year_label"))
```

Explanation: Reads a column value from the current database result row using the requested Java type.

```
0170 |                         .append("\nStrand: ")
```

Explanation: Appends the provided value to the current StringBuilder or collection.

```
0171 |                         .append(rs.getString("strand_code"))
```

Explanation: Reads a column value from the current database result row using the requested Java type.

```
0172 |                         .append(" - ")
```

Explanation: Appends the provided value to the current StringBuilder or collection.

```
0173 |                         .append(rs.getString("strand_name"))
```

Explanation: Reads a column value from the current database result row using the requested Java type.

```
0174 |                         .append("\nGrade Level: ")
```

Explanation: Appends the provided value to the current StringBuilder or collection.

```
0175 |                         .append(rs.getInt("year_level"))
```

Explanation: Reads a column value from the current database result row using the requested Java type.

```
0176 |                         .append("\nEnrollee Type: ")
```

Explanation: Appends the provided value to the current StringBuilder or collection.

```
0177 |                         .append(rs.getString("enrollee_type"))
```

Explanation: Reads a column value from the current database result row using the requested Java type.

```
0178 |                         .append("\nStatus: ")
```

Explanation: Appends the provided value to the current StringBuilder or collection.

```
0179 |                         .append(rs.getString("enrollment_status"))
```

Explanation: Reads a column value from the current database result row using the requested Java type.

```
0180 |                         .append("\nAssigned Section: ");
```

Explanation: Appends the provided value to the current StringBuilder or collection.

```
0181 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0182 |                     String section = rs.getString("assigned_section");
```

Explanation: Reads a column value from the current database result row using the requested Java type.

```
0183 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0184 |                     if (section == null || section.trim().isEmpty()) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0185 |                         output.append("Not yet assigned\n");
```

Explanation: Appends the provided value to the current StringBuilder or collection.

```
0186 |                     } else {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0187 |                         output.append(section).append("\n");
```

Explanation: Appends the provided value to the current StringBuilder or collection.

```
0188 |                         output.append("Section Grade Level: ")
```

Explanation: Appends the provided value to the current StringBuilder or collection.

```
0189 |                             .append(rs.getInt("section_grade_level"))
```

Explanation: Reads a column value from the current database result row using the requested Java type.

```
0190 |                             .append("\n");
```

Explanation: Appends the provided value to the current StringBuilder or collection.

```
0191 |                     }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0192 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0193 |                     output.append("--------------------------------\n");
```

Explanation: Appends the provided value to the current StringBuilder or collection.

```
0194 |                 }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0195 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0196 |                 if (!found) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0197 |                     output.append("No enrollment records found.");
```

Explanation: Appends the provided value to the current StringBuilder or collection.

```
0198 |                 }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0199 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0200 |                 outputArea.setText(output.toString());
```

Explanation: Sets the displayed text of a GUI component.

```
0201 |                 helpLabel.setText(
```

Explanation: Sets the displayed text of a GUI component.

```
0202 |                     "Your enrollment records and assigned sections."
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0203 |                 );
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0204 |             }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0205 |         } catch (Exception ex) {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0206 |             showError(
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0207 |                 "Could not load enrollments: " + ex.getMessage()
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0208 |             );
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0209 |         } finally {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0210 |             try {
```

Explanation: Begins a protected block for code that may throw an exception; resources may be managed by try-with-resources.

```
0211 |                 if (ps != null) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0212 |                     ps.close();
```

Explanation: Closes the associated resource, such as a JDBC ResultSet, Statement, or Connection.

```
0213 |                 }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0214 |             } catch (Exception closeEx) {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0215 |                 System.out.println(closeEx);
```

Explanation: Prints a message or value to the console, commonly for status output or error reporting.

```
0216 |             }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0217 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0218 |             try {
```

Explanation: Begins a protected block for code that may throw an exception; resources may be managed by try-with-resources.

```
0219 |                 if (conn != null) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0220 |                     conn.close();
```

Explanation: Closes the associated resource, such as a JDBC ResultSet, Statement, or Connection.

```
0221 |                 }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0222 |             } catch (Exception closeEx) {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0223 |                 System.out.println(closeEx);
```

Explanation: Prints a message or value to the console, commonly for status output or error reporting.

```
0224 |             }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0225 |         }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0226 |     }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0227 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0228 |     private void viewMyRequirements() {
```

Explanation: Declares the viewMyRequirements method and its return type/parameters; the method performs the operation described by its name and body.

```
0229 |         String sql =
```

Explanation: Stores the SQL command as a string so the program can execute it against the database.

```
0230 |             "SELECT sr.requirement_id, sr.enrollment_id, "
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0231 |             + "sr.requirement_name, sr.requirement_status, "
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0232 |             + "sr.original_file_name, sr.submitted_at "
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0233 |             + "FROM student_requirements sr "
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0234 |             + "JOIN enrollments e "
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0235 |             + "ON sr.enrollment_id = e.enrollment_id "
```

Explanation: Assigns or updates a variable with the value or result on the right-hand side.

```
0236 |             + "JOIN students s ON e.student_id = s.student_id "
```

Explanation: Assigns or updates a variable with the value or result on the right-hand side.

```
0237 |             + "WHERE s.user_id = ? "
```

Explanation: Assigns or updates a variable with the value or result on the right-hand side.

```
0238 |             + "ORDER BY sr.requirement_id DESC";
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0239 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0240 |         Connection conn = null;
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0241 |         PreparedStatement ps = null;
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0242 |         try {
```

Explanation: Begins a protected block for code that may throw an exception; resources may be managed by try-with-resources.

```
0243 |             conn = getConnection();
```

Explanation: Assigns or updates a variable with the value or result on the right-hand side.

```
0244 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0245 |             if (conn == null) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0246 |                 throw new SQLException("Could not connect to the database.");
```

Explanation: Raises an exception so the caller or an enclosing catch block can handle the failure.

```
0247 |             }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0248 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0249 |             ps = conn.prepareStatement(sql);
```

Explanation: Creates a parameterized SQL statement; placeholders can be filled separately to reduce SQL-injection risk.

```
0250 |             ps.setInt(1, user.getUserId());
```

Explanation: Binds a Java value to a placeholder in the prepared SQL statement; the parameter index is one-based.

```
0251 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0252 |             try (ResultSet rs = ps.executeQuery()) {
```

Explanation: Begins a protected block for code that may throw an exception; resources may be managed by try-with-resources.

```
0253 |                 StringBuilder output =
```

Explanation: Creates a mutable string builder so text can be assembled efficiently.

```
0254 |                     new StringBuilder("MY SUBMITTED REQUIREMENTS\n\n");
```

Explanation: Creates an instance of the specified class or object.

```
0255 |                 boolean found = false;
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0256 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0257 |                 while (rs.next()) {
```

Explanation: Repeats the following block while the condition remains true.

```
0258 |                     found = true;
```

Explanation: Assigns or updates a variable with the value or result on the right-hand side.

```
0259 |                     output.append("Requirement ID: ")
```

Explanation: Appends the provided value to the current StringBuilder or collection.

```
0260 |                         .append(rs.getInt("requirement_id"))
```

Explanation: Reads a column value from the current database result row using the requested Java type.

```
0261 |                         .append("\nEnrollment ID: ")
```

Explanation: Appends the provided value to the current StringBuilder or collection.

```
0262 |                         .append(rs.getInt("enrollment_id"))
```

Explanation: Reads a column value from the current database result row using the requested Java type.

```
0263 |                         .append("\nRequirement: ")
```

Explanation: Appends the provided value to the current StringBuilder or collection.

```
0264 |                         .append(rs.getString("requirement_name"))
```

Explanation: Reads a column value from the current database result row using the requested Java type.

```
0265 |                         .append("\nFile: ")
```

Explanation: Appends the provided value to the current StringBuilder or collection.

```
0266 |                         .append(rs.getString("original_file_name"))
```

Explanation: Reads a column value from the current database result row using the requested Java type.

```
0267 |                         .append("\nStatus: ")
```

Explanation: Appends the provided value to the current StringBuilder or collection.

```
0268 |                         .append(rs.getString("requirement_status"))
```

Explanation: Reads a column value from the current database result row using the requested Java type.

```
0269 |                         .append("\nSubmitted: ")
```

Explanation: Appends the provided value to the current StringBuilder or collection.

```
0270 |                         .append(rs.getString("submitted_at"))
```

Explanation: Reads a column value from the current database result row using the requested Java type.

```
0271 |                         .append("\n--------------------------------\n");
```

Explanation: Appends the provided value to the current StringBuilder or collection.

```
0272 |                 }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0273 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0274 |                 if (!found) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0275 |                     output.append(
```

Explanation: Appends the provided value to the current StringBuilder or collection.

```
0276 |                         "You have not submitted any requirements."
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0277 |                     );
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0278 |                 }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0279 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0280 |                 outputArea.setText(output.toString());
```

Explanation: Sets the displayed text of a GUI component.

```
0281 |                 helpLabel.setText(
```

Explanation: Sets the displayed text of a GUI component.

```
0282 |                     "Use the Requirement ID to delete an eligible submission."
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0283 |                 );
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0284 |             }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0285 |         } catch (Exception ex) {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0286 |             showError(
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0287 |                 "Could not load requirements: " + ex.getMessage()
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0288 |             );
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0289 |         } finally {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0290 |             try {
```

Explanation: Begins a protected block for code that may throw an exception; resources may be managed by try-with-resources.

```
0291 |                 if (ps != null) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0292 |                     ps.close();
```

Explanation: Closes the associated resource, such as a JDBC ResultSet, Statement, or Connection.

```
0293 |                 }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0294 |             } catch (Exception closeEx) {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0295 |                 System.out.println(closeEx);
```

Explanation: Prints a message or value to the console, commonly for status output or error reporting.

```
0296 |             }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0297 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0298 |             try {
```

Explanation: Begins a protected block for code that may throw an exception; resources may be managed by try-with-resources.

```
0299 |                 if (conn != null) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0300 |                     conn.close();
```

Explanation: Closes the associated resource, such as a JDBC ResultSet, Statement, or Connection.

```
0301 |                 }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0302 |             } catch (Exception closeEx) {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0303 |                 System.out.println(closeEx);
```

Explanation: Prints a message or value to the console, commonly for status output or error reporting.

```
0304 |             }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0305 |         }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0306 |     }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0307 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0308 |     private void showInstructions() {
```

Explanation: Declares the showInstructions method and its return type/parameters; the method performs the operation described by its name and body.

```
0309 |         outputArea.setText(
```

Explanation: Sets the displayed text of a GUI component.

```
0310 |             "HOW TO USE THE STUDENT DASHBOARD\n\n"
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0311 |             + "1. Submit Enrollment Application.\n"
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0312 |             + "2. View My Enrollments to check your enrollment status "
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0313 |             + "and assigned section.\n"
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0314 |             + "3. Print Enrollment Form to print your enrollment details.\n"
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0315 |             + "4. Upload Enrollment Requirements.\n"
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0316 |             + "5. View My Requirements to check your submissions.\n"
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0317 |             + "6. Delete My Requirement to remove an eligible submission.\n"
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0318 |             + "7. View Payment Requests.\n"
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0319 |             + "8. View Request Breakdown.\n"
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0320 |             + "9. Submit Payment.\n"
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0321 |             + "10. View My Payments.\n\n"
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0322 |             + "Contact enrollment staff if your information is incorrect."
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0323 |         );
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0324 |     }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0325 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0326 |     private void logout() {
```

Explanation: Declares the logout method and its return type/parameters; the method performs the operation described by its name and body.

```
0327 |         int confirm = JOptionPane.showConfirmDialog(
```

Explanation: Displays a Swing dialog to communicate a message, warning, confirmation, or input prompt to the user.

```
0328 |             this,
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0329 |             "Are you sure you want to log out?",
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0330 |             "Logout",
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0331 |             JOptionPane.YES_NO_OPTION
```

Explanation: Displays a Swing dialog to communicate a message, warning, confirmation, or input prompt to the user.

```
0332 |         );
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0333 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0334 |         if (confirm == JOptionPane.YES_OPTION) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0335 |             dispose();
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0336 |             new LoginFrame().setVisible(true);
```

Explanation: Creates an instance of the specified class or object.

```
0337 |         }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0338 |     }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0339 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0340 |     public static void showError(Component parent, String message) {
```

Explanation: Declares the showError method and its return type/parameters; the method performs the operation described by its name and body.

```
0341 |         JOptionPane.showMessageDialog(
```

Explanation: Displays a Swing dialog to communicate a message, warning, confirmation, or input prompt to the user.

```
0342 |             parent,
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0343 |             message,
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0344 |             "Student Dashboard",
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0345 |             JOptionPane.ERROR_MESSAGE
```

Explanation: Displays a Swing dialog to communicate a message, warning, confirmation, or input prompt to the user.

```
0346 |         );
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0347 |     }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0348 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0349 |     private void showError(String message) {
```

Explanation: Declares the showError method and its return type/parameters; the method performs the operation described by its name and body.

```
0350 |         showError(this, message);
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0351 |     }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0352 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0353 |     public static class ChoiceItem {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0354 |         public int id;
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0355 |         public String label;
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0356 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0357 |         public ChoiceItem(int id, String label) {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0358 |             this.id = id;
```

Explanation: Assigns or updates a variable with the value or result on the right-hand side.

```
0359 |             this.label = label;
```

Explanation: Assigns or updates a variable with the value or result on the right-hand side.

```
0360 |         }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0361 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0362 |         @Override
```

Explanation: Applies an annotation to the declaration that follows; the annotation supplies metadata or framework behavior.

```
0363 |         public String toString() {
```

Explanation: Declares the toString method and its return type/parameters; the method performs the operation described by its name and body.

```
0364 |             return label;
```

Explanation: Returns the value or result to the caller and ends the current method.

```
0365 |         }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0366 |     }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0367 | }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

StudentEnrollmentActions.java

Project path: src/StudentEnrollmentActions.java

```
0001 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0002 | import javax.swing.*;
```

Explanation: Imports Swing GUI classes used to build the application's windows and controls.

```
0003 | import java.awt.*;
```

Explanation: Imports AWT classes used for layout, dimensions, colors, fonts, or UI events.

```
0004 | import java.sql.*;
```

Explanation: Imports JDBC classes used to communicate with a relational database.

```
0005 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0006 | public class StudentEnrollmentActions {
```

Explanation: Declares the StudentEnrollmentActions type, which groups related data and behavior.

```
0007 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0008 |     private StudentDashboard dashboard;
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0009 |     private User user;
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0010 |     private JTextArea outputArea;
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0011 |     private JLabel helpLabel;
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0012 |     private StudentDAO studentDAO = new StudentDAO();
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0013 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0014 |     public StudentEnrollmentActions(StudentDashboard dashboard) {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0015 |         this.dashboard = dashboard;
```

Explanation: Assigns or updates a variable with the value or result on the right-hand side.

```
0016 |         this.user = dashboard.user;
```

Explanation: Assigns or updates a variable with the value or result on the right-hand side.

```
0017 |         this.outputArea = dashboard.outputArea;
```

Explanation: Assigns or updates a variable with the value or result on the right-hand side.

```
0018 |         this.helpLabel = dashboard.helpLabel;
```

Explanation: Assigns or updates a variable with the value or result on the right-hand side.

```
0019 |     }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0020 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0021 |     public void submitEnrollmentApplication() {
```

Explanation: Declares the submitEnrollmentApplication method and its return type/parameters; the method performs the operation described by its name and body.

```
0022 |         JTextField lrnField = new JTextField();
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0023 |         JTextField firstNameField = new JTextField();
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0024 |         JTextField middleNameField = new JTextField();
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0025 |         JTextField lastNameField = new JTextField();
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0026 |         JTextField previousSchoolField = new JTextField();
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0027 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0028 |         JComboBox&lt;StudentDashboard.ChoiceItem&gt; schoolYearBox = new JComboBox&lt;&gt;();
```

Explanation: Assigns or updates a variable with the value or result on the right-hand side.

```
0029 |         JComboBox&lt;StudentDashboard.ChoiceItem&gt; strandBox = new JComboBox&lt;&gt;();
```

Explanation: Assigns or updates a variable with the value or result on the right-hand side.

```
0030 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0031 |         JComboBox&lt;String&gt; gradeLevelBox = new JComboBox&lt;&gt;(
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0032 |             new String[]{"11", "12"}
```

Explanation: Creates an instance of the specified class or object.

```
0033 |         );
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0034 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0035 |         JComboBox&lt;String&gt; enrolleeTypeBox = new JComboBox&lt;&gt;(
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0036 |             new String[]{
```

Explanation: Creates an instance of the specified class or object.

```
0037 |                 "New Student",
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0038 |                 "Transferee",
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0039 |                 "Returning Student"
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0040 |             }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0041 |         );
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0042 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0043 |         Connection conn = null;
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0044 |         try {
```

Explanation: Begins a protected block for code that may throw an exception; resources may be managed by try-with-resources.

```
0045 |             conn = dashboard.getConnection();
```

Explanation: Assigns or updates a variable with the value or result on the right-hand side.

```
0046 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0047 |             if (conn == null) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0048 |                 throw new SQLException("Could not connect to the database.");
```

Explanation: Raises an exception so the caller or an enclosing catch block can handle the failure.

```
0049 |             }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0050 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0051 |             String yearSql =
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0052 |                 "SELECT school_year_id, year_label FROM school_years "
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0053 |                 + "ORDER BY school_year_id DESC";
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0054 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0055 |             try (PreparedStatement ps = conn.prepareStatement(yearSql);
```

Explanation: Begins a protected block for code that may throw an exception; resources may be managed by try-with-resources.

```
0056 |                  ResultSet rs = ps.executeQuery()) {
```

Explanation: Executes a SQL query that returns rows, usually a SELECT, and stores them in a ResultSet.

```
0057 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0058 |                 while (rs.next()) {
```

Explanation: Repeats the following block while the condition remains true.

```
0059 |                     schoolYearBox.addItem(new StudentDashboard.ChoiceItem(
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0060 |                         rs.getInt("school_year_id"),
```

Explanation: Reads a column value from the current database result row using the requested Java type.

```
0061 |                         rs.getString("year_label")
```

Explanation: Reads a column value from the current database result row using the requested Java type.

```
0062 |                     ));
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0063 |                 }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0064 |             }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0065 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0066 |             String strandSql =
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0067 |                 "SELECT strand_id, strand_code, strand_name FROM strands "
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0068 |                 + "ORDER BY strand_code";
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0069 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0070 |             try (PreparedStatement ps = conn.prepareStatement(strandSql);
```

Explanation: Begins a protected block for code that may throw an exception; resources may be managed by try-with-resources.

```
0071 |                  ResultSet rs = ps.executeQuery()) {
```

Explanation: Executes a SQL query that returns rows, usually a SELECT, and stores them in a ResultSet.

```
0072 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0073 |                 while (rs.next()) {
```

Explanation: Repeats the following block while the condition remains true.

```
0074 |                     strandBox.addItem(new StudentDashboard.ChoiceItem(
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0075 |                         rs.getInt("strand_id"),
```

Explanation: Reads a column value from the current database result row using the requested Java type.

```
0076 |                         rs.getString("strand_code") + " - "
```

Explanation: Reads a column value from the current database result row using the requested Java type.

```
0077 |                             + rs.getString("strand_name")
```

Explanation: Reads a column value from the current database result row using the requested Java type.

```
0078 |                     ));
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0079 |                 }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0080 |             }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0081 |         } catch (Exception ex) {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0082 |             StudentDashboard.showError(dashboard,
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0083 |                 "Could not load school years and strands: "
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0084 |                 + ex.getMessage()
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0085 |             );
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0086 |             return;
```

Explanation: Ends the current method without returning a value.

```
0087 |         } finally {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0088 |             try {
```

Explanation: Begins a protected block for code that may throw an exception; resources may be managed by try-with-resources.

```
0089 |                 if (conn != null) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0090 |                     conn.close();
```

Explanation: Closes the associated resource, such as a JDBC ResultSet, Statement, or Connection.

```
0091 |                 }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0092 |             } catch (Exception closeEx) {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0093 |                 System.out.println(closeEx);
```

Explanation: Prints a message or value to the console, commonly for status output or error reporting.

```
0094 |             }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0095 |         }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0096 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0097 |         if (schoolYearBox.getItemCount() == 0
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0098 |                 || strandBox.getItemCount() == 0) {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0099 |             StudentDashboard.showError(dashboard,
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0100 |                 "No school years or strands are available. "
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0101 |                 + "Ask the administrator to add them first."
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0102 |             );
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0103 |             return;
```

Explanation: Ends the current method without returning a value.

```
0104 |         }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0105 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0106 |         JPanel form = new JPanel(new GridLayout(0, 2, 8, 8));
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0107 |         form.add(new JLabel("LRN (12 digits):"));
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0108 |         form.add(lrnField);
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0109 |         form.add(new JLabel("First Name:"));
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0110 |         form.add(firstNameField);
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0111 |         form.add(new JLabel("Middle Name (optional):"));
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0112 |         form.add(middleNameField);
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0113 |         form.add(new JLabel("Last Name:"));
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0114 |         form.add(lastNameField);
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0115 |         form.add(new JLabel("Previous School:"));
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0116 |         form.add(previousSchoolField);
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0117 |         form.add(new JLabel("School Year:"));
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0118 |         form.add(schoolYearBox);
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0119 |         form.add(new JLabel("Strand:"));
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0120 |         form.add(strandBox);
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0121 |         form.add(new JLabel("Grade Level:"));
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0122 |         form.add(gradeLevelBox);
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0123 |         form.add(new JLabel("Enrollee Type:"));
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0124 |         form.add(enrolleeTypeBox);
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0125 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0126 |         int result = JOptionPane.showConfirmDialog(
```

Explanation: Displays a Swing dialog to communicate a message, warning, confirmation, or input prompt to the user.

```
0127 |             dashboard,
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0128 |             form,
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0129 |             "Enrollment Application",
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0130 |             JOptionPane.OK_CANCEL_OPTION,
```

Explanation: Displays a Swing dialog to communicate a message, warning, confirmation, or input prompt to the user.

```
0131 |             JOptionPane.PLAIN_MESSAGE
```

Explanation: Displays a Swing dialog to communicate a message, warning, confirmation, or input prompt to the user.

```
0132 |         );
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0133 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0134 |         if (result != JOptionPane.OK_OPTION) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0135 |             return;
```

Explanation: Ends the current method without returning a value.

```
0136 |         }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0137 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0138 |         String lrn = lrnField.getText().trim();
```

Explanation: Reads text currently entered or displayed by a GUI component.

```
0139 |         String firstName = firstNameField.getText().trim();
```

Explanation: Reads text currently entered or displayed by a GUI component.

```
0140 |         String middleName = middleNameField.getText().trim();
```

Explanation: Reads text currently entered or displayed by a GUI component.

```
0141 |         String lastName = lastNameField.getText().trim();
```

Explanation: Reads text currently entered or displayed by a GUI component.

```
0142 |         String previousSchool = previousSchoolField.getText().trim();
```

Explanation: Reads text currently entered or displayed by a GUI component.

```
0143 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0144 |         if (!lrn.matches("\\d{12}")) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0145 |             StudentDashboard.showError(dashboard, "LRN must contain exactly 12 digits.");
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0146 |             return;
```

Explanation: Ends the current method without returning a value.

```
0147 |         }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0148 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0149 |         if (firstName.isEmpty() || lastName.isEmpty()
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0150 |                 || previousSchool.isEmpty()) {
```

Explanation: Checks whether a string or collection has no content.

```
0151 |             StudentDashboard.showError(dashboard,
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0152 |                 "First name, last name, and previous school are required."
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0153 |             );
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0154 |             return;
```

Explanation: Ends the current method without returning a value.

```
0155 |         }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0156 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0157 |         StudentDashboard.ChoiceItem selectedYear =
```

Explanation: Assigns or updates a variable with the value or result on the right-hand side.

```
0158 |             (StudentDashboard.ChoiceItem) schoolYearBox.getSelectedItem();
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0159 |         StudentDashboard.ChoiceItem selectedStrand =
```

Explanation: Assigns or updates a variable with the value or result on the right-hand side.

```
0160 |             (StudentDashboard.ChoiceItem) strandBox.getSelectedItem();
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0161 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0162 |         if (selectedYear == null || selectedStrand == null) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0163 |             StudentDashboard.showError(dashboard, "Please select a school year and strand.");
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0164 |             return;
```

Explanation: Ends the current method without returning a value.

```
0165 |         }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0166 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0167 |         conn = null;
```

Explanation: Assigns or updates a variable with the value or result on the right-hand side.

```
0168 |         try {
```

Explanation: Begins a protected block for code that may throw an exception; resources may be managed by try-with-resources.

```
0169 |             conn = dashboard.getConnection();
```

Explanation: Assigns or updates a variable with the value or result on the right-hand side.

```
0170 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0171 |             if (conn == null) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0172 |                 throw new SQLException("Could not connect to the database.");
```

Explanation: Raises an exception so the caller or an enclosing catch block can handle the failure.

```
0173 |             }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0174 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0175 |             conn.setAutoCommit(false);
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0176 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0177 |             try {
```

Explanation: Begins a protected block for code that may throw an exception; resources may be managed by try-with-resources.

```
0178 |                 int studentId = studentDAO.findOrCreateStudent(
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0179 |                     conn,
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0180 |                     user.getUserId(),
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0181 |                     lrn,
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0182 |                     firstName,
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0183 |                     middleName,
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0184 |                     lastName,
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0185 |                     previousSchool
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0186 |                 );
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0187 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0188 |                 String duplicateSql =
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0189 |                     "SELECT enrollment_id FROM enrollments "
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0190 |                     + "WHERE student_id = ? AND school_year_id = ?";
```

Explanation: Assigns or updates a variable with the value or result on the right-hand side.

```
0191 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0192 |                 try (PreparedStatement ps =
```

Explanation: Begins a protected block for code that may throw an exception; resources may be managed by try-with-resources.

```
0193 |                         conn.prepareStatement(duplicateSql)) {
```

Explanation: Creates a parameterized SQL statement; placeholders can be filled separately to reduce SQL-injection risk.

```
0194 |                     ps.setInt(1, studentId);
```

Explanation: Binds a Java value to a placeholder in the prepared SQL statement; the parameter index is one-based.

```
0195 |                     ps.setInt(2, selectedYear.id);
```

Explanation: Binds a Java value to a placeholder in the prepared SQL statement; the parameter index is one-based.

```
0196 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0197 |                     try (ResultSet rs = ps.executeQuery()) {
```

Explanation: Begins a protected block for code that may throw an exception; resources may be managed by try-with-resources.

```
0198 |                         if (rs.next()) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0199 |                             conn.rollback();
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0200 |                             StudentDashboard.showError(dashboard,
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0201 |                                 "You already have an enrollment "
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0202 |                                 + "application for this school year."
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0203 |                             );
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0204 |                             return;
```

Explanation: Ends the current method without returning a value.

```
0205 |                         }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0206 |                     }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0207 |                 }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0208 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0209 |                 String insertSql =
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0210 |                     "INSERT INTO enrollments "
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0211 |                     + "(student_id, school_year_id, strand_id, section_id, "
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0212 |                     + "year_level, enrollee_type) "
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0213 |                     + "VALUES (?, ?, ?, NULL, ?, ?)";
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0214 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0215 |                 int enrollmentId;
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0216 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0217 |                 try (PreparedStatement ps = conn.prepareStatement(
```

Explanation: Begins a protected block for code that may throw an exception; resources may be managed by try-with-resources.

```
0218 |                         insertSql, Statement.RETURN_GENERATED_KEYS)) {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0219 |                     ps.setInt(1, studentId);
```

Explanation: Binds a Java value to a placeholder in the prepared SQL statement; the parameter index is one-based.

```
0220 |                     ps.setInt(2, selectedYear.id);
```

Explanation: Binds a Java value to a placeholder in the prepared SQL statement; the parameter index is one-based.

```
0221 |                     ps.setInt(3, selectedStrand.id);
```

Explanation: Binds a Java value to a placeholder in the prepared SQL statement; the parameter index is one-based.

```
0222 |                     ps.setInt(
```

Explanation: Binds a Java value to a placeholder in the prepared SQL statement; the parameter index is one-based.

```
0223 |                         4,
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0224 |                         Integer.parseInt(
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0225 |                             gradeLevelBox.getSelectedItem().toString()
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0226 |                         )
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0227 |                     );
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0228 |                     ps.setString(
```

Explanation: Binds a Java value to a placeholder in the prepared SQL statement; the parameter index is one-based.

```
0229 |                         5,
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0230 |                         enrolleeTypeBox.getSelectedItem().toString()
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0231 |                     );
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0232 |                     ps.executeUpdate();
```

Explanation: Executes an INSERT, UPDATE, or DELETE statement and returns the number of affected rows.

```
0233 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0234 |                     try (ResultSet keys = ps.getGeneratedKeys()) {
```

Explanation: Begins a protected block for code that may throw an exception; resources may be managed by try-with-resources.

```
0235 |                         if (!keys.next()) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0236 |                             throw new SQLException(
```

Explanation: Raises an exception so the caller or an enclosing catch block can handle the failure.

```
0237 |                                 "Could not retrieve enrollment ID."
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0238 |                             );
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0239 |                         }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0240 |                         enrollmentId = keys.getInt(1);
```

Explanation: Reads a column value from the current database result row using the requested Java type.

```
0241 |                     }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0242 |                 }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0243 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0244 |                 conn.commit();
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0245 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0246 |                 outputArea.setText(
```

Explanation: Sets the displayed text of a GUI component.

```
0247 |                     "ENROLLMENT APPLICATION SUBMITTED\n\n"
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0248 |                     + "Enrollment ID: " + enrollmentId
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0249 |                     + "\nStudent ID: " + studentId
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0250 |                     + "\nSchool Year: " + selectedYear.label
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0251 |                     + "\nStrand: " + selectedStrand.label
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0252 |                     + "\nGrade Level: "
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0253 |                     + gradeLevelBox.getSelectedItem()
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0254 |                     + "\nStatus: Pending"
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0255 |                     + "\nAssigned Section: Not yet assigned"
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0256 |                     + "\n\nNext step: upload your requirements."
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0257 |                 );
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0258 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0259 |                 helpLabel.setText("Your application has been saved.");
```

Explanation: Sets the displayed text of a GUI component.

```
0260 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0261 |                 JOptionPane.showMessageDialog(
```

Explanation: Displays a Swing dialog to communicate a message, warning, confirmation, or input prompt to the user.

```
0262 |                     dashboard,
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0263 |                     "Enrollment application submitted successfully!\n"
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0264 |                     + "Enrollment ID: " + enrollmentId
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0265 |                 );
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0266 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0267 |             } catch (SQLException ex) {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0268 |                 conn.rollback();
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0269 |                 throw ex;
```

Explanation: Raises an exception so the caller or an enclosing catch block can handle the failure.

```
0270 |             } finally {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0271 |                 conn.setAutoCommit(true);
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0272 |             }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0273 |         } catch (Exception ex) {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0274 |             StudentDashboard.showError(dashboard,
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0275 |                 "Could not submit enrollment application: "
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0276 |                 + ex.getMessage()
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0277 |             );
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0278 |         } finally {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0279 |             try {
```

Explanation: Begins a protected block for code that may throw an exception; resources may be managed by try-with-resources.

```
0280 |                 if (conn != null) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0281 |                     conn.close();
```

Explanation: Closes the associated resource, such as a JDBC ResultSet, Statement, or Connection.

```
0282 |                 }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0283 |             } catch (Exception closeEx) {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0284 |                 System.out.println(closeEx);
```

Explanation: Prints a message or value to the console, commonly for status output or error reporting.

```
0285 |             }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0286 |         }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0287 |     }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0288 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0289 |     public void printEnrollmentForm() {
```

Explanation: Declares the printEnrollmentForm method and its return type/parameters; the method performs the operation described by its name and body.

```
0290 |         String sql =
```

Explanation: Stores the SQL command as a string so the program can execute it against the database.

```
0291 |             "SELECT e.enrollment_id, e.year_level, e.enrollee_type, "
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0292 |             + "e.enrollment_status, sy.year_label, "
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0293 |             + "st.strand_code, st.strand_name, "
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0294 |             + "sec.section_name AS assigned_section, "
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0295 |             + "sec.grade_level AS section_grade_level, "
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0296 |             + "s.lrn, s.first_name, s.middle_name, s.last_name, "
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0297 |             + "s.previous_school "
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0298 |             + "FROM enrollments e "
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0299 |             + "JOIN students s ON e.student_id = s.student_id "
```

Explanation: Assigns or updates a variable with the value or result on the right-hand side.

```
0300 |             + "JOIN school_years sy "
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0301 |             + "ON e.school_year_id = sy.school_year_id "
```

Explanation: Assigns or updates a variable with the value or result on the right-hand side.

```
0302 |             + "JOIN strands st ON e.strand_id = st.strand_id "
```

Explanation: Assigns or updates a variable with the value or result on the right-hand side.

```
0303 |             + "LEFT JOIN sections sec ON e.section_id = sec.section_id "
```

Explanation: Assigns or updates a variable with the value or result on the right-hand side.

```
0304 |             + "WHERE s.user_id = ? "
```

Explanation: Assigns or updates a variable with the value or result on the right-hand side.

```
0305 |             + "ORDER BY e.enrollment_id DESC";
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0306 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0307 |         Connection conn = null;
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0308 |         PreparedStatement ps = null;
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0309 |         try {
```

Explanation: Begins a protected block for code that may throw an exception; resources may be managed by try-with-resources.

```
0310 |             conn = dashboard.getConnection();
```

Explanation: Assigns or updates a variable with the value or result on the right-hand side.

```
0311 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0312 |             if (conn == null) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0313 |                 throw new SQLException("Could not connect to the database.");
```

Explanation: Raises an exception so the caller or an enclosing catch block can handle the failure.

```
0314 |             }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0315 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0316 |             ps = conn.prepareStatement(sql);
```

Explanation: Creates a parameterized SQL statement; placeholders can be filled separately to reduce SQL-injection risk.

```
0317 |             ps.setInt(1, user.getUserId());
```

Explanation: Binds a Java value to a placeholder in the prepared SQL statement; the parameter index is one-based.

```
0318 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0319 |             try (ResultSet rs = ps.executeQuery()) {
```

Explanation: Begins a protected block for code that may throw an exception; resources may be managed by try-with-resources.

```
0320 |                 if (!rs.next()) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0321 |                     StudentDashboard.showError(dashboard,
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0322 |                         "No enrollment record found. "
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0323 |                         + "Submit an enrollment application first."
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0324 |                     );
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0325 |                     return;
```

Explanation: Ends the current method without returning a value.

```
0326 |                 }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0327 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0328 |                 StringBuilder form = new StringBuilder();
```

Explanation: Creates a mutable string builder so text can be assembled efficiently.

```
0329 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0330 |                 form.append("================================================\n");
```

Explanation: Appends the provided value to the current StringBuilder or collection.

```
0331 |                 form.append("          STUDENT ENROLLMENT FORM\n");
```

Explanation: Appends the provided value to the current StringBuilder or collection.

```
0332 |                 form.append("================================================\n\n");
```

Explanation: Appends the provided value to the current StringBuilder or collection.

```
0333 |                 form.append("ENROLLMENT INFORMATION\n");
```

Explanation: Appends the provided value to the current StringBuilder or collection.

```
0334 |                 form.append("------------------------------------------------\n");
```

Explanation: Appends the provided value to the current StringBuilder or collection.

```
0335 |                 form.append("Enrollment ID: ")
```

Explanation: Appends the provided value to the current StringBuilder or collection.

```
0336 |                     .append(rs.getInt("enrollment_id")).append("\n");
```

Explanation: Reads a column value from the current database result row using the requested Java type.

```
0337 |                 form.append("School Year: ")
```

Explanation: Appends the provided value to the current StringBuilder or collection.

```
0338 |                     .append(rs.getString("year_label")).append("\n");
```

Explanation: Reads a column value from the current database result row using the requested Java type.

```
0339 |                 form.append("Enrollment Status: ")
```

Explanation: Appends the provided value to the current StringBuilder or collection.

```
0340 |                     .append(rs.getString("enrollment_status")).append("\n\n");
```

Explanation: Reads a column value from the current database result row using the requested Java type.

```
0341 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0342 |                 form.append("STUDENT INFORMATION\n");
```

Explanation: Appends the provided value to the current StringBuilder or collection.

```
0343 |                 form.append("------------------------------------------------\n");
```

Explanation: Appends the provided value to the current StringBuilder or collection.

```
0344 |                 form.append("LRN: ")
```

Explanation: Appends the provided value to the current StringBuilder or collection.

```
0345 |                     .append(rs.getString("lrn")).append("\n");
```

Explanation: Reads a column value from the current database result row using the requested Java type.

```
0346 |                 form.append("First Name: ")
```

Explanation: Appends the provided value to the current StringBuilder or collection.

```
0347 |                     .append(rs.getString("first_name")).append("\n");
```

Explanation: Reads a column value from the current database result row using the requested Java type.

```
0348 |                 form.append("Middle Name: ")
```

Explanation: Appends the provided value to the current StringBuilder or collection.

```
0349 |                     .append(valueOrBlank(rs.getString("middle_name")))
```

Explanation: Reads a column value from the current database result row using the requested Java type.

```
0350 |                     .append("\n");
```

Explanation: Appends the provided value to the current StringBuilder or collection.

```
0351 |                 form.append("Last Name: ")
```

Explanation: Appends the provided value to the current StringBuilder or collection.

```
0352 |                     .append(rs.getString("last_name")).append("\n");
```

Explanation: Reads a column value from the current database result row using the requested Java type.

```
0353 |                 form.append("Previous School: ")
```

Explanation: Appends the provided value to the current StringBuilder or collection.

```
0354 |                     .append(rs.getString("previous_school")).append("\n\n");
```

Explanation: Reads a column value from the current database result row using the requested Java type.

```
0355 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0356 |                 form.append("ACADEMIC INFORMATION\n");
```

Explanation: Appends the provided value to the current StringBuilder or collection.

```
0357 |                 form.append("-----------------------------------------------\n");
```

Explanation: Appends the provided value to the current StringBuilder or collection.

```
0358 |                 form.append("Grade Level: ")
```

Explanation: Appends the provided value to the current StringBuilder or collection.

```
0359 |                     .append(rs.getInt("year_level")).append("\n");
```

Explanation: Reads a column value from the current database result row using the requested Java type.

```
0360 |                 form.append("Strand: ")
```

Explanation: Appends the provided value to the current StringBuilder or collection.

```
0361 |                     .append(rs.getString("strand_code"))
```

Explanation: Reads a column value from the current database result row using the requested Java type.

```
0362 |                     .append(" - ")
```

Explanation: Appends the provided value to the current StringBuilder or collection.

```
0363 |                     .append(rs.getString("strand_name")).append("\n");
```

Explanation: Reads a column value from the current database result row using the requested Java type.

```
0364 |                 form.append("Enrollee Type: ")
```

Explanation: Appends the provided value to the current StringBuilder or collection.

```
0365 |                     .append(rs.getString("enrollee_type")).append("\n");
```

Explanation: Reads a column value from the current database result row using the requested Java type.

```
0366 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0367 |                 String section = rs.getString("assigned_section");
```

Explanation: Reads a column value from the current database result row using the requested Java type.

```
0368 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0369 |                 form.append("Assigned Section: ");
```

Explanation: Appends the provided value to the current StringBuilder or collection.

```
0370 |                 if (section == null || section.trim().isEmpty()) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0371 |                     form.append("Not yet assigned\n");
```

Explanation: Appends the provided value to the current StringBuilder or collection.

```
0372 |                 } else {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0373 |                     form.append(section).append("\n");
```

Explanation: Appends the provided value to the current StringBuilder or collection.

```
0374 |                     form.append("Section Grade Level: ")
```

Explanation: Appends the provided value to the current StringBuilder or collection.

```
0375 |                         .append(rs.getInt("section_grade_level"))
```

Explanation: Reads a column value from the current database result row using the requested Java type.

```
0376 |                         .append("\n");
```

Explanation: Appends the provided value to the current StringBuilder or collection.

```
0377 |                 }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0378 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0379 |                 form.append("\n");
```

Explanation: Appends the provided value to the current StringBuilder or collection.

```
0380 |                 form.append("Student Signature: _____\n\n");
```

Explanation: Appends the provided value to the current StringBuilder or collection.

```
0381 |                 form.append("Parent/Guardian Signature: _____\n\n");
```

Explanation: Appends the provided value to the current StringBuilder or collection.

```
0382 |                 form.append("Enrollment Staff: _____\n\n");
```

Explanation: Appends the provided value to the current StringBuilder or collection.

```
0383 |                 form.append("Date: _____\n\n");
```

Explanation: Appends the provided value to the current StringBuilder or collection.

```
0384 |                 form.append("================================================\n");
```

Explanation: Appends the provided value to the current StringBuilder or collection.

```
0385 |                 form.append("This form reflects the latest saved enrollment record.\n");
```

Explanation: Appends the provided value to the current StringBuilder or collection.

```
0386 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0387 |                 String formText = form.toString();
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0388 |                 outputArea.setText(formText);
```

Explanation: Sets the displayed text of a GUI component.

```
0389 |                 helpLabel.setText(
```

Explanation: Sets the displayed text of a GUI component.

```
0390 |                     "Enrollment form is ready to print."
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0391 |                 );
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0392 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0393 |                 JTextArea printArea = new JTextArea(formText);
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0394 |                 printArea.setFont(new Font("Monospaced", Font.PLAIN, 12));
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0395 |                 printArea.setEditable(false);
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0396 |                 printArea.setLineWrap(false);
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0397 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0398 |                 try {
```

Explanation: Begins a protected block for code that may throw an exception; resources may be managed by try-with-resources.

```
0399 |                     boolean printed = printArea.print();
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0400 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0401 |                     if (printed) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0402 |                         JOptionPane.showMessageDialog(
```

Explanation: Displays a Swing dialog to communicate a message, warning, confirmation, or input prompt to the user.

```
0403 |                             dashboard,
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0404 |                             "Enrollment form sent to the printer."
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0405 |                         );
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0406 |                     }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0407 |                 } catch (java.awt.print.PrinterException ex) {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0408 |                     StudentDashboard.showError(dashboard,
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0409 |                         "Could not print enrollment form: "
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0410 |                         + ex.getMessage()
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0411 |                     );
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0412 |                 }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0413 |             }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0414 |         } catch (Exception ex) {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0415 |             StudentDashboard.showError(dashboard,
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0416 |                 "Could not prepare enrollment form: " + ex.getMessage()
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0417 |             );
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0418 |         } finally {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0419 |             try {
```

Explanation: Begins a protected block for code that may throw an exception; resources may be managed by try-with-resources.

```
0420 |                 if (ps != null) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0421 |                     ps.close();
```

Explanation: Closes the associated resource, such as a JDBC ResultSet, Statement, or Connection.

```
0422 |                 }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0423 |             } catch (Exception closeEx) {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0424 |                 System.out.println(closeEx);
```

Explanation: Prints a message or value to the console, commonly for status output or error reporting.

```
0425 |             }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0426 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0427 |             try {
```

Explanation: Begins a protected block for code that may throw an exception; resources may be managed by try-with-resources.

```
0428 |                 if (conn != null) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0429 |                     conn.close();
```

Explanation: Closes the associated resource, such as a JDBC ResultSet, Statement, or Connection.

```
0430 |                 }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0431 |             } catch (Exception closeEx) {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0432 |                 System.out.println(closeEx);
```

Explanation: Prints a message or value to the console, commonly for status output or error reporting.

```
0433 |             }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0434 |         }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0435 |     }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0436 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0437 |     private String valueOrBlank(String value) {
```

Explanation: Declares the valueOrBlank method and its return type/parameters; the method performs the operation described by its name and body.

```
0438 |         return value == null ? "" : value;
```

Explanation: Returns the value or result to the caller and ends the current method.

```
0439 |     }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0440 | }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

StudentPaymentActions.java

Project path: src/StudentPaymentActions.java

```
0001 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0002 | import javax.swing.*;
```

Explanation: Imports Swing GUI classes used to build the application's windows and controls.

```
0003 | import java.awt.*;
```

Explanation: Imports AWT classes used for layout, dimensions, colors, fonts, or UI events.

```
0004 | import java.sql.*;
```

Explanation: Imports JDBC classes used to communicate with a relational database.

```
0005 | import java.util.ArrayList;
```

Explanation: Imports Java utility classes such as collections, lists, maps, or helper types.

```
0006 | import java.util.List;
```

Explanation: Imports Java utility classes such as collections, lists, maps, or helper types.

```
0007 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0008 | public class StudentPaymentActions {
```

Explanation: Declares the StudentPaymentActions type, which groups related data and behavior.

```
0009 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0010 |     private StudentDashboard dashboard;
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0011 |     private User user;
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0012 |     private JTextArea outputArea;
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0013 |     private JLabel helpLabel;
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0014 |     private PaymentDAO paymentDAO = new PaymentDAO();
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0015 |     private StudentPaymentDAO studentPaymentDAO = new StudentPaymentDAO();
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0016 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0017 |     public StudentPaymentActions(StudentDashboard dashboard) {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0018 |         this.dashboard = dashboard;
```

Explanation: Assigns or updates a variable with the value or result on the right-hand side.

```
0019 |         this.user = dashboard.user;
```

Explanation: Assigns or updates a variable with the value or result on the right-hand side.

```
0020 |         this.outputArea = dashboard.outputArea;
```

Explanation: Assigns or updates a variable with the value or result on the right-hand side.

```
0021 |         this.helpLabel = dashboard.helpLabel;
```

Explanation: Assigns or updates a variable with the value or result on the right-hand side.

```
0022 |     }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0023 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0024 |     public void submitPayment() {
```

Explanation: Declares the submitPayment method and its return type/parameters; the method performs the operation described by its name and body.

```
0025 |         List&lt;StudentDashboard.ChoiceItem&gt; requests = new ArrayList&lt;&gt;();
```

Explanation: Assigns or updates a variable with the value or result on the right-hand side.

```
0026 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0027 |         String sql =
```

Explanation: Stores the SQL command as a string so the program can execute it against the database.

```
0028 |             "SELECT pr.request_id, pr.amount_due, pr.description "
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0029 |             + "FROM payment_requests pr "
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0030 |             + "JOIN enrollments e "
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0031 |             + "ON pr.enrollment_id = e.enrollment_id "
```

Explanation: Assigns or updates a variable with the value or result on the right-hand side.

```
0032 |             + "JOIN students s ON e.student_id = s.student_id "
```

Explanation: Assigns or updates a variable with the value or result on the right-hand side.

```
0033 |             + "WHERE s.user_id = ? AND pr.request_status = 'Open' "
```

Explanation: Assigns or updates a variable with the value or result on the right-hand side.

```
0034 |             + "ORDER BY pr.request_id DESC";
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0035 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0036 |         Connection conn = null;
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0037 |         PreparedStatement ps = null;
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0038 |         try {
```

Explanation: Begins a protected block for code that may throw an exception; resources may be managed by try-with-resources.

```
0039 |             conn = dashboard.getConnection();
```

Explanation: Assigns or updates a variable with the value or result on the right-hand side.

```
0040 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0041 |             if (conn == null) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0042 |                 throw new SQLException("Could not connect to the database.");
```

Explanation: Raises an exception so the caller or an enclosing catch block can handle the failure.

```
0043 |             }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0044 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0045 |             ps = conn.prepareStatement(sql);
```

Explanation: Creates a parameterized SQL statement; placeholders can be filled separately to reduce SQL-injection risk.

```
0046 |             ps.setInt(1, user.getUserId());
```

Explanation: Binds a Java value to a placeholder in the prepared SQL statement; the parameter index is one-based.

```
0047 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0048 |             try (ResultSet rs = ps.executeQuery()) {
```

Explanation: Begins a protected block for code that may throw an exception; resources may be managed by try-with-resources.

```
0049 |                 while (rs.next()) {
```

Explanation: Repeats the following block while the condition remains true.

```
0050 |                     requests.add(new StudentDashboard.ChoiceItem(
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0051 |                         rs.getInt("request_id"),
```

Explanation: Reads a column value from the current database result row using the requested Java type.

```
0052 |                         "Request #" + rs.getInt("request_id")
```

Explanation: Reads a column value from the current database result row using the requested Java type.

```
0053 |                             + " - PHP "
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0054 |                             + String.format(
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0055 |                                 "%.2f", rs.getDouble("amount_due")
```

Explanation: Reads a column value from the current database result row using the requested Java type.

```
0056 |                             )
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0057 |                             + " - " + rs.getString("description")
```

Explanation: Reads a column value from the current database result row using the requested Java type.

```
0058 |                     ));
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0059 |                 }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0060 |             }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0061 |         } catch (Exception ex) {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0062 |             StudentDashboard.showError(dashboard,
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0063 |                 "Could not load payment requests: " + ex.getMessage()
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0064 |             );
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0065 |             return;
```

Explanation: Ends the current method without returning a value.

```
0066 |         } finally {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0067 |             try {
```

Explanation: Begins a protected block for code that may throw an exception; resources may be managed by try-with-resources.

```
0068 |                 if (ps != null) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0069 |                     ps.close();
```

Explanation: Closes the associated resource, such as a JDBC ResultSet, Statement, or Connection.

```
0070 |                 }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0071 |             } catch (Exception closeEx) {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0072 |                 System.out.println(closeEx);
```

Explanation: Prints a message or value to the console, commonly for status output or error reporting.

```
0073 |             }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0074 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0075 |             try {
```

Explanation: Begins a protected block for code that may throw an exception; resources may be managed by try-with-resources.

```
0076 |                 if (conn != null) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0077 |                     conn.close();
```

Explanation: Closes the associated resource, such as a JDBC ResultSet, Statement, or Connection.

```
0078 |                 }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0079 |             } catch (Exception closeEx) {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0080 |                 System.out.println(closeEx);
```

Explanation: Prints a message or value to the console, commonly for status output or error reporting.

```
0081 |             }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0082 |         }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0083 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0084 |         if (requests.isEmpty()) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0085 |             StudentDashboard.showError(dashboard, "You have no open payment requests.");
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0086 |             return;
```

Explanation: Ends the current method without returning a value.

```
0087 |         }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0088 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0089 |         StudentDashboard.ChoiceItem selectedRequest = (StudentDashboard.ChoiceItem)
```

Explanation: Assigns or updates a variable with the value or result on the right-hand side.

```
0090 |             JOptionPane.showInputDialog(
```

Explanation: Displays a Swing dialog to communicate a message, warning, confirmation, or input prompt to the user.

```
0091 |                 dashboard,
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0092 |                 "Select a payment request:",
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0093 |                 "Submit Payment",
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0094 |                 JOptionPane.QUESTION_MESSAGE,
```

Explanation: Displays a Swing dialog to communicate a message, warning, confirmation, or input prompt to the user.

```
0095 |                 null,
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0096 |                 requests.toArray(),
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0097 |                 requests.get(0)
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0098 |             );
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0099 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0100 |         if (selectedRequest == null) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0101 |             return;
```

Explanation: Ends the current method without returning a value.

```
0102 |         }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0103 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0104 |         JComboBox&lt;String&gt; methodBox = new JComboBox&lt;&gt;(
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0105 |             new String[]{
```

Explanation: Creates an instance of the specified class or object.

```
0106 |                 "Cash",
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0107 |                 "Bank Transfer",
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0108 |                 "GCash",
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0109 |                 "Maya",
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0110 |                 "Other"
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0111 |             }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0112 |         );
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0113 |         JTextField referenceField = new JTextField();
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0114 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0115 |         JPanel form = new JPanel(new GridLayout(0, 2, 8, 8));
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0116 |         form.add(new JLabel("Payment Method:"));
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0117 |         form.add(methodBox);
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0118 |         form.add(new JLabel("Reference Number:"));
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0119 |         form.add(referenceField);
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0120 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0121 |         int result = JOptionPane.showConfirmDialog(
```

Explanation: Displays a Swing dialog to communicate a message, warning, confirmation, or input prompt to the user.

```
0122 |             dashboard,
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0123 |             form,
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0124 |             "Payment Information",
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0125 |             JOptionPane.OK_CANCEL_OPTION,
```

Explanation: Displays a Swing dialog to communicate a message, warning, confirmation, or input prompt to the user.

```
0126 |             JOptionPane.PLAIN_MESSAGE
```

Explanation: Displays a Swing dialog to communicate a message, warning, confirmation, or input prompt to the user.

```
0127 |         );
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0128 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0129 |         if (result != JOptionPane.OK_OPTION) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0130 |             return;
```

Explanation: Ends the current method without returning a value.

```
0131 |         }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0132 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0133 |         String method = methodBox.getSelectedItem().toString();
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0134 |         String reference = referenceField.getText().trim();
```

Explanation: Reads text currently entered or displayed by a GUI component.

```
0135 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0136 |         if (reference.isEmpty()) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0137 |             StudentDashboard.showError(dashboard, "Enter your payment reference number.");
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0138 |             return;
```

Explanation: Ends the current method without returning a value.

```
0139 |         }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0140 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0141 |         int confirm = JOptionPane.showConfirmDialog(
```

Explanation: Displays a Swing dialog to communicate a message, warning, confirmation, or input prompt to the user.

```
0142 |             dashboard,
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0143 |             "Submit payment information?\n\n"
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0144 |                 + selectedRequest.label
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0145 |                 + "\nMethod: " + method
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0146 |                 + "\nReference: " + reference,
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0147 |             "Confirm Payment Submission",
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0148 |             JOptionPane.YES_NO_OPTION
```

Explanation: Displays a Swing dialog to communicate a message, warning, confirmation, or input prompt to the user.

```
0149 |         );
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0150 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0151 |         if (confirm != JOptionPane.YES_OPTION) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0152 |             return;
```

Explanation: Ends the current method without returning a value.

```
0153 |         }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0154 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0155 |         try {
```

Explanation: Begins a protected block for code that may throw an exception; resources may be managed by try-with-resources.

```
0156 |             studentPaymentDAO.addPaymentForStudent(
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0157 |                 selectedRequest.id,
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0158 |                 user.getUserId(),
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0159 |                 method,
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0160 |                 reference
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0161 |             );
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0162 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0163 |             outputArea.setText(
```

Explanation: Sets the displayed text of a GUI component.

```
0164 |                 "PAYMENT INFORMATION SUBMITTED\n\n"
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0165 |                 + selectedRequest.label
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0166 |                 + "\nMethod: " + method
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0167 |                 + "\nReference: " + reference
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0168 |                 + "\n\nYour payment is awaiting verification."
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0169 |             );
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0170 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0171 |             JOptionPane.showMessageDialog(
```

Explanation: Displays a Swing dialog to communicate a message, warning, confirmation, or input prompt to the user.

```
0172 |                 dashboard,
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0173 |                 "Payment information submitted. "
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0174 |                 + "Wait for staff verification."
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0175 |             );
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0176 |         } catch (Exception ex) {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0177 |             StudentDashboard.showError(dashboard,
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0178 |                 "Could not submit payment: " + ex.getMessage()
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0179 |             );
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0180 |         }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0181 |     }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0182 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0183 |     public void viewPaymentRequests() {
```

Explanation: Declares the viewPaymentRequests method and its return type/parameters; the method performs the operation described by its name and body.

```
0184 |         try {
```

Explanation: Begins a protected block for code that may throw an exception; resources may be managed by try-with-resources.

```
0185 |             outputArea.setText("MY PAYMENT REQUESTS\n\n"
```

Explanation: Sets the displayed text of a GUI component.

```
0186 |                 + studentPaymentDAO.getPaymentRequestsForStudent(user.getUserId()));
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0187 |         } catch (SQLException ex) {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0188 |             StudentDashboard.showError(
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0189 |                 dashboard,
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0190 |                 "Could not load payment requests: " + ex.getMessage()
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0191 |             );
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0192 |         }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0193 |     }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0194 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0195 |     public void viewRequestBreakdown() {
```

Explanation: Declares the viewRequestBreakdown method and its return type/parameters; the method performs the operation described by its name and body.

```
0196 |         String input = JOptionPane.showInputDialog(
```

Explanation: Displays a Swing dialog to communicate a message, warning, confirmation, or input prompt to the user.

```
0197 |             dashboard,
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0198 |             "Enter the payment request ID:"
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0199 |         );
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0200 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0201 |         if (input == null || input.trim().isEmpty()) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0202 |             return;
```

Explanation: Ends the current method without returning a value.

```
0203 |         }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0204 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0205 |         int requestId;
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0206 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0207 |         try {
```

Explanation: Begins a protected block for code that may throw an exception; resources may be managed by try-with-resources.

```
0208 |             requestId = Integer.parseInt(input.trim());
```

Explanation: Assigns or updates a variable with the value or result on the right-hand side.

```
0209 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0210 |             if (requestId &lt;= 0) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0211 |                 throw new NumberFormatException();
```

Explanation: Raises an exception so the caller or an enclosing catch block can handle the failure.

```
0212 |             }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0213 |         } catch (NumberFormatException ex) {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0214 |             StudentDashboard.showError(dashboard, "Enter a valid positive request ID.");
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0215 |             return;
```

Explanation: Ends the current method without returning a value.

```
0216 |         }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0217 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0218 |         try {
```

Explanation: Begins a protected block for code that may throw an exception; resources may be managed by try-with-resources.

```
0219 |             outputArea.setText("PAYMENT REQUEST BREAKDOWN\n\n"
```

Explanation: Sets the displayed text of a GUI component.

```
0220 |                 + studentPaymentDAO.getRequestDetailsForStudent(requestId, user.getUserId()));
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0221 |         } catch (SQLException ex) {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0222 |             StudentDashboard.showError(
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0223 |                 dashboard,
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0224 |                 "Could not load request breakdown: " + ex.getMessage()
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0225 |             );
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0226 |         }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0227 |     }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0228 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0229 |     public void viewMyPayments() {
```

Explanation: Declares the viewMyPayments method and its return type/parameters; the method performs the operation described by its name and body.

```
0230 |         try {
```

Explanation: Begins a protected block for code that may throw an exception; resources may be managed by try-with-resources.

```
0231 |             outputArea.setText("MY PAYMENT RECORDS\n\n"
```

Explanation: Sets the displayed text of a GUI component.

```
0232 |                 + paymentDAO.getPaymentsForStudent(user.getUserId()));
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0233 |         } catch (SQLException ex) {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0234 |             StudentDashboard.showError(
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0235 |                 dashboard,
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0236 |                 "Could not load payments: " + ex.getMessage()
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0237 |             );
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0238 |         }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0239 |     }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0240 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0241 | }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

StudentPaymentDAO.java

Project path: src/StudentPaymentDAO.java

```
0001 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0002 | import java.sql.*;
```

Explanation: Imports JDBC classes used to communicate with a relational database.

```
0003 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0004 | public class StudentPaymentDAO {
```

Explanation: Declares the StudentPaymentDAO type, which groups related data and behavior.

```
0005 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0006 |     public String getOpenPaymentRequestsForStudent(int userId) throws SQLException {
```

Explanation: Declares the getOpenPaymentRequestsForStudent method, typically returning the requested field or resource.

```
0007 |         return readRequests(userId, true);
```

Explanation: Returns the value or result to the caller and ends the current method.

```
0008 |     }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0009 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0010 |     public String getPaymentRequestsForStudent(int userId) throws SQLException {
```

Explanation: Declares the getPaymentRequestsForStudent method, typically returning the requested field or resource.

```
0011 |         return readRequests(userId, false);
```

Explanation: Returns the value or result to the caller and ends the current method.

```
0012 |     }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0013 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0014 |     private String readRequests(int userId, boolean onlyUnpaid) throws SQLException {
```

Explanation: Declares the readRequests method and its return type/parameters; the method performs the operation described by its name and body.

```
0015 |         String sql =
```

Explanation: Stores the SQL command as a string so the program can execute it against the database.

```
0016 |                 "SELECT pr.request_id, pr.enrollment_id, pr.amount_due, " +
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0017 |                 "pr.description, pr.request_status, pr.created_at " +
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0018 |                 "FROM payment_requests pr " +
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0019 |                 "JOIN enrollments e ON pr.enrollment_id = e.enrollment_id " +
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0020 |                 "JOIN students s ON e.student_id = s.student_id " +
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0021 |                 "WHERE s.user_id = ? " +
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0022 |                 (onlyUnpaid ? "AND pr.request_status &lt;&gt; 'Paid' " : "") +
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0023 |                 "ORDER BY pr.request_id DESC";
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0024 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0025 |         StringBuilder result = new StringBuilder();
```

Explanation: Creates a mutable string builder so text can be assembled efficiently.

```
0026 |         boolean found = false;
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0027 |         Connection conn = null;
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0028 |         PreparedStatement ps = null;
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0029 |         ResultSet rs = null;
```

Explanation: Declares or assigns a ResultSet that holds rows returned by a database query.

```
0030 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0031 |         try {
```

Explanation: Begins a protected block for code that may throw an exception; resources may be managed by try-with-resources.

```
0032 |             conn = new DbConnection().getConnection();
```

Explanation: Assigns or updates a variable with the value or result on the right-hand side.

```
0033 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0034 |             if (conn == null) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0035 |                 throw new SQLException("Could not connect to the database.");
```

Explanation: Raises an exception so the caller or an enclosing catch block can handle the failure.

```
0036 |             }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0037 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0038 |             ps = conn.prepareStatement(sql);
```

Explanation: Creates a parameterized SQL statement; placeholders can be filled separately to reduce SQL-injection risk.

```
0039 |             ps.setInt(1, userId);
```

Explanation: Binds a Java value to a placeholder in the prepared SQL statement; the parameter index is one-based.

```
0040 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0041 |             rs = ps.executeQuery();
```

Explanation: Executes a SQL query that returns rows, usually a SELECT, and stores them in a ResultSet.

```
0042 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0043 |             while (rs.next()) {
```

Explanation: Repeats the following block while the condition remains true.

```
0044 |                 found = true;
```

Explanation: Assigns or updates a variable with the value or result on the right-hand side.

```
0045 |                 result.append("Request ID: ").append(rs.getInt("request_id"))
```

Explanation: Reads a column value from the current database result row using the requested Java type.

```
0046 |                         .append("\nEnrollment ID: ").append(rs.getInt("enrollment_id"))
```

Explanation: Reads a column value from the current database result row using the requested Java type.

```
0047 |                         .append("\nAmount Due: PHP ")
```

Explanation: Appends the provided value to the current StringBuilder or collection.

```
0048 |                         .append(String.format("%.2f", rs.getDouble("amount_due")))
```

Explanation: Reads a column value from the current database result row using the requested Java type.

```
0049 |                         .append("\nDescription: ").append(rs.getString("description"))
```

Explanation: Reads a column value from the current database result row using the requested Java type.

```
0050 |                         .append("\nStatus: ").append(rs.getString("request_status"))
```

Explanation: Reads a column value from the current database result row using the requested Java type.

```
0051 |                         .append("\nCreated: ").append(rs.getTimestamp("created_at"))
```

Explanation: Appends the provided value to the current StringBuilder or collection.

```
0052 |                         .append("\n----------------------------\n");
```

Explanation: Appends the provided value to the current StringBuilder or collection.

```
0053 |             }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0054 |         } catch (SQLException e) {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0055 |             throw e;
```

Explanation: Raises an exception so the caller or an enclosing catch block can handle the failure.

```
0056 |         } catch (Exception e) {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0057 |             throw new SQLException(e.getMessage(), e);
```

Explanation: Raises an exception so the caller or an enclosing catch block can handle the failure.

```
0058 |         } finally {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0059 |             try {
```

Explanation: Begins a protected block for code that may throw an exception; resources may be managed by try-with-resources.

```
0060 |                 if (rs != null) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0061 |                     rs.close();
```

Explanation: Closes the associated resource, such as a JDBC ResultSet, Statement, or Connection.

```
0062 |                 }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0063 |             } catch (Exception e) {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0064 |                 System.out.println(e);
```

Explanation: Prints a message or value to the console, commonly for status output or error reporting.

```
0065 |             }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0066 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0067 |             try {
```

Explanation: Begins a protected block for code that may throw an exception; resources may be managed by try-with-resources.

```
0068 |                 if (ps != null) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0069 |                     ps.close();
```

Explanation: Closes the associated resource, such as a JDBC ResultSet, Statement, or Connection.

```
0070 |                 }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0071 |             } catch (Exception e) {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0072 |                 System.out.println(e);
```

Explanation: Prints a message or value to the console, commonly for status output or error reporting.

```
0073 |             }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0074 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0075 |             try {
```

Explanation: Begins a protected block for code that may throw an exception; resources may be managed by try-with-resources.

```
0076 |                 if (conn != null) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0077 |                     conn.close();
```

Explanation: Closes the associated resource, such as a JDBC ResultSet, Statement, or Connection.

```
0078 |                 }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0079 |             } catch (Exception e) {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0080 |                 System.out.println(e);
```

Explanation: Prints a message or value to the console, commonly for status output or error reporting.

```
0081 |             }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0082 |         }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0083 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0084 |         if (!found) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0085 |             result.append("No payment requests found.");
```

Explanation: Appends the provided value to the current StringBuilder or collection.

```
0086 |         }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0087 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0088 |         return result.toString();
```

Explanation: Returns the value or result to the caller and ends the current method.

```
0089 |     }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0090 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0091 |     public String getRequestDetailsForStudent(int requestId, int userId)
```

Explanation: Declares the getRequestDetailsForStudent method, typically returning the requested field or resource.

```
0092 |             throws SQLException {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0093 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0094 |         String requestSql =
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0095 |                 "SELECT pr.request_id, pr.enrollment_id, pr.amount_due, " +
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0096 |                 "pr.description, pr.request_status " +
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0097 |                 "FROM payment_requests pr " +
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0098 |                 "JOIN enrollments e ON pr.enrollment_id = e.enrollment_id " +
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0099 |                 "JOIN students s ON e.student_id = s.student_id " +
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0100 |                 "WHERE pr.request_id = ? AND s.user_id = ?";
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0101 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0102 |         String detailSql =
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0103 |                 "SELECT item_name, item_type, amount " +
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0104 |                 "FROM payment_request_details WHERE request_id = ?";
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0105 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0106 |         StringBuilder result = new StringBuilder();
```

Explanation: Creates a mutable string builder so text can be assembled efficiently.

```
0107 |         Connection conn = null;
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0108 |         PreparedStatement ps = null;
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0109 |         ResultSet rs = null;
```

Explanation: Declares or assigns a ResultSet that holds rows returned by a database query.

```
0110 |         PreparedStatement psDetails = null;
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0111 |         ResultSet rsDetails = null;
```

Explanation: Declares or assigns a ResultSet that holds rows returned by a database query.

```
0112 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0113 |         try {
```

Explanation: Begins a protected block for code that may throw an exception; resources may be managed by try-with-resources.

```
0114 |             conn = new DbConnection().getConnection();
```

Explanation: Assigns or updates a variable with the value or result on the right-hand side.

```
0115 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0116 |             if (conn == null) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0117 |                 throw new SQLException("Could not connect to the database.");
```

Explanation: Raises an exception so the caller or an enclosing catch block can handle the failure.

```
0118 |             }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0119 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0120 |             ps = conn.prepareStatement(requestSql);
```

Explanation: Creates a parameterized SQL statement; placeholders can be filled separately to reduce SQL-injection risk.

```
0121 |             ps.setInt(1, requestId);
```

Explanation: Binds a Java value to a placeholder in the prepared SQL statement; the parameter index is one-based.

```
0122 |             ps.setInt(2, userId);
```

Explanation: Binds a Java value to a placeholder in the prepared SQL statement; the parameter index is one-based.

```
0123 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0124 |             rs = ps.executeQuery();
```

Explanation: Executes a SQL query that returns rows, usually a SELECT, and stores them in a ResultSet.

```
0125 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0126 |             if (!rs.next()) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0127 |                 return "Payment request not found for your account.";
```

Explanation: Returns the value or result to the caller and ends the current method.

```
0128 |             }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0129 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0130 |             result.append("Request ID: ").append(rs.getInt("request_id"))
```

Explanation: Reads a column value from the current database result row using the requested Java type.

```
0131 |                     .append("\nEnrollment ID: ").append(rs.getInt("enrollment_id"))
```

Explanation: Reads a column value from the current database result row using the requested Java type.

```
0132 |                     .append("\nDescription: ").append(rs.getString("description"))
```

Explanation: Reads a column value from the current database result row using the requested Java type.

```
0133 |                     .append("\nAmount Due: PHP ")
```

Explanation: Appends the provided value to the current StringBuilder or collection.

```
0134 |                     .append(String.format("%.2f", rs.getDouble("amount_due")))
```

Explanation: Reads a column value from the current database result row using the requested Java type.

```
0135 |                     .append("\nStatus: ").append(rs.getString("request_status"))
```

Explanation: Reads a column value from the current database result row using the requested Java type.

```
0136 |                     .append("\n\nITEM BREAKDOWN\n");
```

Explanation: Appends the provided value to the current StringBuilder or collection.

```
0137 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0138 |             psDetails = conn.prepareStatement(detailSql);
```

Explanation: Creates a parameterized SQL statement; placeholders can be filled separately to reduce SQL-injection risk.

```
0139 |             psDetails.setInt(1, requestId);
```

Explanation: Binds a Java value to a placeholder in the prepared SQL statement; the parameter index is one-based.

```
0140 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0141 |             rsDetails = psDetails.executeQuery();
```

Explanation: Executes a SQL query that returns rows, usually a SELECT, and stores them in a ResultSet.

```
0142 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0143 |             while (rsDetails.next()) {
```

Explanation: Repeats the following block while the condition remains true.

```
0144 |                 result.append(rsDetails.getString("item_name"))
```

Explanation: Reads a column value from the current database result row using the requested Java type.

```
0145 |                         .append(" (").append(rsDetails.getString("item_type"))
```

Explanation: Reads a column value from the current database result row using the requested Java type.

```
0146 |                         .append("): PHP ")
```

Explanation: Appends the provided value to the current StringBuilder or collection.

```
0147 |                         .append(String.format("%.2f", rsDetails.getDouble("amount")))
```

Explanation: Reads a column value from the current database result row using the requested Java type.

```
0148 |                         .append("\n");
```

Explanation: Appends the provided value to the current StringBuilder or collection.

```
0149 |             }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0150 |         } catch (SQLException e) {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0151 |             throw e;
```

Explanation: Raises an exception so the caller or an enclosing catch block can handle the failure.

```
0152 |         } catch (Exception e) {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0153 |             throw new SQLException(e.getMessage(), e);
```

Explanation: Raises an exception so the caller or an enclosing catch block can handle the failure.

```
0154 |         } finally {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0155 |             try {
```

Explanation: Begins a protected block for code that may throw an exception; resources may be managed by try-with-resources.

```
0156 |                 if (rsDetails != null) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0157 |                     rsDetails.close();
```

Explanation: Closes the associated resource, such as a JDBC ResultSet, Statement, or Connection.

```
0158 |                 }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0159 |             } catch (Exception e) {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0160 |                 System.out.println(e);
```

Explanation: Prints a message or value to the console, commonly for status output or error reporting.

```
0161 |             }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0162 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0163 |             try {
```

Explanation: Begins a protected block for code that may throw an exception; resources may be managed by try-with-resources.

```
0164 |                 if (psDetails != null) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0165 |                     psDetails.close();
```

Explanation: Closes the associated resource, such as a JDBC ResultSet, Statement, or Connection.

```
0166 |                 }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0167 |             } catch (Exception e) {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0168 |                 System.out.println(e);
```

Explanation: Prints a message or value to the console, commonly for status output or error reporting.

```
0169 |             }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0170 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0171 |             try {
```

Explanation: Begins a protected block for code that may throw an exception; resources may be managed by try-with-resources.

```
0172 |                 if (rs != null) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0173 |                     rs.close();
```

Explanation: Closes the associated resource, such as a JDBC ResultSet, Statement, or Connection.

```
0174 |                 }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0175 |             } catch (Exception e) {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0176 |                 System.out.println(e);
```

Explanation: Prints a message or value to the console, commonly for status output or error reporting.

```
0177 |             }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0178 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0179 |             try {
```

Explanation: Begins a protected block for code that may throw an exception; resources may be managed by try-with-resources.

```
0180 |                 if (ps != null) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0181 |                     ps.close();
```

Explanation: Closes the associated resource, such as a JDBC ResultSet, Statement, or Connection.

```
0182 |                 }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0183 |             } catch (Exception e) {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0184 |                 System.out.println(e);
```

Explanation: Prints a message or value to the console, commonly for status output or error reporting.

```
0185 |             }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0186 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0187 |             try {
```

Explanation: Begins a protected block for code that may throw an exception; resources may be managed by try-with-resources.

```
0188 |                 if (conn != null) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0189 |                     conn.close();
```

Explanation: Closes the associated resource, such as a JDBC ResultSet, Statement, or Connection.

```
0190 |                 }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0191 |             } catch (Exception e) {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0192 |                 System.out.println(e);
```

Explanation: Prints a message or value to the console, commonly for status output or error reporting.

```
0193 |             }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0194 |         }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0195 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0196 |         return result.toString();
```

Explanation: Returns the value or result to the caller and ends the current method.

```
0197 |     }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0198 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0199 |     public String addPaymentForStudent(
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0200 |             int requestId, int userId, String method, String reference)
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0201 |             throws SQLException {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0202 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0203 |         if (method == null || method.trim().isEmpty()) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0204 |             throw new SQLException("Please select a payment method.");
```

Explanation: Raises an exception so the caller or an enclosing catch block can handle the failure.

```
0205 |         }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0206 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0207 |         try (Connection conn = new DbConnection().getConnection()) {
```

Explanation: Begins a protected block for code that may throw an exception; resources may be managed by try-with-resources.

```
0208 |             if (conn == null) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0209 |                 throw new SQLException("Could not connect to the database.");
```

Explanation: Raises an exception so the caller or an enclosing catch block can handle the failure.

```
0210 |             }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0211 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0212 |             conn.setAutoCommit(false);
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0213 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0214 |             try {
```

Explanation: Begins a protected block for code that may throw an exception; resources may be managed by try-with-resources.

```
0215 |                 double amountDue;
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0216 |                 String status;
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0217 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0218 |                 String requestSql =
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0219 |                         "SELECT pr.amount_due, pr.request_status " +
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0220 |                         "FROM payment_requests pr " +
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0221 |                         "JOIN enrollments e ON pr.enrollment_id = e.enrollment_id " +
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0222 |                         "JOIN students s ON e.student_id = s.student_id " +
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0223 |                         "WHERE pr.request_id = ? AND s.user_id = ? FOR UPDATE";
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0224 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0225 |                 try (PreparedStatement ps = conn.prepareStatement(requestSql)) {
```

Explanation: Begins a protected block for code that may throw an exception; resources may be managed by try-with-resources.

```
0226 |                     ps.setInt(1, requestId);
```

Explanation: Binds a Java value to a placeholder in the prepared SQL statement; the parameter index is one-based.

```
0227 |                     ps.setInt(2, userId);
```

Explanation: Binds a Java value to a placeholder in the prepared SQL statement; the parameter index is one-based.

```
0228 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0229 |                     try (ResultSet rs = ps.executeQuery()) {
```

Explanation: Begins a protected block for code that may throw an exception; resources may be managed by try-with-resources.

```
0230 |                         if (!rs.next()) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0231 |                             throw new SQLException(
```

Explanation: Raises an exception so the caller or an enclosing catch block can handle the failure.

```
0232 |                                     "Payment request not found for your account.");
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0233 |                         }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0234 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0235 |                         amountDue = rs.getDouble("amount_due");
```

Explanation: Reads a column value from the current database result row using the requested Java type.

```
0236 |                         status = rs.getString("request_status");
```

Explanation: Reads a column value from the current database result row using the requested Java type.

```
0237 |                     }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0238 |                 }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0239 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0240 |                 if ("Paid".equalsIgnoreCase(status)) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0241 |                     throw new SQLException("This payment request is already paid.");
```

Explanation: Raises an exception so the caller or an enclosing catch block can handle the failure.

```
0242 |                 }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0243 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0244 |                 if ("Payment Pending".equalsIgnoreCase(status)) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0245 |                     throw new SQLException(
```

Explanation: Raises an exception so the caller or an enclosing catch block can handle the failure.

```
0246 |                             "A payment has already been submitted for this request.");
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0247 |                 }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0248 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0249 |                 String insertSql =
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0250 |                         "INSERT INTO payments " +
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0251 |                         "(request_id, enrollment_id, amount_paid, payment_method, " +
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0252 |                         "reference_number, payment_status) " +
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0253 |                         "SELECT ?, enrollment_id, ?, ?, ?, 'Pending' " +
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0254 |                         "FROM payment_requests WHERE request_id = ?";
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0255 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0256 |                 try (PreparedStatement ps = conn.prepareStatement(insertSql)) {
```

Explanation: Begins a protected block for code that may throw an exception; resources may be managed by try-with-resources.

```
0257 |                     ps.setInt(1, requestId);
```

Explanation: Binds a Java value to a placeholder in the prepared SQL statement; the parameter index is one-based.

```
0258 |                     ps.setDouble(2, amountDue);
```

Explanation: Binds a Java value to a placeholder in the prepared SQL statement; the parameter index is one-based.

```
0259 |                     ps.setString(3, method.trim());
```

Explanation: Binds a Java value to a placeholder in the prepared SQL statement; the parameter index is one-based.

```
0260 |                     ps.setString(4,
```

Explanation: Binds a Java value to a placeholder in the prepared SQL statement; the parameter index is one-based.

```
0261 |                             reference == null || reference.trim().isEmpty()
```

Explanation: Checks whether a string or collection has no content.

```
0262 |                                     ? null : reference.trim());
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0263 |                     ps.setInt(5, requestId);
```

Explanation: Binds a Java value to a placeholder in the prepared SQL statement; the parameter index is one-based.

```
0264 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0265 |                     if (ps.executeUpdate() != 1) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0266 |                         throw new SQLException("Could not save the payment.");
```

Explanation: Raises an exception so the caller or an enclosing catch block can handle the failure.

```
0267 |                     }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0268 |                 }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0269 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0270 |                 String updateSql =
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0271 |                         "UPDATE payment_requests SET request_status = 'Payment Pending' " +
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0272 |                         "WHERE request_id = ?";
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0273 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0274 |                 try (PreparedStatement ps = conn.prepareStatement(updateSql)) {
```

Explanation: Begins a protected block for code that may throw an exception; resources may be managed by try-with-resources.

```
0275 |                     ps.setInt(1, requestId);
```

Explanation: Binds a Java value to a placeholder in the prepared SQL statement; the parameter index is one-based.

```
0276 |                     ps.executeUpdate();
```

Explanation: Executes an INSERT, UPDATE, or DELETE statement and returns the number of affected rows.

```
0277 |                 }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0278 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0279 |                 conn.commit();
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0280 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0281 |                 return "Payment information submitted successfully.\n" +
```

Explanation: Returns the value or result to the caller and ends the current method.

```
0282 |                         "Request ID: " + requestId + "\n" +
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0283 |                         "Amount: PHP " + String.format("%.2f", amountDue) + "\n" +
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0284 |                         "Payment Status: Pending\n" +
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0285 |                         "The cashier must verify the payment.";
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0286 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0287 |             } catch (Exception ex) {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0288 |                 conn.rollback();
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0289 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0290 |                 if (ex instanceof SQLException) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0291 |                     throw (SQLException) ex;
```

Explanation: Raises an exception so the caller or an enclosing catch block can handle the failure.

```
0292 |                 }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0293 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0294 |                 throw new SQLException("Could not submit payment.", ex);
```

Explanation: Raises an exception so the caller or an enclosing catch block can handle the failure.

```
0295 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0296 |             } finally {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0297 |                 conn.setAutoCommit(true);
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0298 |             }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0299 |         }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0300 |     }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0301 | }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

StudentRequirementActions.java

Project path: src/StudentRequirementActions.java

```
0001 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0002 | import javax.swing.*;
```

Explanation: Imports Swing GUI classes used to build the application's windows and controls.

```
0003 | import java.io.File;
```

Explanation: Imports a class or package so its types can be referenced in this source file.

```
0004 | import java.io.IOException;
```

Explanation: Imports a class or package so its types can be referenced in this source file.

```
0005 | import java.nio.file.Files;
```

Explanation: Imports NIO utilities used for character encoding or byte handling.

```
0006 | import java.nio.file.Path;
```

Explanation: Imports NIO utilities used for character encoding or byte handling.

```
0007 | import java.nio.file.StandardCopyOption;
```

Explanation: Imports NIO utilities used for character encoding or byte handling.

```
0008 | import java.sql.*;
```

Explanation: Imports JDBC classes used to communicate with a relational database.

```
0009 | import java.util.ArrayList;
```

Explanation: Imports Java utility classes such as collections, lists, maps, or helper types.

```
0010 | import java.util.List;
```

Explanation: Imports Java utility classes such as collections, lists, maps, or helper types.

```
0011 | import java.util.UUID;
```

Explanation: Imports Java utility classes such as collections, lists, maps, or helper types.

```
0012 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0013 | public class StudentRequirementActions {
```

Explanation: Declares the StudentRequirementActions type, which groups related data and behavior.

```
0014 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0015 |     private StudentDashboard dashboard;
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0016 |     private User user;
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0017 |     private JTextArea outputArea;
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0018 |     private JLabel helpLabel;
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0019 |     private DeleteDAO deleteDAO = new DeleteDAO();
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0020 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0021 |     public StudentRequirementActions(StudentDashboard dashboard) {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0022 |         this.dashboard = dashboard;
```

Explanation: Assigns or updates a variable with the value or result on the right-hand side.

```
0023 |         this.user = dashboard.user;
```

Explanation: Assigns or updates a variable with the value or result on the right-hand side.

```
0024 |         this.outputArea = dashboard.outputArea;
```

Explanation: Assigns or updates a variable with the value or result on the right-hand side.

```
0025 |         this.helpLabel = dashboard.helpLabel;
```

Explanation: Assigns or updates a variable with the value or result on the right-hand side.

```
0026 |     }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0027 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0028 |     public void submitRequirement() {
```

Explanation: Declares the submitRequirement method and its return type/parameters; the method performs the operation described by its name and body.

```
0029 |         List&lt;StudentDashboard.ChoiceItem&gt; enrollments = new ArrayList&lt;&gt;();
```

Explanation: Assigns or updates a variable with the value or result on the right-hand side.

```
0030 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0031 |         String enrollmentSql =
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0032 |             "SELECT e.enrollment_id, sy.year_label FROM enrollments e "
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0033 |             + "JOIN students s ON e.student_id = s.student_id "
```

Explanation: Assigns or updates a variable with the value or result on the right-hand side.

```
0034 |             + "JOIN school_years sy "
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0035 |             + "ON e.school_year_id = sy.school_year_id "
```

Explanation: Assigns or updates a variable with the value or result on the right-hand side.

```
0036 |             + "WHERE s.user_id = ? ORDER BY e.enrollment_id DESC";
```

Explanation: Assigns or updates a variable with the value or result on the right-hand side.

```
0037 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0038 |         Connection conn = null;
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0039 |         PreparedStatement ps = null;
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0040 |         try {
```

Explanation: Begins a protected block for code that may throw an exception; resources may be managed by try-with-resources.

```
0041 |             conn = dashboard.getConnection();
```

Explanation: Assigns or updates a variable with the value or result on the right-hand side.

```
0042 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0043 |             if (conn == null) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0044 |                 throw new SQLException("Could not connect to the database.");
```

Explanation: Raises an exception so the caller or an enclosing catch block can handle the failure.

```
0045 |             }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0046 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0047 |             ps = conn.prepareStatement(enrollmentSql);
```

Explanation: Creates a parameterized SQL statement; placeholders can be filled separately to reduce SQL-injection risk.

```
0048 |             ps.setInt(1, user.getUserId());
```

Explanation: Binds a Java value to a placeholder in the prepared SQL statement; the parameter index is one-based.

```
0049 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0050 |             try (ResultSet rs = ps.executeQuery()) {
```

Explanation: Begins a protected block for code that may throw an exception; resources may be managed by try-with-resources.

```
0051 |                 while (rs.next()) {
```

Explanation: Repeats the following block while the condition remains true.

```
0052 |                     enrollments.add(new StudentDashboard.ChoiceItem(
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0053 |                         rs.getInt("enrollment_id"),
```

Explanation: Reads a column value from the current database result row using the requested Java type.

```
0054 |                         "Enrollment #" + rs.getInt("enrollment_id")
```

Explanation: Reads a column value from the current database result row using the requested Java type.

```
0055 |                             + " - " + rs.getString("year_label")
```

Explanation: Reads a column value from the current database result row using the requested Java type.

```
0056 |                     ));
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0057 |                 }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0058 |             }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0059 |         } catch (Exception ex) {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0060 |             StudentDashboard.showError(dashboard,
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0061 |                 "Could not load enrollments: " + ex.getMessage()
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0062 |             );
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0063 |             return;
```

Explanation: Ends the current method without returning a value.

```
0064 |         } finally {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0065 |             try {
```

Explanation: Begins a protected block for code that may throw an exception; resources may be managed by try-with-resources.

```
0066 |                 if (ps != null) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0067 |                     ps.close();
```

Explanation: Closes the associated resource, such as a JDBC ResultSet, Statement, or Connection.

```
0068 |                 }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0069 |             } catch (Exception closeEx) {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0070 |                 System.out.println(closeEx);
```

Explanation: Prints a message or value to the console, commonly for status output or error reporting.

```
0071 |             }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0072 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0073 |             try {
```

Explanation: Begins a protected block for code that may throw an exception; resources may be managed by try-with-resources.

```
0074 |                 if (conn != null) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0075 |                     conn.close();
```

Explanation: Closes the associated resource, such as a JDBC ResultSet, Statement, or Connection.

```
0076 |                 }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0077 |             } catch (Exception closeEx) {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0078 |                 System.out.println(closeEx);
```

Explanation: Prints a message or value to the console, commonly for status output or error reporting.

```
0079 |             }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0080 |         }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0081 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0082 |         if (enrollments.isEmpty()) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0083 |             StudentDashboard.showError(dashboard, "Submit an enrollment application first.");
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0084 |             return;
```

Explanation: Ends the current method without returning a value.

```
0085 |         }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0086 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0087 |         StudentDashboard.ChoiceItem selectedEnrollment = (StudentDashboard.ChoiceItem)
```

Explanation: Assigns or updates a variable with the value or result on the right-hand side.

```
0088 |             JOptionPane.showInputDialog(
```

Explanation: Displays a Swing dialog to communicate a message, warning, confirmation, or input prompt to the user.

```
0089 |                 dashboard,
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0090 |                 "Select the enrollment for this document:",
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0091 |                 "Select Enrollment",
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0092 |                 JOptionPane.QUESTION_MESSAGE,
```

Explanation: Displays a Swing dialog to communicate a message, warning, confirmation, or input prompt to the user.

```
0093 |                 null,
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0094 |                 enrollments.toArray(),
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0095 |                 enrollments.get(0)
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0096 |             );
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0097 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0098 |         if (selectedEnrollment == null) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0099 |             return;
```

Explanation: Ends the current method without returning a value.

```
0100 |         }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0101 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0102 |         String[] requirementTypes = {
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0103 |             "PSA Birth Certificate",
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0104 |             "Form 138 / Report Card",
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0105 |             "Good Moral Certificate",
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0106 |             "Certificate of Transfer",
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0107 |             "Other Requirement"
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0108 |         };
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0109 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0110 |         String requirementType = (String) JOptionPane.showInputDialog(
```

Explanation: Displays a Swing dialog to communicate a message, warning, confirmation, or input prompt to the user.

```
0111 |             dashboard,
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0112 |             "Select the requirement type:",
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0113 |             "Requirement Type",
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0114 |             JOptionPane.QUESTION_MESSAGE,
```

Explanation: Displays a Swing dialog to communicate a message, warning, confirmation, or input prompt to the user.

```
0115 |             null,
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0116 |             requirementTypes,
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0117 |             requirementTypes[0]
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0118 |         );
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0119 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0120 |         if (requirementType == null) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0121 |             return;
```

Explanation: Ends the current method without returning a value.

```
0122 |         }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0123 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0124 |         JFileChooser chooser = new JFileChooser();
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0125 |         chooser.setDialogTitle("Choose Your Requirement Document");
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0126 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0127 |         if (chooser.showOpenDialog(dashboard) != JFileChooser.APPROVE_OPTION) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0128 |             return;
```

Explanation: Ends the current method without returning a value.

```
0129 |         }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0130 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0131 |         File selectedFile = chooser.getSelectedFile();
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0132 |         String lowerName = selectedFile.getName().toLowerCase();
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0133 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0134 |         if (!(lowerName.endsWith(".pdf")
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0135 |                 || lowerName.endsWith(".jpg")
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0136 |                 || lowerName.endsWith(".jpeg")
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0137 |                 || lowerName.endsWith(".png"))) {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0138 |             StudentDashboard.showError(dashboard, "Only PDF, JPG, JPEG, and PNG files are allowed.");
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0139 |             return;
```

Explanation: Ends the current method without returning a value.

```
0140 |         }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0141 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0142 |         if (!selectedFile.isFile() || selectedFile.length() == 0) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0143 |             StudentDashboard.showError(dashboard, "Select a valid, non-empty file.");
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0144 |             return;
```

Explanation: Ends the current method without returning a value.

```
0145 |         }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0146 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0147 |         if (selectedFile.length() &gt; 10L * 1024 * 1024) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0148 |             StudentDashboard.showError(dashboard, "The file must not exceed 10 MB.");
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0149 |             return;
```

Explanation: Ends the current method without returning a value.

```
0150 |         }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0151 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0152 |         int confirm = JOptionPane.showConfirmDialog(
```

Explanation: Displays a Swing dialog to communicate a message, warning, confirmation, or input prompt to the user.

```
0153 |             dashboard,
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0154 |             "Submit this requirement?\n\nType: " + requirementType
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0155 |                 + "\nFile: " + selectedFile.getName(),
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0156 |             "Confirm Submission",
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0157 |             JOptionPane.YES_NO_OPTION
```

Explanation: Displays a Swing dialog to communicate a message, warning, confirmation, or input prompt to the user.

```
0158 |         );
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0159 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0160 |         if (confirm != JOptionPane.YES_OPTION) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0161 |             return;
```

Explanation: Ends the current method without returning a value.

```
0162 |         }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0163 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0164 |         File destination = null;
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0165 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0166 |         conn = null;
```

Explanation: Assigns or updates a variable with the value or result on the right-hand side.

```
0167 |         try {
```

Explanation: Begins a protected block for code that may throw an exception; resources may be managed by try-with-resources.

```
0168 |             conn = dashboard.getConnection();
```

Explanation: Assigns or updates a variable with the value or result on the right-hand side.

```
0169 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0170 |             if (conn == null) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0171 |                 throw new SQLException("Could not connect to the database.");
```

Explanation: Raises an exception so the caller or an enclosing catch block can handle the failure.

```
0172 |             }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0173 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0174 |             String duplicateSql =
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0175 |                 "SELECT sr.requirement_id FROM student_requirements sr "
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0176 |                 + "JOIN enrollments e "
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0177 |                 + "ON sr.enrollment_id = e.enrollment_id "
```

Explanation: Assigns or updates a variable with the value or result on the right-hand side.

```
0178 |                 + "JOIN students s ON e.student_id = s.student_id "
```

Explanation: Assigns or updates a variable with the value or result on the right-hand side.

```
0179 |                 + "WHERE s.user_id = ? AND sr.enrollment_id = ? "
```

Explanation: Assigns or updates a variable with the value or result on the right-hand side.

```
0180 |                 + "AND sr.requirement_name = ?";
```

Explanation: Assigns or updates a variable with the value or result on the right-hand side.

```
0181 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0182 |             try (PreparedStatement psDuplicate =
```

Explanation: Begins a protected block for code that may throw an exception; resources may be managed by try-with-resources.

```
0183 |                     conn.prepareStatement(duplicateSql)) {
```

Explanation: Creates a parameterized SQL statement; placeholders can be filled separately to reduce SQL-injection risk.

```
0184 |                 psDuplicate.setInt(1, user.getUserId());
```

Explanation: Binds a Java value to a placeholder in the prepared SQL statement; the parameter index is one-based.

```
0185 |                 psDuplicate.setInt(2, selectedEnrollment.id);
```

Explanation: Binds a Java value to a placeholder in the prepared SQL statement; the parameter index is one-based.

```
0186 |                 psDuplicate.setString(3, requirementType);
```

Explanation: Binds a Java value to a placeholder in the prepared SQL statement; the parameter index is one-based.

```
0187 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0188 |                 try (ResultSet rs = psDuplicate.executeQuery()) {
```

Explanation: Begins a protected block for code that may throw an exception; resources may be managed by try-with-resources.

```
0189 |                     if (rs.next()) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0190 |                         StudentDashboard.showError(dashboard,
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0191 |                             "This requirement already has a record. "
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0192 |                             + "Contact staff if you need to replace it."
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0193 |                         );
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0194 |                         return;
```

Explanation: Ends the current method without returning a value.

```
0195 |                     }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0196 |                 }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0197 |             }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0198 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0199 |             File uploadFolder = new File("uploads/requirements");
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0200 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0201 |             if (!uploadFolder.exists() &amp;amp;&amp;amp; !uploadFolder.mkdirs()) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0202 |                 throw new IOException(
```

Explanation: Raises an exception so the caller or an enclosing catch block can handle the failure.

```
0203 |                     "Could not create the upload folder."
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0204 |                 );
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0205 |             }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0206 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0207 |             String extension = getFileExtension(selectedFile.getName());
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0208 |             String savedName = UUID.randomUUID() + extension;
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0209 |             destination = new File(uploadFolder, savedName);
```

Explanation: Assigns or updates a variable with the value or result on the right-hand side.

```
0210 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0211 |             Files.copy(
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0212 |                 selectedFile.toPath(),
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0213 |                 destination.toPath(),
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0214 |                 StandardCopyOption.REPLACE_EXISTING
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0215 |             );
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0216 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0217 |             String insertSql =
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0218 |                 "INSERT INTO student_requirements "
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0219 |                 + "(enrollment_id, requirement_name, requirement_status, "
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0220 |                 + "original_file_name, file_path, submitted_at) "
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0221 |                 + "VALUES (?, ?, 'Submitted', ?, ?, CURRENT_TIMESTAMP)";
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0222 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0223 |             try (PreparedStatement psInsert =
```

Explanation: Begins a protected block for code that may throw an exception; resources may be managed by try-with-resources.

```
0224 |                     conn.prepareStatement(insertSql)) {
```

Explanation: Creates a parameterized SQL statement; placeholders can be filled separately to reduce SQL-injection risk.

```
0225 |                 psInsert.setInt(1, selectedEnrollment.id);
```

Explanation: Binds a Java value to a placeholder in the prepared SQL statement; the parameter index is one-based.

```
0226 |                 psInsert.setString(2, requirementType);
```

Explanation: Binds a Java value to a placeholder in the prepared SQL statement; the parameter index is one-based.

```
0227 |                 psInsert.setString(3, selectedFile.getName());
```

Explanation: Binds a Java value to a placeholder in the prepared SQL statement; the parameter index is one-based.

```
0228 |                 psInsert.setString(4, destination.getAbsolutePath());
```

Explanation: Binds a Java value to a placeholder in the prepared SQL statement; the parameter index is one-based.

```
0229 |                 psInsert.executeUpdate();
```

Explanation: Executes an INSERT, UPDATE, or DELETE statement and returns the number of affected rows.

```
0230 |             } catch (SQLException ex) {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0231 |                 Files.deleteIfExists(destination.toPath());
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0232 |                 throw ex;
```

Explanation: Raises an exception so the caller or an enclosing catch block can handle the failure.

```
0233 |             }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0234 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0235 |             outputArea.setText(
```

Explanation: Sets the displayed text of a GUI component.

```
0236 |                 "REQUIREMENT SUBMITTED\n\n"
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0237 |                 + "Enrollment: " + selectedEnrollment.label
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0238 |                 + "\nRequirement: " + requirementType
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0239 |                 + "\nFile: " + selectedFile.getName()
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0240 |                 + "\nStatus: Submitted"
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0241 |             );
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0242 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0243 |             helpLabel.setText("Requirement submitted successfully.");
```

Explanation: Sets the displayed text of a GUI component.

```
0244 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0245 |             JOptionPane.showMessageDialog(
```

Explanation: Displays a Swing dialog to communicate a message, warning, confirmation, or input prompt to the user.

```
0246 |                 dashboard,
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0247 |                 "Your requirement has been submitted successfully."
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0248 |             );
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0249 |         } catch (SQLException | IOException ex) {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0250 |             if (destination != null &amp;amp;&amp;amp; destination.exists()) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0251 |                 try {
```

Explanation: Begins a protected block for code that may throw an exception; resources may be managed by try-with-resources.

```
0252 |                     Files.deleteIfExists(destination.toPath());
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0253 |                 } catch (IOException ignored) {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0254 |                 }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0255 |             }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0256 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0257 |             StudentDashboard.showError(dashboard,
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0258 |                 "Could not submit requirement: " + ex.getMessage()
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0259 |             );
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0260 |         } finally {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0261 |             try {
```

Explanation: Begins a protected block for code that may throw an exception; resources may be managed by try-with-resources.

```
0262 |                 if (conn != null) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0263 |                     conn.close();
```

Explanation: Closes the associated resource, such as a JDBC ResultSet, Statement, or Connection.

```
0264 |                 }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0265 |             } catch (Exception closeEx) {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0266 |                 System.out.println(closeEx);
```

Explanation: Prints a message or value to the console, commonly for status output or error reporting.

```
0267 |             }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0268 |         }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0269 |     }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0270 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0271 |     public void deleteMyRequirement() {
```

Explanation: Declares the deleteMyRequirement method and its return type/parameters; the method performs the operation described by its name and body.

```
0272 |         String input = JOptionPane.showInputDialog(
```

Explanation: Displays a Swing dialog to communicate a message, warning, confirmation, or input prompt to the user.

```
0273 |             dashboard,
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0274 |             "Enter the Requirement ID shown in View My Requirements:"
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0275 |         );
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0276 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0277 |         if (input == null || input.trim().isEmpty()) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0278 |             return;
```

Explanation: Ends the current method without returning a value.

```
0279 |         }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0280 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0281 |         int requirementId;
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0282 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0283 |         try {
```

Explanation: Begins a protected block for code that may throw an exception; resources may be managed by try-with-resources.

```
0284 |             requirementId = Integer.parseInt(input.trim());
```

Explanation: Assigns or updates a variable with the value or result on the right-hand side.

```
0285 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0286 |             if (requirementId &lt;= 0) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0287 |                 throw new NumberFormatException();
```

Explanation: Raises an exception so the caller or an enclosing catch block can handle the failure.

```
0288 |             }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0289 |         } catch (NumberFormatException ex) {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0290 |             StudentDashboard.showError(dashboard, "Enter a valid positive Requirement ID.");
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0291 |             return;
```

Explanation: Ends the current method without returning a value.

```
0292 |         }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0293 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0294 |         String filePath;
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0295 |         String status;
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0296 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0297 |         Connection conn = null;
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0298 |         try {
```

Explanation: Begins a protected block for code that may throw an exception; resources may be managed by try-with-resources.

```
0299 |             conn = dashboard.getConnection();
```

Explanation: Assigns or updates a variable with the value or result on the right-hand side.

```
0300 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0301 |             if (conn == null) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0302 |                 throw new SQLException("Could not connect to the database.");
```

Explanation: Raises an exception so the caller or an enclosing catch block can handle the failure.

```
0303 |             }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0304 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0305 |             String findSql =
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0306 |                 "SELECT sr.file_path, sr.requirement_status "
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0307 |                 + "FROM student_requirements sr "
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0308 |                 + "JOIN enrollments e "
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0309 |                 + "ON sr.enrollment_id = e.enrollment_id "
```

Explanation: Assigns or updates a variable with the value or result on the right-hand side.

```
0310 |                 + "JOIN students s ON e.student_id = s.student_id "
```

Explanation: Assigns or updates a variable with the value or result on the right-hand side.

```
0311 |                 + "WHERE sr.requirement_id = ? AND s.user_id = ?";
```

Explanation: Assigns or updates a variable with the value or result on the right-hand side.

```
0312 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0313 |             try (PreparedStatement ps =
```

Explanation: Begins a protected block for code that may throw an exception; resources may be managed by try-with-resources.

```
0314 |                     conn.prepareStatement(findSql)) {
```

Explanation: Creates a parameterized SQL statement; placeholders can be filled separately to reduce SQL-injection risk.

```
0315 |                 ps.setInt(1, requirementId);
```

Explanation: Binds a Java value to a placeholder in the prepared SQL statement; the parameter index is one-based.

```
0316 |                 ps.setInt(2, user.getUserId());
```

Explanation: Binds a Java value to a placeholder in the prepared SQL statement; the parameter index is one-based.

```
0317 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0318 |                 try (ResultSet rs = ps.executeQuery()) {
```

Explanation: Begins a protected block for code that may throw an exception; resources may be managed by try-with-resources.

```
0319 |                     if (!rs.next()) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0320 |                         StudentDashboard.showError(dashboard,
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0321 |                             "Requirement not found in your account."
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0322 |                         );
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0323 |                         return;
```

Explanation: Ends the current method without returning a value.

```
0324 |                     }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0325 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0326 |                     filePath = rs.getString("file_path");
```

Explanation: Reads a column value from the current database result row using the requested Java type.

```
0327 |                     status = rs.getString("requirement_status");
```

Explanation: Reads a column value from the current database result row using the requested Java type.

```
0328 |                 }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0329 |             }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0330 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0331 |             if (status == null
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0332 |                     || !(status.equalsIgnoreCase("Submitted")
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0333 |                     || status.equalsIgnoreCase("Incomplete"))) {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0334 |                 StudentDashboard.showError(dashboard,
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0335 |                     "You can delete only unreviewed requirements. "
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0336 |                     + "Contact enrollment staff if this document "
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0337 |                     + "has already been reviewed."
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0338 |                 );
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0339 |                 return;
```

Explanation: Ends the current method without returning a value.

```
0340 |             }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0341 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0342 |             int confirm = JOptionPane.showConfirmDialog(
```

Explanation: Displays a Swing dialog to communicate a message, warning, confirmation, or input prompt to the user.

```
0343 |                 dashboard,
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0344 |                 "Delete requirement ID " + requirementId + "?",
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0345 |                 "Confirm Requirement Deletion",
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0346 |                 JOptionPane.YES_NO_OPTION,
```

Explanation: Displays a Swing dialog to communicate a message, warning, confirmation, or input prompt to the user.

```
0347 |                 JOptionPane.WARNING_MESSAGE
```

Explanation: Displays a Swing dialog to communicate a message, warning, confirmation, or input prompt to the user.

```
0348 |             );
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0349 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0350 |             if (confirm != JOptionPane.YES_OPTION) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0351 |                 return;
```

Explanation: Ends the current method without returning a value.

```
0352 |             }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0353 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0354 |             if (!deleteDAO.deleteOwnRequirement(requirementId, user.getUserId())) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0355 |                 StudentDashboard.showError(dashboard,
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0356 |                     "The requirement could not be deleted. "
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0357 |                     + "It may have been reviewed or removed already."
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0358 |                 );
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0359 |                 return;
```

Explanation: Ends the current method without returning a value.

```
0360 |             }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0361 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0362 |             boolean fileRemoved = false;
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0363 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0364 |             if (filePath != null &amp;amp;&amp;amp; !filePath.trim().isEmpty()) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0365 |                 try {
```

Explanation: Begins a protected block for code that may throw an exception; resources may be managed by try-with-resources.

```
0366 |                     Path uploadDirectory =
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0367 |                         new File("uploads/requirements")
```

Explanation: Creates an instance of the specified class or object.

```
0368 |                             .getCanonicalFile().toPath();
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0369 |                     Path savedPath =
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0370 |                         new File(filePath).getCanonicalFile().toPath();
```

Explanation: Creates an instance of the specified class or object.

```
0371 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0372 |                     if (savedPath.startsWith(uploadDirectory)
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0373 |                             &amp;&amp; !savedPath.equals(uploadDirectory)) {
```

Explanation: Compares values for equality using the class's equals method.

```
0374 |                         fileRemoved = Files.deleteIfExists(savedPath);
```

Explanation: Assigns or updates a variable with the value or result on the right-hand side.

```
0375 |                     }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0376 |                 } catch (IOException ignored) {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0377 |                     fileRemoved = false;
```

Explanation: Assigns or updates a variable with the value or result on the right-hand side.

```
0378 |                 }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0379 |             }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0380 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0381 |             outputArea.setText(
```

Explanation: Sets the displayed text of a GUI component.

```
0382 |                 "REQUIREMENT DELETED\n\n"
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0383 |                 + "Requirement ID: " + requirementId
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0384 |                 + "\nDatabase record: Deleted"
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0385 |                 + "\nUploaded file: "
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0386 |                 + (fileRemoved ? "Deleted" : "Not found or not removed")
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0387 |             );
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0388 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0389 |             helpLabel.setText("Requirement deletion completed.");
```

Explanation: Sets the displayed text of a GUI component.

```
0390 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0391 |             JOptionPane.showMessageDialog(
```

Explanation: Displays a Swing dialog to communicate a message, warning, confirmation, or input prompt to the user.

```
0392 |                 dashboard,
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0393 |                 "The requirement record was deleted."
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0394 |                 + (fileRemoved
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0395 |                     ? "\nThe uploaded file was deleted too."
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0396 |                     : "\nThe uploaded file may still exist in the upload folder.")
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0397 |             );
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0398 |         } catch (Exception ex) {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0399 |             StudentDashboard.showError(dashboard,
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0400 |                 "Could not delete requirement: " + ex.getMessage()
```

Explanation: Provides a string literal used as text, a message, or part of a query.

```
0401 |             );
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0402 |         } finally {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0403 |             try {
```

Explanation: Begins a protected block for code that may throw an exception; resources may be managed by try-with-resources.

```
0404 |                 if (conn != null) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0405 |                     conn.close();
```

Explanation: Closes the associated resource, such as a JDBC ResultSet, Statement, or Connection.

```
0406 |                 }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0407 |             } catch (Exception closeEx) {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0408 |                 System.out.println(closeEx);
```

Explanation: Prints a message or value to the console, commonly for status output or error reporting.

```
0409 |             }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0410 |         }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0411 |     }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0412 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0413 |     private String getFileExtension(String fileName) {
```

Explanation: Declares the getFileExtension method, typically returning the requested field or resource.

```
0414 |         int dot = fileName.lastIndexOf('.');
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0415 |         return dot &gt;= 0 ? fileName.substring(dot) : "";
```

Explanation: Returns the value or result to the caller and ends the current method.

```
0416 |     }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0417 | }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

StudentUser.java

Project path: src/StudentUser.java

```
0001 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0002 | public class StudentUser extends User {
```

Explanation: Declares the StudentUser type, which groups related data and behavior.

```
0003 |     public StudentUser(int userId, String username, String email, String role) {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0004 |         super(userId, username, email, "Student/Parent");
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0005 |     }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0006 | }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

User.java

Project path: src/User.java

```
0001 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0002 | public class User {
```

Explanation: Declares the User type, which groups related data and behavior.

```
0003 |     private int userId;
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0004 |     private String username;
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0005 |     private String email;
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0006 |     private String role;
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0007 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0008 |     public User(int userId, String username, String email, String role) {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0009 |         this.userId = userId;
```

Explanation: Assigns or updates a variable with the value or result on the right-hand side.

```
0010 |         this.username = username;
```

Explanation: Assigns or updates a variable with the value or result on the right-hand side.

```
0011 |         this.email = email;
```

Explanation: Assigns or updates a variable with the value or result on the right-hand side.

```
0012 |         this.role = role;
```

Explanation: Assigns or updates a variable with the value or result on the right-hand side.

```
0013 |     }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0014 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0015 |     public int getUserId() {
```

Explanation: Declares the getUserId method, typically returning the requested field or resource.

```
0016 |         return userId;
```

Explanation: Returns the value or result to the caller and ends the current method.

```
0017 |     }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0018 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0019 |     public String getUsername() {
```

Explanation: Declares the getUsername method, typically returning the requested field or resource.

```
0020 |         return username;
```

Explanation: Returns the value or result to the caller and ends the current method.

```
0021 |     }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0022 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0023 |     public String getEmail() {
```

Explanation: Declares the getEmail method, typically returning the requested field or resource.

```
0024 |         return email;
```

Explanation: Returns the value or result to the caller and ends the current method.

```
0025 |     }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0026 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0027 |     public String getRole() {
```

Explanation: Declares the getRole method, typically returning the requested field or resource.

```
0028 |         return role;
```

Explanation: Returns the value or result to the caller and ends the current method.

```
0029 |     }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0030 | }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

UserDAO.java

Project path: src/UserDAO.java

```
0001 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0002 | import java.sql.*;
```

Explanation: Imports JDBC classes used to communicate with a relational database.

```
0003 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0004 | public class UserDAO {
```

Explanation: Declares the UserDAO type, which groups related data and behavior.

```
0005 |     private DbConnection db = new DbConnection();
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0006 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0007 |     public User login(String username, String passwordHash) {
```

Explanation: Declares the login method, which implements this part of the application's authentication or registration workflow.

```
0008 |         Connection conn = null;
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0009 |         PreparedStatement ps = null;
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0010 |         ResultSet rs = null;
```

Explanation: Declares or assigns a ResultSet that holds rows returned by a database query.

```
0011 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0012 |         try {
```

Explanation: Begins a protected block for code that may throw an exception; resources may be managed by try-with-resources.

```
0013 |             conn = db.getConnection();
```

Explanation: Assigns or updates a variable with the value or result on the right-hand side.

```
0014 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0015 |             if (conn == null) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0016 |                 return null;
```

Explanation: Returns the value or result to the caller and ends the current method.

```
0017 |             }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0018 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0019 |             String sql = "SELECT * FROM users WHERE username = ? AND password_hash = ?";
```

Explanation: Stores the SQL command as a string so the program can execute it against the database.

```
0020 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0021 |             ps = conn.prepareStatement(sql);
```

Explanation: Creates a parameterized SQL statement; placeholders can be filled separately to reduce SQL-injection risk.

```
0022 |             ps.setString(1, username);
```

Explanation: Binds a Java value to a placeholder in the prepared SQL statement; the parameter index is one-based.

```
0023 |             ps.setString(2, passwordHash);
```

Explanation: Binds a Java value to a placeholder in the prepared SQL statement; the parameter index is one-based.

```
0024 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0025 |             rs = ps.executeQuery();
```

Explanation: Executes a SQL query that returns rows, usually a SELECT, and stores them in a ResultSet.

```
0026 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0027 |             if (rs.next()) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0028 |                 return new User(
```

Explanation: Returns the value or result to the caller and ends the current method.

```
0029 |                     rs.getInt("user_id"),
```

Explanation: Reads a column value from the current database result row using the requested Java type.

```
0030 |                     rs.getString("username"),
```

Explanation: Reads a column value from the current database result row using the requested Java type.

```
0031 |                     rs.getString("email"),
```

Explanation: Reads a column value from the current database result row using the requested Java type.

```
0032 |                     rs.getString("role")
```

Explanation: Reads a column value from the current database result row using the requested Java type.

```
0033 |                 );
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0034 |             }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0035 |         } catch (Exception e) {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0036 |             System.out.println("Login error: " + e.getMessage());
```

Explanation: Prints a message or value to the console, commonly for status output or error reporting.

```
0037 |         } finally {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0038 |             try {
```

Explanation: Begins a protected block for code that may throw an exception; resources may be managed by try-with-resources.

```
0039 |                 if (rs != null) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0040 |                     rs.close();
```

Explanation: Closes the associated resource, such as a JDBC ResultSet, Statement, or Connection.

```
0041 |                 }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0042 |             } catch (Exception e) {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0043 |                 System.out.println(e);
```

Explanation: Prints a message or value to the console, commonly for status output or error reporting.

```
0044 |             }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0045 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0046 |             try {
```

Explanation: Begins a protected block for code that may throw an exception; resources may be managed by try-with-resources.

```
0047 |                 if (ps != null) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0048 |                     ps.close();
```

Explanation: Closes the associated resource, such as a JDBC ResultSet, Statement, or Connection.

```
0049 |                 }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0050 |             } catch (Exception e) {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0051 |                 System.out.println(e);
```

Explanation: Prints a message or value to the console, commonly for status output or error reporting.

```
0052 |             }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0053 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0054 |             try {
```

Explanation: Begins a protected block for code that may throw an exception; resources may be managed by try-with-resources.

```
0055 |                 if (conn != null) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0056 |                     conn.close();
```

Explanation: Closes the associated resource, such as a JDBC ResultSet, Statement, or Connection.

```
0057 |                 }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0058 |             } catch (Exception e) {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0059 |                 System.out.println(e);
```

Explanation: Prints a message or value to the console, commonly for status output or error reporting.

```
0060 |             }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0061 |         }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0062 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0063 |         return null;
```

Explanation: Returns the value or result to the caller and ends the current method.

```
0064 |     }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0065 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0066 |     public boolean addUser(String username, String passwordHash,
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0067 |                             String email, String role) {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0068 |         Connection conn = null;
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0069 |         PreparedStatement ps = null;
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0070 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0071 |         try {
```

Explanation: Begins a protected block for code that may throw an exception; resources may be managed by try-with-resources.

```
0072 |             conn = db.getConnection();
```

Explanation: Assigns or updates a variable with the value or result on the right-hand side.

```
0073 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0074 |             if (conn == null) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0075 |                 return false;
```

Explanation: Returns the value or result to the caller and ends the current method.

```
0076 |             }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0077 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0078 |             String sql = "INSERT INTO users "
```

Explanation: Stores the SQL command as a string so the program can execute it against the database.

```
0079 |                     + "(username, password_hash, email, role) VALUES (?, ?, ?, ?)";
```

Explanation: Executes this statement; its expression or method call performs the operation shown in the source.

```
0080 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0081 |             ps = conn.prepareStatement(sql);
```

Explanation: Creates a parameterized SQL statement; placeholders can be filled separately to reduce SQL-injection risk.

```
0082 |             ps.setString(1, username);
```

Explanation: Binds a Java value to a placeholder in the prepared SQL statement; the parameter index is one-based.

```
0083 |             ps.setString(2, passwordHash);
```

Explanation: Binds a Java value to a placeholder in the prepared SQL statement; the parameter index is one-based.

```
0084 |             ps.setString(3, email);
```

Explanation: Binds a Java value to a placeholder in the prepared SQL statement; the parameter index is one-based.

```
0085 |             ps.setString(4, role);
```

Explanation: Binds a Java value to a placeholder in the prepared SQL statement; the parameter index is one-based.

```
0086 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0087 |             int rowInserted = ps.executeUpdate();
```

Explanation: Executes an INSERT, UPDATE, or DELETE statement and returns the number of affected rows.

```
0088 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0089 |             return rowInserted &gt; 0;
```

Explanation: Returns the value or result to the caller and ends the current method.

```
0090 |         } catch (Exception e) {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0091 |             System.out.println("User creation error: " + e.getMessage());
```

Explanation: Prints a message or value to the console, commonly for status output or error reporting.

```
0092 |             return false;
```

Explanation: Returns the value or result to the caller and ends the current method.

```
0093 |         } finally {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0094 |             try {
```

Explanation: Begins a protected block for code that may throw an exception; resources may be managed by try-with-resources.

```
0095 |                 if (ps != null) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0096 |                     ps.close();
```

Explanation: Closes the associated resource, such as a JDBC ResultSet, Statement, or Connection.

```
0097 |                 }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0098 |             } catch (Exception e) {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0099 |                 System.out.println(e);
```

Explanation: Prints a message or value to the console, commonly for status output or error reporting.

```
0100 |             }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0101 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0102 |             try {
```

Explanation: Begins a protected block for code that may throw an exception; resources may be managed by try-with-resources.

```
0103 |                 if (conn != null) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0104 |                     conn.close();
```

Explanation: Closes the associated resource, such as a JDBC ResultSet, Statement, or Connection.

```
0105 |                 }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0106 |             } catch (Exception e) {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0107 |                 System.out.println(e);
```

Explanation: Prints a message or value to the console, commonly for status output or error reporting.

```
0108 |             }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0109 |         }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0110 |     }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0111 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0112 |     public int getUserId(String username) {
```

Explanation: Declares the getUserId method, typically returning the requested field or resource.

```
0113 |         Connection conn = null;
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0114 |         PreparedStatement ps = null;
```

Explanation: Declares a field or local variable with its data type; an initializer assigns its starting value if present.

```
0115 |         ResultSet rs = null;
```

Explanation: Declares or assigns a ResultSet that holds rows returned by a database query.

```
0116 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0117 |         try {
```

Explanation: Begins a protected block for code that may throw an exception; resources may be managed by try-with-resources.

```
0118 |             conn = db.getConnection();
```

Explanation: Assigns or updates a variable with the value or result on the right-hand side.

```
0119 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0120 |             if (conn == null) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0121 |                 return -1;
```

Explanation: Returns the value or result to the caller and ends the current method.

```
0122 |             }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0123 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0124 |             String sql = "SELECT user_id FROM users WHERE username = ?";
```

Explanation: Stores the SQL command as a string so the program can execute it against the database.

```
0125 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0126 |             ps = conn.prepareStatement(sql);
```

Explanation: Creates a parameterized SQL statement; placeholders can be filled separately to reduce SQL-injection risk.

```
0127 |             ps.setString(1, username);
```

Explanation: Binds a Java value to a placeholder in the prepared SQL statement; the parameter index is one-based.

```
0128 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0129 |             rs = ps.executeQuery();
```

Explanation: Executes a SQL query that returns rows, usually a SELECT, and stores them in a ResultSet.

```
0130 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0131 |             if (rs.next()) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0132 |                 return rs.getInt("user_id");
```

Explanation: Returns the value or result to the caller and ends the current method.

```
0133 |             }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0134 |         } catch (Exception e) {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0135 |             System.out.println("User lookup error: " + e.getMessage());
```

Explanation: Prints a message or value to the console, commonly for status output or error reporting.

```
0136 |         } finally {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0137 |             try {
```

Explanation: Begins a protected block for code that may throw an exception; resources may be managed by try-with-resources.

```
0138 |                 if (rs != null) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0139 |                     rs.close();
```

Explanation: Closes the associated resource, such as a JDBC ResultSet, Statement, or Connection.

```
0140 |                 }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0141 |             } catch (Exception e) {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0142 |                 System.out.println(e);
```

Explanation: Prints a message or value to the console, commonly for status output or error reporting.

```
0143 |             }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0144 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0145 |             try {
```

Explanation: Begins a protected block for code that may throw an exception; resources may be managed by try-with-resources.

```
0146 |                 if (ps != null) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0147 |                     ps.close();
```

Explanation: Closes the associated resource, such as a JDBC ResultSet, Statement, or Connection.

```
0148 |                 }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0149 |             } catch (Exception e) {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0150 |                 System.out.println(e);
```

Explanation: Prints a message or value to the console, commonly for status output or error reporting.

```
0151 |             }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0152 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0153 |             try {
```

Explanation: Begins a protected block for code that may throw an exception; resources may be managed by try-with-resources.

```
0154 |                 if (conn != null) {
```

Explanation: Checks a condition; the following block runs only when the condition evaluates to true.

```
0155 |                     conn.close();
```

Explanation: Closes the associated resource, such as a JDBC ResultSet, Statement, or Connection.

```
0156 |                 }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0157 |             } catch (Exception e) {
```

Explanation: Continues the surrounding declaration, expression, argument list, or control structure; interpret it together with the adjacent lines.

```
0158 |                 System.out.println(e);
```

Explanation: Prints a message or value to the console, commonly for status output or error reporting.

```
0159 |             }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0160 |         }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0161 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0162 |         return -1;
```

Explanation: Returns the value or result to the caller and ends the current method.

```
0163 |     }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

```
0164 | }
```

Explanation: Opens or closes a code block. The surrounding declaration or control statement determines the block's purpose.

database_tables.sql

Project path: database/database_tables.sql

```
0001 | CREATE TABLE IF NOT EXISTS users (
```

Explanation: Begins a table definition and lists the columns, data types, and constraints that follow.

```
0002 |     user_id        INT AUTO_INCREMENT PRIMARY KEY,
```

Explanation: Configures the numeric identifier to be generated automatically for new rows.

```
0003 |     username       VARCHAR(50)  NOT NULL UNIQUE,
```

Explanation: Requires this column to have a value rather than SQL NULL.

```
0004 |     password_hash  VARCHAR(64)  NOT NULL,
```

Explanation: Requires this column to have a value rather than SQL NULL.

```
0005 |     email          VARCHAR(100) NOT NULL UNIQUE,
```

Explanation: Requires this column to have a value rather than SQL NULL.

```
0006 |     role           VARCHAR(20)  NOT NULL,
```

Explanation: Requires this column to have a value rather than SQL NULL.

```
0007 |     is_active      TINYINT(1)   NOT NULL DEFAULT 1
```

Explanation: Requires this column to have a value rather than SQL NULL.

```
0008 | );
```

Explanation: Closes the table or SQL definition opened above.

```
0009 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0010 | CREATE TABLE IF NOT EXISTS school_years (
```

Explanation: Begins a table definition and lists the columns, data types, and constraints that follow.

```
0011 |     school_year_id INT AUTO_INCREMENT PRIMARY KEY,
```

Explanation: Configures the numeric identifier to be generated automatically for new rows.

```
0012 |     year_label     VARCHAR(20) NOT NULL,
```

Explanation: Requires this column to have a value rather than SQL NULL.

```
0013 |     is_active      TINYINT(1)  NOT NULL DEFAULT 0
```

Explanation: Requires this column to have a value rather than SQL NULL.

```
0014 | );
```

Explanation: Closes the table or SQL definition opened above.

```
0015 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0016 | CREATE TABLE IF NOT EXISTS strands (
```

Explanation: Begins a table definition and lists the columns, data types, and constraints that follow.

```
0017 |     strand_id    INT AUTO_INCREMENT PRIMARY KEY,
```

Explanation: Configures the numeric identifier to be generated automatically for new rows.

```
0018 |     strand_code  VARCHAR(20)  NOT NULL,
```

Explanation: Requires this column to have a value rather than SQL NULL.

```
0019 |     strand_name  VARCHAR(100) NOT NULL
```

Explanation: Requires this column to have a value rather than SQL NULL.

```
0020 | );
```

Explanation: Closes the table or SQL definition opened above.

```
0021 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0022 | CREATE TABLE IF NOT EXISTS students (
```

Explanation: Begins a table definition and lists the columns, data types, and constraints that follow.

```
0023 |     student_id      INT AUTO_INCREMENT PRIMARY KEY,
```

Explanation: Configures the numeric identifier to be generated automatically for new rows.

```
0024 |     user_id         INT NOT NULL,
```

Explanation: Requires this column to have a value rather than SQL NULL.

```
0025 |     lrn             VARCHAR(12)  NOT NULL UNIQUE,
```

Explanation: Requires this column to have a value rather than SQL NULL.

```
0026 |     first_name      VARCHAR(50)  NOT NULL,
```

Explanation: Requires this column to have a value rather than SQL NULL.

```
0027 |     middle_name     VARCHAR(50),
```

Explanation: Defines the current column or constraint; the comma indicates another definition follows.

```
0028 |     last_name       VARCHAR(50)  NOT NULL,
```

Explanation: Requires this column to have a value rather than SQL NULL.

```
0029 |     previous_school VARCHAR(150),
```

Explanation: Defines the current column or constraint; the comma indicates another definition follows.

```
0030 |     FOREIGN KEY (user_id) REFERENCES users(user_id)
```

Explanation: Defines a relationship to a key in another table, helping preserve referential integrity.

```
0031 | );
```

Explanation: Closes the table or SQL definition opened above.

```
0032 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0033 | CREATE TABLE IF NOT EXISTS sections (
```

Explanation: Begins a table definition and lists the columns, data types, and constraints that follow.

```
0034 |     section_id     INT AUTO_INCREMENT PRIMARY KEY,
```

Explanation: Configures the numeric identifier to be generated automatically for new rows.

```
0035 |     section_name   VARCHAR(50) NOT NULL,
```

Explanation: Requires this column to have a value rather than SQL NULL.

```
0036 |     grade_level    INT NOT NULL,
```

Explanation: Requires this column to have a value rather than SQL NULL.

```
0037 |     strand_id      INT NOT NULL,
```

Explanation: Requires this column to have a value rather than SQL NULL.

```
0038 |     school_year_id INT NOT NULL,
```

Explanation: Requires this column to have a value rather than SQL NULL.

```
0039 |     slot_limit     INT NOT NULL,
```

Explanation: Requires this column to have a value rather than SQL NULL.

```
0040 |     is_active      TINYINT(1) NOT NULL DEFAULT 1,
```

Explanation: Requires this column to have a value rather than SQL NULL.

```
0041 |     FOREIGN KEY (strand_id)      REFERENCES strands(strand_id),
```

Explanation: Defines a relationship to a key in another table, helping preserve referential integrity.

```
0042 |     FOREIGN KEY (school_year_id) REFERENCES school_years(school_year_id)
```

Explanation: Defines a relationship to a key in another table, helping preserve referential integrity.

```
0043 | );
```

Explanation: Closes the table or SQL definition opened above.

```
0044 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0045 | CREATE TABLE IF NOT EXISTS enrollments (
```

Explanation: Begins a table definition and lists the columns, data types, and constraints that follow.

```
0046 |     enrollment_id     INT AUTO_INCREMENT PRIMARY KEY,
```

Explanation: Configures the numeric identifier to be generated automatically for new rows.

```
0047 |     student_id        INT NOT NULL,
```

Explanation: Requires this column to have a value rather than SQL NULL.

```
0048 |     school_year_id    INT NOT NULL,
```

Explanation: Requires this column to have a value rather than SQL NULL.

```
0049 |     strand_id         INT NOT NULL,
```

Explanation: Requires this column to have a value rather than SQL NULL.

```
0050 |     section_id        INT NULL,
```

Explanation: Defines the current column or constraint; the comma indicates another definition follows.

```
0051 |     year_level        INT NOT NULL,
```

Explanation: Requires this column to have a value rather than SQL NULL.

```
0052 |     enrollee_type     VARCHAR(30) NOT NULL,
```

Explanation: Requires this column to have a value rather than SQL NULL.

```
0053 |     enrollment_status VARCHAR(20) NOT NULL DEFAULT 'Pending',
```

Explanation: Requires this column to have a value rather than SQL NULL.

```
0054 |     FOREIGN KEY (student_id)     REFERENCES students(student_id),
```

Explanation: Defines a relationship to a key in another table, helping preserve referential integrity.

```
0055 |     FOREIGN KEY (school_year_id) REFERENCES school_years(school_year_id),
```

Explanation: Defines a relationship to a key in another table, helping preserve referential integrity.

```
0056 |     FOREIGN KEY (strand_id)      REFERENCES strands(strand_id),
```

Explanation: Defines a relationship to a key in another table, helping preserve referential integrity.

```
0057 |     FOREIGN KEY (section_id)     REFERENCES sections(section_id)
```

Explanation: Defines a relationship to a key in another table, helping preserve referential integrity.

```
0058 | );
```

Explanation: Closes the table or SQL definition opened above.

```
0059 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0060 | CREATE TABLE IF NOT EXISTS student_requirements (
```

Explanation: Begins a table definition and lists the columns, data types, and constraints that follow.

```
0061 |     requirement_id     INT AUTO_INCREMENT PRIMARY KEY,
```

Explanation: Configures the numeric identifier to be generated automatically for new rows.

```
0062 |     enrollment_id      INT NOT NULL,
```

Explanation: Requires this column to have a value rather than SQL NULL.

```
0063 |     requirement_name   VARCHAR(100) NOT NULL,
```

Explanation: Requires this column to have a value rather than SQL NULL.

```
0064 |     requirement_status VARCHAR(20)  NOT NULL,
```

Explanation: Requires this column to have a value rather than SQL NULL.

```
0065 |     original_file_name VARCHAR(255),
```

Explanation: Defines the current column or constraint; the comma indicates another definition follows.

```
0066 |     file_path          VARCHAR(500),
```

Explanation: Defines the current column or constraint; the comma indicates another definition follows.

```
0067 |     submitted_at       TIMESTAMP NULL,
```

Explanation: Defines the current column or constraint; the comma indicates another definition follows.

```
0068 |     FOREIGN KEY (enrollment_id) REFERENCES enrollments(enrollment_id)
```

Explanation: Defines a relationship to a key in another table, helping preserve referential integrity.

```
0069 | );
```

Explanation: Closes the table or SQL definition opened above.

```
0070 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0071 | CREATE TABLE IF NOT EXISTS requirements (
```

Explanation: Begins a table definition and lists the columns, data types, and constraints that follow.

```
0072 |     requirement_id   INT AUTO_INCREMENT PRIMARY KEY,
```

Explanation: Configures the numeric identifier to be generated automatically for new rows.

```
0073 |     requirement_name VARCHAR(100) NOT NULL
```

Explanation: Requires this column to have a value rather than SQL NULL.

```
0074 | );
```

Explanation: Closes the table or SQL definition opened above.

```
0075 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0076 | CREATE TABLE IF NOT EXISTS submitted_requirements (
```

Explanation: Begins a table definition and lists the columns, data types, and constraints that follow.

```
0077 |     enrollment_id  INT NOT NULL,
```

Explanation: Requires this column to have a value rather than SQL NULL.

```
0078 |     requirement_id INT NOT NULL,
```

Explanation: Requires this column to have a value rather than SQL NULL.

```
0079 |     status         VARCHAR(20) NOT NULL,
```

Explanation: Requires this column to have a value rather than SQL NULL.

```
0080 |     UNIQUE (enrollment_id, requirement_id),
```

Explanation: Requires values in this column or constraint to be unique.

```
0081 |     FOREIGN KEY (enrollment_id)  REFERENCES enrollments(enrollment_id),
```

Explanation: Defines a relationship to a key in another table, helping preserve referential integrity.

```
0082 |     FOREIGN KEY (requirement_id) REFERENCES requirements(requirement_id)
```

Explanation: Defines a relationship to a key in another table, helping preserve referential integrity.

```
0083 | );
```

Explanation: Closes the table or SQL definition opened above.

```
0084 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0085 | CREATE TABLE IF NOT EXISTS fees (
```

Explanation: Begins a table definition and lists the columns, data types, and constraints that follow.

```
0086 |     fee_id         INT AUTO_INCREMENT PRIMARY KEY,
```

Explanation: Configures the numeric identifier to be generated automatically for new rows.

```
0087 |     fee_name       VARCHAR(100) NOT NULL,
```

Explanation: Requires this column to have a value rather than SQL NULL.

```
0088 |     amount         DECIMAL(10,2) NOT NULL,
```

Explanation: Requires this column to have a value rather than SQL NULL.

```
0089 |     school_year_id INT NOT NULL,
```

Explanation: Requires this column to have a value rather than SQL NULL.

```
0090 |     strand_id      INT NOT NULL,
```

Explanation: Requires this column to have a value rather than SQL NULL.

```
0091 |     FOREIGN KEY (school_year_id) REFERENCES school_years(school_year_id),
```

Explanation: Defines a relationship to a key in another table, helping preserve referential integrity.

```
0092 |     FOREIGN KEY (strand_id)      REFERENCES strands(strand_id)
```

Explanation: Defines a relationship to a key in another table, helping preserve referential integrity.

```
0093 | );
```

Explanation: Closes the table or SQL definition opened above.

```
0094 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0095 | CREATE TABLE IF NOT EXISTS discounts (
```

Explanation: Begins a table definition and lists the columns, data types, and constraints that follow.

```
0096 |     discount_id    INT AUTO_INCREMENT PRIMARY KEY,
```

Explanation: Configures the numeric identifier to be generated automatically for new rows.

```
0097 |     discount_name  VARCHAR(100) NOT NULL,
```

Explanation: Requires this column to have a value rather than SQL NULL.

```
0098 |     discount_type  VARCHAR(20)  NOT NULL,
```

Explanation: Requires this column to have a value rather than SQL NULL.

```
0099 |     discount_value DECIMAL(10,2) NOT NULL
```

Explanation: Requires this column to have a value rather than SQL NULL.

```
0100 | );
```

Explanation: Closes the table or SQL definition opened above.

```
0101 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0102 | CREATE TABLE IF NOT EXISTS payment_requests (
```

Explanation: Begins a table definition and lists the columns, data types, and constraints that follow.

```
0103 |     request_id     INT AUTO_INCREMENT PRIMARY KEY,
```

Explanation: Configures the numeric identifier to be generated automatically for new rows.

```
0104 |     enrollment_id  INT NOT NULL,
```

Explanation: Requires this column to have a value rather than SQL NULL.

```
0105 |     amount_due     DECIMAL(10,2) NOT NULL,
```

Explanation: Requires this column to have a value rather than SQL NULL.

```
0106 |     description    VARCHAR(255),
```

Explanation: Defines the current column or constraint; the comma indicates another definition follows.

```
0107 |     request_status VARCHAR(20) NOT NULL DEFAULT 'Open',
```

Explanation: Requires this column to have a value rather than SQL NULL.

```
0108 |     created_at     TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
```

Explanation: Defines the current column or constraint; the comma indicates another definition follows.

```
0109 |     FOREIGN KEY (enrollment_id) REFERENCES enrollments(enrollment_id)
```

Explanation: Defines a relationship to a key in another table, helping preserve referential integrity.

```
0110 | );
```

Explanation: Closes the table or SQL definition opened above.

```
0111 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0112 | CREATE TABLE IF NOT EXISTS payment_request_details (
```

Explanation: Begins a table definition and lists the columns, data types, and constraints that follow.

```
0113 |     request_id INT NOT NULL,
```

Explanation: Requires this column to have a value rather than SQL NULL.

```
0114 |     item_name  VARCHAR(100) NOT NULL,
```

Explanation: Requires this column to have a value rather than SQL NULL.

```
0115 |     item_type  VARCHAR(20)  NOT NULL,
```

Explanation: Requires this column to have a value rather than SQL NULL.

```
0116 |     amount     DECIMAL(10,2) NOT NULL,
```

Explanation: Requires this column to have a value rather than SQL NULL.

```
0117 |     FOREIGN KEY (request_id) REFERENCES payment_requests(request_id)
```

Explanation: Defines a relationship to a key in another table, helping preserve referential integrity.

```
0118 | );
```

Explanation: Closes the table or SQL definition opened above.

```
0119 |
```

Explanation: Blank line used to visually separate related parts of the source code.

```
0120 | CREATE TABLE IF NOT EXISTS payments (
```

Explanation: Begins a table definition and lists the columns, data types, and constraints that follow.

```
0121 |     payment_id       INT AUTO_INCREMENT PRIMARY KEY,
```

Explanation: Configures the numeric identifier to be generated automatically for new rows.

```
0122 |     request_id       INT NULL,
```

Explanation: Defines the current column or constraint; the comma indicates another definition follows.

```
0123 |     enrollment_id    INT NOT NULL,
```

Explanation: Requires this column to have a value rather than SQL NULL.

```
0124 |     amount_paid      DECIMAL(10,2) NOT NULL,
```

Explanation: Requires this column to have a value rather than SQL NULL.

```
0125 |     payment_method   VARCHAR(30)  NOT NULL,
```

Explanation: Requires this column to have a value rather than SQL NULL.

```
0126 |     reference_number VARCHAR(100),
```

Explanation: Defines the current column or constraint; the comma indicates another definition follows.

```
0127 |     payment_status   VARCHAR(20)  NOT NULL DEFAULT 'Pending',
```

Explanation: Requires this column to have a value rather than SQL NULL.

```
0128 |     paid_at          TIMESTAMP NULL,
```

Explanation: Defines the current column or constraint; the comma indicates another definition follows.

```
0129 |     FOREIGN KEY (request_id)    REFERENCES payment_requests(request_id),
```

Explanation: Defines a relationship to a key in another table, helping preserve referential integrity.

```
0130 |     FOREIGN KEY (enrollment_id) REFERENCES enrollments(enrollment_id)
```

Explanation: Defines a relationship to a key in another table, helping preserve referential integrity.

```
0131 | );
```

Explanation: Closes the table or SQL definition opened above.